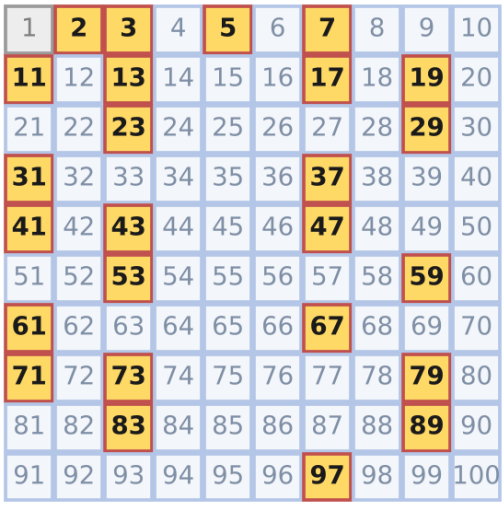
Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

SỐ HỌC

Từ cơ bản đến nâng cao

TẬP 1 · LÝ THUYẾT



$$a = b \cdot q + r$$

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

$$a^{p-1} \equiv 1 \pmod{p}$$

$$C(n, k) = n! \,/\, (k!\,(n-k)!)$$

$$ax + by = \gcd(a, b)$$

Chương 1. CHIA HẾT VÀ ƯỚC SỐ
Chương 2. SỐ NGUYÊN TỐ
Chương 3. SỐ HỌC ĐỒNG DƯ
Chương 4. TỔ HỢP VÀ ĐẾM
Chương 5. KĨ THUẬT SỐ HỌC TRONG LẬP TRÌNH

Dành cho học sinh THCS, THPT ôn luyện học sinh giỏi Tin học các cấp

LỜI NÓI ĐẦU

Khoảng năm 300 trước Công nguyên, trong bộ sách *Cơ sở*, Euclid mô tả một cách tìm ước chung lớn nhất của hai số chỉ bằng những phép trừ liên tiếp. Hơn hai nghìn năm sau, chính thuật toán ấy chạy hàng tỉ lần mỗi ngày bên trong các máy tính: trong phần mềm mã hóa bảo vệ tài khoản ngân hàng, trong các thư viện tính toán, và trong gần như mọi đề thi học sinh giỏi Tin học có một chút "mùi số học".

Học sinh thường gặp ba khó khăn với các bài số học. Thứ nhất, **số quá lớn**: đề cho n tới 10^{12}, 10^{18}, có khi là một số dài cả triệu chữ số, và mọi vòng lặp "thử từng số" đều chạy không kịp. Thứ hai, **những cái bẫy âm thầm**: tràn số, số dư âm, chia trong đồng dư..., chương trình không báo lỗi mà chỉ in ra kết quả sai. Thứ ba, các công thức thường được học thuộc như những "câu thần chú", nên khi đề bài thay đổi một chút là không biết bắt đầu từ đâu.

Cuốn sách này được viết để giải quyết cả ba khó khăn ấy. Mỗi bài học không đưa ra công thức ngay. Em sẽ được đặt trước một bài toán cụ thể, viết chương trình ngây thơ (có sơ đồ khối và code đầy đủ), đếm số phép tính hoặc chỉ ra chỗ tràn số để thấy bế tắc, rồi được dẫn từng bước tới ý tưởng tốt, kèm chứng minh vì sao nó đúng. Mọi ví dụ đều được chạy tay bằng bảng và hình vẽ; các con số trong bảng được sinh ra bằng cách chạy chính thuật toán, nên em có thể tin từng con số.

Bộ tài liệu gồm ba tập đi cùng nhau:

- **Tập 1: Lý thuyết** (cuốn sách này) gồm 20 bài học trong 5 chương: Chia hết và ước số; Số nguyên tố; Số học đồng dư; Tổ hợp và đếm; Kĩ thuật số học trong lập trình.
- **Tập 2: Bài tập** gồm các bài tập theo từng bài học, ở ba mức độ (cơ bản, trung bình, nâng cao), có ràng buộc theo subtask như đề thi thật. Tên các bài tập được giới thiệu ở mục *Luyện tập* cuối mỗi bài học.
- **Tập 3: Hướng dẫn giải** phân tích từng bài tập từ cách ngây thơ đến cách tối ưu, kèm chương trình hoàn chỉnh và bảng điểm theo thuật toán.

Chương trình được viết bằng C++ theo chuẩn C++14, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến. Mọi chương trình trong sách đều đã được kiểm tra bằng cách đối chiếu với chương trình ngây thơ (hoặc với phép tính số lớn chính xác) trên hàng trăm bộ dữ liệu ngẫu nhiên.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học đi theo cùng một con đường, giống như cách một người giỏi suy nghĩ khi gặp bài mới:

Phần	Nội dung
Khởi động	Một câu chuyện, một câu đố gần gũi để em bắt đầu nghĩ.
Kiến thức nền	Định nghĩa, định lí, hình vẽ trực quan và ví dụ tính tay.
Bài toán	Đề bài cụ thể, có ví dụ được sinh bằng chính chương trình trong sách.
Ý tưởng ngây thơ	Cách làm đầu tiên ai cũng nghĩ ra, kèm sơ đồ khối, chương trình đầy đủ và phần đếm số phép tính.
Bế tắc	Chỉ ra chính xác vì sao cách ngây thơ chậm (hoặc sai, hoặc tràn số), và đặt câu hỏi dẫn tới ý tưởng mới.
Phá bế tắc	Từng bước: ý tưởng, chạy tay bằng hình và bảng, chứng minh, chương trình.
Mở rộng	Các biến thể và ứng dụng thường gặp trong đề thi.
Bẫy thường gặp	Những lỗi đã làm mất điểm của nhiều thế hệ học sinh.
Ghi nhớ	Tóm tắt những điều cốt lõi nhất.
Luyện tập	Giới thiệu các bài tập tương ứng ở Tập 2 và vài câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
BÀI TOÁN	Đề bài chính thức.
ĐỊNH NGHĨA	Khái niệm mới cần nắm chính xác.
ĐỊNH LÍ	Kết quả quan trọng, thường kèm chứng minh.
THUẬT TOÁN	Các bước của thuật toán, viết bằng lời.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp: $O(\sqrt{n})$, $O(\log n)$, $O(n \log \log n)$...
BẾ TẮC	Chỗ cách làm hiện tại không vượt qua được.
VÌ SAO ĐÚNG?	Lập luận chứng minh tính đúng đắn.
HÃY HÌNH DUNG	Hình ảnh trực quan giúp nhớ lâu.
VÍ DỤ	Một ví dụ tính tay ngắn.
LƯU Ý	Chi tiết nhỏ nhưng dễ sai.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.

Quy ước màu trong hình vẽ

Màu	Ý nghĩa
Ô vàng viền đỏ	Giá trị đang xét ở bước đó, hoặc kết quả cuối cùng.
Ô xanh dương	Giá trị đã tính xong.
Ô cam nhạt	Dữ liệu vào (chữ số, số hạng...).
Ô xám	Giá trị khởi tạo, hoặc số đã bị loại (ví dụ: hợp số trong sàng).
Chữ đỏ	Kết luận, đáp số, hoặc chỗ cần chú ý.

Kí hiệu dùng trong sách

Kí hiệu	Ý nghĩa
$a \bmod m$	Số dư (không âm) của a khi chia cho m.
$\lfloor x \rfloor$, $\lceil x \rceil$	Phần nguyên dưới (số nguyên lớn nhất $\le x$), phần nguyên trên (số nguyên nhỏ nhất $\ge x$).
$b \mid a$	b chia hết a (a chia hết cho b).
$a \equiv b \pmod m$	a và b đồng dư theo mô-đun m: $m \mid (a - b)$.
gcd(a, b), lcm(a, b)	Ước chung lớn nhất, bội chung nhỏ nhất (trong lời văn viết ƯCLN, BCNN).
p	Thường dùng cho số nguyên tố.
d(n), σ(n), φ(n)	Số ước, tổng các ước, số các số trong 1..n nguyên tố cùng nhau với n.
C(n, k)	Số cách chọn k phần tử từ n phần tử (tổ hợp chập k của n).
O(...)	Độ phức tạp thời gian. Máy tính làm khoảng 10^8 phép tính đơn giản mỗi giây.

LỜI KHUYÊN KHI HỌC

Hãy đọc với giấy và bút bên cạnh. Mỗi khi gặp một bảng chạy tay, hãy che bảng lại và tự tính trước. Mỗi khi gặp một định lí, hãy thử nó với vài số nhỏ trước khi đọc chứng minh. Và mỗi khi viết một chương trình số học, hãy tự hỏi ba câu: **giá trị lớn nhất có thể là bao nhiêu? có số âm không? có phép chia nào không?** Ba câu hỏi ấy chặn được phần lớn các lỗi số học.

MỤC LỤC

LỜI NÓI ĐẦU **2**

CÁCH HỌC VỚI TÀI LIỆU NÀY **3**

CHƯƠNG 1. CHIA HẾT VÀ ƯỚC SỐ **6**

Bài 1. Phép chia có dư và đồng dư 7
Bài 2. Ước số: liệt kê, đếm và tính tổng 16
Bài 3. Ước chung lớn nhất và bội chung nhỏ nhất 23
Bài 4. Euclid mở rộng và phương trình ax + by = c 31

CHƯƠNG 2. SỐ NGUYÊN TỐ **39**

Bài 5. Số nguyên tố và kiểm tra nguyên tố 40
Bài 6. Sàng Eratosthenes 53
Bài 7. Phân tích thừa số nguyên tố 65
Bài 8. Sàng trên đoạn 75

CHƯƠNG 3. SỐ HỌC ĐỒNG DƯ **85**

Bài 9. Lũy thừa nhanh 86
Bài 10. Nghịch đảo mô-đun 98
Bài 11. Hàm phi Euler và định lí Euler 110
Bài 12. Định lí số dư Trung Hoa 122

CHƯƠNG 4. TỔ HỢP VÀ ĐẾM **133**

Bài 13. Tổ hợp và hệ số nhị thức 134
Bài 14. Nguyên lí bù trừ 148
Bài 15. Dãy truy hồi và lũy thừa ma trận 160
Bài 16. Kĩ thuật chia khối ⌊n / i⌋ 172

CHƯƠNG 5. KĨ THUẬT SỐ HỌC TRONG LẬP TRÌNH **182**

Bài 17. Số nguyên lớn 183
Bài 18. Hệ cơ số và biểu diễn nhị phân 196
Bài 19. Tổng tiền tố và số dư 207
Bài 20. Điểm nguyên và hình học số nguyên 217

PHỤ LỤC A. TỔNG HỢP THUẬT TOÁN VÀ CÔNG THỨC **227**

PHỤ LỤC B. NHỮNG CON SỐ ĐÁNG NHỚ **230**

CHƯƠNG 1. CHIA HẾT VÀ ƯỚC SỐ

Mọi thứ trong số học bắt đầu từ một phép tính em đã biết từ năm lớp 3: phép chia có dư. Từ đó nảy ra những câu hỏi rất "Tin học": số dư của một số dài một triệu chữ số là bao nhiêu? Một số có bao nhiêu ước? Làm sao tìm ước chung lớn nhất của hai số cỡ 10^{18} trong nháy mắt? Phương trình $3x + 5y = 2026$ có nghiệm nguyên không?

Chương này xây nền móng cho cả cuốn sách: đồng dư, ước số, ước chung lớn nhất và thuật toán Euclid. Thuật toán Euclid ra đời hơn 2300 năm trước, nhưng đến nay vẫn là một trong những thuật toán được dùng nhiều nhất trong lập trình thi đấu.

TRONG CHƯƠNG NÀY

Bài 1. Phép chia có dư và đồng dư: thương, số dư và số dư của một số rất lớn

Bài 2. Ước số: liệt kê, đếm và tính tổng: các ước đi theo cặp, chỉ cần duyệt tới $\sqrt{n}$

Bài 3. Ước chung lớn nhất và bội chung nhỏ nhất: thuật toán Euclid

Bài 4. Euclid mở rộng và phương trình $ax + by = c$: định lí Bézout và họ nghiệm

BÀI 1 Phép chia có dư và đồng dư

Học xong bài này, em sẽ:

- ✓ Hiểu chính xác **thương** và **số dư** của phép chia, kể cả khi số bị chia là số âm.
- ✓ Biết phép `/` và `%` của C++ khác quy ước toán học ở chỗ nào, và cách sửa trong một dòng.
- ✓ Nắm khái niệm **đồng dư** $a \equiv b \pmod m$ và các tính chất cộng, trừ, nhân của nó.
- ✓ Tính được số dư của một số có tới hàng triệu chữ số trong O(độ dài), không cần lưu cả số.

KHỞI ĐỘNG

Hôm nay là **thứ Hai**. Cô giáo hỏi: "1000 ngày nữa là thứ mấy? Còn 100 ngày trước là thứ mấy?" Cả lớp đếm trên lịch một lúc rồi bỏ cuộc. Một bạn ngồi cuối lớp trả lời sau mười giây: "Chủ nhật, và thứ Bảy ạ."

Bạn ấy không đếm từng ngày. Bạn ấy chỉ cần biết 1000 **chia cho 7 dư bao nhiêu**. Bài học này cho em cách nghĩ ấy, rồi dùng nó để tính số dư của một số dài một triệu chữ số, thứ mà không kiểu số nào của C++ chứa nổi.

1. Phép chia có dư

Chia 17 cái kẹo cho 5 bạn, mỗi bạn được 3 cái và còn thừa 2 cái. Ta viết $17 = 5 \cdot 3 + 2$: **3 là thương**, **2 là số dư**. Số dư phải nhỏ hơn số bạn: nếu còn thừa từ 5 cái trở lên thì chia thêm được một lượt nữa.

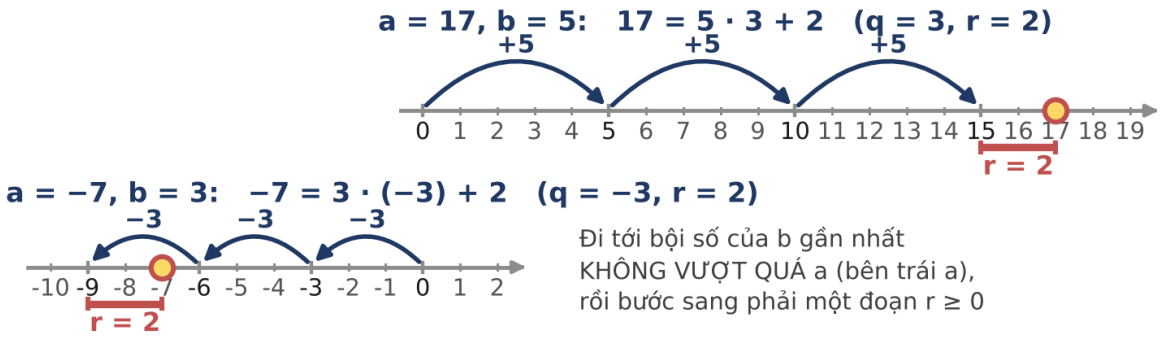
ĐỊNH NGHĨA (PHÉP CHIA CÓ DƯ)

Với số nguyên a và số nguyên dương b, luôn tồn tại **duy nhất** một cặp số nguyên q, r sao cho:

$$\mathbf{a = b \cdot q + r \quad và \quad 0 \le r < b}$$

q gọi là **thương**, r gọi là **số dư** của phép chia a cho b. Kí hiệu: $q = \lfloor a / b \rfloor$ (phần nguyên dưới, tức số nguyên lớn nhất không vượt quá a / b), $r = a \bmod b$. Nếu $r = 0$ ta nói a **chia hết** cho b, viết $b \mid a$.

Trên trục số, phép chia có dư có một hình ảnh rất dễ nhớ: đi từ 0 theo những bước nhảy dài b, dừng ở **bội số của b gần a nhất mà không vượt quá a**. Số bước nhảy là q, đoạn còn lại tới a là r. Cách nhìn này đúng cả khi a âm: khi đó ta nhảy sang trái.



Hình 1.1. *Thương là số bước nhảy, số dư là đoạn còn lại; số dư luôn thuộc [0, b).*

VÌ SAO CẶP (q, r) LUÔN TỒN TẠI VÀ DUY NHẤT?

Tồn tại: các bội số của b chia trục số thành những đoạn dài b liền nhau. Số a rơi vào đúng một đoạn $[b \cdot q, b \cdot q + b)$, nên $r = a - b \cdot q$ thỏa $0 \le r < b$.

Duy nhất: giả sử $a = b \cdot q + r = b \cdot q' + r'$ với $0 \le r, r' < b$. Trừ vế: $b \cdot (q - q') = r' - r$. Vế phải nằm trong khoảng $(-b, b)$ mà lại là bội của b, nên nó bằng 0. Vậy $r = r'$, $q = q'$.

2. Phép chia trong C++ và số âm

Với $a \ge 0$, phép `a / b` và `a % b` của C++ cho đúng q và r. Nhưng với a âm, C++ **làm tròn thương về phía 0** chứ không làm tròn xuống, nên số dư có thể âm:

a	b	a / b (C++)	a % b (C++)	q (toán học)	r (toán học)
17	5	3	2	3	2
−17	5	**−3**	**−2**	−4	3
−7	3	**−2**	**−1**	−3	2
−6	3	−2	0	−2	0

Quy tắc của C++ (từ chuẩn C++11): `a / b` bỏ phần thập phân (làm tròn về 0), còn `a % b` mang **dấu của a** và luôn thỏa `a == (a / b) * b + a % b`. Khi đề bài cần số dư theo nghĩa toán học (luôn không âm), hãy dùng công thức:

$$\mathbf{r = ((a \ \%\ m) + m) \ \%\ m}$$

Nếu a % m đã không âm thì cộng m rồi lấy dư lần nữa không làm thay đổi gì; nếu a % m âm (thuộc khoảng $(-m, 0)$) thì cộng m đưa nó về đúng khoảng $[0, m)$.

VÍ DỤ: THỨ TRONG TUẦN

Đánh số thứ Hai = 0, thứ Ba = 1, ..., Chủ nhật = 6. Sau (hoặc trước) k ngày kể từ ngày t thì thứ là (t + k) mod 7, với k âm nghĩa là lùi về quá khứ.

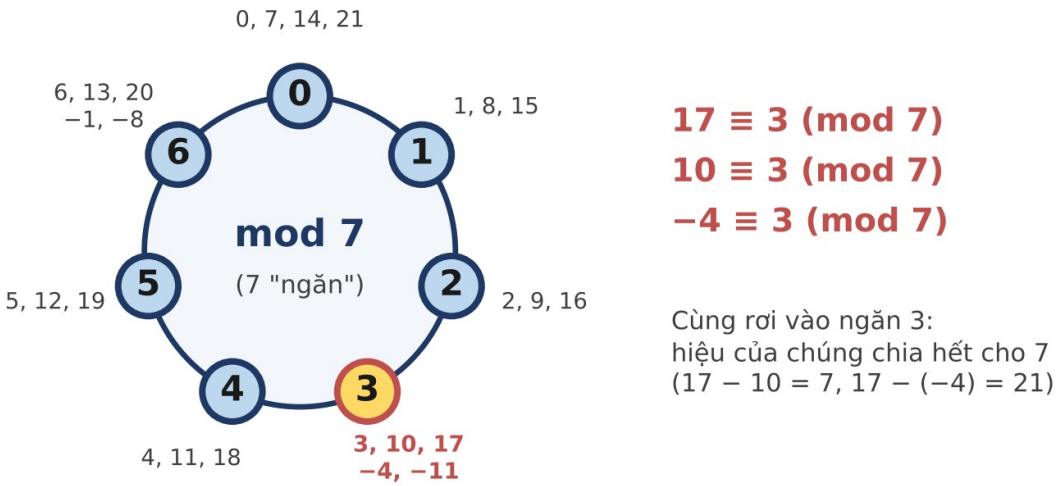
- 1000 ngày nữa: $1000 = 7 \cdot 142 + 6$, ra 6: **Chủ nhật**.
- 100 ngày trước: $-100 = 7 \cdot (-15) + 5$, ra 5: **thứ Bảy**. Trong C++, `-100 % 7` bằng −2: phải sửa thành `((-100 % 7) + 7) % 7 = 5`.
- 10^{18} ngày nữa: $10^{18} \bmod 7 = 1$: **thứ Ba**. (Em sẽ tự tính được số này ở cuối bài.)

3. Đồng dư

Trên "đồng hồ 7 ngăn" ở Hình 1.2, các số 3, 10, 17, −4 đều rơi vào cùng ngăn 3. Chúng khác nhau, nhưng với câu hỏi "thứ mấy?" thì chúng **như nhau**. Toán học có một từ riêng cho chữ "như nhau" ấy.

ĐỊNH NGHĨA (ĐỒNG DƯ)

Cho số nguyên dương m. Hai số nguyên a, b gọi là **đồng dư theo mô-đun m**, viết $a \equiv b \pmod m$, nếu chúng có cùng số dư khi chia cho m; tương đương: **m chia hết a − b**.



Hình 1.2. *Mỗi số nguyên rơi vào đúng một trong m "ngăn" 0, 1, ..., m − 1. Các số cùng ngăn thì đồng dư với nhau.*

Sức mạnh của đồng dư nằm ở chỗ: khi chỉ quan tâm đến số dư, ta được phép **thay một số bằng số dư của nó** ở bất kì đâu trong một biểu thức chỉ có cộng, trừ, nhân.

ĐỊNH LÍ (TÍNH CHẤT CỦA ĐỒNG DƯ)

Nếu $a \equiv a' \pmod m$ và $b \equiv b' \pmod m$ thì:

- $a + b \equiv a' + b' \pmod m$
- $a - b \equiv a' - b' \pmod m$
- $a \cdot b \equiv a' \cdot b' \pmod m$
- $a^k \equiv a'^k \pmod m$ với mọi số tự nhiên k.

Dạng hay dùng trong lập trình: $(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$, và $(a \cdot b) \bmod m = ((a \bmod m) \cdot (b \bmod m)) \bmod m$.

VÌ SAO ĐÚNG?

Đặt $a = a' + m \cdot s$ và $b = b' + m \cdot t$. Khi đó $a + b = (a' + b') + m \cdot (s + t)$, nên $a + b$ và $a' + b'$ chênh nhau một bội của m. Với phép nhân: $a \cdot b = a' \cdot b' + m \cdot (a' \cdot t + b' \cdot s + m \cdot s \cdot t)$: lại chênh nhau một bội của m. Lũy thừa là nhân nhiều lần.

CẨN THẬN VỚI PHÉP CHIA

Đồng dư **không** cho phép chia hai vế một cách tùy tiện: $2 \cdot 3 \equiv 2 \cdot 8 \pmod{10}$ (vì 6 và 16 cùng dư 6), nhưng $3 \not\equiv 8 \pmod{10}$. Phép "chia" trong đồng dư cần một công cụ riêng là **nghịch đảo mô-đun**, em sẽ học ở Bài 10.

4. Bài toán SODU

BÀI TOÁN: SODU

Cho một số nguyên không âm N rất lớn, có tới 10^6 chữ số, và số nguyên dương m. Hãy tính số dư của N khi chia cho m.

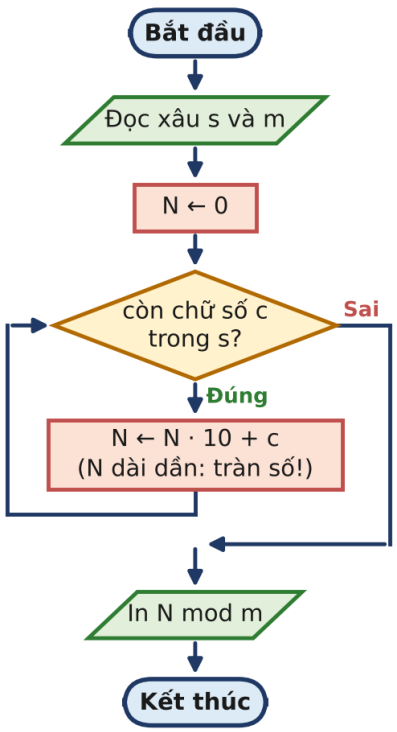
Dữ liệu vào: Dòng đầu chứa N (không có chữ số 0 thừa ở đầu). Dòng thứ hai chứa m ($1 \le m \le 10^9$).

Kết quả: Một số nguyên là $N \bmod m$.

SODU.INP	SODU.OUT	Giải thích
2026 7	3	$2026 = 7 \cdot 289 + 3$.
12345678901234567890 1234567890 1000000007	197434842	N có 30 chữ số, vượt xa giới hạn của mọi kiểu số nguyên trong C++.
7 9	7	Số bị chia nhỏ hơn số chia: thương bằng 0, số dư chính là N.

5. Ý tưởng ngây thơ: Dựng lại cả số N

Cách tự nhiên nhất: đọc N, đổi ra một số nguyên, rồi lấy dư. Vì N được cho dưới dạng các chữ số nên ta đọc vào xâu và dựng lại N từ trái sang phải: mỗi chữ số mới làm N gấp 10 lần rồi cộng thêm chữ số đó.



Hình 1.3. *Sơ đồ khối của cách dựng lại cả số N.*

SODU.CPP (ngây thơ): dựng lại N bằng unsigned long long

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("SODU.INP", "r")) {
        freopen("SODU.INP", "r", stdin);
        freopen("SODU.OUT", "w", stdout);
    }
    string s;
    long long m;
    cin >> s >> m;
    unsigned long long N = 0;    // dựng lại cả số N rồi mới chia
    for (char c : s) N = N * 10 + (c - '0');  // tràn khi N > 19 chữ số!
    cout << N % m << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Chương trình chỉ có một vòng lặp qua các chữ số: O(L) phép tính với L là độ dài của N, rất nhanh. Vấn đề không nằm ở thời gian mà ở **độ lớn**: `unsigned long long` chỉ chứa được số nhỏ hơn $2^{64} \approx 1{,}8 \cdot 10^{19}$, tức khoảng 19 chữ số.

BẾ TẮC

Với ví dụ 2 (30 chữ số), N bị **tràn số** từ chữ số thứ 20: máy tính âm thầm bỏ đi phần vượt quá 2^{64} và chương trình in ra một con số sai mà không báo lỗi gì. Một số có 10^6 chữ số cần khoảng 3,3 triệu bit, không kiểu số nào của C++ chứa nổi. Nhưng ta có thật sự cần **cả** số N không? Đề bài chỉ hỏi số dư của nó.

6. Phá bế tắc: Chỉ giữ lại số dư

Bước 1 Viết N theo sơ đồ Horner

Cách dựng N từ trái sang phải chính là cách viết N theo **sơ đồ Horner**. Ví dụ với N = 2026:

$$\mathbf{2026 = ((2 \cdot 10 + 0) \cdot 10 + 2) \cdot 10 + 6}$$

Biểu thức này chỉ gồm phép nhân và phép cộng. Theo định lí về tính chất đồng dư, *ở* **mỗi bước** ta được thay kết quả trung gian bằng số dư của nó khi chia cho m, mà số dư cuối cùng không đổi.

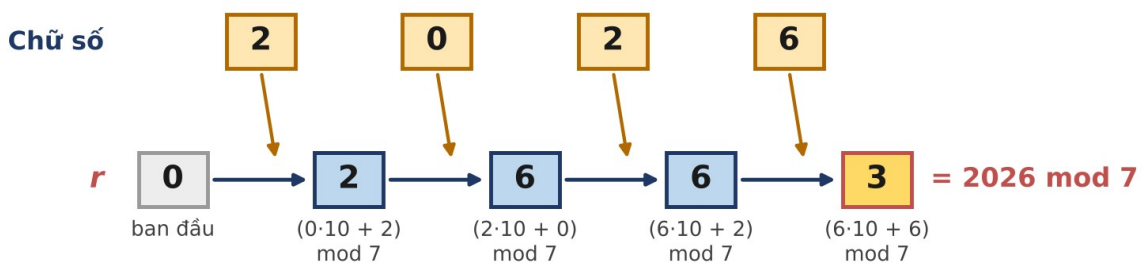
Bước 2 Duy trì số dư của phần đầu

Gọi r là số dư của phần đầu của N (các chữ số đã đọc) khi chia cho m. Ban đầu chưa đọc chữ số nào: r = 0. Đọc thêm chữ số c thì phần đầu mới bằng (phần đầu cũ) · 10 + c, nên:

$$\mathbf{r \leftarrow (r \cdot 10 + c) \bmod m}$$

Vì luôn có $0 \le r < m$ nên $r \cdot 10 + c < 10 \cdot m \le 10^{10}$: vừa vặn trong `long long`, không bao giờ tràn.

Bước 3 Chạy tay với N = 2026, m = 7



Hình 1.4. *Mỗi chữ số mới đẩy r sang một giá trị mới; r luôn nhỏ hơn 7.*

Chữ số đọc	Phần đầu của N	r · 10 + c	r mới (mod 7)
2	2	0 · 10 + 2 = 2	2
0	20	2 · 10 + 0 = 20	20 mod 7 = **6**
2	202	6 · 10 + 2 = 62	62 mod 7 = **6**
6	2026	6 · 10 + 6 = 66	66 mod 7 = **3**

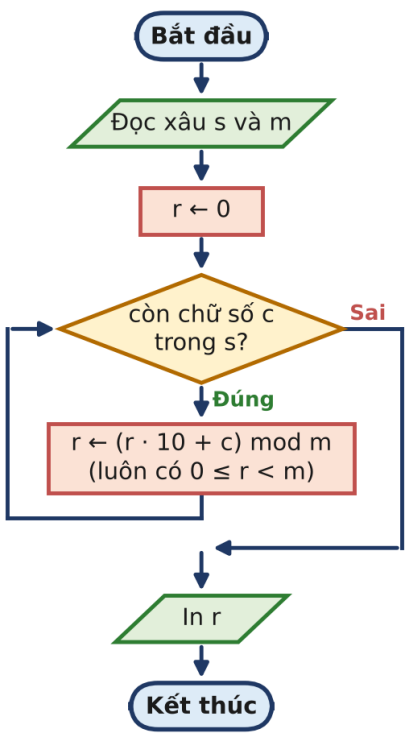
Kiểm tra lại cột thứ hai: 20 mod 7 = 6, 202 = 7 · 28 + 6, 2026 = 7 · 289 + 3. Đúng như bảng: r luôn là số dư của phần đầu, dù ta không hề lưu phần đầu ấy.

VÌ SAO ĐÚNG?

Bất biến: sau khi đọc k chữ số đầu, r là số dư của số tạo bởi k chữ số đó khi chia cho m.

Đúng khi k = 0 (r = 0). Giả sử đúng tới k, gọi phần đầu là A, ta có A ≡ r (mod m). Phần đầu mới là A · 10 + c ≡ r · 10 + c (mod m) theo tính chất đồng dư, nên (r · 10 + c) mod m chính là số dư của phần đầu mới. Khi k = L, phần đầu chính là N.

Bước 4 Chương trình



Hình 1.5. Sơ đồ khối của cách chỉ giữ số dư.

SODU.CPP: sơ đồ Horner, chỉ giữ số dư

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("SODU.INP", "r")) {
        freopen("SODU.INP", "r", stdin);
        freopen("SODU.OUT", "w", stdout);
    }
    string s;               // N dài tới 10^6 chữ số: đọc vào xâu
    long long m;
    cin >> s >> m;
    long long r = 0;        // r = (phần đầu đã đọc của N) mod m
    for (char c : s)
        r = (r * 10 + (c - '0')) % m;   // thêm một chữ số bên phải
    cout << r << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Mỗi chữ số tốn một phép nhân, một phép cộng, một phép lấy dư: O(L) thời gian và O(1) bộ nhớ ngoài xâu đầu vào. Với L = 10^6, chương trình chạy chưa tới một phần trăm giây.

7. Mở rộng

Mở rộng 1: Dấu hiệu chia hết cho 3, 9 và 11

Vì $10 \equiv 1 \pmod 9$, mọi lũy thừa $10^k \equiv 1 \pmod 9$. Do đó một số đồng dư với **tổng các chữ số** của nó theo mô-đun 9 (và theo mô-đun 3). Đây là lí do của dấu hiệu chia hết cho 3 và 9 em đã học ở tiểu học. Tương tự, $10 \equiv -1 \pmod{11}$, nên $10^k \equiv (-1)^k \pmod{11}$: một số đồng dư với **tổng đan dấu** các chữ số của nó tính từ hàng đơn vị. Ví dụ 918 082: $2 - 8 + 0 - 8 + 1 - 9 = -22$, chia hết cho 11, nên 918 082 chia hết cho 11.

Mở rộng 2: Số dư của tổng và tích rất dài

Khi cần $(a_1 + a_2 + \ldots + a_n) \bmod m$ hay $(a_1 \cdot a_2 \cdot \ldots \cdot a_n) \bmod m$, hãy lấy dư **sau mỗi phép tính**, đúng như trong SODU. Với tích, nhớ rằng hai số nhỏ hơn $m \le 10^9$ nhân nhau được số tới 10^{18}: vẫn vừa `long long`. Khi m tới 10^{18}, tích hai số dư có thể tới 10^{36}: phải dùng `__int128` hoặc phép nhân đặc biệt ở Bài 9.

Mở rộng 3: Chữ số tận cùng và tính tuần hoàn

Chữ số tận cùng của a là a mod 10. Các lũy thừa 2^1, 2^2, 2^3, ... có chữ số tận cùng 2, 4, 8, 6, 2, 4, 8, 6, ...: lặp lại với **chu kì** 4, vì chỉ có hữu hạn số dư nên dãy số dư sớm muộn phải lặp. Ví dụ $2026 \bmod 4 = 2$, nên 2^{2026} có chữ số tận cùng giống 2^2, tức là **4**.

BẪY THƯỜNG GẶP

- **Số dư âm:** trong C++, `-7 % 3` bằng −1, không phải 2. Dùng `((a % m) + m) % m` khi a có thể âm.
- **Tràn số trung gian:** dùng `int` cho r khi m = 10^9 thì r · 10 + c có thể tới khoảng $10^{10} > 2^{31}$. Hãy dùng `long long`.
- **Lấy dư quá muộn:** cộng dồn cả dãy rồi mới lấy dư thì tổng đã tràn từ trước. Lấy dư ngay sau mỗi phép tính.
- **Chia hai vế của đồng dư:** $2 \cdot 3 \equiv 2 \cdot 8 \pmod{10}$ nhưng $3 \not\equiv 8 \pmod{10}$. Đừng "giản ước" khi chưa học Bài 10.
- **Đọc N vào kiểu số:** `cin >> N` với N 30 chữ số không báo lỗi rõ ràng, chỉ cho kết quả sai. Đọc vào xâu.

GHI NHỚ

- $a = b \cdot q + r$ với $0 \le r < b$; $q = \lfloor a / b \rfloor$ là bội số lớn nhất của b không vượt quá a (tính theo số lần b).
- C++ làm tròn thương về 0; số dư theo nghĩa toán học là `((a % m) + m) % m`.
- $a \equiv b \pmod{m}$ khi và chỉ khi $m \mid (a - b)$. Đồng dư giữ nguyên qua cộng, trừ, nhân, lũy thừa; không giữ qua phép chia.
- Sơ đồ Horner: $r \leftarrow (r \cdot 10 + c) \bmod m$ cho số dư của số rất lớn trong O(độ dài).

Luyện tập

Bài	Tên	Gợi ý liên hệ
1.1	**NGÀY TRONG TUẦN**	Thứ của ngày cách hôm nay k ngày, $\lvert k \rvert \le 10^{18}$, k có thể âm. Số dư không âm.
1.2	**CHỮ SỐ CUỐI**	Chữ số tận cùng của a^b với a, b cho dưới dạng xâu rất dài: dùng tính tuần hoàn (chu kì 4) và số dư của số lớn.
1.3	**VIẾT LIỀN**	Số dư khi chia cho m của số 123456789101112...n (viết liền 1..n): sơ đồ Horner theo từng số, nhân với $10^{(\text{số chữ số})}$.

Tự kiểm tra nhanh

1) Theo quy ước toán học, (−23) mod 5 bằng bao nhiêu? Trong C++, `-23 % 5` cho kết quả gì? *(Đáp số: 2 và −3)*

2) Hôm nay là thứ Tư. Sau 2026 ngày là thứ mấy? *(Đáp số: 2026 mod 7 = 3, nên là thứ Bảy)*

3) Không dùng máy tính, tìm số dư của 123 456 789 khi chia cho 9 và khi chia cho 11. *(Đáp số: tổng chữ số 45 nên dư 0 khi chia cho 9; tổng đan dấu 9 − 8 + 7 − 6 + 5 − 4 + 3 − 2 + 1 = 5 nên dư 5 khi chia cho 11)*

4) Chạy tay SODU với $N = 10^{18}$ (chữ số 1 và 18 chữ số 0), m = 7. Vì sao chỉ cần để ý rằng $10 \equiv 3$ và $3^6 \equiv 1 \pmod 7$? *(Đáp số: $10^{18} \equiv 3^{18} = (3^6)^3 \equiv 1 \pmod 7$)*

BÀI 2 Ước số: liệt kê, đếm và tính tổng

Học xong bài này, em sẽ:

- ✓ Hiểu vì sao các ước của n luôn **đi theo cặp** (d, n / d) và trong mỗi cặp có một số không vượt quá √n.
- ✓ Đếm số ước và tính tổng các ước của $n \le 10^{12}$ trong O(√n) thay vì O(n).
- ✓ Tránh được ba lỗi kinh điển: đếm trùng khi n là số chính phương, dùng `sqrt` số thực, tràn số.
- ✓ Biết cách tính số ước của **mọi** số từ 1 tới N cùng lúc bằng "sàng điều hòa" O(N log N).

KHỞI ĐỘNG

Cô giáo có 36 học sinh và muốn xếp cả lớp thành một hình chữ nhật đều đặn: mỗi hàng có cùng số bạn, không bạn nào đứng lẻ. Có bao nhiêu cách xếp? 1 hàng 36 bạn, 2 hàng 18 bạn, 3 hàng 12 bạn... Mỗi cách xếp ứng với một **ước** của 36.

Với 36 bạn, em đếm nhẩm được. Nhưng nếu là 10^{12} (một nghìn tỉ) "bạn" thì sao? Thử từng số từ 1 tới 10^{12} mất gần một giờ. Bài học này chỉ ra rằng em chỉ cần thử tới **một triệu** số, và chương trình chạy xong trong vài phần nghìn giây.

1. Ước số và cặp ước

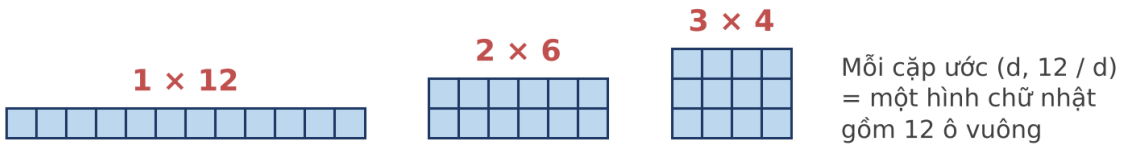
ĐỊNH NGHĨA

Số nguyên dương d gọi là **ước** của số nguyên dương n nếu n chia hết cho d (n mod d = 0). Khi đó n gọi là **bội** của d. Kí hiệu d(n) là **số ước** của n, σ(n) (đọc là "xích-ma") là **tổng các ước** của n, kể cả 1 và chính n.

Ví dụ, các ước của 12 là 1, 2, 3, 4, 6, 12, nên d(12) = 6 và σ(12) = 1 + 2 + 3 + 4 + 6 + 12 = 28. Hãy để ý cách các ước "bắt cặp" với nhau: 1 · 12 = 2 · 6 = 3 · 4 = 12.

HÃY HÌNH DUNG

Mỗi ước d của n là một cách xếp n ô vuông thành hình chữ nhật d hàng, n / d cột. Hình chữ nhật d × (n / d) và hình (n / d) × d là cùng một hình xoay ngang, nên các ước luôn đi thành từng **cặp** (d, n / d).



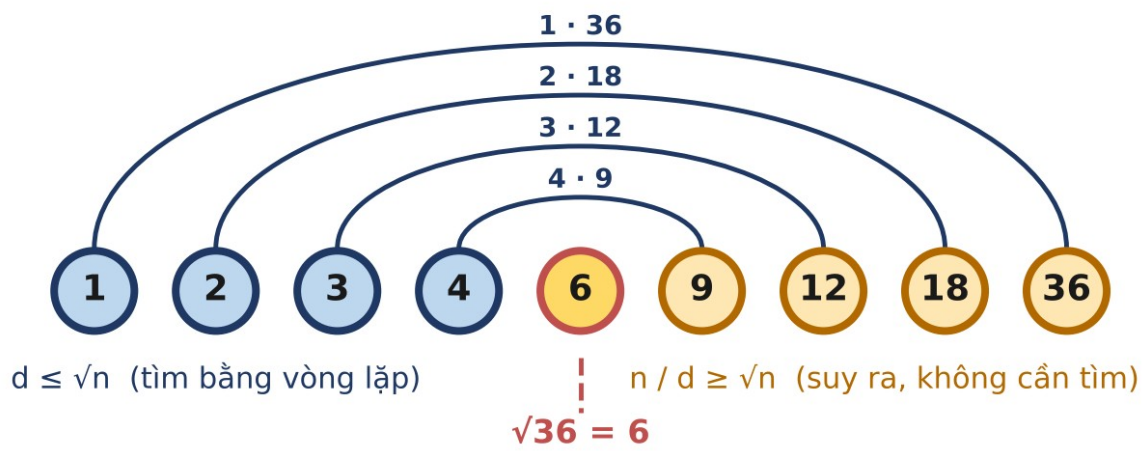
Hình 2.1. *Ba hình chữ nhật gồm 12 ô vuông ứng với ba cặp ước (1, 12), (2, 6), (3, 4).*

ĐỊNH LÍ (CẶP ƯỚC)

Nếu d là ước của n thì n / d cũng là ước của n. Trong hai số d và n / d, có ít nhất một số **không vượt quá √n**.

VÌ SAO ĐÚNG?

n / d là số nguyên và d · (n / d) = n, nên n / d là ước của n. Nếu cả hai số đều lớn hơn √n thì tích của chúng lớn hơn √n · √n = n: vô lí. Vậy có ít nhất một số ≤ √n.



Hình 2.2. *Chín ước của 36 bắt cặp đối xứng qua √36 = 6. Ước 6 tự bắt cặp với chính nó.*

2. Bài toán DEMUOC

BÀI TOÁN: DEMUOC

Cho số nguyên dương n. Hãy đếm số ước của n và tính tổng các ước của n.

Dữ liệu vào: Một dòng chứa n ($1 \le n \le 10^{12}$).

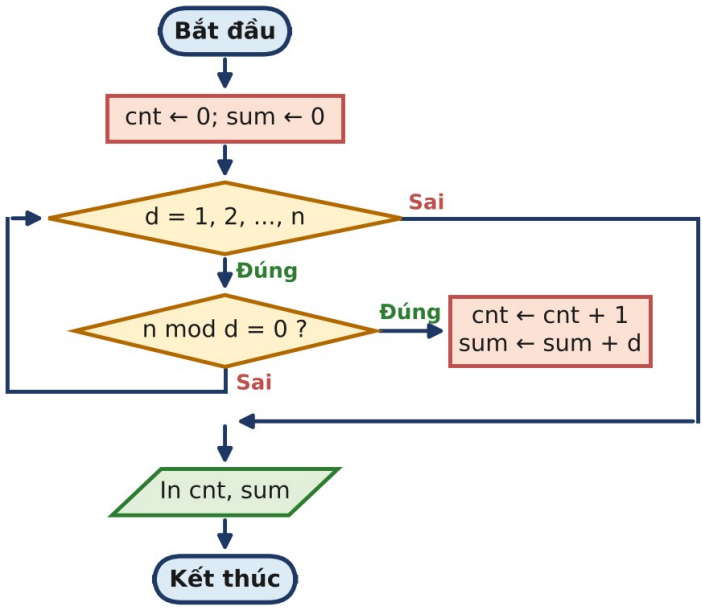
Kết quả: Hai số nguyên d(n) và σ(n), cách nhau một dấu cách.

DEMUOC.INP	DEMUOC.OUT	Giải thích
12	6 28	Các ước 1, 2, 3, 4, 6, 12.
36	9 91	36 là số chính phương: ước 6 chỉ được đếm một lần.

DEMUOC.INP	DEMUOC.OUT	Giải thích
1	1 1	Số 1 chỉ có một ước là chính nó.
1000000000000	169 2499694822171	$10^{12} = 2^{12} \cdot 5^{12}$ có $13 \cdot 13 = 169$ ước (công thức này em sẽ học ở Bài 7).

3. Ý tưởng ngây thơ: Thử mọi số từ 1 tới n

Định nghĩa nói d là ước khi n mod d = 0. Vậy cứ thử lần lượt d = 1, 2, ..., n: số nào chia hết thì đếm và cộng vào tổng.



Hình 2.3. Sơ đồ khối của cách thử mọi số.

DEMUOC.CPP (ngây thơ): thử d = 1, 2, ..., n

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DEMUOC.INP", "r")) {
        freopen("DEMUOC.INP", "r", stdin);
        freopen("DEMUOC.OUT", "w", stdout);
    }
    long long n;
    cin >> n;
    long long cnt = 0, sum = 0;
    for (long long d = 1; d <= n; d++)          // thử mọi số từ 1 tới n
        if (n % d == 0) { cnt++; sum += d; }
    cout << cnt << " " << sum << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Vòng lặp chạy đúng n lần, mỗi lần một phép chia lấy dư: O(n). Trên máy tính thông thường, chương trình mất khoảng **3,2 giây** với $n = 10^9$. Với $n = 10^{12}$ gấp một nghìn lần, tức gần **một giờ**.

BẾ TẮC

Hầu hết các số d mà ta thử đều không phải ước: 10^{12} chỉ có 169 ước trong một nghìn tỉ số được thử. Hơn nữa, mỗi khi tìm được một ước d, ta "biết luôn" ước n / d, nhưng chương trình vẫn đi tìm lại nó một lần nữa ở rất xa phía sau. Liệu có thể dừng sớm hơn nhiều không?

4. Phá bế tắc: Chỉ duyệt tới √n

Bước 1 **Mỗi ước nhỏ cho ta một ước lớn**

Theo định lí cặp ước, mỗi cặp (d, n / d) có một số ≤ √n. Vậy chỉ cần duyệt d từ 1 tới √n: mỗi khi d là ước, ta ghi nhận **cả hai** ước d và n / d. Mọi ước lớn hơn √n đều được tìm thấy qua "bạn đôi" nhỏ của nó.

Bước 2 **Cẩn thận với số chính phương**

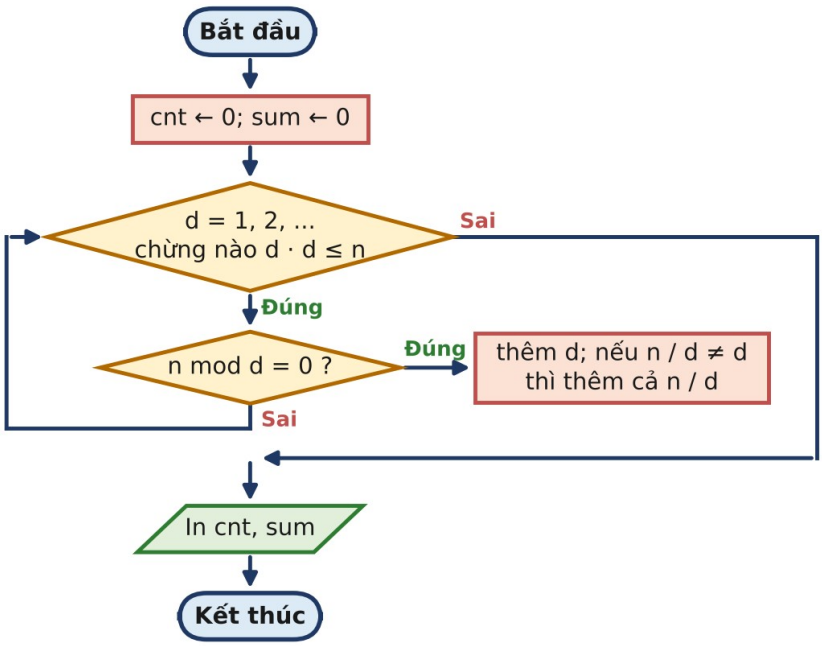
Nếu n là số chính phương, ước d = √n bắt cặp với chính nó (6 · 6 = 36). Khi đó d = n / d và ta chỉ được đếm **một lần**. Điều kiện dừng viết là `d * d <= n`, hoàn toàn bằng số nguyên, để khỏi dùng `sqrt` cho kết quả số thực có thể sai lệch.

Bước 3 **Chạy tay với n = 36**

d	d · d ≤ 36?	36 mod d	Ghi nhận	cnt, sum
1	có	0	1 và 36	2, 37
2	có	0	2 và 18	4, 57
3	có	0	3 và 12	6, 72
4	có	0	4 và 9	8, 85
5	có	1	không	8, 85
6	có	0	**chỉ 6**	9, 91
7	**không** (49 > 36)		dừng	**9, 91**

Chỉ 6 lần thử (thay vì 36) đã tìm đủ 9 ước, tổng 91.

Bước 4 Chương trình



Hình 2.4. *Sơ đồ khối của cách duyệt tới √n.*

DEMUOC.CPP: duyệt d ≤ √n, ghi nhận cả cặp ước

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DEMUOC.INP", "r")) {
        freopen("DEMUOC.INP", "r", stdin);
        freopen("DEMUOC.OUT", "w", stdout);
    }
    long long n;
    cin >> n;
    long long cnt = 0, sum = 0;
    for (long long d = 1; d * d <= n; d++)      // chỉ duyệt d ≤ √n
        if (n % d == 0) {
            long long e = n / d;                // ước "bạn đôi" của d
            cnt++; sum += d;
            if (e != d) { cnt++; sum += e; }    // n chính phương: d = e
        }
    cout << cnt << " " << sum << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Vòng lặp chạy khoảng √n lần: với n = 10^{12} chỉ là 10^6 lần, chương trình chạy trong khoảng **0,006 giây** thay vì gần một giờ. Đây là một trong những cải tiến đơn giản mà hiệu quả nhất của số học: từ O(n) xuống O(√n).

Tổng các ước của n ≤ 10^{12} có thể vượt 2^{31} (với n = 10^{12} là 2 499 694 822 171), nên cnt và sum phải là `long long`.

5. Mở rộng

Mở rộng 1: Liệt kê các ước theo thứ tự tăng dần

Khi duyệt d tăng dần, các ước nhỏ d xuất hiện theo thứ tự **tăng**, còn các ước lớn n / d xuất hiện theo thứ tự **giảm**. Hãy đưa chúng vào hai mảng riêng, rồi in mảng thứ nhất, sau đó in mảng thứ hai theo chiều ngược lại. Không cần sắp xếp.

Mở rộng 2: Số ước của mọi số từ 1 tới N (sàng điều hòa)

Nếu cần d(k) và σ(k) cho **mọi** k ≤ N (chẳng hạn N = 10^6), gọi O(√k) cho từng số tốn tổng cộng khoảng N√N ≈ 10^9 phép tính. Hãy đảo ngược câu hỏi: thay vì hỏi "k có những ước nào?", hãy hỏi "d là ước của những số nào?". Câu trả lời là d, 2d, 3d, ...: chỉ việc nhảy theo bước d và cộng d vào các số ấy.

Sàng điều hòa: số ước và tổng ước của mọi số 1..N

```cpp
#include <bits/stdc++.h>
using namespace std;

// Số ước và tổng ước của MỌI số 1..N bằng "sàng điều hòa"
int main() {
    int N;
    cin >> N;
    vector<int> cnt(N + 1, 0);
    vector<long long> sum(N + 1, 0);
    for (int d = 1; d <= N; d++)        // d là ước của d, 2d, 3d, ...
        for (int k = d; k <= N; k += d)
            cnt[k]++, sum[k] += d;
    for (int k = 1; k <= N; k++)
        cout << k << ": " << cnt[k] << " " << sum[k] << "\n";
    return 0;
}
```

Số phép tính là N/1 + N/2 + N/3 + ... + N/N = N · (1 + 1/2 + ... + 1/N) ≈ N ln N. Với N = 10^6, tổng này đúng bằng 13 970 034: chưa tới $1{,}4 \cdot 10^7$ phép tính. Tên gọi "điều hòa" đến từ chuỗi 1 + 1/2 + 1/3 + ..., gọi là chuỗi điều hòa.

Mở rộng 3: Số hoàn hảo và số ước lẻ

- Số **hoàn hảo** là số bằng tổng các ước thực sự (khác chính nó), tức σ(n) = 2n: 6 = 1 + 2 + 3, 28, 496, 8128.

- n có **số ước lẻ** khi và chỉ khi n là số chính phương: các ước bắt cặp hai-hai, chỉ trừ ước $\sqrt{n}$ tự bắt cặp với mình.
- Một số $\le 10^9$ có không quá 1344 ước, một số $\le 10^{18}$ có không quá 103 680 ước (Phụ lục B). Con số này giúp em ước lượng khi phải duyệt qua mọi ước.

BẪY THƯỜNG GẶP

- **Đếm trùng ước $\sqrt{n}$:** với n = 36, ước 6 bị đếm hai lần nếu quên kiểm tra `d != n / d`.
- **Dùng `sqrt(n)` số thực:** với n cỡ 10^{12} trở lên, `(long long)sqrt(n)` có thể lệch 1 do sai số làm tròn. Viết `d * d <= n` an toàn hơn.
- **`int` cho d:** khi n tới 10^{18}, d tới 10^9 và `d * d` tràn `int`. Dùng `long long` cho d.
- **`int` cho tổng ước:** $\sigma(n)$ với $n \le 10^{12}$ có thể vượt $2 \cdot 10^{12}$.
- **Quên ước 1 và chính n:** định nghĩa trong bài đếm cả hai. Một số đề chỉ đếm **ước thực sự** (khác n): đọc kĩ đề.

GHI NHỚ

- Các ước đi theo cặp (d, n / d), mỗi cặp có một số $\le \sqrt{n}$.
- Duyệt `for (d = 1; d * d <= n; d++)`, mỗi ước d ghi nhận cả n / d, trừ khi d = n / d: $O(\sqrt{n})$.
- Sàng điều hòa: với mỗi d, cộng vào d, 2d, 3d, ...: O(N log N) cho mọi số tới N.
- n có số ước lẻ $\Leftrightarrow$ n là số chính phương.

Luyện tập

Bài	Tên	Gợi ý liên hệ
2.1	**XẾP HÀNG**	Đếm số cách xếp n học sinh thành hình chữ nhật có ít nhất 2 hàng và 2 cột: d(n) trừ đi các trường hợp suy biến.
2.2	**ƯỚC TĂNG DẦN**	In mọi ước của $n \le 10^{12}$ theo thứ tự tăng dần; Mở rộng 1.
2.3	**SỐ BẠN BÈ**	Đếm các cặp (a, b) với $a < b \le N$, $\sigma(a) - a = b$ và $\sigma(b) - b = a$ (cặp số bạn bè): sàng điều hòa.

Tự kiểm tra nhanh

1) Tính số ước và tổng các ước của 360. *(Đáp số: 24 ước, tổng 1170)*

2) Vòng lặp trong DEMUOC.CPP chạy bao nhiêu lần với $n = 10^{12}$? Với n = 999 999 999 989 (một số nguyên tố)? *(Đáp số: 10^6 lần trong cả hai trường hợp (d chạy tới khi $d \cdot d > n$))*

3) Những số nào từ 1 đến 100 có số ước lẻ? *(Đáp số: các số chính phương 1, 4, 9, ..., 100 (10 số))*

BÀI 3 Ước chung lớn nhất và bội chung nhỏ nhất

Học xong bài này, em sẽ:

- ✓ Hiểu ƯCLN và BCNN, và hệ thức gcd(a, b) · lcm(a, b) = a · b.
- ✓ Chứng minh và cài đặt **thuật toán Euclid**: gcd(a, b) = gcd(b, a mod b).
- ✓ Giải thích vì sao Euclid chỉ cần không quá vài chục bước kể cả với số cỡ 10^{18}.
- ✓ Tính BCNN mà không bị tràn số, và phát hiện khi BCNN vượt giới hạn.

KHỞI ĐỘNG

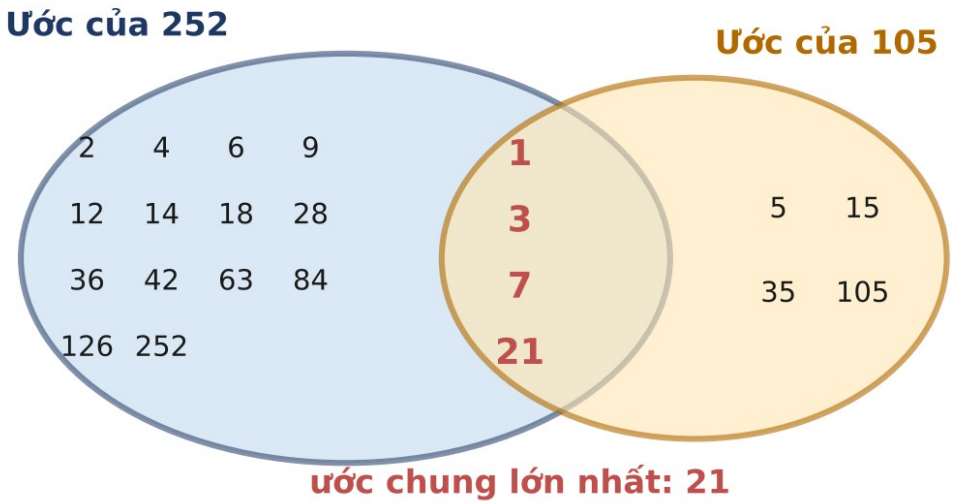
Một tấm gạch men hình chữ nhật 252 cm × 105 cm cần được cắt thành những miếng **hình vuông bằng nhau**, không thừa mẩu nào. Miếng vuông lớn nhất có cạnh bao nhiêu?

Cạnh hình vuông phải là ước của cả 252 lẫn 105, và ta muốn nó lớn nhất: đó là **ước chung lớn nhất**. Hơn 2300 năm trước, Euclid đã tìm ra một cách cắt rất khéo cho câu hỏi này, và cách cắt ấy chính là thuật toán nhanh nhất mà em sẽ học hôm nay.

1. ƯCLN và BCNN

ĐỊNH NGHĨA

Ước chung lớn nhất của hai số nguyên dương a, b, kí hiệu gcd(a, b) (viết tắt của *greatest common divisor*), là số lớn nhất chia hết cả a và b. **Bội chung nhỏ nhất** lcm(a, b) (*least common multiple*) là số dương nhỏ nhất chia hết cho cả a và b. Hai số có gcd bằng 1 gọi là **nguyên tố cùng nhau**.



Hình 3.1. *Các ước chung của 252 và 105 là 1, 3, 7, 21; lớn nhất là 21.*

Một quan sát quan trọng: **mọi ước chung đều là ước của ƯCLN** (ở hình trên: 1, 3, 7 đều là ước của 21). Và giữa ƯCLN và BCNN có một hệ thức rất đẹp:

$$\gcd(a, b) \cdot \operatorname{lcm}(a, b) = a \cdot b$$

Ví dụ: gcd(252, 105) = 21, lcm(252, 105) = 1260, và 21 · 1260 = 26 460 = 252 · 105. Nhờ hệ thức này, tính được ƯCLN là tính được BCNN: lcm(a, b) = a / gcd(a, b) · b.

2. Bài toán UCLN

BÀI TOÁN: UCLN

Cho hai số nguyên dương a và b. Hãy tính ƯCLN và BCNN của chúng.

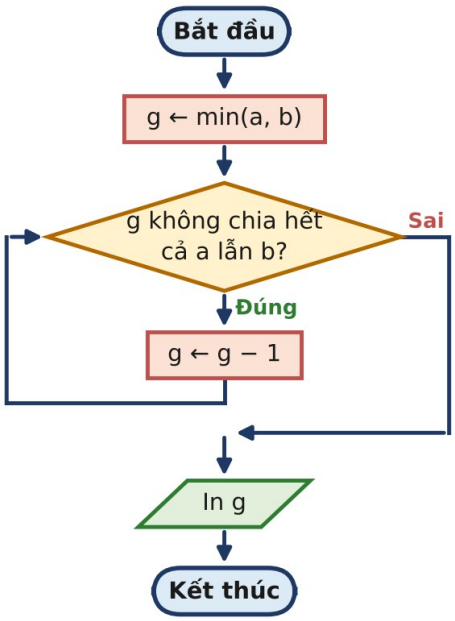
Dữ liệu vào: Một dòng chứa a và b ($1 \le a, b \le 10^{18}$).

Kết quả: Dòng 1: gcd(a, b). Dòng 2: lcm(a, b) nếu lcm(a, b) ≤ 10^{18}, ngược lại in −1.

UCLN.INP	UCLN.OUT	Giải thích
`252 105`	`21` `1260`	Như ở phần khởi động.
`17 5`	`1` `85`	17 và 5 nguyên tố cùng nhau: BCNN là tích 85.
`1000000000000000000` `999999999999999999`	`1` `-1`	Hai số liên tiếp luôn nguyên tố cùng nhau; BCNN gần 10^{36} quá lớn.
`24 24`	`24` `24`	gcd(a, a) = lcm(a, a) = a.

3. Ý tưởng ngây thơ: Thử giảm dần

ƯCLN không vượt quá số nhỏ hơn. Vậy thử g = min(a, b), rồi giảm dần g cho tới khi g chia hết cả a và b. Số đầu tiên tìm được chính là ƯCLN (vì ta thử từ lớn xuống).



Hình 3.2. *Sơ đồ khối của cách thử giảm dần.*

UCLN.CPP (ngây thơ): thử g giảm dần từ min(a, b)

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

int main() {
    if (fopen("UCLN.INP", "r")) {
        freopen("UCLN.INP", "r", stdin);
        freopen("UCLN.OUT", "w", stdout);
    }
    ll a, b;
    cin >> a >> b;
    ll g = min(a, b);
    while (a % g != 0 || b % g != 0) g--;       // thử giảm dần từ min(a, b)
    const ll LIM = 1000000000000000000LL;
    ll x = a / g;
    cout << g << "\n";
    if (x > LIM / b) cout << -1 << "\n";
    else cout << x * b << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Trường hợp xấu nhất là khi a, b nguyên tố cùng nhau: vòng lặp chạy min(a, b) lần, tới 10^{18} lần ở ví dụ 3. Kể cả máy tính nhanh nhất thế giới cũng không chạy xong trong đời người.

BẾ TẮC

Thử từng ứng viên là vô vọng. Ta cần một cách **thu nhỏ bài toán**: thay cặp (a, b) bằng một cặp số nhỏ hơn mà có **cùng ƯCLN**, rồi lặp lại cho tới khi câu trả lời hiển nhiên.

4. Phá bế tắc: Thuật toán Euclid

Bước 1 Trừ số lớn cho số nhỏ

Nếu d chia hết a và b thì d chia hết a − b. Ngược lại nếu d chia hết b và a − b thì d chia hết (a − b) + b = a. Vậy hai cặp (a, b) và (a − b, b) có **cùng tập ước chung**, nên cùng ƯCLN:

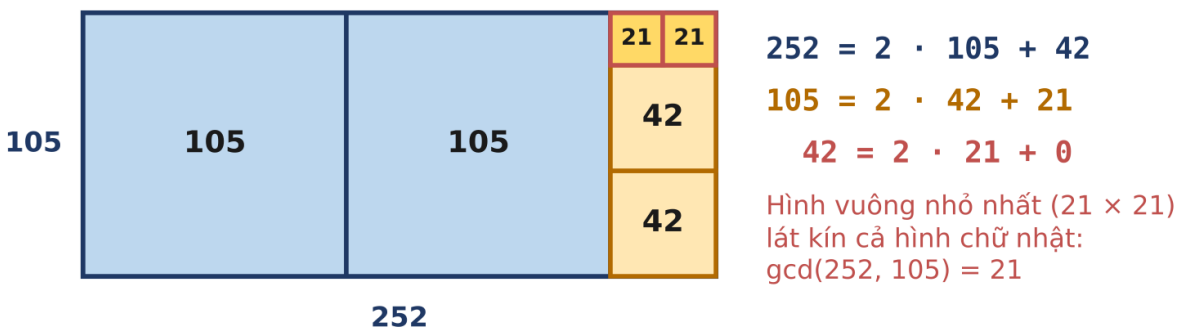
$$\gcd(a, b) = \gcd(a - b, b)$$

Ở bài toán cắt gạch: cắt bỏ một hình vuông b × b khỏi tấm a × b, phần còn lại (a − b) × b vẫn cắt được bằng đúng những miếng vuông như cũ.

Bước 2 Trừ nhiều lần một lúc: phép chia lấy dư

Khi a lớn hơn b nhiều, ta trừ b liên tiếp q lần cho tới khi phần còn lại nhỏ hơn b. Phần còn lại ấy chính là a mod b. Thay cả chuỗi phép trừ bằng một phép chia:

$$\gcd(a, b) = \gcd(b, a \bmod b), \quad \gcd(a, 0) = a$$



Hình 3.3. *Cắt tấm 252 × 105 thành hình vuông: mỗi bước Euclid cắt đi các hình vuông lớn nhất có thể.*

Bước 3 Chạy tay với a = 252, b = 105

a	b	q = a / b	r = a mod b	Cặp mới (b, r)
252	105	2	42	(105, 42)
105	42	2	21	(42, 21)
42	21	2	0	(21, 0)

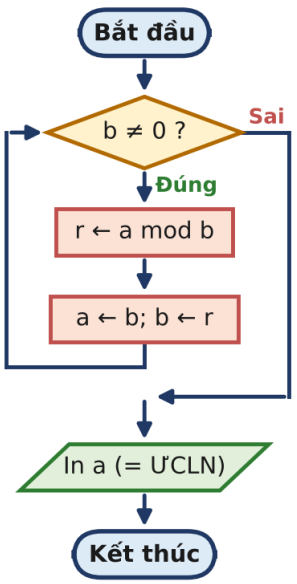
a	b	q = a / b	r = a mod b	Cặp mới (b, r)
21	0			dừng: gcd = **21**

VÌ SAO ĐÚNG? VÌ SAO DỪNG?

Đúng: $a = b \cdot q + r$ nên $r = a - q \cdot b$. Mọi ước chung của a, b chia hết r; mọi ước chung của b, r chia hết $a = b \cdot q + r$. Hai cặp có cùng tập ước chung nên cùng ƯCLN. Khi $b = 0$, mọi số chia hết 0, nên $\gcd(a, 0) = a$.

Dừng: số thứ hai giảm nghiêm ngặt ($r < b$) và luôn không âm, nên sau hữu hạn bước nó về 0.

Bước 4 Chương trình



Hình 3.4. *Sơ đồ khối của thuật toán Euclid.*

UCLN.CPP: thuật toán Euclid, BCNN không tràn số

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

ll gcd_(ll a, ll b) {                          // thuật toán Euclid
    while (b != 0) {
        ll r = a % b;                          // (a, b) -> (b, a mod b)
        a = b; b = r;
    }
    return a;
}

int main() {
    if (fopen("UCLN.INP", "r")) {
        freopen("UCLN.INP", "r", stdin);
        freopen("UCLN.OUT", "w", stdout);
    }
    ll a, b;
    cin >> a >> b;
    ll g = gcd_(a, b);
    const ll LIM = 1000000000000000000LL;      // 10^18
    ll x = a / g;                              // chia trước, nhân sau
    cout << g << "\n";
    if (x > LIM / b) cout << -1 << "\n";       // x · b > 10^18: không in
    else cout << x * b << "\n";
    return 0;
}
```

Để tính BCNN, ta **chia trước, nhân sau**: x = a / gcd(a, b) là số nguyên, rồi lcm = x · b. Trước khi nhân, kiểm tra $x > 10^{18} / b$: nếu đúng thì $x \cdot b > 10^{18}$ và ta in −1, không bao giờ để phép nhân tràn số.

ĐẾM SỐ PHÉP TÍNH: EUCLID NHANH ĐẾN MỨC NÀO?

Sau **hai** bước liên tiếp, số thứ nhất giảm ít nhất một nửa. Thật vậy, xét a mod b: nếu $b \le a / 2$ thì $a \bmod b < b \le a / 2$; còn nếu $b > a / 2$ thì $a \bmod b = a - b < a / 2$. Vậy số bước không quá $2 \cdot \log_2 a \approx 120$ với $a \le 10^{18}$.

Thực tế còn tốt hơn: trường hợp xấu nhất là hai số Fibonacci liên tiếp (định lí Lamé). Với các số $\le 10^{18}$, cặp xấu nhất là $F_{87} = 679\,891\,637\,638\,612\,258$ và $F_{86} = 420\,196\,140\,727\,489\,673$, cần **85 bước**. Độ phức tạp O(log min(a, b)).

5. Mở rộng

Mở rộng 1: ƯCLN và BCNN của cả một dãy

$\gcd(a_1, a_2, ..., a_n) = \gcd(\gcd(a_1, a_2), a_3, ...)$: gộp dần từ trái sang phải. Thời gian thực tế gần O(n + log max), vì giá trị gcd chỉ giảm và mỗi lần giảm là ít nhất chia đôi. Với BCNN của dãy thì gộp tương tự, nhưng BCNN tăng rất nhanh: phải kiểm tra tràn sau mỗi bước (hoặc tính theo mô-đun bằng phân tích thừa số ở Bài 7).

Mở rộng 2: Bỏ đi một phần tử

Bài toán hay gặp: bỏ đi đúng một phần tử của dãy để gcd các phần tử còn lại lớn nhất. Tính trước gcd tiền tố L[i] = gcd(a_1..a_i) và gcd hậu tố R[i] = gcd(a_i..a_n). Bỏ phần tử i thì gcd còn lại là gcd(L[i − 1], R[i + 1]): O(n log max) cho cả dãy.

Mở rộng 3: Hàm có sẵn và các tính chất hay dùng

- Trong C++14 có hàm `__gcd(a, b)` (thư viện GNU); từ C++17 có `gcd` và `lcm` trong `<numeric>`.
- gcd(a, 0) = a; gcd(a, b) = gcd(|a|, |b|) với số âm.
- gcd(k · a, k · b) = k · gcd(a, b); nếu gcd(a, b) = g thì a / g và b / g nguyên tố cùng nhau.
- Với dãy Fibonacci: gcd(F_m, F_n) = $F_{gcd(m, n)}$, một sự liên hệ bất ngờ giữa Euclid và Fibonacci.

BẪY THƯỜNG GẶP

- **Tính BCNN bằng a · b / gcd:** tích a · b tràn số trước khi kịp chia. Luôn chia trước: `a / g * b`.
- **Không kiểm tra BCNN vượt giới hạn:** với a, b ≤ 10^{18}, BCNN có thể tới 10^{36}. Kiểm tra `x > LIM / b` trước khi nhân.
- **Gọi gcd với số âm** mà không lấy trị tuyệt đối: `%` của C++ với số âm có thể cho kết quả âm.
- **Dùng `int`** cho a, b cỡ 10^{18}.
- **Dùng phép trừ thay cho phép chia** (bản Euclid cổ): gcd(10^{18}, 1) cần 10^{18} phép trừ.

GHI NHỚ

- gcd(a, b) = gcd(b, a mod b), gcd(a, 0) = a: thuật toán Euclid, O(log min(a, b)).
- Mọi ước chung là ước của ƯCLN; gcd(a, b) · lcm(a, b) = a · b.
- BCNN: chia trước, nhân sau, và kiểm tra tràn: `x = a / g; if (x > LIM / b) ...`.
- Trường hợp xấu nhất của Euclid là hai số Fibonacci liên tiếp: vẫn chỉ khoảng 85 bước với số 10^{18}.

Luyện tập

Bài	Tên	Gợi ý liên hệ
3.1	CẮT GẠCH	Cắt tấm a × b thành hình vuông bằng nhau lớn nhất: in cạnh và số miếng, a, b ≤ 10^{18}.
3.2	BỎ MỘT SỐ	Bỏ một phần tử của dãy để ƯCLN còn lại lớn nhất: gcd tiền tố và hậu tố (Mở rộng 2).

Bài	Tên	Gợi ý liên hệ
3.3	**ĐÈN NHÁY**	n đèn, đèn i nháy sau mỗi t_i giây; lần đầu mọi đèn cùng nháy: BCNN của dãy, phát hiện vượt 10^{18}.

Tự kiểm tra nhanh

1) Chạy tay Euclid với a = 1071, b = 462. Có bao nhiêu bước và ƯCLN bằng bao nhiêu? *(Đáp số: 1071 → 462 → 147 → 21 → 0, ba phép chia, gcd = 21)*

2) Vì sao gcd(n, n + 1) luôn bằng 1? *(Đáp số: gcd(n + 1, n) = gcd(n, 1) = 1)*

3) Tính lcm(4, 6, 10) theo cách gộp dần. *(Đáp số: lcm(4, 6) = 12, lcm(12, 10) = 60)*

BÀI 4 Euclid mở rộng và phương trình ax + by = c

Học xong bài này, em sẽ:

- ✓ Hiểu **định lí Bézout**: ax + by = c có nghiệm nguyên khi và chỉ khi gcd(a, b) chia hết c.
- ✓ Cài đặt **thuật toán Euclid mở rộng** để tìm x, y với ax + by = gcd(a, b).
- ✓ Viết được **mọi** nghiệm của phương trình từ một nghiệm, và chọn ra nghiệm theo yêu cầu.
- ✓ Trả lời 10^5 phương trình mỗi giây, thay vì thử từng giá trị của x.

KHỞI ĐỘNG

Một cửa hàng chỉ có hai loại can: can 3 lít và can 5 lít. Có đong được đúng 7 lít nước bằng cách đổ đầy và đổ đi những chiếc can ấy không? Đổ đầy 4 lần can 3 lít vào bể (12 lít), rồi múc ra một lần can 5 lít: còn đúng 7 lít. Ta vừa giải phương trình 3x + 5y = 7 với x = 4, y = −1.

Nếu chỉ có can 6 lít và can 4 lít thì sao? Thử mãi cũng không ra 7 lít. Vì sao vậy? Và với những chiếc can dung tích cả tỉ lít, làm sao tìm được cách đong ngay lập tức? Câu trả lời nằm trong một phiên bản "mở rộng" của thuật toán Euclid.

1. Định lí Bézout

Phương trình ax + by = c với x, y là **số nguyên** (có thể âm) gọi là **phương trình Diophantine tuyến tính**. Một quan sát đơn giản: g = gcd(a, b) chia hết a và b, nên chia hết ax + by với mọi x, y. Vậy nếu g không chia hết c thì phương trình **vô nghiệm**. Với can 6 lít và 4 lít: gcd(6, 4) = 2 không chia hết 7, nên không bao giờ đong được 7 lít.

ĐỊNH LÍ (BÉZOUT)

Với hai số nguyên dương a, b và g = gcd(a, b), luôn tồn tại các số nguyên x, y sao cho ax + by = g. Hệ quả: phương trình ax + by = c có nghiệm nguyên **khi và chỉ khi g chia hết c**.

Nếu tìm được x_0, y_0 với $ax_0 + by_0 = g$ thì nhân cả hai vế với c / g, ta được một nghiệm của ax + by = c. Vậy việc chính là tìm x_0, y_0, và thuật toán Euclid sẽ làm việc đó (đồng thời chứng minh luôn định lí).

2. Bài toán PTNGUYEN

BÀI TOÁN: PTNGUYEN

Cho Q phương trình ax + by = c. Với mỗi phương trình, hãy tìm nghiệm nguyên (x, y) có **x nhỏ nhất mà không âm**, hoặc cho biết phương trình vô nghiệm.

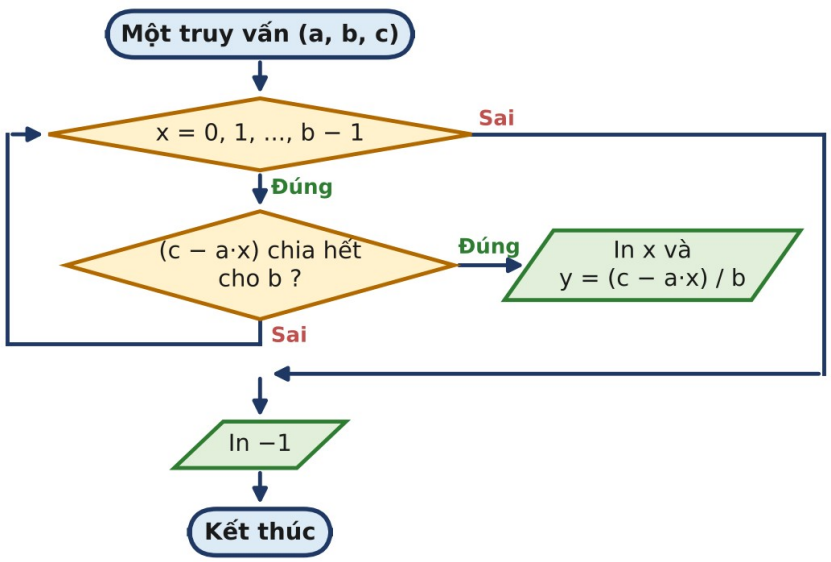
Dữ liệu vào: Dòng đầu chứa Q ($1 \le Q \le 10^5$). Mỗi dòng trong Q dòng tiếp theo chứa a, b, c ($1 \le a, b \le 10^9$, $|c| \le 10^{18}$).

Kết quả: Với mỗi phương trình, in x và y của nghiệm cần tìm, hoặc −1 nếu vô nghiệm.

PTNGUYEN.INP	PTNGUYEN.OUT	Giải thích
3 30 21 12 6 4 7 3 5 7	6 -8 -1 4 -1	30 · 6 + 21 · (−8) = 12. Không có nghiệm nào với 0 ≤ x < 6. gcd(6, 4) = 2 không chia hết 7: vô nghiệm. 3 · 4 + 5 · (−1) = 7: cách đong của phần khởi động.

3. Ý tưởng ngây thơ: Thử x = 0, 1, 2, ...

Với mỗi x, phương trình có nghiệm nguyên y khi và chỉ khi c − ax chia hết cho b. Thử x = 0, 1, 2, ...: giá trị đầu tiên thỏa mãn chính là x nhỏ nhất không âm. Nên thử tới đâu? Khi x tăng thêm b, số dư của c − ax khi chia cho b lặp lại như cũ; vậy nếu thử hết x = 0, 1, ..., b − 1 mà không được thì vô nghiệm.



Hình 4.1. Sơ đồ khối của cách thử x.

PTNGUYEN.CPP (ngây thơ): thử x = 0, 1, ..., b − 1

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

int main() {
    if (fopen("PTNGUYEN.INP", "r")) {
        freopen("PTNGUYEN.INP", "r", stdin);
        freopen("PTNGUYEN.OUT", "w", stdout);
    }
    int q;
    cin >> q;
    while (q--) {
        ll a, b, c;
        cin >> a >> b >> c;
        ll ans = -1;
        for (ll x = 0; x < b; x++)              // thử x = 0, 1, ..., b − 1
            if (((c - a * x) % b + b) % b == 0) { ans = x; break; }
        if (ans < 0) cout << -1 << "\n";
        else cout << ans << " " << (c - a * ans) / b << "\n";
    }
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Mỗi phương trình tốn tới $b \le 10^9$ lần thử; với $Q = 10^5$ phương trình là 10^{14} lần thử: hoàn toàn không thể.

BẾ TẮC

Ta đang mò từng x mà không hề dùng đến cấu trúc của a và b. Thuật toán Euclid "biết" rất nhiều về a và b: nó biến (a, b) thành (b, a mod b) cho tới (g, 0). Liệu có thể **lần ngược** các bước ấy để dựng ra x, y?

4. Phá bế tắc: Euclid mở rộng

Bước 1 Nghiệm ở cuối chuỗi là hiển nhiên

Euclid kết thúc ở cặp (g, 0), và với cặp này phương trình $g \cdot x + 0 \cdot y = g$ có nghiệm hiển nhiên **$= 1, y = 0$**.

Bước 2 Lần ngược một bước

Giả sử với cặp (b, a mod b) ta đã có x_1, y_1 thỏa $b \cdot x_1 + (a \bmod b) \cdot y_1 = g$. Thay $a \bmod b = a - \lfloor a / b \rfloor \cdot b$:

$$\mathbf{b \cdot x_1 + (a - \lfloor a/b \rfloor \cdot b) \cdot y_1 = a \cdot y_1 + b \cdot (x_1 - \lfloor a/b \rfloor \cdot y_1) = g}$$

Vậy với cặp (a, b): $\mathbf{x = y_1, y = x_1 - \lfloor a / b \rfloor \cdot y_1}$. Mỗi bước lần ngược chỉ tốn vài phép tính, và số bước bằng số bước của Euclid.

Bước 3 Chạy tay với a = 30, b = 21

Ba cột đầu là các bước Euclid thường (đi từ trên xuống). Cột thứ tư được điền **từ dưới lên**: mỗi dòng dùng cặp (x, y) của dòng ngay dưới nó làm (x_1, y_1).

a	b	⌊a / b⌋	x, y (lần ngược)	Kiểm tra a · x + b · y
30	21	1	x = −2, y = 1 − 1 · (−2) = **3**	30 · (−2) + 21 · 3 = 3
21	9	2	x = 1, y = 0 − 2 · 1 = −2	21 · 1 + 9 · (−2) = 3
9	3	3	x = 0, y = 1 − 3 · 0 = 1	9 · 0 + 3 · 1 = 3
3	0		x = 1, y = 0 (khởi đầu)	3 · 1 + 0 · 0 = 3

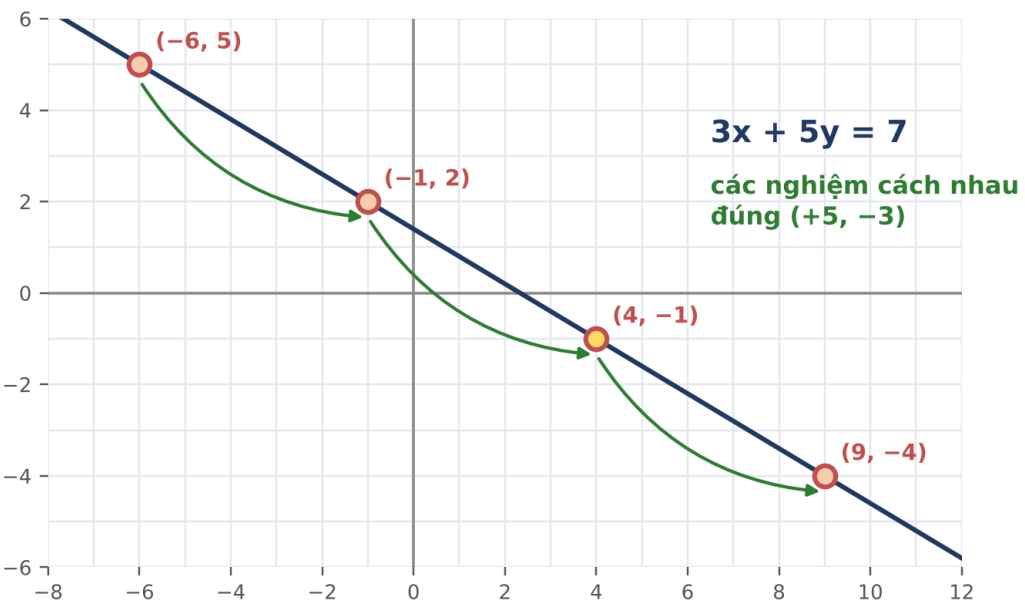
VÌ SAO ĐÚNG? (CŨNG LÀ CHỨNG MINH ĐỊNH LÍ BÉZOUT)

Quy nạp ngược theo các bước của Euclid: ở bước cuối, (1, 0) là nghiệm của g · x + 0 · y = g. Nếu (x_1, y_1) là nghiệm cho cặp (b, a mod b) thì phép biến đổi ở Bước 2 cho nghiệm của cặp (a, b). Vì Euclid luôn dừng, ta luôn dựng được x, y với ax + by = g. Đó chính là định lí Bézout.

Bước 4 Từ một nghiệm tới mọi nghiệm

Có một nghiệm (x_0, y_0) của ax + by = c rồi, ta tìm được mọi nghiệm khác. Nếu (x, y) cũng là nghiệm thì $a(x - x_0) = -b(y - y_0)$. Chia hai vế cho g: $(a / g)(x - x_0) = -(b / g)(y - y_0)$, mà a / g, b / g nguyên tố cùng nhau, nên b / g chia hết $x - x_0$. Vậy:

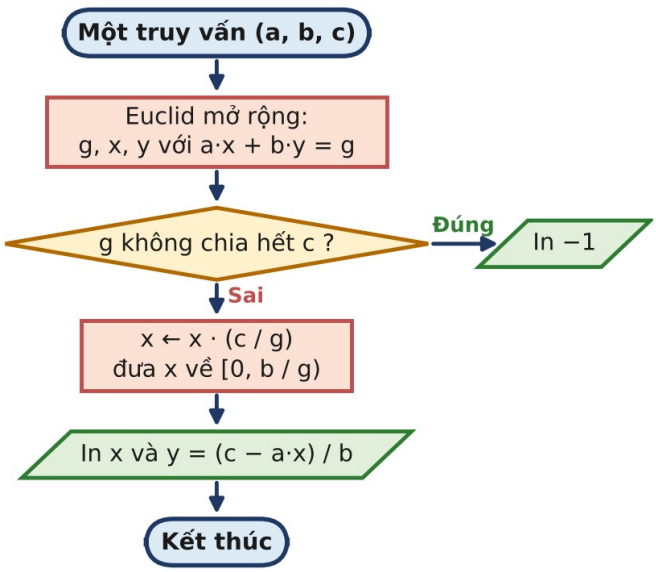
$$\mathbf{x = x_0 + (b / g) \cdot t, \quad y = y_0 - (a / g) \cdot t, \quad t \in \mathbb{Z}}$$



Hình 4.2. Các nghiệm nguyên của 3x + 5y = 7 nằm cách đều trên đường thẳng, mỗi bước (+5, −3).

Các giá trị x của nghiệm cách nhau đúng b / g, nên x nhỏ nhất không âm là $\mathbf{x_0 \bmod (b / g)}$ (lấy số dư không âm, như ở Bài 1). Với ví dụ 1: $30 \cdot (-2) + 21 \cdot 3 = 3$, nhân với $12 / 3 = 4$ được $x_0 = -8$, $y_0 = 12$; $b / g = 7$, nên $x = (-8) \bmod 7 = 6$ và $y = (12 - 30 \cdot 6) / 21 = -8$.

Bước 5 **Chương trình**



Hình 4.3. *Sơ đồ khối của lời giải bằng Euclid mở rộng.*

PTNGUYEN.CPP: Euclid mở rộng và họ nghiệm

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

// Trả về g = gcd(a, b) và tìm x, y với a·x + b·y = g
ll extgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) { x = 1; y = 0; return a; }     // a·1 + 0·0 = a
    ll x1, y1;
    ll g = extgcd(b, a % b, x1, y1);            // b·x1 + (a mod b)·y1 = g
    x = y1;                                     // a mod b = a − (a/b)·b
    y = x1 - (a / b) * y1;
    return g;
}

int main() {
    if (fopen("PTNGUYEN.INP", "r")) {
        freopen("PTNGUYEN.INP", "r", stdin);
        freopen("PTNGUYEN.OUT", "w", stdout);
    }
    int q;
    cin >> q;
    while (q--) {
        ll a, b, c, x, y;
        cin >> a >> b >> c;
        ll g = extgcd(a, b, x, y);
        if (c % g != 0) { cout << -1 << "\n"; continue; }   // Bézout
        ll bg = b / g;                          // x tăng giảm theo bước b/g
        ll x0 = (ll)((__int128)x * (c / g) % bg);
        if (x0 < 0) x0 += bg;                   // x nhỏ nhất không âm
        ll y0 = (ll)(((__int128)c - (__int128)a * x0) / b);
        cout << x0 << " " << y0 << "\n";
    }
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Mỗi phương trình tốn O(log min(a, b)) cho Euclid mở rộng, tức vài chục phép tính: 10^5 phương trình chạy trong khoảng một phần trăm giây.

Chú ý tràn số: x_0 có thể lớn cỡ $b \le 10^9$ và c / g cỡ 10^{18}, tích cỡ 10^{27} vượt `long long`. Chương trình dùng `__int128` cho phép nhân này rồi lấy dư ngay.

5. Mở rộng

Mở rộng 1: Đếm số nghiệm tự nhiên

Muốn đếm số nghiệm với **x ≥ 0 và y ≥ 0** (số cách đong bằng can, chỉ đổ vào mà không múc ra), dùng họ nghiệm: $x = x_0 + (b / g)t \ge 0$ và $y = y_0 - (a / g)t \ge 0$ cho một khoảng các giá trị t, đếm số nguyên trong khoảng ấy. Ví dụ 3x + 5y = 30 có ba nghiệm tự nhiên: (0, 6), (5, 3), (10, 0).

Mở rộng 2: Số lớn nhất không đổi được (bài toán đồng xu Frobenius)

Với a, b nguyên tố cùng nhau, mọi số $c > ab - a - b$ đều viết được thành $ax + by$ với $x, y \geq 0$, còn $ab - a - b$ thì không. Với can 3 lít và 5 lít: $3 \cdot 5 - 3 - 5 = 7$, nên 7 lít là lượng lớn nhất **không** đong được nếu chỉ được đổ vào; mọi lượng từ 8 lít trở lên đều đong được.

Mở rộng 3: Phương trình đồng dư $ax \equiv c \pmod m$

$ax \equiv c \pmod m$ nghĩa là $ax - c = my$ với y nguyên nào đó, tức $ax + m(-y) = c$: chính là bài toán PTNGUYEN. Trường hợp đặc biệt $ax \equiv 1 \pmod m$ cho ta **nghịch đảo** của a theo mô-đun m, công cụ để "chia" trong đồng dư mà em sẽ học ở Bài 10.

BẪY THƯỜNG GẶP

- **Quên kiểm tra g chia hết c:** khi đó mọi "nghiệm" in ra đều sai.
- **Tràn số khi nhân $x_0 \cdot (c / g)$:** dùng `__int128` hoặc lấy dư theo b / g trước khi nhân.
- **Lấy dư ra số âm:** $x_0 \bmod (b / g)$ trong C++ có thể âm; cộng thêm b / g.
- **Chia cho b thay vì b / g** khi chuẩn hóa x: được một nghiệm đúng, nhưng chưa chắc là nghiệm có x nhỏ nhất.
- **Nhầm thứ tự x, y khi lần ngược:** công thức là $x = y_1$, $y = x_1 - \lfloor a / b \rfloor \cdot y_1$. Hãy luôn thử lại $a \cdot x + b \cdot y$.

GHI NHỚ

- Bézout: $ax + by = c$ có nghiệm nguyên $\Leftrightarrow \gcd(a, b) \mid c$.
- Euclid mở rộng: ở đáy (g, 0) có (1, 0); lần ngược $x = y_1$, $y = x_1 - \lfloor a / b \rfloor \cdot y_1$.
- Họ nghiệm: $x = x_0 + (b / g)t$, $y = y_0 - (a / g)t$.
- x nhỏ nhất không âm: lấy $x_0 \bmod (b / g)$ theo nghĩa không âm.

Luyện tập

Bài	Tên	Gợi ý liên hệ
4.1	**ĐONG NƯỚC**	Có đong được c lít bằng hai loại can a, b lít (được đổ vào và múc ra) không: Bézout.
4.2	**MUA HÀNG**	Đếm số cách mua đúng c đồng bằng hai loại hàng giá a, b ($x, y \geq 0$): Mở rộng 1.
4.3	**ĐỒNG HỒ HỎNG**	Kim đồng hồ nhảy a nấc mỗi lần trên mặt m nấc; sau ít nhất bao nhiêu lần nhảy thì tới nấc c: phương trình đồng dư $ax \equiv c \pmod m$.

Tự kiểm tra nhanh

1) Phương trình $6x + 9y = 20$ có nghiệm nguyên không? Còn $6x + 9y = 21$? *(Đáp số: không (gcd = 3 không chia hết 20); có, ví dụ $x = 2, y = 1$)*

2) Dùng Euclid mở rộng, tìm x, y với $30x + 21y = 3$. *(Đáp số: $x = -2, y = 3$)*

3) $3x + 5y = 30$ có bao nhiêu nghiệm tự nhiên? *(Đáp số: 3 nghiệm: (0, 6), (5, 3), (10, 0))*

4) Với can 3 lít và 5 lít (chỉ đổ vào), lượng nước lớn nhất không đong được là bao nhiêu? *(Đáp số: 7 lít)*

CHƯƠNG 2. SỐ NGUYÊN TỐ

Số nguyên tố là những "nguyên tử" của số học: mọi số tự nhiên lớn hơn 1 đều được ghép từ chúng theo một cách duy nhất. Chúng xuất hiện khắp nơi trong Tin học, từ mô-đun $10^9 + 7$ quen thuộc của các đề thi, bảng băm, cho tới hệ mật mã RSA bảo vệ các giao dịch ngân hàng.

Chương này trả lời bốn câu hỏi, mỗi câu một bài: một số có phải số nguyên tố không? Có bao nhiêu số nguyên tố trong một đoạn? Một số được ghép từ những số nguyên tố nào? Và làm sao tìm số nguyên tố ở vùng số cỡ 10^{12}? Câu trả lời đi từ phép thử chia tới $\sqrt{n}$, qua cái sàng 2300 năm tuổi của Eratosthenes, tới những kĩ thuật sàng mà các chương trình thi đấu dùng hằng ngày.

TRONG CHƯƠNG NÀY

Bài 5. Số nguyên tố và kiểm tra nguyên tố: vì sao chỉ cần thử tới $\sqrt{n}$, và vì sao chỉ thử $6k \pm 1$

Bài 6. Sàng Eratosthenes: tìm mọi số nguyên tố tới 10^7 trong nháy mắt

Bài 7. Phân tích thừa số nguyên tố: định lí cơ bản của số học và sàng ước nguyên tố nhỏ nhất

Bài 8. Sàng trên đoạn: tìm số nguyên tố trong [L, R] khi R tới 10^{12}

BÀI 5 Số nguyên tố và kiểm tra nguyên tố

Học xong bài này, em sẽ:

- ✓ Nắm chắc định nghĩa **số nguyên tố**, **hợp số**, và vì sao số 1 không thuộc loại nào.
- ✓ Chứng minh được: hợp số n luôn có một ước nằm trong [2, √n]; từ đó kiểm tra nguyên tố trong O(√n).
- ✓ Biết hai mẹo giảm số phép thử: bỏ số chẵn và chỉ thử các số dạng **6k ± 1**.
- ✓ Viết đúng vòng lặp `i * i <= n` mà không sợ tràn số hay sai số của `sqrt`.
- ✓ Biết ý tưởng của **Miller–Rabin** để kiểm tra số tới 10^{18} chỉ với vài trăm phép nhân.

KHỞI ĐỘNG

Trong các đề thi Tin học, em sẽ gặp đi gặp lại con số **1 000 000 007**. Thầy cô bảo: "Đó là một số nguyên tố." Nhưng làm sao em tự kiểm tra được điều ấy? Thử chia nó cho 2, 3, 4, 5, ... tới 1 000 000 006 thì mất cả tỉ phép chia.

Một bạn nói: "Chỉ cần thử tới khoảng 31 623 thôi." Một bạn khác còn bảo: "Chỉ cần thử khoảng mười nghìn số." Cả hai đều đúng, và bài học này giải thích vì sao. Cũng chính những ý ấy cho phép máy tính kiểm tra một số cỡ 10^{12} trong khoảng một phần nghìn giây.

1. Số nguyên tố và hợp số

Có những số chỉ "xếp hàng" được theo một kiểu: 7 viên bi chỉ xếp được thành 1 hàng 7 viên (hoặc 7 hàng 1 viên). Còn 12 viên bi thì xếp được thành 2 · 6, 3 · 4. Những số "không tách nhỏ được" như 7 là viên gạch để xây nên mọi số khác, và có tên riêng.

ĐỊNH NGHĨA (SỐ NGUYÊN TỐ, HỢP SỐ)

Số tự nhiên p > 1 gọi là **số nguyên tố** nếu nó chỉ có đúng hai ước dương là 1 và p. Số tự nhiên n > 1 không phải số nguyên tố gọi là **hợp số**: khi đó n = a · b với 1 < a, b < n.

Số **0 và 1 không phải số nguyên tố, cũng không phải hợp số.**

Bảng sau liệt kê các ước của 12 số đầu tiên. Cột cuối cho thấy: số nguyên tố là số có **đúng 2 ước**; số 1 chỉ có 1 ước nên bị loại ra.

n	Các ước dương	Số ước	Loại
1	1	1	không thuộc loại nào
2	1, 2	2	**nguyên tố**
3	1, 3	2	**nguyên tố**

n	Các ước dương	Số ước	Loại
4	1, 2, 4	3	hợp số
5	1, 5	2	**nguyên tố**
6	1, 2, 3, 6	4	hợp số
7	1, 7	2	**nguyên tố**
8	1, 2, 4, 8	4	hợp số
9	1, 3, 9	3	hợp số
10	1, 2, 5, 10	4	hợp số
11	1, 11	2	**nguyên tố**
12	1, 2, 3, 4, 6, 12	6	hợp số

Có 15 số nguyên tố nhỏ hơn 50: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47. Hãy để ý: **là số nguyên tố chẵn duy nhất**, vì mọi số chẵn lớn hơn 2 đều có ước 2.

Vì sao người ta không coi 1 là số nguyên tố? Ở Bài 7 em sẽ thấy: mọi số lớn hơn 1 phân tích được thành tích các số nguyên tố theo **đúng một cách**. Nếu 1 là số nguyên tố thì $6 = 2 \cdot 3 = 1 \cdot 2 \cdot 3 = 1 \cdot 1 \cdot 2 \cdot 3 = ...$, tính "duy nhất" ấy mất đi.

ĐỊNH LÍ (ƯỚC NHỎ NHẤT LÀ SỐ NGUYÊN TỐ)

Với mọi $n > 1$, ước nhỏ nhất lớn hơn 1 của n là một số nguyên tố. Đặc biệt, mọi hợp số đều có ít nhất một ước nguyên tố.

VÌ SAO ĐÚNG?

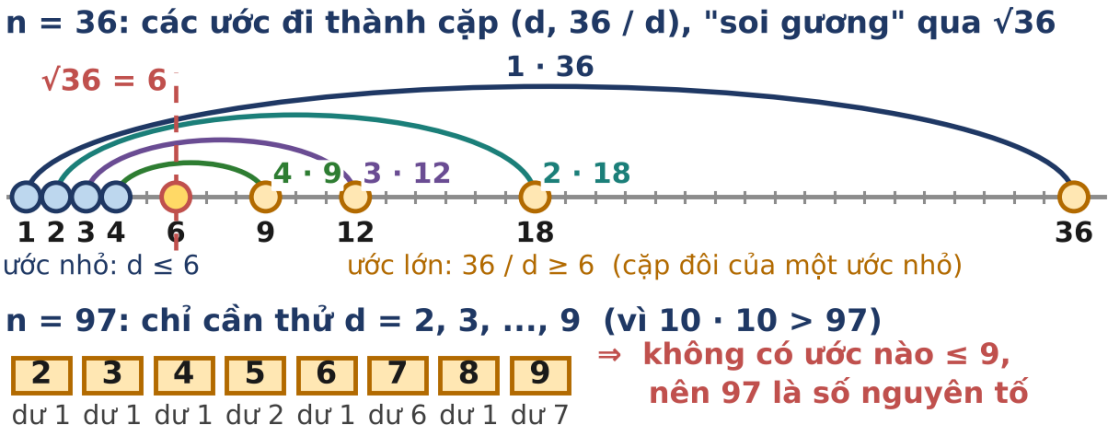
Gọi d là ước nhỏ nhất lớn hơn 1 của n. Nếu d là hợp số thì $d = a \cdot b$ với $1 < a < d$. Khi đó $a \mid d$ và $d \mid n$ nên $a \mid n$: a là một ước của n, lớn hơn 1 mà lại nhỏ hơn d. Mâu thuẫn với cách chọn d. Vậy d nguyên tố.

2. Ước nhỏ không vượt quá √n

Muốn biết n có phải số nguyên tố không, ta tìm xem n có ước nào trong khoảng $[2, n - 1]$ không. Nhưng có thật phải nhìn hết cả khoảng ấy? Hãy xem các ước của 36 xếp trên trục số.

HÃY HÌNH DUNG

Các ước của n luôn đi thành **cặp** $(d, n / d)$. Trong mỗi cặp, một số đứng bên trái $\sqrt{n}$ và số kia đứng bên phải, như hai hình ảnh soi gương qua một vạch ở vị trí $\sqrt{n}$. Vậy nếu bên trái vạch không có ước nào (ngoài 1), thì bên phải cũng không có ước nào (ngoài n).



Hình 5.1. *Mỗi ước lớn là "cặp đôi" của một ước nhỏ. Để kết luận 97 nguyên tố, chỉ cần thử 8 số từ 2 tới 9.*

ĐỊNH LÍ (THỬ TỚI √n LÀ ĐỦ)

Nếu n là hợp số thì n có một ước d thỏa $2 \le d \le \sqrt{n}$. Nói cách khác: nếu $n > 1$ không chia hết cho số nào trong $[2, \lfloor\sqrt{n}\rfloor]$ thì n là số nguyên tố.

VÌ SAO ĐÚNG?

Hợp số n viết được thành $n = a \cdot b$ với $1 < a \le b < n$ (đặt a là thừa số nhỏ hơn). Nếu $a > \sqrt{n}$ thì $b \ge a > \sqrt{n}$, suy ra $a \cdot b > \sqrt{n} \cdot \sqrt{n} = n$: vô lí. Vậy $a \le \sqrt{n}$, và a chính là ước cần tìm.

Kết hợp với định lí ở mục 1: ước nhỏ nhất của hợp số n vừa là số nguyên tố, vừa không vượt quá $\sqrt{n}$.

Với $n = 1\,000\,000\,007$ thì $\sqrt{n} \approx 31\,622{,}8$, nên chỉ cần thử 31 621 số từ 2 tới 31 622, thay vì một tỉ số. Đây chính là câu trả lời của bạn thứ nhất trong phần Khởi động.

3. Bài toán KTNT

BÀI TOÁN: KTNT

Cho Q số nguyên dương. Với mỗi số n, hãy cho biết n có phải là số nguyên tố hay không.

Dữ liệu vào: Dòng đầu chứa Q ($1 \le Q \le 1000$). Q dòng tiếp theo, mỗi dòng chứa một số n ($1 \le n \le 10^{12}$).

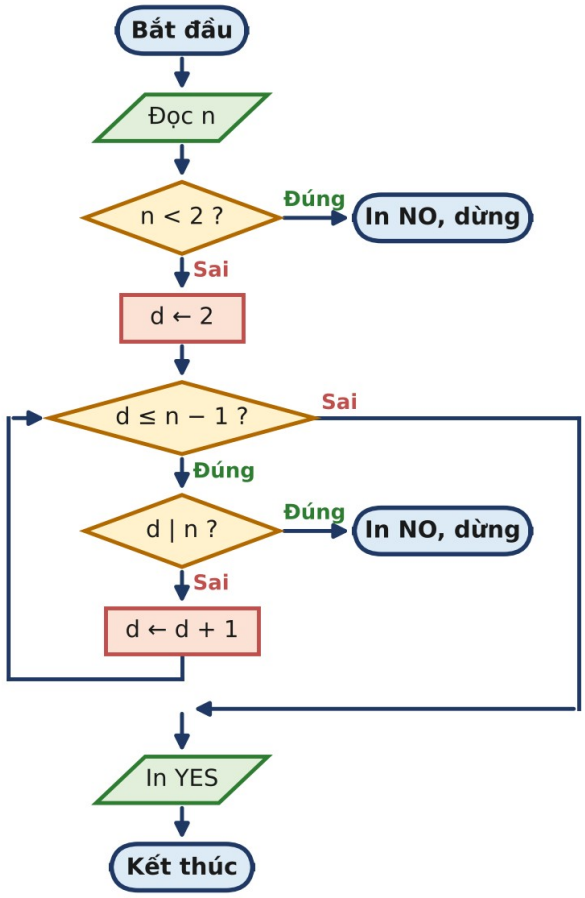
Kết quả: Q dòng, dòng thứ i ghi `YES` nếu số thứ i là số nguyên tố, ngược lại ghi `NO`.

KTNT.INP	KTNT.OUT	Giải thích
4 1 2 97 391	NO YES YES NO	1 không nguyên tố; 2 là số nguyên tố chẵn duy nhất; 97 nguyên tố; 391 = 17 · 23.

KTNT.INP	KTNT.OUT	Giải thích
3 999999999989 999999999999 1000000007	YES NO YES	999 999 999 989 là số nguyên tố lớn nhất nhỏ hơn 10^{12}; 999 999 999 999 chia hết cho 3; 1 000 000 007 nguyên tố.
2 999966000289 561	NO NO	999 966 000 289 = $999\,983^2$ (bình phương một số nguyên tố); 561 = 3 · 11 · 17.

4. Ý tưởng ngây thơ: Thử mọi ước từ 2 tới n − 1

Định nghĩa nói gì, ta làm đúng như vậy: với d chạy từ 2 tới n − 1, nếu n chia hết cho d thì n có ước thật sự, nó là hợp số. Nếu chạy hết mà không gặp ước nào thì n nguyên tố. Trường hợp n = 1 phải xử lí riêng.



Hình 5.2. *Sơ đồ khối của cách thử mọi ước từ 2 tới n − 1 (cho một số n).*

KTNT.CPP (ngây thơ): thử mọi ước từ 2 tới n − 1

```cpp
#include <bits/stdc++.h>
using namespace std;

// thử mọi ước d từ 2 tới n − 1: đúng nhưng rất chậm
bool laNT(long long n) {
    if (n < 2) return false;
    for (long long d = 2; d < n; d++)
        if (n % d == 0) return false;       // tìm thấy ước thật sự
    return true;
}

int main() {
    if (fopen("KTNT.INP", "r")) {
        freopen("KTNT.INP", "r", stdin);
        freopen("KTNT.OUT", "w", stdout);
    }
    int q;
    cin >> q;
    while (q--) {
        long long n;
        cin >> n;
        cout << (laNT(n) ? "YES" : "NO") << "\n";
    }
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Với hợp số, vòng lặp dừng sớm ngay ở ước nhỏ nhất (với 391 thì dừng ở d = 17). Trường hợp xấu nhất là khi n **là** số nguyên tố: phải làm đủ n − 2 phép chia. Với n = 999 999 999 989, đó là gần 10^{12} phép chia. Máy tính làm được khoảng $3 \cdot 10^8$ phép chia số 64 bit mỗi giây, nên **một** số đã tốn gần 54 phút; Q = 1000 số thì mất hơn một tháng.

BẾ TẮC

Chương trình đúng nhưng vô vọng với n cỡ 10^{12}. Phần lớn công sức bị phí vào việc thử những ước "bên phải √n", mà như Hình 5.1 cho thấy, nếu chúng là ước thì cặp đôi của chúng đã bị phát hiện từ trước rồi.

5. Phá bế tắc: Chỉ thử tới √n, rồi bỏ bớt ứng viên

Bước 1 **Dừng ở √n**

Theo định lí ở mục 2, chỉ cần thử d từ 2 tới ⌊√n⌋. Có một chi tiết kĩ thuật: thay vì so sánh `d <= sqrt(n)`, ta viết **`d * d <= n`**. Hàm `sqrt` trả về số thực, với n gần 10^{12} hoặc 10^{18} có thể bị sai lệch ở chữ số cuối cùng; còn phép nhân số nguyên thì chính xác tuyệt đối. Chỉ cần nhớ d phải là `long long`, vì d · d có thể tới 10^{12}.

Bước 2 Bỏ các số chẵn

Nếu n lẻ thì n không chia hết cho bất kì số chẵn nào. Vậy chỉ cần thử d = 2 một lần, rồi thử các số lẻ 3, 5, 7, ...: số phép thử giảm một nửa. Với n = 999 999 999 989, ta chỉ còn phải thử 500 000 số thay vì gần một triệu.

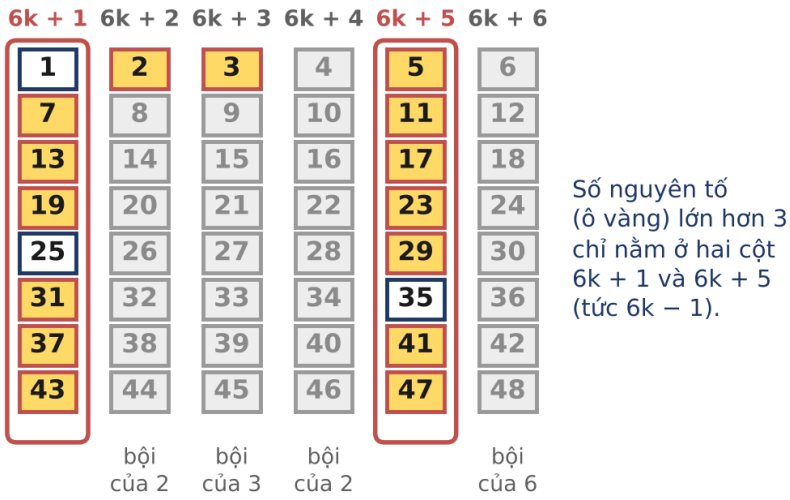
Bước 3 Chỉ thử các số dạng 6k ± 1

Đi xa thêm một bước: sau khi đã thử 2 và 3, những số chia hết cho 2 hoặc 3 không cần thử nữa. Xếp các số tự nhiên thành 6 cột theo số dư khi chia cho 6, ta thấy chỉ còn hai cột "đáng ngờ".

VÌ SAO MỌI SỐ NGUYÊN TỐ p > 3 CÓ DẠNG 6k ± 1?

Chia p cho 6, số dư r ∈ {0, 1, 2, 3, 4, 5}. Nếu r ∈ {0, 2, 4} thì p chẵn; nếu r = 3 thì p = 6k + 3 = 3 · (2k + 1) chia hết cho 3. Cả hai trường hợp đều mâu thuẫn với việc p là số nguyên tố lớn hơn 3. Vậy r = 1 hoặc r = 5, tức p = 6k + 1 hoặc p = 6k + 5 = 6(k + 1) − 1.

Áp dụng cho việc tìm ước: ước nhỏ nhất của n là số nguyên tố, nên nếu n không chia hết cho 2 và 3 thì ước ấy (nếu ≤ √n) phải nằm trong dãy 5, 7, 11, 13, 17, 19, ..., tức là các cặp (i, i + 2) với i = 5, 11, 17, ...



Hình 5.3. *Bốn cột 6k + 2, 6k + 3, 6k + 4, 6k + 6 toàn là bội của 2 hoặc 3. Mọi số nguyên tố lớn hơn 3 nằm ở cột 6k + 1 hoặc 6k + 5.*

Thử thừa vài số như 25 hay 35 (không nguyên tố) cũng không sao, chỉ tốn công một chút: điều quan trọng là **không bỏ sót** ước nguyên tố nào. Bảng sau so sánh số phép chia khi n = 999 999 999 989 là số nguyên tố, trường hợp xấu nhất:

Cách thử	Các số được thử	Số phép chia	Thời gian (ước lượng)
Mọi d từ 2 tới n − 1	2, 3, 4, ..., n − 1	999 999 999 987	khoảng 54 phút
d từ 2 tới √n	2, 3, ..., 999 999	999 998	3,2 ms
2 rồi các số lẻ tới √n	2, 3, 5, 7, ..., 999 999	500 000	1,6 ms

Cách thử	Các số được thử	Số phép chia	Thời gian (ước lượng)
2, 3 rồi 6k ± 1 tới √n	2, 3, 5, 7, 11, 13, ...	**333 334**	**1,1 ms**

Bước 4 Chạy tay với n = 97 và n = 391

Cả hai số đều lẻ và không chia hết cho 3 (97 mod 3 = 1, 391 mod 3 = 1), nên đi vào vòng lặp với i = 5. Mỗi vòng thử hai ước i và i + 2, rồi tăng i thêm 6.

n	i	i + 2	i · i ≤ n ?	n mod i	n mod (i + 2)	Kết luận
97	5	7	25 ≤ 97: có	2	6	thử tiếp
97	11	13	121 > 97: **không**			**YES**
391	5	7	25 ≤ 391: có	1	6	thử tiếp
391	11	13	121 ≤ 391: có	6	1	thử tiếp
391	17	19	289 ≤ 391: có	**0**		**NO**

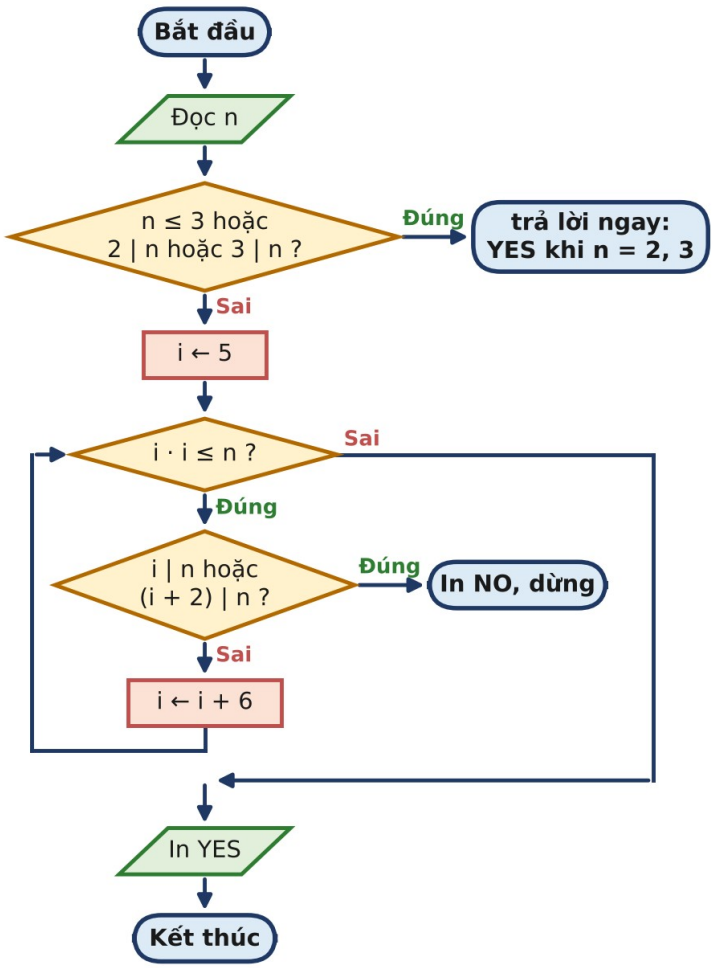
Với 97, chương trình chỉ làm 4 phép chia (cho 2, 3, 5, 7) là kết luận được. Với 391, ước 17 lộ ra ở vòng thứ ba: 391 = 17 · 23. Để ý rằng 23 > √391 ≈ 19,8 nằm "bên phải gương" nên ta không bao giờ cần thử tới nó.

VÌ SAO CHƯƠNG TRÌNH ĐÚNG?

Trả lời NO thì đúng: chương trình chỉ trả lời NO khi n < 2, hoặc khi n > 3 và tìm thấy một ước d của n (d = 2, d = 3, hoặc 5 ≤ d ≤ √n). Ước ấy luôn nhỏ hơn n, nên n là hợp số.

Trả lời YES thì đúng: nếu n ≥ 5 là hợp số, ước nguyên tố nhỏ nhất p của n thỏa p ≤ √n (mục 2). p = 2 hoặc 3 thì đã bị bắt trước vòng lặp; nếu p ≥ 5 thì p có dạng 6k ± 1 nên p = i hoặc p = i + 2 với một i nào đó mà i ≤ p ≤ √n, tức i · i ≤ n: vòng lặp chắc chắn đi tới i ấy và trả lời NO. Vậy khi chương trình trả lời YES, n không thể là hợp số.

Bước 5 Chương trình



Hình 5.4. *Sơ đồ khối của cách thử 2, 3 rồi các cặp 6k ± 1 tới √n.*

KTNT.CPP: thử chia tới √n với các số 6k ± 1

```cpp
#include <bits/stdc++.h>
using namespace std;

// n có phải số nguyên tố không? Chỉ thử các ước 6k ± 1 không vượt quá √n
bool laNT(long long n) {
    if (n < 2) return false;                // 0 và 1 không nguyên tố
    if (n < 4) return true;                 // 2 và 3
    if (n % 2 == 0 || n % 3 == 0) return false;
    for (long long i = 5; i * i <= n; i += 6)   // i = 5, 11, 17, ...
        if (n % i == 0 || n % (i + 2) == 0) return false;
    return true;
}

int main() {
    if (fopen("KTNT.INP", "r")) {
        freopen("KTNT.INP", "r", stdin);
        freopen("KTNT.OUT", "w", stdout);
    }
    int q;
    cin >> q;
    while (q--) {
        long long n;
        cin >> n;
        cout << (laNT(n) ? "YES" : "NO") << "\n";
    }
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

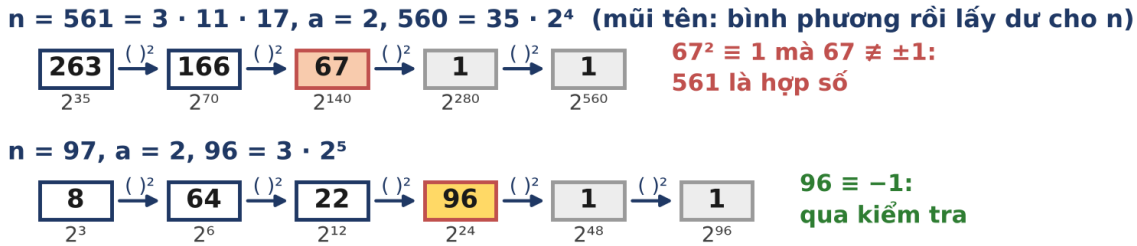
Mỗi số tốn tối đa khoảng √n / 3 phép chia, tức 333 334 phép với $n \approx 10^{12}$. Với Q = 1000 số nguyên tố lớn nhất có thể, tổng cộng khoảng $3{,}3 \cdot 10^8$ phép chia: đo thực tế mất khoảng 1,1 giây. Độ phức tạp $O(Q \cdot \sqrt{n})$. Nếu Q tăng lên 10^4, cách này không còn đủ nhanh: em cần Miller–Rabin ở Mở rộng 1.

6. Mở rộng

Mở rộng 1: Kiểm tra số tới 10^{18} bằng Miller–Rabin

Với $n \approx 10^{18}$, $\sqrt{n} = 10^9$: thử chia mất cả giây cho mỗi số. Các chương trình thi đấu dùng một ý tưởng hoàn toàn khác, dựa trên **định lí Fermat nhỏ** (em sẽ chứng minh ở Bài 10): nếu p nguyên tố và a không chia hết cho p thì $a^{p-1} \equiv 1 \pmod p$. Vậy nếu tìm được a mà $a^{n-1} \not\equiv 1 \pmod n$ thì n chắc chắn là hợp số, dù ta không biết ước của nó!

Đáng tiếc chiều ngược lại không đúng: có những hợp số "giả làm" số nguyên tố trước phép thử Fermat với mọi a, gọi là **số Carmichael**, nhỏ nhất là $561 = 3 \cdot 11 \cdot 17$ ($2^{560} \equiv 1 \pmod{561}$). Miller–Rabin vá lỗ hổng này bằng một nhận xét: theo mô-đun số nguyên tố p, phương trình $x^2 \equiv 1$ chỉ có hai nghiệm $x \equiv \pm 1$. Viết $n - 1 = d \cdot 2^s$ với d lẻ, rồi xét dãy a^d, a^{2d}, a^{4d}, ..., a^{n-1} (mỗi số là bình phương của số trước). Nếu n nguyên tố, dãy phải bắt đầu bằng 1, hoặc chứa giá trị $n - 1$ trước khi gặp số 1 đầu tiên.



Hình 5.5. *Dãy bình phương liên tiếp với a = 2. Với 561, số 67 bình phương thành 1 mà 67 ≠ ±1: lộ mặt hợp số. Với 97, dãy đi qua 96 ≡ −1 như một số nguyên tố "đúng mực".*

Người ta đã kiểm chứng: chỉ cần thử với 12 cơ sở a là 12 số nguyên tố đầu tiên (2, 3, 5, ..., 37) thì kết luận đúng cho **mọi** $n < 2^{64}$. Mỗi cơ sở cần O(log n) phép nhân theo mô-đun (lũy thừa nhanh, Bài 9), và phép nhân hai số cỡ 10^{18} cần `__int128` để không tràn.

KTNT_MR.CPP: Miller–Rabin tất định cho mọi số 64 bit

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef unsigned long long ull;

ull mul(ull a, ull b, ull m) { return (unsigned __int128)a * b % m; }
ull pw(ull a, ull e, ull m) {               // a^e mod m (Bài 9)
    ull r = 1;
    for (a %= m; e; e >>= 1, a = mul(a, a, m))
        if (e & 1) r = mul(r, a, m);
    return r;
}
// Miller–Rabin với 12 cơ sở cố định: đúng với mọi n < 2^64
bool laNT(ull n) {
    const ull A[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37};
    if (n < 2) return false;
    for (ull p : A)
        if (n % p == 0) return n == p;
    ull d = n - 1;
    int s = 0;
    while (d % 2 == 0) d /= 2, s++;         // n − 1 = d · 2^s, d lẻ
    for (ull a : A) {
        ull x = pw(a, d, n);
        if (x == 1 || x == n - 1) continue;
        bool ok = false;
        for (int r = 1; r < s && !ok; r++) {
            x = mul(x, x, n);
            ok = (x == n - 1);
        }
        if (!ok) return false;              // a là "nhân chứng": hợp số
    }
    return true;
}

int main() {
    if (fopen("KTNT.INP", "r")) {
        freopen("KTNT.INP", "r", stdin);
        freopen("KTNT.OUT", "w", stdout);
    }
    int q;
    cin >> q;
    while (q--) {
        ull n;
        cin >> n;
        cout << (laNT(n) ? "YES" : "NO") << "\n";
    }
    return 0;
}
```

Độ phức tạp: khoảng $12 \cdot 64$ phép bình phương cho mỗi số, tức vài nghìn phép nhân: 10^4 hay 10^5 số cỡ 10^{18} đều xong trong nháy mắt.

Mở rộng 2: Có vô số số nguyên tố

Khoảng 300 năm trước Công nguyên, Euclid đã chứng minh: **không có số nguyên tố lớn nhất**. Giả sử chỉ có hữu hạn số nguyên tố $p_1, p_2, ..., p_k$. Xét $N = p_1 \cdot p_2 \cdot ... \cdot p_k + 1$. $N > 1$ nên có một ước nguyên tố q (mục 1). Nhưng N chia cho mỗi p_i đều dư 1, nên q khác mọi p_i: ta tìm được một số nguyên tố nằm ngoài danh sách, mâu thuẫn.

Cẩn thận: chứng minh **không** nói N là số nguyên tố. Ví dụ $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 + 1 = 30\,031 = 59 \cdot 509$, là hợp số; nhưng các ước nguyên tố 59, 509 của nó quả thật nằm ngoài danh sách 2, 3, 5, 7, 11, 13.

Mở rộng 3: Chỉ thử chia cho các số nguyên tố

Cặp $6k \pm 1$ vẫn còn thử thừa các số như 25, 35, 49, ... Nếu có sẵn danh sách mọi số nguyên tố $\le 10^6$ (có 78 498 số), ta chỉ thử chia cho chúng: mỗi số $n \le 10^{12}$ tốn tối đa 78 498 phép chia thay vì 333 334, nhanh hơn hơn 4 lần. Danh sách ấy được tạo ra chỉ trong vài phần trăm giây bằng **sàng Eratosthenes**, nội dung của Bài 6.

BẪY THƯỜNG GẶP

- **Coi 1 là số nguyên tố:** vòng lặp thử ước không chạy lần nào khi n = 1 nên trả lời YES. Luôn xử lí riêng $n < 2$.
- **Tràn số ở `i * i`:** nếu i là `int` thì $i \cdot i$ với $i \approx 10^6$ đã vượt 2^{31}. Khai báo `long long i`.
- **Dùng `sqrt` số thực:** `i <= sqrt(n)` có thể sai ở n lớn do sai số làm tròn. Viết `i * i <= n`.
- **Dùng `<` thay cho `<=`:** với `i * i < n`, chương trình kết luận 25 và $999\,966\,000\,289 = 999\,983^2$ là số nguyên tố. Bình phương của số nguyên tố là bẫy kinh điển trong test.
- **Quên 2 và 3 khi dùng $6k \pm 1$:** vòng lặp bắt đầu từ 5, nên n = 2, 3 (nguyên tố) và các bội của 2, 3 phải được xử lí trước vòng lặp.
- **Tin phép thử Fermat:** $a^{n-1} \equiv 1 \pmod n$ với mọi a không chứng tỏ n nguyên tố (561, 1105, 1729, ...). Dùng Miller–Rabin.

GHI NHỚ

- Số nguyên tố là số > 1 có đúng hai ước 1 và chính nó; 2 là số nguyên tố chẵn duy nhất; 1 không nguyên tố.
- Ước nhỏ nhất (> 1) của n là số nguyên tố; hợp số n có ước nguyên tố $\le \sqrt{n}$.
- Kiểm tra nguyên tố: thử 2, 3 rồi các cặp i, i + 2 với i = 5, 11, 17, ... trong khi $i \cdot i \le n$: $O(\sqrt{n})$.
- Mọi số nguyên tố $p > 3$ có dạng $6k \pm 1$.
- Với n tới 10^{18} hoặc rất nhiều truy vấn: Miller–Rabin với 12 cơ sở cố định, nhân bằng `__int128`.

Luyện tập

Bài	Tên	Gợi ý liên hệ
5.1	**NGUYÊN TỐ GẦN NHẤT**	Tìm số nguyên tố nhỏ nhất không nhỏ hơn n ($n \le 10^{12}$): kiểm tra n, n + 1, ... bằng KTNT; khoảng cách giữa hai số nguyên tố liên tiếp ở vùng này chỉ vài trăm.

Bài	Tên	Gợi ý liên hệ
5.2	**TỔNG HAI SỐ NGUYÊN TỐ**	Viết số chẵn $n \le 10^{12}$ thành tổng hai số nguyên tố $p + q$ (giả thuyết Goldbach): thử p tăng dần, kiểm tra cả p và $n - p$.
5.3	**NGUYÊN TỐ 64 BIT**	$Q \le 10^5$ số tới 10^{18}: Miller–Rabin, lũy thừa nhanh (Bài 9) và phép nhân `__int128`.

Tự kiểm tra nhanh

1) Trong các số 1, 91, 97, 221, 223, số nào là số nguyên tố? *(Đáp số: 97 và 223; $91 = 7 \cdot 13$, $221 = 13 \cdot 17$)*

2) Khi kiểm tra $n = 10\,007$ (một số nguyên tố) bằng KTNT.CPP, vòng lặp `for` chạy bao nhiêu lần? *(Đáp số: $\lfloor\sqrt{10007}\rfloor = 100$, $i = 5, 11, \ldots, 95$: 16 lần)*

3) Nếu sửa điều kiện vòng lặp thành `i * i < n`, chương trình trả lời gì với $n = 25$? Vì sao? *(Đáp số: YES (sai), vì vòng lặp dừng trước khi thử $i = 5 = \sqrt{25}$)*

4) Số $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 + 1$ có phải số nguyên tố không? *(Đáp số: không, $30\,031 = 59 \cdot 509$)*

BÀI 6 Sàng Eratosthenes

Học xong bài này, em sẽ:

- ✓ Hiểu ý tưởng "đảo chiều" của cái sàng: thay vì hỏi từng số, để mỗi số nguyên tố **gạch bỏ** các bội của nó.
- ✓ Chứng minh được vì sao chỉ cần sàng với $p \le \sqrt{N}$ và bắt đầu gạch từ p^2.
- ✓ Biết vì sao sàng chạy trong $O(N \log \log N)$, gần như tuyến tính.
- ✓ Kết hợp sàng với **mảng tiền tố** để trả lời mỗi truy vấn đếm trong $O(1)$.
- ✓ Biết sàng tuyến tính và cách tiết kiệm bộ nhớ khi N rất lớn.

KHỞI ĐỘNG

Khoảng năm 240 trước Công nguyên, nhà bác học Hi Lạp Eratosthenes, người đầu tiên đo được chu vi Trái Đất, đề xuất một cách tìm số nguyên tố mà không cần chia một lần nào. Viết các số từ 2 tới 100 lên bảng. Khoanh số 2, rồi gạch mọi bội của 2. Số đầu tiên chưa bị gạch là 3: khoanh nó, gạch mọi bội của 3. Cứ thế tiếp tục.

Những số còn sót lại sau khi "rây" chính là các số nguyên tố, giống như hạt gạo nằm lại trên cái sàng sau khi cát đã lọt xuống hết. Bài học hôm nay biến cái sàng ấy thành một chương trình đếm số nguyên tố trên một triệu đoạn [a, b] khác nhau trong chưa tới nửa giây.

1. Ý tưởng của cái sàng

Ở Bài 5, mỗi số tự "trình diện" để bị kiểm tra: "Em có ước nào không?" Cái sàng đảo ngược câu hỏi ấy: **mỗi số nguyên tố p tự tìm đến các bội của mình** và đánh dấu chúng là hợp số. Một số không bị số nguyên tố nhỏ hơn nào đánh dấu thì nó không có ước nguyên tố nhỏ hơn nó, nên nó là số nguyên tố.

THUẬT TOÁN (SÀNG ERATOSTHENES)

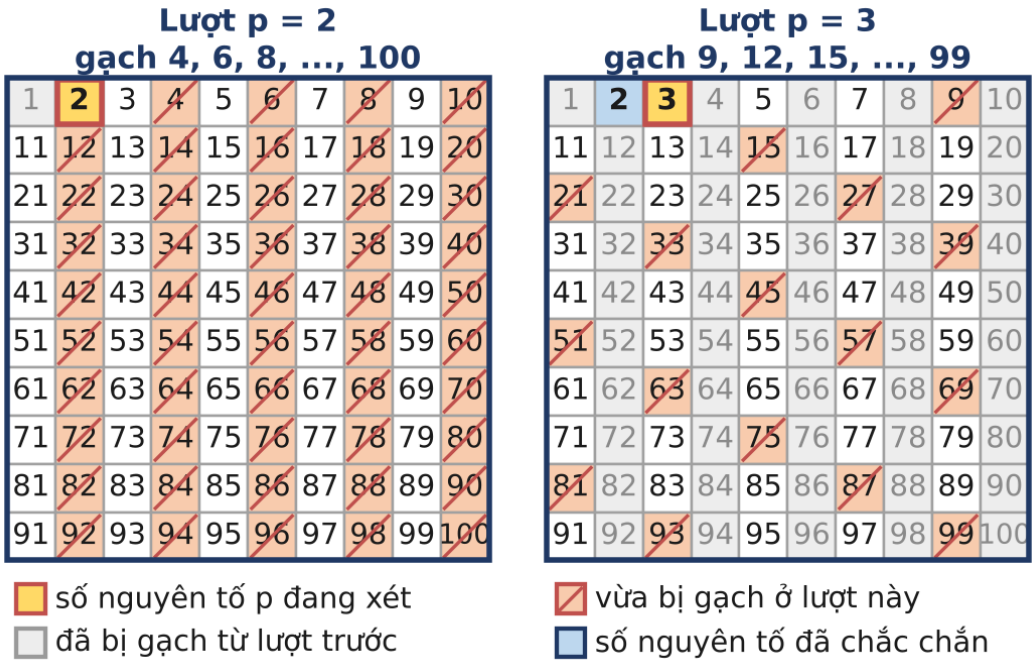
1) Đánh dấu mọi số 2, 3, ..., N là "chưa bị gạch"; gạch sẵn 0 và 1.

2) Với $p = 2, 3, 4, \ldots$ trong khi $p \cdot p \le N$: nếu p chưa bị gạch thì p là số nguyên tố; gạch các số p^2, $p^2 + p$, $p^2 + 2p$, ... $\le N$.

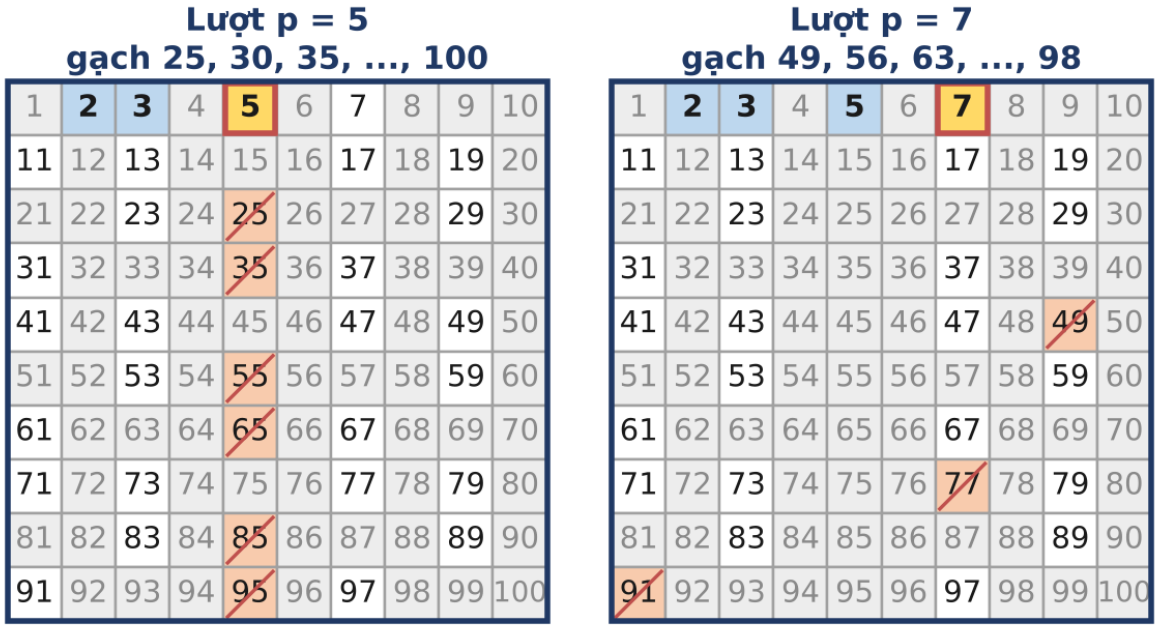
3) Kết thúc: các số chưa bị gạch chính là các số nguyên tố trong [2, N].

HÃY HÌNH DUNG

Hãy hình dung bảng 1..100 xếp thành 10 hàng. Bội của 2 tạo thành 5 cột dọc; bội của 3 tạo thành những đường chéo; bội của 5 lại là 2 cột. Mỗi lượt gạch, bảng "thưa" dần, và sau đúng bốn lượt ($p = 2, 3, 5, 7$), chỉ còn 25 số nguyên tố đứng lại.



Hình 6.1. *Hai lượt gạch đầu tiên trên bảng 1..100. Lượt p = 3 chỉ có 16 số thật sự mới bị gạch; các số như 12, 18 đã bị gạch ở lượt p = 2.*



Hình 6.2. *Lượt p = 5 và p = 7. Lượt 7 bắt đầu từ 49 và chỉ gạch mới được 3 số: 49, 77, 91.*



Hình 6.3. *Kết quả: 25 số nguyên tố không vượt quá 100.*

Bảng sau ghi lại từng lượt. Cột "số phép gạch" đếm cả những lần gạch lại một số đã bị gạch, còn cột "mới bị gạch" chỉ đếm số lần ô trắng chuyển thành ô gạch.

Lượt p	Bắt đầu gạch từ	Số phép gạch	Số mới bị gạch	Số chưa bị gạch (≥ 2)
2	4	49	49	50
3	9	31	16	34
5	25	16	6	28
7	49	8	3	**25**

ĐỊNH LÍ (TÍNH ĐÚNG CỦA SÀNG)

Sau khi xử lí mọi số nguyên tố $p \le \sqrt{N}$, một số $x \in [2, N]$ chưa bị gạch khi và chỉ khi x là số nguyên tố.

VÌ SAO ĐÚNG?

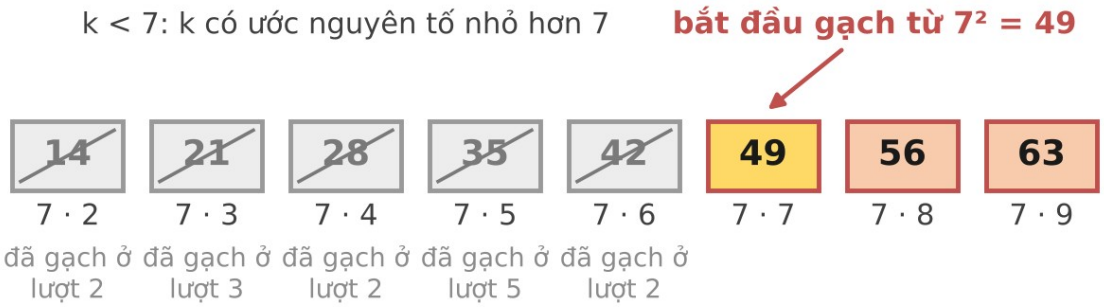
Số nguyên tố không bị gạch: mỗi số bị gạch có dạng $p \cdot k$ với $k \ge p \ge 2$, tức là có ước p với $1 < p < p \cdot k$: đó là hợp số.

Hợp số bị gạch: hợp số $x \le N$ có ước nguyên tố nhỏ nhất $p \le \sqrt{x} \le \sqrt{N}$ (Bài 5). Viết $x = p \cdot k$ thì mọi ước nguyên tố của k đều $\ge p$, nên $k \ge p$ và $x \ge p^2$. Vậy x là một trong các số p^2, $p^2 + p$, ... mà lượt p đã gạch.

Vòng ngoài chọn đúng số nguyên tố: khi tới lượt p, mọi hợp số $< p^2$ đều có ước nguyên tố $< p$ nên đã bị gạch ở lượt trước. Vì vậy p chưa bị gạch thì p nguyên tố.

2. Vì sao bắt đầu gạch từ p^2?

Ở lượt p = 7, các bội 14, 21, 28, 35, 42 có cần gạch nữa không? Mỗi số ấy bằng 7 · k với k < 7, và k có một ước nguyên tố nhỏ hơn 7. Bội 7 · k vì thế đã bị gạch ở lượt của ước ấy rồi.



Hình 6.4. *Các bội 7 · k với k < 7 đều đã bị gạch ở lượt 2, 3 hoặc 5. Bội đầu tiên còn có thể "trắng" là 7 · 7 = 49.*

Nhận xét này cho ta hai điều. Thứ nhất, mỗi lượt bắt đầu từ p^2 thay vì 2p, đỡ được một ít công. Thứ hai, quan trọng hơn: khi $p^2 > N$ thì lượt p **không gạch gì cả**, nên vòng ngoài dừng ngay khi p · p > N. Với N = 100 ta chỉ cần 4 lượt; với $N = 10^7$ chỉ cần các p ≤ 3162.

3. Bài toán DEMNT

BÀI TOÁN: DEMNT

Cho Q truy vấn, mỗi truy vấn gồm hai số a, b. Với mỗi truy vấn, hãy đếm số số nguyên tố trong đoạn [a, b].

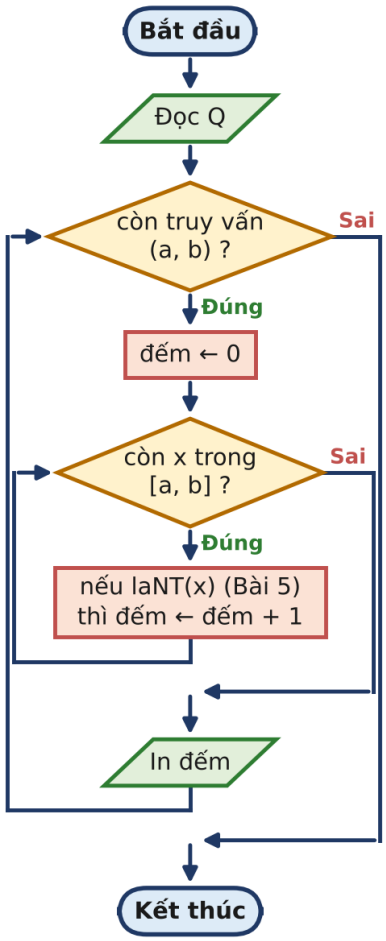
Dữ liệu vào: Dòng đầu chứa Q ($1 \le Q \le 10^6$). Q dòng tiếp theo, mỗi dòng chứa hai số a, b ($1 \le a \le b \le 10^7$).

Kết quả: Q dòng, mỗi dòng là đáp số của một truy vấn, theo thứ tự trong dữ liệu vào.

DEMNT.INP	DEMNT.OUT	Giải thích
3 1 10 10 30 1 100	4 6 25	[1, 10] có 2, 3, 5, 7; [10, 30] có 11, 13, 17, 19, 23, 29; [1, 100] có 25 số nguyên tố.
2 1 10000000 9999900 10000000	664579 9	Có 664 579 số nguyên tố không vượt quá 10^7; đoạn 101 số cuối có 9 số.
3 1 1 2 2 14 16	0 1 0	Các trường hợp biên: 1 không nguyên tố, 2 nguyên tố, [14, 16] không có số nào.

4. Ý tưởng ngây thơ: Kiểm tra từng số trong đoạn

Ta đã có hàm kiểm tra nguyên tố của Bài 5. Với mỗi truy vấn, duyệt x từ a tới b, gọi hàm kiểm tra và đếm.



Hình 6.5. *Sơ đồ khối của cách kiểm tra từng số trong mỗi đoạn.*

DEMNT.CPP (ngây thơ): kiểm tra từng số bằng cách thử chia

```cpp
#include <bits/stdc++.h>
using namespace std;

bool laNT(int n) {                          // thử ước tới √n (Bài 5)
    if (n < 2) return false;
    for (int d = 2; (long long)d * d <= n; d++)
        if (n % d == 0) return false;
    return true;
}

int main() {
    if (fopen("DEMNT.INP", "r")) {
        freopen("DEMNT.INP", "r", stdin);
        freopen("DEMNT.OUT", "w", stdout);
    }
    int q;
    cin >> q;
    while (q--) {
        int a, b, dem = 0;
        cin >> a >> b;
        for (int x = a; x <= b; x++)        // kiểm tra lại từng số
            if (laNT(x)) dem++;
        cout << dem << "\n";
    }
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Đếm thực tế: riêng **một** truy vấn $[1, 10^7]$ đã cần 1 746 210 133 phép chia (khoảng $1{,}7 \cdot 10^9$), vì mỗi số nguyên tố x phải thử đủ √x ước. Với $Q = 10^6$ truy vấn như thế, tổng số phép tính lên tới cỡ 10^{15}: hàng chục ngày.

BẾ TẮC

Có hai sự lãng phí. Mỗi số bị kiểm tra **lại từ đầu** ở mọi truy vấn chứa nó. Và mỗi lần kiểm tra lại không dùng chút thông tin nào từ những số khác: biết 7 là số nguyên tố chẳng giúp gì khi kiểm tra 49. Ta cần tính trước "x có nguyên tố không" cho **mọi** $x \le 10^7$, một lần duy nhất và thật rẻ.

5. Phá bế tắc: Sàng một lần, trả lời bằng mảng tiền tố

Bước 1 Sàng tới N = max b

Đọc hết các truy vấn, lấy N là giá trị b lớn nhất (không cần sàng thừa), rồi sàng Eratosthenes trên mảng `gach[0..N]` kiểu `char` (mỗi phần tử 1 byte). Chạy tay với N = 30:

Lượt p	p^2	Các số bị gạch	Mới bị gạch	Còn lại (≥ 2)
2	4	4, 6, 8, ..., 30 (14 số)	14 số	15
3	9	9, 12, 15, 18, 21, 24, 27, 30	9, 15, 21, 27	11

Lượt p	p^2	Các số bị gạch	Mới bị gạch	Còn lại (≥ 2)
5	25	25, 30	25	10
7	49 > 30	dừng		

Còn lại 10 số nguyên tố: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29. Tổng cộng có 14 + 8 + 2 = 24 phép gạch.

Bước 2 Đếm số phép gạch

Lượt p gạch khoảng N / p số. Cộng lại theo mọi số nguyên tố $p \le \sqrt{N}$, tổng số phép gạch xấp xỉ $N \cdot (1/2 + 1/3 + 1/5 + 1/7 + ...)$. Một kết quả nổi tiếng của Euler và Mertens nói rằng tổng nghịch đảo các số nguyên tố không vượt quá N tăng **cực kì chậm**, xấp xỉ ln ln N. Vậy sàng Eratosthenes tốn O(N log log N) phép tính. Bảng sau đếm bằng cách chạy thật:

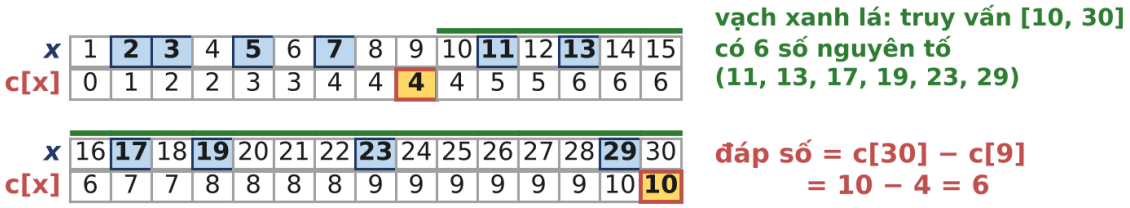
N	π(N): số SNT ≤ N	Số phép gạch (đếm thật)	N · ln ln N
10^3	168	1 411	1 933
10^4	1 229	16 981	22 203
10^5	9 592	193 078	244 347
10^6	78 498	2 122 048	2 625 792
10^7	664 579	22 850 051	27 799 426
10^8	5 761 455	242 570 204	291 347 399

Với $N = 10^7$, ln ln N ≈ 2,8: số phép gạch chỉ khoảng 2,3 lần N. Trên thực tế, sàng tới 10^7 chỉ mất vài phần trăm giây.

Bước 3 Mảng tiền tố

Sàng cho biết từng số có nguyên tố không, nhưng mỗi truy vấn có thể dài tới 10^7 số; không thể đếm lại từng ô. Ta dựng mảng **tiền tố** c[x] = số số nguyên tố trong [1, x], tính bằng c[x] = c[x − 1] + (x nguyên tố ? 1 : 0). Khi đó số số nguyên tố trong [a, b] là:

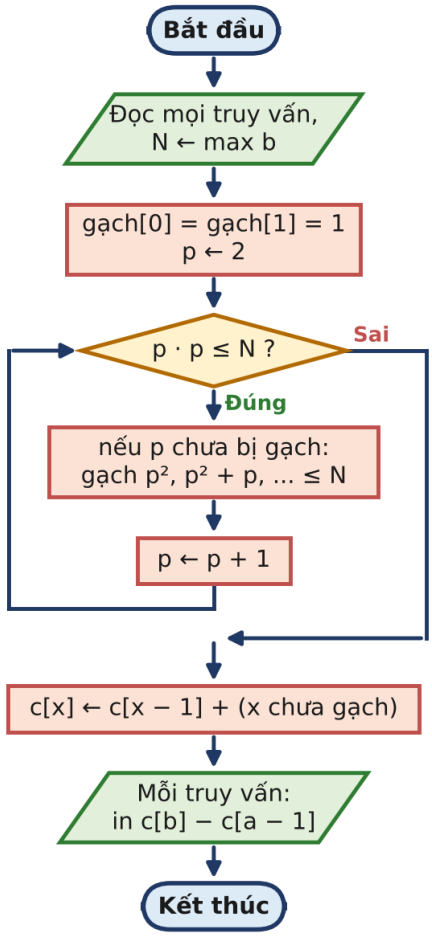
$$\mathbf{c[b] - c[a - 1]}$$



Hình 6.6. Mảng tiền tố c[x] với x ≤ 30. Truy vấn [10, 30] được trả lời bằng một phép trừ.

Lấy c[a − 1] chứ không phải c[a], vì a cũng thuộc đoạn cần đếm. Với a = 1, c[0] = 0 nên công thức vẫn đúng mà không cần xử lí riêng.

Bước 4 Chương trình



Hình 6.7. *Sơ đồ khối của cách sàng một lần rồi trả lời bằng mảng tiền tố.*

DEMNT.CPP: sàng Eratosthenes + mảng tiền tố

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DEMNT.INP", "r")) {
        freopen("DEMNT.INP", "r", stdin);
        freopen("DEMNT.OUT", "w", stdout);
    }
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    vector<int> a(q), b(q);
    int N = 1;                                  // chỉ cần sàng tới max b
    for (int i = 0; i < q; i++) {
        cin >> a[i] >> b[i];
        N = max(N, b[i]);
    }
    vector<char> gach(N + 1, 0);        // gach[x] = 1: x không nguyên tố
    gach[0] = gach[1] = 1;
    for (long long p = 2; p * p <= N; p++)
        if (!gach[p])                           // p nguyên tố: gạch bội từ p^2
            for (long long j = p * p; j <= N; j += p) gach[j] = 1;
    vector<int> c(N + 1, 0);                    // c[x] = số SNT trong [1, x]
    for (int x = 1; x <= N; x++) c[x] = c[x - 1] + !gach[x];
    for (int i = 0; i < q; i++)
        cout << c[b[i]] - c[a[i] - 1] << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Sàng O(N log log N), mảng tiền tố O(N), mỗi truy vấn O(1): tổng O(N log log N + Q). Bộ nhớ: mảng `gach` 10^7 byte (10 MB) và mảng `c` kiểu `int` 40 MB. Với N = 10^7 và Q = 10^6, chương trình chạy khoảng 0,4 giây, phần lớn là thời gian đọc và in. Vì dữ liệu lớn, nhớ dùng `ios::sync_with_stdio(false)` và in `"\n"` thay cho `endl`.

6. Mở rộng

Mở rộng 1: Sàng tuyến tính

Trong sàng Eratosthenes, 30 bị gạch ba lần (bởi 2, 3 và 5). Sàng tuyến tính (sàng Euler) sắp xếp sao cho **mỗi hợp số bị gạch đúng một lần**, bởi ước nguyên tố nhỏ nhất của nó. Ý tưởng: mỗi hợp số x viết duy nhất được thành x = i · p với p = lp[x] là ước nguyên tố nhỏ nhất của x, và p ≤ lp[i]. Vậy với mỗi i, ta chỉ gạch các số i · p với p nguyên tố chạy từ 2 tới lp[i].

DEMNT_TUYENTINH.CPP: sàng tuyến tính, mỗi hợp số gạch một lần

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DEMNT.INP", "r")) {
        freopen("DEMNT.INP", "r", stdin);
        freopen("DEMNT.OUT", "w", stdout);
    }
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    vector<int> a(q), b(q);
    int N = 1;
    for (int i = 0; i < q; i++) {
        cin >> a[i] >> b[i];
        N = max(N, b[i]);
    }
    vector<int> lp(N + 1, 0), pr;           // lp[x]: ước NT nhỏ nhất của x
    for (int i = 2; i <= N; i++) {
        if (lp[i] == 0) { lp[i] = i; pr.push_back(i); }
        // gạch i · p với mọi SNT p ≤ lp[i]: mỗi hợp số bị gạch đúng 1 lần
        for (int p : pr) {
            if (p > lp[i] || (long long)i * p > N) break;
            lp[i * p] = p;
        }
    }
    vector<int> c(N + 1, 0);
    for (int x = 1; x <= N; x++) c[x] = c[x - 1] + (x > 1 && lp[x] == x);
    for (int i = 0; i < q; i++)
        cout << c[b[i]] - c[a[i] - 1] << "\n";
    return 0;
}
```

N	Eratosthenes (gạch từ p^2)	Tuyến tính: N − π(N) − 1
30	24	19
10^6	2 122 048	921 501
10^7	22 850 051	9 335 420
10^8	242 570 204	94 238 544

Sàng tuyến tính gạch ít hơn khoảng 2,5 lần, nhưng tốn thêm mảng `lp` kiểu `int` và các phép truy cập mảng phức tạp hơn, nên trên thực tế không nhanh hơn bao nhiêu. Giá trị thật của nó là mảng lp[x], **ước nguyên tố nhỏ nhất** của mọi x: đó chính là chìa khóa của Bài 7.

Mở rộng 2: Tiết kiệm bộ nhớ khi N rất lớn

Với N = 10^9, mảng `char` cần 1 GB, vượt giới hạn bộ nhớ thông thường. Hai mẹo hay dùng: (1) **chỉ lưu số lẻ**, vì số chẵn duy nhất là số nguyên tố là 2; ô i ứng với số 2i + 1. (2) Dùng **`bitset`** hay `vector<bool>`, mỗi số chỉ chiếm 1 bit.

Sàng chỉ số lẻ

```
// chỉ lưu số lẻ: ô i ứng với số 2i + 1
vector<char> h(N / 2 + 1, 0);          // h[i] = 1: 2i + 1 là hợp số
for (long long i = 1; (2 * i + 1) * (2 * i + 1) <= N; i++)
    if (!h[i]) {
        long long p = 2 * i + 1;    // gạch p², p² + 2p, ... (bỏ bội chẵn)
        for (long long j = p * p; j <= N; j += 2 * p) h[j / 2] = 1;
    }
// x lẻ ≥ 3 là SNT khi h[x / 2] == 0; số 2 xét riêng
```

Cách lưu	$N = 10^7$	$N = 10^8$	$N = 10^9$
`char` cho mọi số	10 MB	100 MB	1 000 MB
`char`, chỉ số lẻ	5 MB	50 MB	500 MB
bit, chỉ số lẻ	0,6 MB	6,25 MB	62,5 MB

Mở rộng 3: Có bao nhiêu số nguyên tố?

Kí hiệu π(N) là số số nguyên tố không vượt quá N. **Định lí số nguyên tố** (chứng minh năm 1896) khẳng định π(N) xấp xỉ N / ln N, theo nghĩa tỉ số của chúng tiến về 1. Nói nôm na: quanh số N, cứ khoảng ln N số thì có một số nguyên tố. Quanh 10^{12}, ln N ≈ 27,6: các số nguyên tố vẫn còn khá "dày", điều mà Bài 8 sẽ tận dụng.

N	π(N)	N / ln N (làm tròn)	Tỉ số
10^3	168	145	1,161
10^4	1 229	1 086	1,132
10^5	9 592	8 686	1,104
10^6	78 498	72 382	1,084
10^7	664 579	620 421	1,071
10^8	5 761 455	5 428 681	1,061

BẪY THƯỜNG GẶP

- **Quên gạch 0 và 1:** nếu không gạch sẵn, mảng tiền tố đếm cả số 1 là số nguyên tố.
- **Dừng vòng ngoài nhưng quên vòng trong phải chạy tới N:** điều kiện $p \cdot p \le N$ chỉ dành cho vòng ngoài; vòng gạch đi tới tận N.
- **Tràn số:** với N gần $2 \cdot 10^9$, `j += p` hoặc `p * p` có thể vượt quá `int`. Dùng `long long` cho p và j.
- **Lấy c[b] − c[a] thay vì c[b] − c[a − 1]:** bỏ sót chính số a khi a là số nguyên tố.
- **In bằng `endl`:** với 10^6 dòng, mỗi `endl` đẩy bộ đệm ra màn hình, có thể chậm hơn vài lần. Dùng `"\n"`.

GHI NHỚ

- Sàng Eratosthenes: với mỗi p chưa bị gạch, $p \cdot p \le N$, gạch p^2, $p^2 + p$, ..., N. Các số còn lại (≥ 2) là số nguyên tố.
- Bắt đầu gạch từ p^2 vì các bội nhỏ hơn đã bị gạch bởi số nguyên tố nhỏ hơn; vòng ngoài dừng khi $p^2 > N$.
- Độ phức tạp O(N log log N): với $N = 10^7$ chỉ khoảng $2{,}3 \cdot 10^7$ phép gạch.
- Đếm trên đoạn: c[x] = c[x − 1] + [x nguyên tố], đáp số c[b] − c[a − 1].
- $\pi(N) \approx N / \ln N$; sàng tuyến tính cho thêm ước nguyên tố nhỏ nhất của mọi số.

Luyện tập

Bài	Tên	Gợi ý liên hệ
6.1	**ĐẾM SỐ NGUYÊN TỐ**	In $\pi(n)$ và số nguyên tố thứ k với $n \le 10^7$: sàng một lần, lưu danh sách các số nguyên tố.
6.2	**SỐ NGUYÊN TỐ SINH ĐÔI**	Đếm các cặp (p, p + 2) cùng nguyên tố trong [a, b] với nhiều truy vấn: mảng tiền tố trên dấu hiệu "p và p + 2 đều nguyên tố".
6.3	**SỐ CÓ ĐÚNG BA ƯỚC**	Với $n \le 10^{12}$, n có đúng 3 ước khi và chỉ khi $n = p^2$ với p nguyên tố: sàng tới 10^6 rồi kiểm tra $\sqrt{n}$.

Tự kiểm tra nhanh

1) Khi sàng với N = 100, số 91 bị gạch ở lượt p nào? *(Đáp số: lượt p = 7, vì 91 = 7 · 13)*

2) Sàng tới N = 1000 thì lượt cuối cùng có gạch là lượt p nào? *(Đáp số: p = 31 ($31^2 = 961 \le 1000 < 37^2$))*

3) Biết c[20] = 8 và c[50] = 15. Đoạn [21, 50] có bao nhiêu số nguyên tố? *(Đáp số: 15 − 8 = 7)*

4) Với N = 30, sàng Eratosthenes (gạch từ p^2) và sàng tuyến tính lần lượt thực hiện bao nhiêu phép gạch? *(Đáp số: 24 và 30 − 10 − 1 = 19)*

BÀI 7 Phân tích thừa số nguyên tố

Học xong bài này, em sẽ:

- ✓ Phát biểu và hiểu chứng minh **định lí cơ bản của số học**: mọi số > 1 phân tích được thành tích các số nguyên tố theo đúng một cách.
- ✓ Phân tích một số bằng cách thử chia tới √n, và biết vì sao phần còn lại (nếu > 1) là số nguyên tố.
- ✓ Dựng mảng **ước nguyên tố nhỏ nhất** spf bằng sàng, rồi phân tích mỗi số trong O(log n).
- ✓ Từ phân tích, tính được số ước, tổng các ước, ƯCLN, BCNN và số mũ của p trong n!.

KHỞI ĐỘNG

Các nhà hóa học biết rằng mọi chất đều được tạo từ một bảng hữu hạn các nguyên tố hóa học, và mỗi phân tử có một "công thức" duy nhất: nước luôn là H_2O. Số học cũng có bảng nguyên tố của riêng mình: các **số nguyên tố**. Mỗi số tự nhiên lớn hơn 1 có một "công thức phân tử" duy nhất, chẳng hạn $360 = 2^3 \cdot 3^2 \cdot 5$.

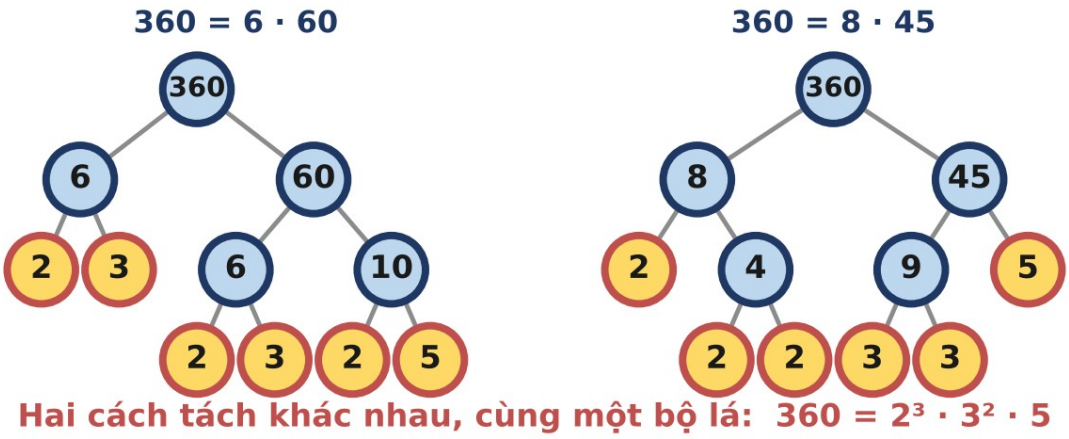
Biết công thức ấy, em trả lời ngay được 360 có bao nhiêu ước (24), tổng các ước bằng bao nhiêu (1170), hay 360 và 84 có ƯCLN là mấy (12). Câu hỏi của bài học: làm sao tìm công thức ấy cho **một triệu** số cỡ 10^7 trong chưa tới một giây?

1. Định lí cơ bản của số học

Thử tách 360 thành tích hai số, rồi tách tiếp từng số cho tới khi chỉ còn các số nguyên tố. Mỗi người có thể bắt đầu theo một cách khác nhau, nhưng điều kì lạ là ai cũng đến cùng một kết quả.

HÃY HÌNH DUNG

Hãy hình dung mỗi lần tách là một nhánh cây: số ở nút cha bằng tích hai nút con. Cây dừng mọc khi mọi lá đều là số nguyên tố. Hai cái cây trông khác nhau hoàn toàn, nhưng khi gom các lá lại thì được đúng cùng một bộ.



Hình 7.1. *Hai cây phân tích của 360. Nút xanh là hợp số còn tách tiếp; lá vàng là số nguyên tố.*

ĐỊNH LÍ CƠ BẢN CỦA SỐ HỌC

Mọi số tự nhiên $n > 1$ đều viết được thành tích các số nguyên tố, và cách viết ấy là **duy nhất** nếu không kể thứ tự các thừa số. Gom các thừa số giống nhau, ta được **dạng phân tích chính tắc**:

$$n = p_1^{e_1} \cdot p_2^{e_2} \cdot \ldots \cdot p_k^{e_k}, \quad p_1 < p_2 < \ldots < p_k, \quad e_i \geq 1$$

VÌ SAO ĐÚNG?

Tồn tại. Quy nạp theo n. Nếu n nguyên tố thì xong. Nếu không, theo Bài 5, n có ước nguyên tố p, và $n = p \cdot m$ với $1 < m < n$. Theo giả thiết quy nạp, m là tích các số nguyên tố, nên n cũng vậy.

Bổ đề Euclid. Nếu p nguyên tố và $p \mid a \cdot b$ thì $p \mid a$ hoặc $p \mid b$. Thật vậy, nếu p không chia hết a thì $\gcd(p, a) = 1$ (ước dương của p chỉ có 1 và p), nên theo định lí Bézout (Bài 4) có x, y với $p \cdot x + a \cdot y = 1$. Nhân hai vế với b: $b = p \cdot b \cdot x + (a \cdot b) \cdot y$, cả hai số hạng đều chia hết cho p, nên $p \mid b$.

Duy nhất. Giả sử $p_1 \cdot p_2 \cdot \ldots \cdot p_r = q_1 \cdot q_2 \cdot \ldots \cdot q_s$ (đều là số nguyên tố). Vì p_1 chia hết vế phải, áp dụng bổ đề Euclid nhiều lần, p_1 chia hết một q_j nào đó; mà q_j nguyên tố nên $p_1 = q_j$. Giản ước hai vế cho p_1 rồi lặp lại: hai bộ thừa số trùng nhau.

Dạng chính tắc của một số thường **rất ngắn**. Vì mỗi thừa số ít nhất bằng 2, số thừa số (tính cả lặp) không vượt quá $\log_2 n$; với $n \leq 10^7$ là tối đa 23 (đạt ở $2^{23} = 8\,388\,608$). Số ước nguyên tố khác nhau còn ít hơn: tích tám số nguyên tố đầu tiên là 9 699 690, tích chín số đã là 223 092 870 $> 10^7$.

n	Dạng chính tắc	Số SNT khác nhau	Tổng số mũ
360	$2^3 \cdot 3^2 \cdot 5$	3	6

n	Dạng chính tắc	Số SNT khác nhau	Tổng số mũ
1 001	$7 \cdot 11 \cdot 13$	3	3
1 024	2^{10}	1	10
2 026	$2 \cdot 1013$	2	2
9 699 690	$2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19$	**8**	8
8 388 608	2^{23}	1	**23**
10 000 000	$2^7 \cdot 5^7$	2	14

2. Bài toán PTTS

BÀI TOÁN: PTTS

Cho Q số nguyên. Hãy phân tích mỗi số ra thừa số nguyên tố.

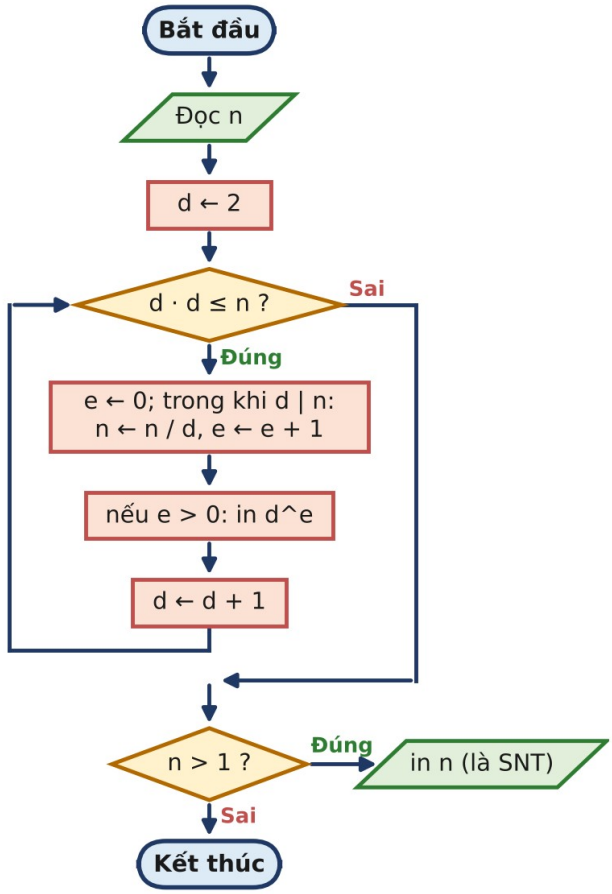
Dữ liệu vào: Dòng đầu chứa Q ($1 \le Q \le 10^6$). Q dòng tiếp theo, mỗi dòng chứa một số n ($2 \le n \le 10^7$).

Kết quả: Q dòng, mỗi dòng có dạng `n = p1^e1 * p2^e2 * ...` với các thừa số nguyên tố tăng dần; số mũ bằng 1 thì không ghi.

PTTS.INP	PTTS.OUT	Giải thích
3 360 97 1001	360 = 2^3 * 3^2 * 5 97 = 97 1001 = 7 * 11 * 13	Ví dụ chạy tay trong bài. Số nguyên tố 97 chỉ có một thừa số là chính nó.
3 9699690 8388608 9999991	9699690 = 2 * 3 * 5 * 7 * 11 * 13 * 17 * 19 8388608 = 2^23 9999991 = 9999991	Số có nhiều ước nguyên tố khác nhau nhất và số có tổng số mũ lớn nhất trong giới hạn; 9 999 991 là số nguyên tố.
2 2 10000000	2 = 2 10000000 = 2^7 * 5^7	Hai số nhỏ nhất và lớn nhất có thể.

3. Ý tưởng ngây thơ: Thử chia tới √n

Cách tự nhiên: thử d = 2, 3, 4, ... Mỗi khi d | n thì **chia n cho d tới hết** và ghi lại số lần chia được (số mũ). Vì các ước nhỏ đã bị "vắt kiệt" trước, khi n chia hết cho d thì d chắc chắn là số nguyên tố: nếu d = a · b với a, b < d thì n đã không còn chia hết cho a. Ta dừng khi d · d > n (n lúc này đã bị chia nhỏ đi); nếu n còn lại lớn hơn 1 thì nó là một thừa số nguyên tố cuối cùng.



Hình 7.2. *Sơ đồ khối của cách thử chia tới √n cho một số n.*

PTTS.CPP (ngây thơ): thử chia tới √n cho từng số

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("PTTS.INP", "r")) {
        freopen("PTTS.INP", "r", stdin);
        freopen("PTTS.OUT", "w", stdout);
    }
    int q;
    cin >> q;
    while (q--) {
        int n;
        cin >> n;
        cout << n << " =";
        bool dau = true;
        for (int d = 2; (long long)d * d <= n; d++) {   // thử chia tới √n
            if (n % d) continue;
            int e = 0;
            while (n % d == 0) n /= d, e++;  // chia hết mức có thể
            cout << (dau ? " " : " * ") << d;
            if (e > 1) cout << "^" << e;
            dau = false;
        }
        if (n > 1) cout << (dau ? " " : " * ") << n;  // phần còn lại là SNT
        cout << "\n";
    }
    return 0;
}
```

VÌ SAO PHẦN CÒN LẠI LÀ SỐ NGUYÊN TỐ?

Khi vòng lặp dừng, n hiện tại không còn ước nào trong $[2, d-1]$ và $d \cdot d > n$. Nếu $n > 1$ là hợp số thì n có ước nguyên tố $p \le \sqrt{n} < d$, mâu thuẫn. Vậy $n = 1$ hoặc n là số nguyên tố. Ví dụ 2026: ngay ở $d = 2$, n đã thành 1013; vòng lặp dừng ở $d = 32$ vì $32 \cdot 32 > 1013$, và 1013 là thừa số nguyên tố cuối cùng.

ĐẾM SỐ PHÉP TÍNH

Trường hợp xấu nhất là n nguyên tố gần 10^7: phải thử khoảng $\sqrt{n} \approx 3162$ số. Với $Q = 10^6$ số như thế: khoảng $3{,}2 \cdot 10^9$ phép chia. Đo thực tế với một triệu số nguyên tố trong $[9 \cdot 10^6, 10^7]$: **15 giây**.

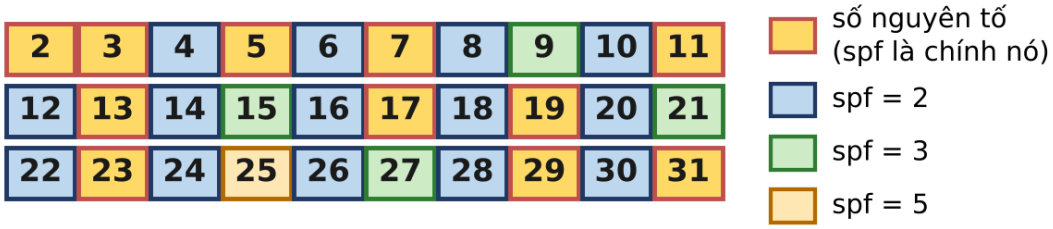
BẾ TẮC

Vẫn là căn bệnh của Bài 6: mỗi số được phân tích riêng rẽ từ đầu. Trong khi đó, nếu ta **biết sẵn một ước nguyên tố của n** thì chỉ một phép chia là n đã nhỏ đi ít nhất một nửa. Liệu có thể tính sẵn một ước nguyên tố cho mọi số $\le 10^7$?

4. Phá bế tắc: Sàng ước nguyên tố nhỏ nhất

Bước 1 Mảng spf

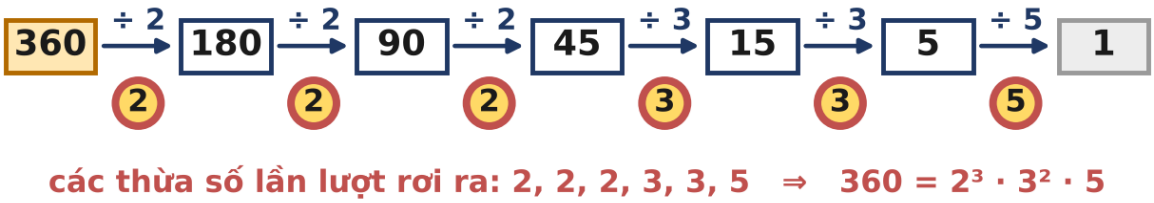
Gọi **spf[x]** (smallest prime factor) là ước nguyên tố nhỏ nhất của x. Sàng Eratosthenes tính spf gần như miễn phí: khi lượt p gạch số j **lần đầu tiên**, p chính là ước nguyên tố nhỏ nhất của j (vì các lượt chạy theo p tăng dần). Vậy thay vì ghi `gach[j] = 1`, ta ghi `spf[j] = p` nếu spf[j] chưa có giá trị. Số nguyên tố p thì spf[p] = p.



Hình 7.3. Mảng spf với x từ 2 tới 31, tô màu theo giá trị spf. Ước nguyên tố nhỏ nhất của 25 là 5, của 27 là 3.

Bước 2 Nhảy theo spf

Có spf rồi, việc phân tích chỉ còn là: lấy p = spf[n], chia n cho p, lặp lại tới khi n = 1. Các thừa số rơi ra **theo thứ tự tăng dần**, vì n / p chỉ còn các ước nguyên tố ≥ p.

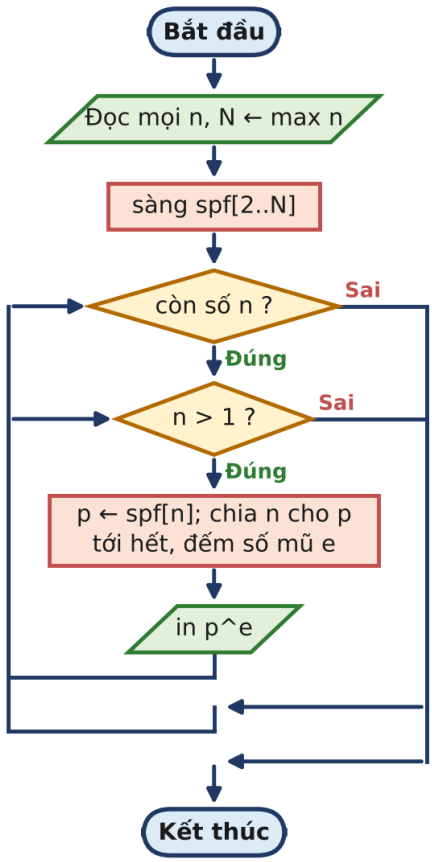


Hình 7.4. Phân tích 360 bằng sáu bước nhảy n ← n / spf[n].

n	spf[n]	n / spf[n]	Ghi nhận
360	2	180	2 (lần 1)
180	2	90	2 (lần 2)
90	2	45	2 (lần 3): xong p = 2, in $\mathbf{2^3}$
45	3	15	3 (lần 1)
15	3	5	3 (lần 2): xong p = 3, in $\mathbf{3^2}$
5	5	1	5 (lần 1): in **5**; n = 1, dừng

Trong chương trình, vòng `while (n % p == 0)` gom các lần chia cho cùng một p để đếm số mũ e.

Bước 3 Chương trình



Hình 7.5. Sơ đồ khối của cách sàng spf một lần rồi nhảy theo spf.

VÌ SAO ĐÚNG?

Đúng: spf[n] luôn là một ước nguyên tố của n hiện tại, nên mỗi lần chia ta tách ra đúng một thừa số nguyên tố, và tích các thừa số đã tách nhân với n hiện tại luôn bằng số ban đầu (bất biến). Khi n = 1, các thừa số đã tách chính là phân tích cần tìm.

Nhanh: mỗi lần chia, n giảm ít nhất 2 lần (vì $p \geq 2$), nên số lần chia không vượt quá $\log_2 n \leq 23$ với $n \leq 10^7$.

PTTS.CPP: sàng spf rồi phân tích trong O(log n)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("PTTS.INP", "r")) {
        freopen("PTTS.INP", "r", stdin);
        freopen("PTTS.OUT", "w", stdout);
    }
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    vector<int> a(q);
    int N = 2;
    for (int i = 0; i < q; i++) {
        cin >> a[i];
        N = max(N, a[i]);
    }
    vector<int> spf(N + 1, 0);              // ước nguyên tố nhỏ nhất
    for (long long i = 2; i <= N; i++)
        if (spf[i] == 0) {                  // i nguyên tố
            spf[i] = i;
            for (long long j = i * i; j <= N; j += i)
                if (spf[j] == 0) spf[j] = i;    // i là SNT đầu tiên gạch j
        }
    for (int i = 0; i < q; i++) {
        int n = a[i];
        cout << n << " =";
        bool dau = true;
        while (n > 1) {
            int p = spf[n], e = 0;          // p chắc chắn là ước nguyên tố
            while (n % p == 0) n /= p, e++;
            cout << (dau ? " " : " * ") << p;
            if (e > 1) cout << "^" << e;
            dau = false;
        }
        cout << "\n";
    }
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Sàng spf: O(N log log N) với N = max n. Mỗi số: O(log n), tối đa 23 phép chia. Với Q = 10^6 số nguyên tố gần 10^7 như trên: **0,43 giây** so với 15 giây của cách ngây thơ, và phần lớn thời gian là in kết quả. Bộ nhớ: mảng spf kiểu `int`, 40 MB với N = 10^7.

5. Mở rộng

Mở rộng 1: Số ước và tổng các ước

Nếu $n = p_1^{e_1} \cdot \ldots \cdot p_k^{e_k}$ thì mọi ước của n có dạng $p_1^{f_1} \cdot \ldots \cdot p_k^{f_k}$ với $0 \le f_i \le e_i$ (theo tính duy nhất của phân tích). Mỗi f_i có $e_i + 1$ cách chọn, nên:

$$\mathbf{d(n) = (e_1 + 1) \cdot (e_2 + 1) \cdot \ldots \cdot (e_k + 1)}$$

$$\mathbf{\sigma(n) = (1 + p_1 + \ldots + p_1^{e_1}) \cdot \ldots \cdot (1 + p_k + \ldots + p_k^{e_k})}$$

Công thức thứ hai có được khi khai triển tích: mỗi cách chọn một số hạng từ mỗi ngoặc cho đúng một ước. Với $360 = 2^3 \cdot 3^2 \cdot 5$:

p	e	e + 1	$1 + p + \ldots + p^e$
2	3	4	1 + 2 + 4 + 8 = 15
3	2	3	1 + 3 + 9 = 13
5	1	2	1 + 5 = 6
Tích		d(360) = **24**	σ(360) = 15 · 13 · 6 = **1170**

Mở rộng 2: ƯCLN và BCNN theo số mũ

Nếu viết a và b theo cùng một bộ số nguyên tố (cho phép số mũ 0) thì gcd(a, b) lấy **số mũ nhỏ nhất**, lcm(a, b) lấy **số mũ lớn nhất** của từng số nguyên tố. Ví dụ $360 = 2^3 \cdot 3^2 \cdot 5^1 \cdot 7^0$ và $84 = 2^2 \cdot 3^1 \cdot 5^0 \cdot 7^1$ cho gcd = $2^2 \cdot 3 = 12$, lcm = $2^3 \cdot 3^2 \cdot 5 \cdot 7 = 2520$. Vì min(x, y) + max(x, y) = x + y nên gcd(a, b) · lcm(a, b) = a · b. Cách này đặc biệt hữu ích khi cần **BCNN của cả dãy số theo mô-đun**: không thể chia lấy dư, nhưng có thể giữ số mũ lớn nhất của từng số nguyên tố.

Mở rộng 3: Công thức Legendre và số chữ số 0 của n!

Số mũ của số nguyên tố p trong n! bằng số lần p xuất hiện trong các thừa số 1, 2, ..., n: có $\lfloor n/p \rfloor$ số chia hết cho p, trong đó $\lfloor n/p^2 \rfloor$ số góp thêm một thừa số p nữa, và cứ thế:

$$e_p(n!) = \lfloor n / p \rfloor + \lfloor n / p^2 \rfloor + \lfloor n / p^3 \rfloor + \ldots$$

Công thức Legendre (chia dần n thay vì tính lũy thừa p, tránh tràn)

```
long long somu(long long n, long long p) {  // số mũ của p trong n!
    long long s = 0;
    while (n) { n /= p; s += n; }          // [n/p] + [n/p²] + ...
    return s;
}
```

Số chữ số 0 tận cùng của n! là số mũ của 10 = 2 · 5, tức $\min(e_2, e_5) = e_5$ (vì thừa số 2 luôn nhiều hơn). Với n = 2026: 405 + 81 + 16 + 3 = **505** chữ số 0; với n = 100 là 20 + 4 = 24.

BẪY THƯỜNG GẶP

- **Quên thừa số cuối cùng:** trong cách thử chia, sau vòng lặp nếu n > 1 thì n còn lại là một số nguyên tố, phải in ra. Ví dụ 2026 = 2 · 1013.
- **Tràn số trong sàng spf:** `i * i` với i tới 10^7 vượt `int`; khai báo `long long i` hoặc so sánh `i <= N / i`.
- **Ghi đè spf:** thiếu điều kiện `if (spf[j] == 0)` thì spf[j] thành ước nguyên tố **lớn nhất** của j, thứ tự in sai.
- **Sàng spf cho n quá lớn:** mảng `int` tới 10^8 tốn 400 MB. Với vài số n tới 10^{12}, hãy thử chia tới √n.
- **Tính p^k trong Legendre:** dễ tràn khi n ≈ 10^{18}; dùng vòng `n /= p`.

GHI NHỚ

- Mọi $n > 1$ có duy nhất một dạng $n = p_1^{e_1} \cdot \ldots \cdot p_k^{e_k}$; chứng minh tính duy nhất dựa vào bổ đề Euclid (từ Bézout).
- Thử chia tới $\sqrt{n}$, vắt kiệt từng ước; phần còn lại > 1 là số nguyên tố: $O(\sqrt{n})$ mỗi số.
- Sàng spf: lần gạch đầu tiên cho ước nguyên tố nhỏ nhất. Phân tích bằng $n \leftarrow n / \text{spf}[n]$: $O(\log n)$ mỗi số.
- Với $n = p_1^{e_1} \cdot \ldots \cdot p_k^{e_k}$: $d(n) = (e_1 + 1) \cdot \ldots \cdot (e_k + 1)$, $\sigma(n)$ là tích các tổng $1 + p + \ldots + p^e$; gcd lấy min, lcm lấy max số mũ.
- Số mũ của p trong $n!$ là $\Sigma \lfloor n / p^k \rfloor$; số chữ số 0 tận cùng của $n!$ là số mũ của 5.

Luyện tập

Bài	Tên	Gợi ý liên hệ
7.1	**SỐ ƯỚC**	Với 10^6 số $n \le 10^7$, in số ước và tổng các ước: sàng spf rồi áp dụng công thức $d(n)$, $\sigma(n)$.
7.2	**CHỮ SỐ 0 CỦA GIAI THỪA**	Số chữ số 0 tận cùng của $n!$ trong hệ cơ số b: phân tích b, dùng công thức Legendre cho từng $p \mid b$, lấy min của $e_p(n!) /$ (số mũ của p trong b).
7.3	**BCNN CỦA DÃY**	BCNN của $n \le 10^5$ số $a_i \le 10^7$ theo mô-đun $10^9 + 7$: phân tích từng số bằng spf, giữ số mũ lớn nhất của mỗi số nguyên tố.

Tự kiểm tra nhanh

1) Phân tích 2026 ra thừa số nguyên tố. *(Đáp số: $2026 = 2 \cdot 1013$)*

2) spf[91] bằng bao nhiêu? Phân tích 91 bằng mảng spf cần mấy lần chia? *(Đáp số: spf[91] = 7; hai lần chia ($91 \to 13 \to 1$))*

3) Số 720 có bao nhiêu ước, tổng các ước bằng bao nhiêu? *(Đáp số: $720 = 2^4 \cdot 3^2 \cdot 5$, $d = 5 \cdot 3 \cdot 2 = 30$, $\sigma = 31 \cdot 13 \cdot 6 = 2418$)*

4) 1000! có bao nhiêu chữ số 0 tận cùng? *(Đáp số: $200 + 40 + 8 + 1 = 249$)*

BÀI 8 Sàng trên đoạn

Học xong bài này, em sẽ:

- ✓ Hiểu vì sao sàng thường không dùng được khi R tới 10^{12}, và vì sao chỉ cần các số nguyên tố ≤ √R.
- ✓ Cài đặt **sàng trên đoạn**: gạch bội của mỗi p trong [L, R], bắt đầu từ max(p^2, ⌈L / p⌉ · p).
- ✓ Xử lí đúng các chi tiết: chỉ số x − L, số 1, số nguyên tố p nằm ngay trong đoạn.
- ✓ Biết các số nguyên tố "dày" tới đâu quanh 10^{12} và cách dùng sàng đoạn cho những câu hỏi khác.

KHỞI ĐỘNG

Một nhóm bạn muốn tìm các số nguyên tố trong đoạn [999 999 000 000, 10^{12}]. Bạn An đề nghị: "Sàng Eratosthenes tới 10^{12} rồi đọc kết quả." Bạn Bình tính nhẩm: mảng một nghìn tỉ phần tử, mỗi phần tử 1 byte, là **1 TB** bộ nhớ, và hàng chục nghìn tỉ phép gạch. Không máy tính thi đấu nào chịu nổi.

Nhưng đoạn cần xét chỉ dài một triệu số. Liệu có thể "sàng" riêng một triệu số ấy, như đặt một ô cửa sổ nhỏ lên trục số khổng lồ, mà bỏ qua phần trước nó? Câu trả lời là có, và chỉ tốn vài mili giây.

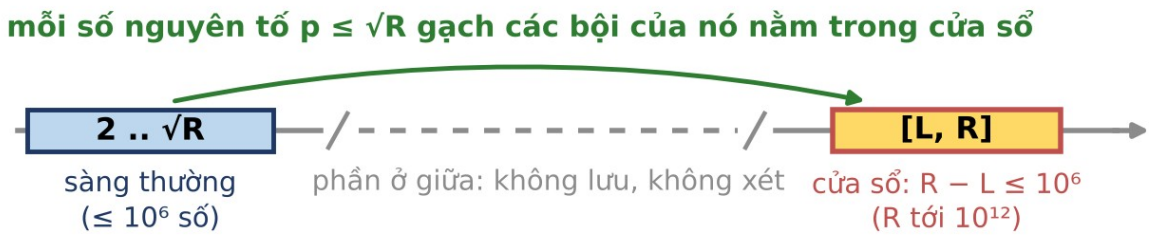
1. Ý tưởng: chỉ cần các số nguyên tố ≤ √R

Hãy nhớ lại điều cốt lõi của Bài 5: **hợp số x có một ước nguyên tố ≤ √x**. Với mọi x trong [L, R] thì √x ≤ √R ≤ 10^6. Vậy muốn biết x có là hợp số không, ta chỉ cần biết các số nguyên tố không vượt quá 10^6, những số mà sàng thường tìm ra trong nháy mắt.

Ngược lại, để loại các hợp số trong [L, R], ta cho mỗi số nguyên tố p ≤ √R **gạch các bội của nó nằm trong đoạn**, giống hệt sàng Eratosthenes nhưng trên một "cửa sổ" thay vì từ 2.

HÃY HÌNH DUNG

Trục số kéo dài tới 10^{12}. Ta chỉ giữ trong bộ nhớ hai mẩu nhỏ: mẩu đầu [2, √R] để tìm các số nguyên tố "thợ gạch", và mẩu cửa sổ [L, R]. Mỗi thợ gạch p nhảy những bước dài p, chỉ "đặt chân" xuống trong cửa sổ.



Hình 8.1. *Hai mẩu được lưu trong bộ nhớ; toàn bộ phần ở giữa bị bỏ qua.*

	Sàng thường tới R	Sàng trên đoạn [L, R]
Bộ nhớ	R byte (1 TB khi $R = 10^{12}$)	$\sqrt{R} + (R - L + 1)$ byte (khoảng 2 MB)
Phép gạch	khoảng $R \cdot \ln\ln R$	khoảng $\sqrt{R} \cdot \ln\ln\sqrt{R} + (R - L) \cdot \ln\ln R$
Dùng được khi	$R \le$ khoảng 10^8	$R \le 10^{12}$ (thậm chí 10^{14}), $R - L \le 10^7$

2. Bài toán SANGDOAN

BÀI TOÁN: SANGDOAN

Cho hai số L, R. Hãy liệt kê mọi số nguyên tố trong đoạn [L, R].

Dữ liệu vào: Một dòng chứa hai số L, R ($1 \le L \le R \le 10^{12}$, $R - L \le 10^6$).

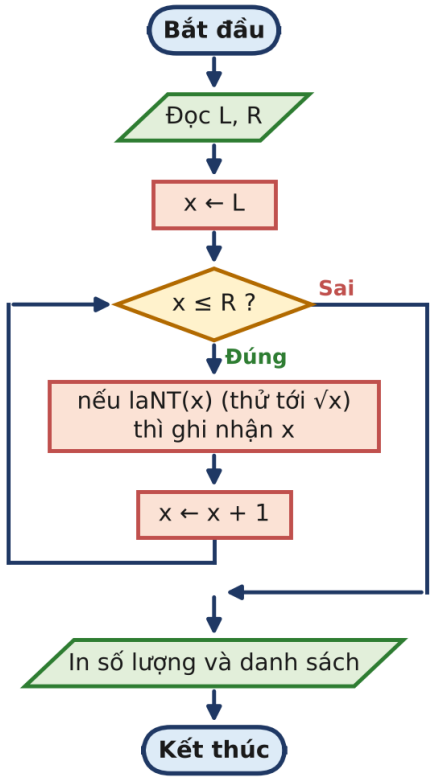
Kết quả: Dòng đầu ghi số lượng số nguyên tố trong [L, R]. Dòng thứ hai ghi các số nguyên tố đó theo thứ tự tăng dần, cách nhau một dấu cách (không có dòng này nếu không có số nào).

SANGDOAN.INP	SANGDOAN.OUT	Giải thích
`100 130`	`6` `101 103 107 109 113 127`	Ví dụ chạy tay trong bài.
`1 20`	`8` `2 3 5 7 11 13 17 19`	L = 1: số 1 không nguyên tố; các số nguyên tố 2, 3 nằm ngay trong đoạn và không được tự gạch mình.

`999999999900`
`1000000000000`
	`4`
	`999999999937 999999999959`
	`999999999961 999999999989`
		Đoạn sát giới hạn 10^{12}; 999 999 999 989 là số nguyên tố lớn nhất không vượt quá 10^{12}.

3. Ý tưởng ngây thơ: Kiểm tra từng số trong đoạn

Duyệt x từ L tới R, kiểm tra từng số bằng cách thử chia tới $\sqrt{x}$ như Bài 5.



Hình 8.2. *Sơ đồ khối của cách kiểm tra từng số trong đoạn.*

SANGDOAN.CPP (ngây thơ): thử chia từng số tới √x

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

bool laNT(ll n) {                              // thử ước tới √n (Bài 5)
    if (n < 2) return false;
    for (ll d = 2; d * d <= n; d++)
        if (n % d == 0) return false;
    return true;
}

int main() {
    if (fopen("SANGDOAN.INP", "r")) {
        freopen("SANGDOAN.INP", "r", stdin);
        freopen("SANGDOAN.OUT", "w", stdout);
    }
    ll L, R;
    cin >> L >> R;
    vector<ll> kq;
    for (ll x = L; x <= R; x++)                // kiểm tra riêng từng số
        if (laNT(x)) kq.push_back(x);
    cout << kq.size() << "\n";
    for (size_t i = 0; i < kq.size(); i++)
        cout << kq[i] << (i + 1 < kq.size() ? " " : "\n");
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Hợp số thường lộ ước nhỏ rất sớm, nhưng mỗi số nguyên tố bắt ta thử đủ $\sqrt{x} \approx 10^6$ ước. Đoạn $[10^{12} - 10^6, 10^{12}]$ có 36 400 số nguyên tố, nên tổng cộng khoảng $3{,}6 \cdot 10^{10}$ phép chia: ước lượng khoảng hai phút; kể cả khi dùng mẹo $6k \pm 1$ cũng còn hơn nửa phút.

BẾ TẮC

Cùng một số nguyên tố nhỏ, chẳng hạn 7, được đem ra thử chia với **từng** số trong đoạn, dù các bội của 7 trong đoạn cách đều nhau và ta biết trước chúng nằm ở đâu. Thay vì hỏi một triệu lần "x có chia hết cho 7 không?", hãy để 7 nhảy thẳng tới các bội của nó.

4. Phá bế tắc: Sàng trên cửa sổ [L, R]

Bước 1 Sàng thường các số nguyên tố ≤ √R

Tính $S = \lfloor\sqrt{R}\rfloor$ (dùng `sqrtl` rồi chỉnh lại bằng phép nhân số nguyên cho chắc chắn), sàng Eratosthenes trên $[2, S]$ và lưu danh sách các số nguyên tố. Với $R = 10^{12}$, $S = 10^6$ và có 78 498 số nguyên tố.

Bước 2 Gạch bội của p trong đoạn

Mảng `gach` có $R - L + 1$ phần tử; số x ứng với ô **x − L**. Với mỗi số nguyên tố $p \le S$, bội đầu tiên của p không nhỏ hơn L là $\lceil L / p \rceil \cdot p$, tính bằng số nguyên `(L + p - 1) / p * p`. Nhưng ta bắt đầu gạch từ:

$$\mathbf{\max(p^2, \lceil L / p \rceil \cdot p)}$$

Lí do thứ nhất giống Bài 6: các bội nhỏ hơn p^2 đã bị số nguyên tố nhỏ hơn gạch. Lí do thứ hai **bắt buộc**: nếu p nằm trong đoạn (như khi L = 1), bội đầu tiên $\lceil L / p \rceil \cdot p$ chính là p, và ta sẽ gạch nhầm một số nguyên tố. Lấy max với p^2 tránh được điều đó.

VÍ DỤ: CỬA SỔ SÁT 10^{12}

Với L = 999 999 999 900, R = 10^{12} (ví dụ 3), cửa sổ có 101 số. Bội đầu tiên ≥ L và số phép gạch của vài số nguyên tố:

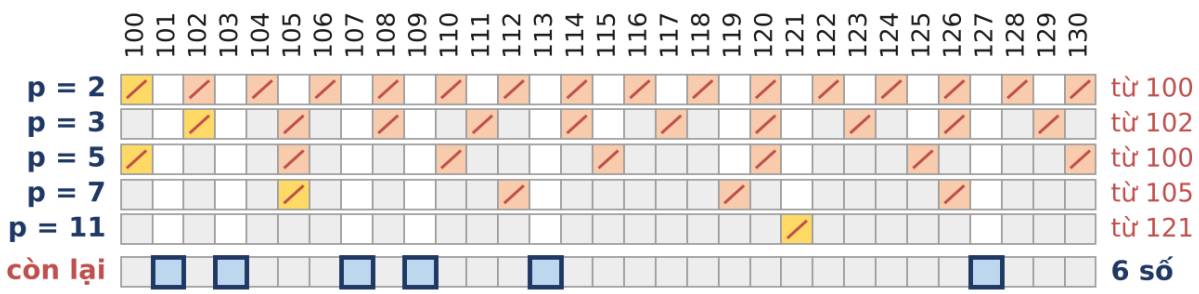
p	⌈L / p⌉ · p	Ô bắt đầu (x − L)	Số phép gạch trong đoạn
2	999 999 999 900	0	51
3	999 999 999 900	0	34
7	999 999 999 901	1	15
997	999 999 999 919	19	1
999 983	1 000 000 999 694	> R	0

Các số nguyên tố lớn gần √R hầu như không gạch gì, nhưng vẫn phải được xét: một số trong đoạn có thể là bình phương hay tích của hai số nguyên tố cỡ 10^6. Lưu ý (L + p − 1) / p · p ≤ R + p nên không bao giờ tràn `long long`.

Bước 3 **Chạy tay với [L, R] = [100, 130]**

S = ⌊√130⌋ = 11, các số nguyên tố "thợ gạch" là 2, 3, 5, 7, 11.

p	⌈L / p⌉ · p	p^2	Bắt đầu	Các số bị gạch trong [100, 130]
2	100	4	100	100, 102, 104, ..., 130 (16 số)
3	102	9	102	102, 105, 108, ..., 129 (10 số)
5	100	25	100	100, 105, 110, ..., 130 (7 số)
7	105	49	105	105, 112, 119, 126
11	110	121	**121**	121



Hình 8.3. Mỗi hàng là một lượt gạch; ô vàng là bội đầu tiên được gạch. Hàng cuối: 6 số còn lại là 101, 103, 107, 109, 113, 127.

Để ý hàng p = 11: bội đầu tiên ≥ 100 của 11 là 110, nhưng 110 = 11 · 10 đã bị 2 và 5 gạch; lượt 11 bắt đầu từ $121 = 11^2$, số duy nhất trong đoạn mà chỉ 11 mới gạch được. Ví dụ 2 (L = 1, R = 20) cho thấy vì sao phải lấy max với p^2:

p	⌈L / p⌉ · p	p^2	Bắt đầu	Các số bị gạch trong [1, 20]
2	**2**	4	4	4, 6, 8, 10, 12, 14, 16, 18, 20
3	**3**	9	9	9, 12, 15, 18
(số 1)				gạch riêng vì L = 1

Nếu bắt đầu từ ⌈L / p⌉ · p thì lượt 2 gạch luôn số 2 và lượt 3 gạch luôn số 3: hai số nguyên tố bị mất. Bắt đầu từ p^2 thì còn lại đúng 8 số 2, 3, 5, 7, 11, 13, 17, 19.

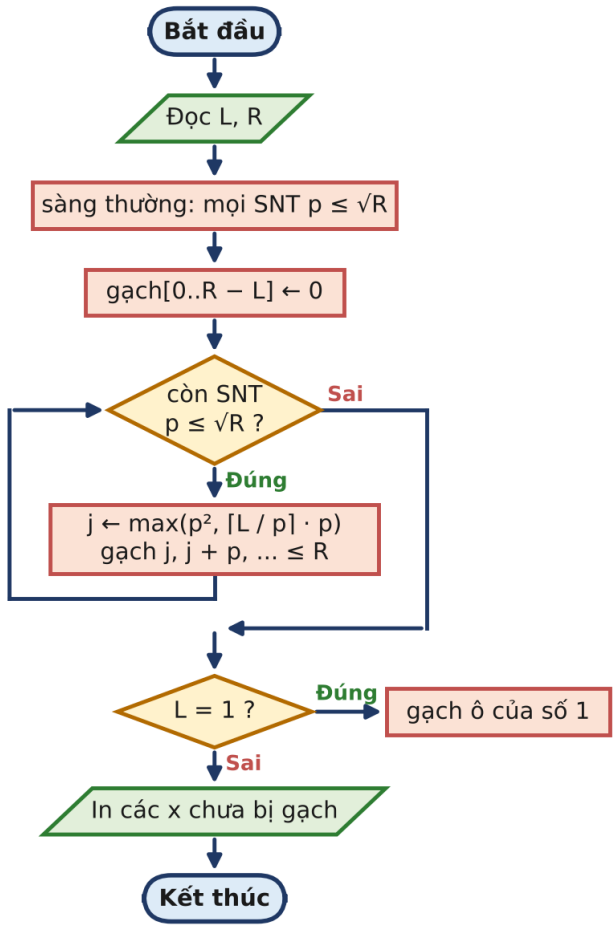
VÌ SAO ĐÚNG?

Không gạch nhầm số nguyên tố: mọi số bị gạch có dạng $j = p \cdot k$ với $j \geq p^2$, tức $k \geq p \geq 2$, nên j là hợp số.

Gạch hết hợp số: hợp số $x \in [L, R]$ có ước nguyên tố nhỏ nhất $p \leq \sqrt{x} \leq \sqrt{R}$, và $x \geq p^2$ (Bài 6). x là bội của p, $x \geq L$ và $x \geq p^2$, nên $x \geq \max(p^2, \lceil L / p \rceil \cdot p)$: lượt p chắc chắn đi qua x.

Số 1: không có p nào gạch được 1, nên khi L = 1 phải gạch riêng ô 0.

Bước 4 Chương trình



Hình 8.4. *Sơ đồ khối của sàng trên đoạn.*

SANGDOAN.CPP: sàng trên đoạn [L, R]

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

int main() {
    if (fopen("SANGDOAN.INP", "r")) {
        freopen("SANGDOAN.INP", "r", stdin);
        freopen("SANGDOAN.OUT", "w", stdout);
    }
    ll L, R;
    cin >> L >> R;
    ll S = sqrtl(R);                          // S = ⌊√R⌋, chỉnh lại cho chắc
    while (S * S > R) S--;
    while ((S + 1) * (S + 1) <= R) S++;
    // Bước 1: sàng Eratosthenes thường cho các số ≤ √R
    vector<char> hop(S + 1, 0);
    vector<ll> ps;
    for (ll i = 2; i <= S; i++)
        if (!hop[i]) {
            ps.push_back(i);
            for (ll j = i * i; j <= S; j += i) hop[j] = 1;
        }
    // Bước 2: gạch bội của từng p trong [L, R]; số x ứng với ô x − L
    vector<char> gach(R - L + 1, 0);
    for (ll p : ps) {
        ll st = max(p * p, (L + p - 1) / p * p);    // bội đầu tiên cần gạch
        for (ll j = st; j <= R; j += p) gach[j - L] = 1;
    }
    if (L == 1) gach[0] = 1;                  // 1 không phải số nguyên tố
    vector<ll> kq;
    for (ll x = L; x <= R; x++)
        if (!gach[x - L]) kq.push_back(x);
    cout << kq.size() << "\n";
    for (size_t i = 0; i < kq.size(); i++)
        cout << kq[i] << (i + 1 < kq.size() ? " " : "\n");
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Sàng tới √R: $O(\sqrt{R} \log\log \sqrt{R})$, khoảng $2{,}1 \cdot 10^6$ phép gạch. Gạch trong đoạn: mỗi p tốn khoảng $(R - L) / p + 1$ phép, tổng $O((R - L) \log\log R + \pi(\sqrt{R}))$; đếm thật trên đoạn $[10^{12} - 10^6, 10^{12}]$ là 2 887 484 phép gạch. Chương trình chạy khoảng 0,01 giây (chưa kể in kết quả), so với vài phút của cách ngây thơ. Bộ nhớ chỉ vài MB.

5. Mở rộng

Mở rộng 1: Khoảng cách giữa hai số nguyên tố liên tiếp

Sàng trên đoạn cho phép "nhìn" vào những vùng số rất lớn. Bảng sau lấy các đoạn dài 10^6 ở ba độ lớn khác nhau:

Đoạn bắt đầu từ L	Số SNT trong [L, L + 10^6]	Khoảng cách lớn nhất	Đạt giữa hai số nguyên tố	ln L
10^6	70 435	132	1 357 201 và 1 357 333	13,8

Đoạn bắt đầu từ L	Số SNT trong [L, L + 10^6]	Khoảng cách lớn nhất	Đạt giữa hai số nguyên tố	ln L
10^9	48 155	176	1 000 097 621 và 1 000 097 797	20,7
10^{12}	36 249	236	1 000 000 494 263 và 1 000 000 494 499	27,6

Khoảng cách trung bình gần đúng bằng ln L (với L = 10^{12}: trung bình 27,59, còn ln L ≈ 27,63), đúng như định lí số nguyên tố ở Bài 6. Khoảng cách lớn nhất cũng chỉ vài trăm. Đây là lí do bài 5.1 (tìm số nguyên tố gần nhất) chạy nhanh. Tuy vậy, khoảng trống có thể **dài tùy ý**: với mọi m, các số (m + 1)! + 2, (m + 1)! + 3, ..., (m + 1)! + (m + 1) là m hợp số liên tiếp, vì (m + 1)! + k chia hết cho k với 2 ≤ k ≤ m + 1.

Mở rộng 2: Đếm số không có ước nguyên tố nhỏ

Chỉ cần thay "các số nguyên tố ≤ √R" bằng "các số nguyên tố ≤ B" và gạch từ ⌈L / p⌉ · p (gạch cả p nếu p nằm trong đoạn), ta đếm được các số trong [L, R] **không có ước nguyên tố nào ≤ B**. Ví dụ trong [1, 100] có 26 số không chia hết cho 2, 3, 5 (kể cả số 1). Trong đoạn [10^{12}, 10^{12} + 10^6], có 120 358 số không có ước nguyên tố nào ≤ 100, rất gần với ước lượng (10^6 + 1) · Π(1 − 1/p) ≈ 120 317 (tích lấy theo 25 số nguyên tố p ≤ 100). Kĩ thuật này dùng để lọc nhanh ứng viên trước khi chạy một phép kiểm tra đắt tiền như Miller–Rabin.

Mở rộng 3: Sàng phân đoạn để tiết kiệm bộ nhớ

Ý tưởng cửa sổ còn dùng được ngược lại: muốn đếm số nguyên tố tới N = 10^9 mà không có 1 GB bộ nhớ, ta chia [2, N] thành các khối dài khoảng 2^{15} đến 2^{18} (vừa với bộ nhớ đệm của CPU) và sàng lần lượt từng khối bằng các số nguyên tố ≤ √N. Bộ nhớ chỉ còn O(√N + kích thước khối), và nhờ dữ liệu nằm gọn trong bộ đệm, chương trình thường còn **nhanh hơn** sàng một mảng lớn.

BẪY THƯỜNG GẶP

- **Gạch nhầm chính p:** bắt đầu từ ⌈L / p⌉ · p mà không lấy max với p^2 thì khi L ≤ p ≤ R, số nguyên tố p bị gạch.
- **Quên số 1:** khi L = 1, không p nào gạch ô của số 1; phải gạch riêng.
- **Dùng chỉ số x thay vì x − L:** mảng chỉ có R − L + 1 phần tử; truy cập `gach[x]` với x ≈ 10^{12} làm chương trình sập.
- **Tràn số và sai số:** p · p, ⌈L / p⌉ · p đều cần `long long`; tính ⌊√R⌋ bằng `sqrt` rồi chỉnh lại bằng phép nhân số nguyên.
- **Công thức trần sai:** `L / p * p` là bội **không vượt quá** L, có thể nhỏ hơn L; bội đầu tiên ≥ L là `(L + p - 1) / p * p`.

GHI NHỚ

- Hợp số x có ước nguyên tố ≤ √x: để sàng [L, R] chỉ cần các số nguyên tố ≤ √R.
- Mỗi p gạch các bội trong đoạn, bắt đầu từ max(p^2, ⌈L / p⌉ · p); số x ứng với ô x − L.
- Khi L = 1 phải loại riêng số 1.
- Độ phức tạp O(√R log log √R + (R − L) log log R), bộ nhớ O(√R + R − L).
- Quanh N, khoảng cách trung bình giữa hai số nguyên tố liên tiếp xấp xỉ ln N.

Luyện tập

Bài	Tên	Gợi ý liên hệ
8.1	**NGUYÊN TỐ TRÊN ĐOẠN**	T ≤ 10 truy vấn [L, R] với R ≤ 10^{12}, R − L ≤ 10^5: sàng tới 10^6 một lần, mỗi truy vấn sàng đoạn riêng.
8.2	**KHOẢNG CÁCH LỚN NHẤT**	Tìm hai số nguyên tố liên tiếp trong [L, R] có hiệu lớn nhất (và nhỏ nhất): sàng đoạn rồi duyệt các số còn lại.
8.3	**SỐ ƯỚC TRÊN ĐOẠN**	Tính số ước của mọi x ∈ [L, R], R ≤ 10^{12}: lưu phần còn lại của từng x, chia dần cho mỗi p ≤ 10^6 trong đoạn; phần còn lại > 1 là một số nguyên tố.

Tự kiểm tra nhanh

1) Với L = 1000 và p = 7, bội đầu tiên của 7 không nhỏ hơn L là số nào? Lượt p = 7 bắt đầu gạch từ đâu? *(Đáp số: ⌈1000 / 7⌉· 7 = 1001; bắt đầu từ max(49, 1001) = 1001)*

2) Sàng đoạn [1, 30] cần những số nguyên tố "thợ gạch" nào? *(Đáp số: các số nguyên tố ≤ ⌊√30⌋ = 5: 2, 3, 5)*

3) Nếu bắt đầu gạch từ ⌈L / p⌉ · p (không lấy max với p^2), chương trình in số lượng bao nhiêu với L = 2, R = 10? Đáp số đúng là bao nhiêu? *(Đáp số: in 2 (chỉ còn 5, 7), đúng phải là 4 (2, 3, 5, 7))*

4) Đoạn [10^{12}, 10^{12} + 100] có bao nhiêu số nguyên tố? *(Đáp số: 4 số (10^{12} + 39, + 61, + 63, + 91))*

CHƯƠNG 3. SỐ HỌC ĐỒNG DƯ

Ở Bài 1 em đã gặp đồng dư: thay mỗi số bằng số dư của nó rồi cộng, trừ, nhân thoải mái. Chương này đi tiếp con đường ấy với bốn câu hỏi lớn. Làm sao tính $a^b \bmod m$ khi số mũ b tới 10^{18}? Làm sao "chia" khi chỉ có số dư? Có bao nhiêu số nguyên tố cùng nhau với n, và điều đó nói gì về lũy thừa? Làm sao tìm lại một số khi chỉ biết số dư của nó khi chia cho vài số khác nhau?

Câu trả lời là bốn công cụ có mặt trong gần như mọi đề thi có dòng chữ "in kết quả theo mô-đun $10^9 + 7$": lũy thừa nhanh, nghịch đảo mô-đun, hàm phi Euler và định lí số dư Trung Hoa. Chúng nối với nhau thành một chuỗi: lũy thừa nhanh cho ta nghịch đảo qua định lí Fermat, định lí Fermat mở rộng thành định lí Euler, còn định lí số dư Trung Hoa ghép mọi thứ lại khi mô-đun là hợp số.

TRONG CHƯƠNG NÀY

Bài 9. Lũy thừa nhanh: $a^b \bmod m$ với b tới 10^{18} chỉ trong khoảng 60 bước

Bài 10. Nghịch đảo mô-đun: phép chia theo mô-đun và định lí Fermat nhỏ

Bài 11. Hàm phi Euler và định lí Euler: đếm số nguyên tố cùng nhau với n, rút gọn số mũ

Bài 12. Định lí số dư Trung Hoa: tìm một số khi biết số dư của nó theo nhiều mô-đun

BÀI 9 Lũy thừa nhanh

Học xong bài này, em sẽ:

- ✓ Tính $a^b \bmod m$ với b tới 10^{18} chỉ bằng khoảng 60 lượt lặp, theo hai cách: **đệ quy** và **vòng lặp trên các bit** của b.
- ✓ Hiểu vì sao thuật toán đúng (bất biến $r \cdot a^b$) và đếm chính xác số phép nhân.
- ✓ Nhân hai số cỡ 10^{18} theo mô-đun mà không tràn số: dùng `__int128` hoặc **nhân Ấn Độ**.
- ✓ Nhận ra ý tưởng bình phương liên tiếp ở những nơi khác: lũy thừa ma trận, tổng cấp số nhân, số mũ cho dưới dạng xâu.

KHỞI ĐỘNG

Chuyện kể rằng người phát minh ra cờ vua xin nhà vua một phần thưởng "nhỏ": ô thứ nhất của bàn cờ đặt 1 hạt thóc, ô thứ hai 2 hạt, ô thứ ba 4 hạt, cứ thế gấp đôi cho tới ô thứ 64. Ô cuối cùng có $2^{63} \approx 9{,}2 \cdot 10^{18}$ hạt, nhiều hơn lượng thóc cả thế giới làm ra trong hàng nghìn năm. Nhà vua đành chịu thua.

Nhưng để biết số hạt ở ô cuối, có cần gấp đôi 63 lần không? Bình phương 2 được 2^2, bình phương tiếp được 2^4, 2^8, 2^{16}, 2^{32}: chỉ 5 phép nhân đã đi được hơn nửa đường. Bài học này biến mẹo nhỏ ấy thành **lũy thừa nhanh**, thuật toán được dùng nhiều nhất của cả chương: tính $a^b \bmod m$ với b tới 10^{18} chỉ trong khoảng 60 bước.

1. Lũy thừa và số dư của lũy thừa

Lũy thừa a^b là tích của b thừa số a: $a^b = a \cdot a \cdot \ldots \cdot a$. Quy ước $a^0 = 1$ (tích của không thừa số nào), kể cả khi $a = 0$. Hai quy tắc em đã học từ lớp 6 sẽ là chìa khóa của cả bài:

$$\mathbf{a^{x+y} = a^x \cdot a^y} \qquad \mathbf{a^{2x} = (a^x)^2}$$

Lũy thừa lớn rất nhanh: 2^{64} đã vượt `unsigned long long`, còn 3^{1000} có tới 478 chữ số. Nhưng theo định lí ở Bài 1, đồng dư giữ nguyên qua phép nhân, nên muốn biết $a^b \bmod m$ ta chỉ cần nhân lần lượt và **lấy dư sau mỗi phép nhân**: mọi số trung gian luôn nhỏ hơn m.

VÍ DỤ: BA CHỮ SỐ CUỐI CỦA 3 MŨ 13

Ba chữ số cuối của một số chính là số dư của nó khi chia cho 1000. Nhân dần với 3 và chỉ giữ ba chữ số cuối: 3, 9, 27, 81, 243, 729, 2187 → 187, 561, 1683 → 683, 2049 → 49, 147, 441, 1323 → **323**.

Vậy $3^{13} = 1\,594\,323$ có ba chữ số cuối là 323, và ta chưa bao giờ phải làm việc với số lớn hơn 3000. Nhưng để ý: tới 3^{13} ta đã tốn 12 phép nhân. Với số mũ 10^{18} thì sao?

2. Bài toán LUYTHUA

BÀI TOÁN: LUYTHUA

Cho ba số nguyên a, b, m. Hãy tính $a^b \bmod m$ (quy ước $a^0 = 1$).

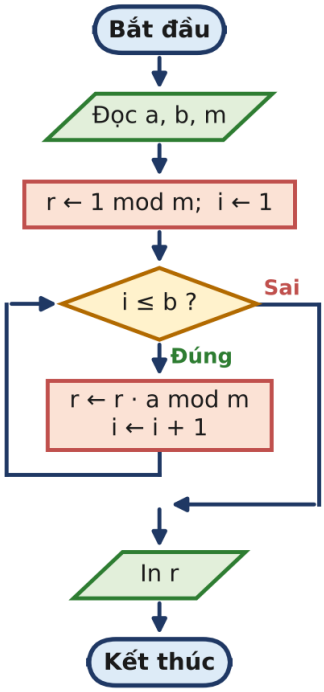
Dữ liệu vào: Dòng đầu chứa a và b, dòng thứ hai chứa m ($0 \le a, b \le 10^{18}$, $1 \le m \le 10^{18}$).

Kết quả: Một số nguyên là $a^b \bmod m$.

LUYTHUA.INP	LUYTHUA.OUT	Giải thích
`3 13` `1000`	`323`	$3^{13} = 1\,594\,323$, ba chữ số cuối là 323.
`2 1000000000000000000` `1000000007`	`719476260`	Số mũ 10^{18}: không thể nhân 10^{18} lần.
`123456789012345678 987654321098765432` `999999999999999989`	`681120143778430554`	Cả ba số cỡ 10^{18}: tích hai số dư lên tới 10^{36}.
`7 0` `1`	`0`	Mọi số chia cho 1 đều dư 0, kể cả $7^0 = 1$.

3. Ý tưởng ngây thơ: Nhân b lần

Làm đúng như định nghĩa: bắt đầu với r = 1 rồi nhân r với a đúng b lần, lấy dư sau mỗi lần nhân như ví dụ ba chữ số cuối ở trên. Hai chi tiết nhỏ cần để ý ngay từ đầu: khởi tạo r = 1 mod m (không phải 1) để đúng cả khi m = 1; và vì r, a đều có thể gần 10^{18} nên tích r · a phải tính trong `__int128` (ta sẽ quay lại chuyện này ở Bước 3).



Hình 9.1. *Sơ đồ khối của cách nhân b lần.*

LUYTHUA.CPP (ngây thơ): nhân b lần

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

int main() {
    if (fopen("LUYTHUA.INP", "r")) {
        freopen("LUYTHUA.INP", "r", stdin);
        freopen("LUYTHUA.OUT", "w", stdout);
    }
    ll a, b, m;
    cin >> a >> b >> m;
    a %= m;
    ll r = 1 % m;
    for (ll i = 1; i <= b; i++)             // nhân b lần
        r = (__int128)r * a % m;            // r = a^i mod m
    cout << r << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Vòng lặp chạy đúng b lần, mỗi lần một phép nhân và một phép lấy dư: O(b). Với b = 10^6 chương trình xong trong nháy mắt; với b = 10^9 đã mất vài giây, vì phép chia trên `__int128` khá chậm.

BẾ TẮC

Với $b = 10^{18}$, kể cả khi mỗi giây làm được 10^8 phép nhân, chương trình cần khoảng 10^{10} giây, tức là **hơn 300 năm**. Cách ngây thơ đi tới a^b bằng những bước nhỏ nhất có thể: mỗi bước số mũ chỉ tăng thêm 1. Có cách nào để số mũ tăng nhanh hơn, như cách bình phương trong câu chuyện bàn cờ?

4. Phá bế tắc: Bình phương liên tiếp

Bước 1 Chia đôi số mũ

Hãy nhìn lại quy tắc $a^{2x} = (a^x)^2$. Muốn có 3^{12}, ta không cần 11 phép nhân: tính 3^6 rồi bình phương lên. Muốn có 3^6, tính 3^3 rồi bình phương. Còn 3^3 có số mũ lẻ: $3^3 = (3^1)^2 \cdot 3$. Tổng quát, với $h = a^{\lfloor b/2 \rfloor}$:

$$\mathbf{a^b = h^2 \text{ nếu } b \text{ chẵn}, \qquad a^b = h^2 \cdot a \text{ nếu } b \text{ lẻ}}$$

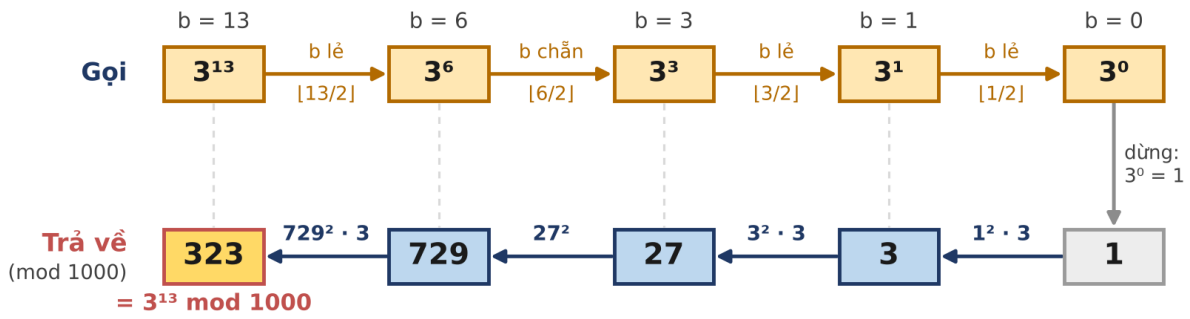
Mỗi lần áp dụng công thức, số mũ giảm **một nửa**. Từ $b = 10^{18}$, sau khoảng 60 lần chia đôi ta về tới $b = 0$, nơi đáp số là $a^0 = 1$. Đó chính là một hàm đệ quy:

LUYTHUA.CPP (đệ quy): chia đôi số mũ

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

// a^b mod m bằng đệ quy: a^b = (a^(b/2))^2, nhân thêm a nếu b lẻ
ll power(ll a, ll b, ll m) {
    if (b == 0) return 1 % m;
    ll h = power(a, b / 2, m);              // h = a^(b/2) mod m
    ll r = (__int128)h * h % m;
    if (b % 2 == 1) r = (__int128)r * (a % m) % m;
    return r;
}

int main() {
    if (fopen("LUYTHUA.INP", "r")) {
        freopen("LUYTHUA.INP", "r", stdin);
        freopen("LUYTHUA.OUT", "w", stdout);
    }
    ll a, b, m;
    cin >> a >> b >> m;
    cout << power(a, b, m) << "\n";
    return 0;
}
```



Hình 9.2. *Tính 3^{13} mod 1000 bằng đệ quy: đi xuống thì chia đôi số mũ, đi lên thì bình phương (và nhân thêm 3 nếu số mũ lẻ).*

Kiểm tra lại hàng dưới của Hình 9.2: $3^2 \cdot 3 = 27$, $27^2 = 729$, còn $729^2 = 531\,441$ nên $729^2 \cdot 3 \equiv 441 \cdot 3 = 1323 \equiv 323 \pmod{1000}$. Khớp với ví dụ 1, mà chỉ dùng 4 lần bình phương và 3 lần nhân thêm 3, thay vì 12 phép nhân.

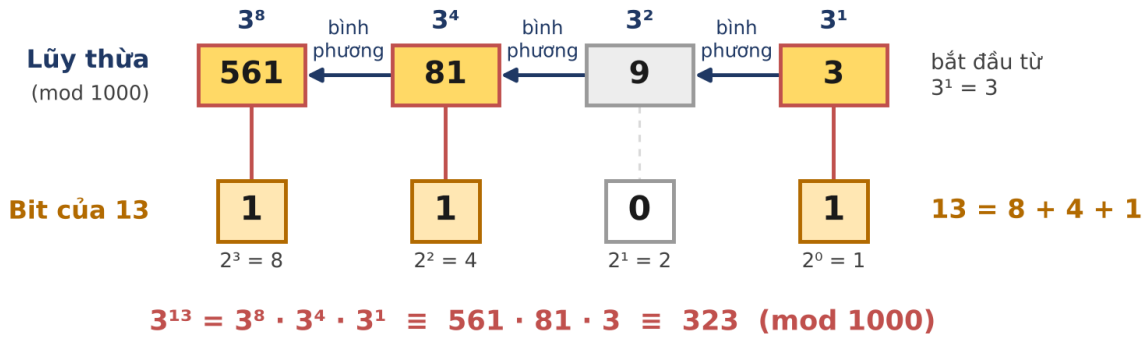
> **LƯU Ý: GỌI ĐỆ QUY MỘT LẦN**
>
> Phải lưu h = power(a, b / 2, m) vào một biến rồi mới tính h · h. Nếu viết `power(a, b / 2, m) * power(a, b / 2, m)`, mỗi lời gọi sinh ra hai lời gọi con, và tổng số lời gọi lại tăng lên cỡ b: công chia đôi đổ sông đổ biển.

Bước 2 Nhìn số mũ ở hệ nhị phân

Đệ quy rất dễ hiểu, nhưng còn một cách nhìn đẹp hơn, cho ta một vòng lặp không cần đệ quy. Viết số mũ ở hệ nhị phân: $13 = 1101_2 = 8 + 4 + 1$. Theo quy tắc $a^{x+y} = a^x \cdot a^y$:

$$3^{13} = 3^{8+4+1} = 3^8 \cdot 3^4 \cdot 3^1$$

Các lũy thừa $3^1, 3^2, 3^4, 3^8, \ldots$ thì rất dễ có: số sau là **bình phương** của số trước. Vậy ta chỉ cần đi qua các bit của b từ phải sang trái, vừa đi vừa bình phương, và nhân vào kết quả những lũy thừa ứng với bit 1.



Hình 9.3. *Số mũ $13 = 1101_2$: lấy các lũy thừa $3^1, 3^4, 3^8$ ứng với bit 1, bỏ qua 3^2 ứng với bit 0.*

THUẬT TOÁN (LŨY THỪA NHANH, BÌNH PHƯƠNG LIÊN TIẾP)

1) $r \leftarrow 1 \bmod m$; $a \leftarrow a \bmod m$.

2) Trong khi $b > 0$: nếu b lẻ (bit cuối bằng 1) thì $r \leftarrow r \cdot a \bmod m$; sau đó $a \leftarrow a \cdot a \bmod m$ và $b \leftarrow \lfloor b / 2 \rfloor$ (bỏ bit cuối).

3) Trả về r.

Bước 3 Chạy tay với a = 3, b = 13, m = 1000

Lượt	b (nhị phân)	Bit cuối	r sau lượt	a sau lượt	b mới
1	13 (1101)	1	$1 \cdot 3 = 3$	$3 \cdot 3 = 9$	6
2	6 (110)	0	3 (giữ nguyên)	$9 \cdot 9 = 81$	3
3	3 (11)	1	$3 \cdot 81 = 243$	$6561 \rightarrow 561$	1
4	1 (1)	1	$243 \cdot 561 = 136\,323 \rightarrow$ **323**	$314\,721 \rightarrow 721$	0

Sau lượt 4, b = 0 và vòng lặp dừng với r = 323. Cột a lần lượt nhận các giá trị 3^2, 3^4, 3^8, 3^{16} (mod 1000); lần bình phương cuối cùng là thừa, nhưng viết như vậy chương trình gọn hơn. Cột r chỉ thay đổi ở các lượt ứng với bit 1.

VÌ SAO ĐÚNG?

Bất biến: gọi A, B là giá trị ban đầu của a, b. Ở đầu mỗi lượt luôn có $r \cdot a^b \equiv A^B \pmod m$.

Ban đầu r = 1, a = A, b = B: đúng. Nếu b chẵn, $b = 2c$, thì $a^b = (a^2)^c$: thay a bằng a^2 và b bằng c không đổi tích $r \cdot a^b$. Nếu b lẻ, $b = 2c + 1$, thì $r \cdot a^b = (r \cdot a) \cdot (a^2)^c$: thay r bằng $r \cdot a$, a bằng a^2, b bằng c cũng không đổi tích. Khi vòng lặp dừng, $b = 0$ nên $r \cdot a^0 = r$ chính là $A^B \bmod m$.

Mỗi lượt b giảm ít nhất một nửa, nên vòng lặp chạy đúng $\lfloor \log_2 B \rfloor + 1$ lượt (số chữ số của B ở hệ nhị phân).

Bước 4 Nhân hai số cỡ 10^{18} mà không tràn

Trong bài này r và a đều có thể gần 10^{18}, nên tích $r \cdot a$ lên tới 10^{36}, vượt xa giới hạn của `long long` (khoảng $9{,}2 \cdot 10^{18}$). Nếu chỉ viết `r * a % m`, kết quả sai âm thầm, đúng như chuyện tràn số ở Bài 1. Có hai cách chữa:

- **Dùng `__int128`**, kiểu số nguyên 128 bit của g++ (chứa tới khoảng $1{,}7 \cdot 10^{38}$): ép **một thừa số** sang `__int128` rồi nhân, lấy dư, kết quả lại vừa `long long`. Gọn và nhanh, nhưng `cin`/`cout` không đọc, in trực tiếp kiểu này được.
- **Nhân Ấn Độ:** tính $a \cdot b \bmod m$ chỉ bằng phép cộng, theo đúng ý tưởng bình phương liên tiếp nhưng thay "nhân" bằng "cộng" và "bình phương" bằng "nhân đôi": $a \cdot 13 = a \cdot 8 + a \cdot 4 + a \cdot 1$. Mọi số trung gian nhỏ hơn $2m \le 2 \cdot 10^{18} < 2^{63}$, vừa `long long`.

LUYTHUA.CPP (nhân Ấn Độ): không cần __int128

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

// Nhân Ấn Độ: a · b mod m chỉ bằng phép cộng và nhân đôi.
// Mọi số trung gian < 2m ≤ 2 · 10^18 < 2^63: không tràn long long.
ll mulmod(ll a, ll b, ll m) {
    ll r = 0;
    a %= m;
    while (b > 0) {
        if (b & 1) r = (r + a) % m;       // cộng a·2^i khi bit i bằng 1
        a = (a + a) % m;                  // a, 2a, 4a, 8a, ...
        b >>= 1;
    }
    return r;
}

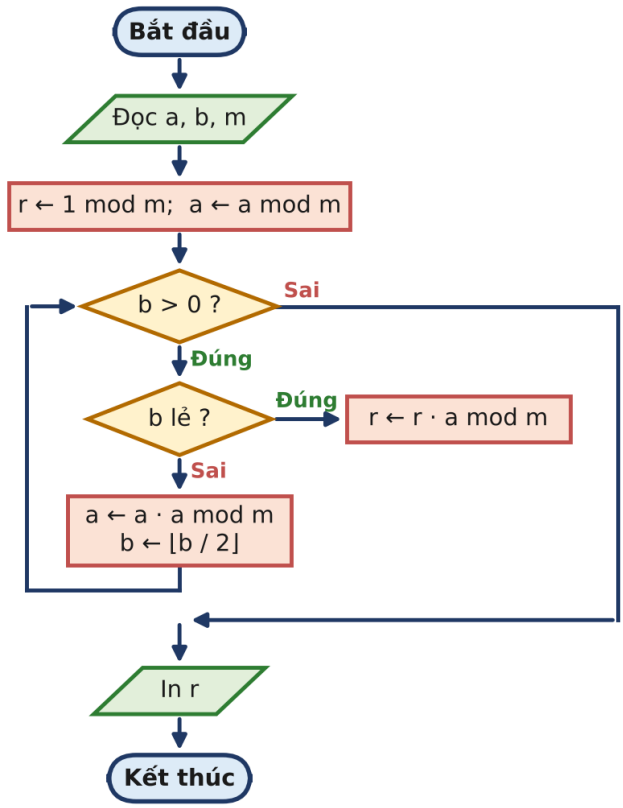
ll power(ll a, ll b, ll m) {
    ll r = 1 % m;
    a %= m;
    while (b > 0) {
        if (b & 1) r = mulmod(r, a, m);
        a = mulmod(a, a, m);
        b >>= 1;
    }
    return r;
}

int main() {
    if (fopen("LUYTHUA.INP", "r")) {
        freopen("LUYTHUA.INP", "r", stdin);
        freopen("LUYTHUA.OUT", "w", stdout);
    }
    ll a, b, m;
    cin >> a >> b >> m;
    cout << power(a, b, m) << "\n";
    return 0;
}
```

Mô-đun m	Tích hai số dư (< m^2)	Cách nhân
$m \le 3 \cdot 10^9$ (ví dụ $10^9 + 7$)	$< 9 \cdot 10^{18} < 2^{63}$	`a * b % m` trong `long long`
$3 \cdot 10^9 < m \le 10^{18}$	tới 10^{36}	`(__int128)a * b % m`, hoặc nhân Ấn Độ (chậm hơn vài chục lần)

Ngưỡng $3 \cdot 10^9$ đến từ $\sqrt{(2^{63})} \approx 3{,}04 \cdot 10^9$. Với mô-đun quen thuộc $10^9 + 7$, cứ nhân bình thường trong `long long`.

Bước 5 Chương trình



Hình 9.4. *Sơ đồ khối của lũy thừa nhanh (vòng lặp trên các bit của b).*

LUYTHUA.CPP: lũy thừa nhanh, nhân bằng __int128

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

// a · b mod m với a, b < m ≤ 10^18: tích tới 10^36 nên nhân bằng __int128
ll mulmod(ll a, ll b, ll m) {
    return (__int128)a * b % m;
}

ll power(ll a, ll b, ll m) {
    ll r = 1 % m;               // m = 1 thì mọi số dư đều bằng 0
    a %= m;
    while (b > 0) {
        if (b & 1) r = mulmod(r, a, m);   // bit cuối của b là 1
        a = mulmod(a, a, m);              // a^1, a^2, a^4, a^8, ...
        b >>= 1;                          // bỏ bit cuối của b
    }
    return r;
}

int main() {
    if (fopen("LUYTHUA.INP", "r")) {
        freopen("LUYTHUA.INP", "r", stdin);
        freopen("LUYTHUA.OUT", "w", stdout);
    }
    ll a, b, m;
    cin >> a >> b >> m;
    cout << power(a, b, m) << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Vòng lặp chạy $\lfloor \log_2 b \rfloor + 1$ lượt, mỗi lượt một phép bình phương và có thể thêm một phép nhân vào r (ở lượt có bit 1). Bảng dưới đếm số phép nhân của hai cách:

b	Ngây thơ	Số lượt (số bit)	Lũy thừa nhanh
10^3	1 000	10	10 + 6 = 16
10^6	1 000 000	20	20 + 7 = 27
10^9	1 000 000 000	30	30 + 13 = 43
10^{18}	10^{18}	60	60 + 24 = 84

Cột cuối là số lượt (số lần bình phương) cộng số bit 1 của b (số lần nhân vào r). Từ hơn 300 năm xuống còn 84 phép nhân!

Độ phức tạp: O(log b) thời gian, O(1) bộ nhớ.

5. Mở rộng

Mở rộng 1: Lũy thừa ma trận

Thuật toán bình phương liên tiếp không dùng gì đặc biệt của số nguyên, ngoài tính chất **kết hợp** của phép nhân: $(x \cdot y) \cdot z = x \cdot (y \cdot z)$. Vì vậy nó dùng được cho mọi phép "nhân" có tính kết hợp, mà quan trọng nhất là **nhân ma trận**. Dãy Fibonacci $F_0 = 0$, $F_1 = 1$, $F_{n+1} = F_n + F_{n-1}$ thỏa mãn:

$$\mathbf{M = [[1, 1], [1, 0]],} \qquad \mathbf{M^n = [[F_{n+1}, F_n], [F_n, F_{n-1}]]}$$

Thay số a bằng ma trận M, thay phép nhân số bằng phép nhân hai ma trận 2×2 (8 phép nhân số), ta tính được $F_n \bmod m$ với $n = 10^{18}$ chỉ sau khoảng 60 lần bình phương ma trận. Chẳng hạn $F_n \bmod (10^9 + 7)$ với $n = 10^{18}$ bằng 209 783 453. Bài 15 sẽ học kĩ kĩ thuật này.

Mở rộng 2: Tổng cấp số nhân theo mô-đun bất kì

Cần $S(n) = 1 + a + a^2 + \ldots + a^{n-1} \bmod m$. Công thức $S(n) = (a^n - 1) / (a - 1)$ đòi hỏi phép chia, mà chia theo mô-đun chỉ làm được trong một số trường hợp (Bài 10). Ta lại chia đôi:

$$\mathbf{S(2k) = S(k) \cdot (1 + a^k)} \qquad \mathbf{S(n) = 1 + a \cdot S(n - 1)}$$

Đẳng thức đầu đúng vì nửa sau $a^k + \ldots + a^{2k-1}$ chính là nửa đầu nhân với a^k. Hàm dưới đây trả về đồng thời $S = S(n)$ và $P = a^n$ (cần cho đẳng thức đầu), mỗi lần số lẻ thì bớt 1, số chẵn thì chia đôi, nên chỉ tốn $O(\log n)$. Ví dụ $1 + 2 + \ldots + 2^9 = 1023$, theo mô-đun 1000 bằng 23.

Tổng cấp số nhân bằng chia đôi

```
// S = 1 + a + ... + a^(n-1) mod m, đồng thời P = a^n mod m (a < m)
void geo(ll a, ll n, ll m, ll &S, ll &P) {
    if (n == 0) { S = 0; P = 1 % m; return; }
    if (n % 2 == 1) {                    // S(n) = 1 + a·S(n-1)
        geo(a, n - 1, m, S, P);
        S = (1 + (__int128)a * S) % m;
        P = (__int128)P * a % m;
        return;
    }
    geo(a, n / 2, m, S, P);              // S(2k) = S(k)·(1 + a^k)
    S = (__int128)S * ((1 + P) % m) % m;
    P = (__int128)P * P % m;
}
```

Mở rộng 3: Số mũ cho dưới dạng xâu rất dài

Nếu số mũ b có tới 10^5 chữ số, ta không đọc được b vào một biến. Hãy dùng sơ đồ Horner của Bài 1, nhưng trên số mũ: nếu x là phần đầu đã đọc của b và d là chữ số tiếp theo thì phần đầu mới là $10x + d$, và $a^{10x+d} = (a^x)^{10} \cdot a^d$. Vậy chỉ cần $r \leftarrow r^{10} \cdot a^d \bmod m$ với mỗi chữ số d:

Lũy thừa với số mũ là xâu

```
ll r = 1 % m;                        // r = a^(phần đầu của b) mod m
for (char c : s)                     // s là xâu chữ số của b
    r = mulmod(power(r, 10, m), power(a, c - '0', m), m);
```

Ví dụ $2^{12345} \bmod 1000$: r lần lượt bằng 2, 96, 608, 184, **832** (ứng với số mũ 1, 12, 123, 1234, 12345).

BẪY THƯỜNG GẶP

- **Khởi tạo r = 1:** với m = 1 và b = 0, vòng lặp không chạy và chương trình in 1, trong khi đáp số là 0. Viết `r = 1 % m`.
- **Ép kiểu muộn:** `(__int128)(a * b) % m` vẫn tràn, vì a · b đã được tính trong `long long` trước khi ép. Phải ép một thừa số: `(__int128)a * b % m`.
- **Gọi đệ quy hai lần:** `power(a, b/2, m) * power(a, b/2, m)` làm thuật toán chậm trở lại O(b). Lưu kết quả vào biến rồi bình phương.
- **Dùng hàm `pow` của `<cmath>`:** nó tính bằng số thực `double`, chỉ chính xác khoảng 15–16 chữ số và không lấy dư được. Luôn tự viết hàm lũy thừa nhanh.
- **Khai báo b là `int`:** số mũ tới 10^{18} cần `long long`; đọc vào `int` sẽ sai ngay từ đầu.

GHI NHỚ

- $a^b = (a^{\lfloor b/2 \rfloor})^2$ (nhân thêm a nếu b lẻ): mỗi bước số mũ giảm một nửa, chỉ O(log b) phép nhân.
- Vòng lặp: đi qua các bit của b từ phải sang trái; bit 1 thì nhân a vào r; luôn bình phương a; bất biến $r \cdot a^b$ không đổi.
- Lấy dư sau mỗi phép nhân; khởi tạo r = 1 mod m.
- $m \le 3 \cdot 10^9$: nhân trong `long long` là đủ; m tới 10^{18}: nhân bằng `__int128` hoặc nhân Ấn Độ.
- Bình phương liên tiếp dùng được cho mọi phép nhân có tính kết hợp, ví dụ nhân ma trận.

Luyện tập

Bài	Tên	Gợi ý liên hệ
9.1	**LŨY THỪA**	Q ≤ 10^5 truy vấn a^b mod (10^9 + 7) với a, b ≤ 10^{18}: lũy thừa nhanh, nhân trong `long long`.
9.2	**SỐ MŨ KHỔNG LỒ**	Số mũ cho dưới dạng xâu 10^5 chữ số: sơ đồ Horner trên số mũ (Mở rộng 3), hoặc rút gọn số mũ bằng định lí Fermat, Euler (Bài 10, 11).
9.3	**TỔNG LŨY THỪA**	1 + a + a^2 + ... + a^n mod m với n, m ≤ 10^{18}, m bất kì: chia đôi như Mở rộng 2, nhân bằng `__int128`.

Tự kiểm tra nhanh

1) Viết 100 ở hệ nhị phân. Khi tính 3^{100} bằng vòng lặp lũy thừa nhanh, vòng lặp chạy bao nhiêu lượt và nhân vào r bao nhiêu lần? *(Đáp số: $100 = 1100100_2$; 7 lượt, nhân vào r 3 lần vì $3^{100} = 3^{64} \cdot 3^{32} \cdot 3^4$)*

2) Tính 2^{100} mod 13 bằng tay theo cách bình phương liên tiếp. *(Đáp số: 3)*

3) Tìm ba chữ số cuối của 7^{222}. *(Đáp số: 7^{222} mod 1000 = 49, nên ba chữ số cuối là 049)*

4) Với a = 5, b = 0, m = 1, chương trình LUYTHUA in ra gì? Nếu khởi tạo r = 1 (thay cho r = 1 % m) thì sao? *(Đáp số: in 0; nếu viết r = 1 thì in sai 1, vì vòng lặp không chạy lượt nào)*

BÀI 10 Nghịch đảo mô-đun

Học xong bài này, em sẽ:

- ✓ Hiểu "phép chia" theo mô-đun là phép nhân với **nghịch đảo**, và biết khi nào nghịch đảo tồn tại.
- ✓ Chứng minh **định lí Fermat nhỏ** bằng hình ảnh hoán vị các số dư.
- ✓ Tính nghịch đảo theo mô-đun nguyên tố bằng lũy thừa nhanh, theo mô-đun bất kì bằng Euclid mở rộng.
- ✓ Tính nghịch đảo của mọi số 1..n trong O(n) và giải phương trình đồng dư bậc nhất $a \cdot x \equiv c \pmod m$.

KHỞI ĐỘNG

Một đề thi hỏi: "Gieo hai con xúc xắc. Tính xác suất để tổng hai mặt bằng 7, in kết quả theo mô-đun $10^9 + 7$." Có 6 trong 36 khả năng cho tổng 7, nên đáp số là 1/6: một phân số! Vậy "1/6 theo mô-đun $10^9 + 7$" nghĩa là gì?

Đó là số x thỏa $6 \cdot x \equiv 1 \pmod{10^9 + 7}$, giống hệt cách 1/6 là số mà nhân với 6 thì được 1. Số ấy là 166 666 668, vì $6 \cdot 166\,666\,668 = 1\,000\,000\,008$. Bài học này cho em biết vì sao số như vậy luôn tồn tại khi mô-đun là số nguyên tố, và tìm nó chỉ bằng một lời gọi hàm lũy thừa nhanh của Bài 9.

1. Phép chia trong đồng dư

Ở Bài 1 ta đã thấy: đồng dư giữ nguyên qua cộng, trừ, nhân, nhưng **không** cho phép chia tùy tiện. Vậy làm sao "chia" khi chỉ làm việc với số dư? Hãy nhớ lại: trong số thực, chia cho 4 chính là **nhân với 1/4**, số mà nhân với 4 thì được 1. Ta mượn đúng ý này.

ĐỊNH NGHĨA (NGHỊCH ĐẢO MÔ-ĐUN)

Cho số nguyên dương m. Số nguyên x gọi là **nghịch đảo** của b theo mô-đun m nếu $b \cdot x \equiv 1 \pmod m$. Kí hiệu $x = b^{-1} \bmod m$. Khi đó "chia a cho b theo mô-đun m" được hiểu là tính $a \cdot b^{-1} \bmod m$.

Ví dụ, $3 \cdot 5 = 15 \equiv 1 \pmod 7$ nên $3^{-1} \equiv 5 \pmod 7$. Phép chia mới này khớp với phép chia quen thuộc khi chia hết: $12 / 4 = 3$; theo mô-đun 7 thì $12 \equiv 5$ và $4^{-1} \equiv 2$ (vì $4 \cdot 2 = 8 \equiv 1$), nên $12 / 4 \equiv 5 \cdot 2 = 10 \equiv 3$. Đúng là 3.

Số nào có nghịch đảo? Hãy nhìn bảng nhân: b có nghịch đảo khi và chỉ khi hàng b của bảng có chứa số 1.

Nhân theo mô-đun 7

·	1	2	3	4	5	6
1	**1**	2	3	4	5	6
2	2	4	6	**1**	3	5
3	3	6	2	5	**1**	4
4	4	**1**	5	2	6	3
5	5	3	**1**	6	4	2
6	6	5	4	3	2	**1**

Mỗi hàng có đúng một số 1:
mọi b = 1..6 đều có nghịch đảo

Nhân theo mô-đun 6

·	1	2	3	4	5
1	**1**	2	3	4	5
2	2	4	0	2	4
3	3	0	3	0	3
4	4	2	0	4	2
5	5	4	3	2	**1**

Hàng 2, 3, 4
không có số 1:
2, 3, 4 không có
nghịch đảo
theo mô-đun 6

Hình 10.1. *Bảng nhân theo mô-đun 7 và theo mô-đun 6. Ô vàng: $b \cdot x \equiv 1$, tức x là nghịch đảo của b.*

Theo mô-đun 7, mọi hàng đều có đúng một số 1. Theo mô-đun 6 thì các hàng 2, 3, 4 không có: chẳng hạn $2 \cdot x$ luôn chẵn, mà số chẵn chia 6 không bao giờ dư 1. Quy luật chung như sau.

ĐỊNH LÍ (ĐIỀU KIỆN TỒN TẠI NGHỊCH ĐẢO)

b có nghịch đảo theo mô-đun m khi và chỉ khi gcd(b, m) = 1. Khi đó nghịch đảo là **duy nhất** theo mô-đun m.

VÌ SAO ĐÚNG?

Điều kiện cần: nếu $b \cdot x \equiv 1 \pmod m$ thì $b \cdot x - m \cdot y = 1$ với một số nguyên y nào đó. Gọi g = gcd(b, m): g chia hết vế trái nên g | 1, tức g = 1.

Điều kiện đủ: nếu gcd(b, m) = 1 thì theo định lí Bézout (Bài 4) có các số nguyên x, y với $b \cdot x + m \cdot y = 1$. Lấy đồng dư theo m được $b \cdot x \equiv 1$.

Duy nhất: nếu $b \cdot x \equiv 1$ và $b \cdot x' \equiv 1$ thì $x \equiv x \cdot (b \cdot x') = (x \cdot b) \cdot x' \equiv x' \pmod m$.

Đặc biệt, theo mô-đun **nguyên tố** p, mọi số b không chia hết cho p đều có nghịch đảo: đó là lí do các đề thi thích mô-đun $10^9 + 7$.

2. Bài toán CHIAMOD

BÀI TOÁN: CHIAMOD

Cho số nguyên tố p và Q truy vấn, mỗi truy vấn gồm hai số nguyên a, b với b không chia hết cho p. Với mỗi truy vấn, hãy tính (a / b) mod p, tức là $a \cdot b^{-1} \bmod p$.

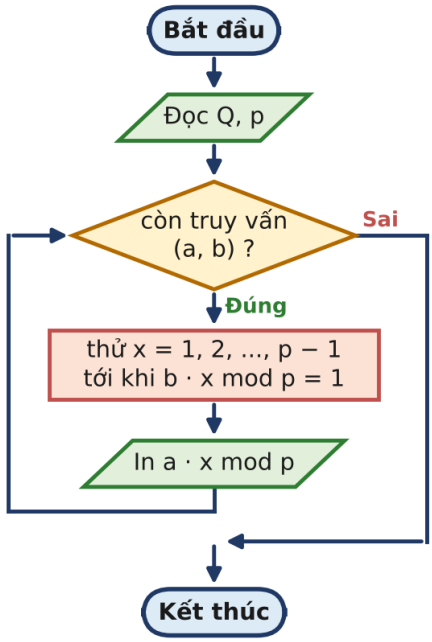
Dữ liệu vào: Dòng đầu chứa Q và p ($1 \le Q \le 10^5$, $2 \le p \le 2 \cdot 10^9$, p nguyên tố). Mỗi dòng trong Q dòng tiếp theo chứa a và b ($0 \le a, b \le 10^{18}$, b không chia hết cho p).

Kết quả: Q dòng, mỗi dòng là đáp số của một truy vấn.

CHIAMOD.INP	CHIAMOD.OUT	Giải thích
3 7 1 3 2 3 6 4	5 3 5	$3^{-1} \equiv 5$ vì $3 \cdot 5 = 15 \equiv 1$; $2/3 \equiv 2 \cdot 5 = 10 \equiv 3$; $6/4 \equiv 6 \cdot 2 = 12 \equiv 5 \pmod 7$.
3 1000000007 1 2 10 4 100000000000000000 999999999999	500000004 500000006 81416941	$1/2 \equiv 500\,000\,004$ vì $2 \cdot 500\,000\,004 = 10^9 + 8 \equiv 1$. Còn $10/4 = 5/2$ ứng với 500 000 006.
1 2 5 3	1	Theo mô-đun 2, mọi số lẻ đều đồng dư với 1.

3. Ý tưởng ngây thơ: Thử mọi x

Nghịch đảo của b (đã lấy dư cho p) là một số trong khoảng 1..p − 1, nên cứ thử lần lượt x = 1, 2, ... cho tới khi $b \cdot x \bmod p = 1$. Định lí ở trên bảo đảm vòng lặp luôn tìm thấy.



Hình 10.2. *Sơ đồ khối của cách thử mọi x.*

CHIAMOD.CPP (ngây thơ): thử x = 1, 2, ..., p − 1

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

int main() {
    if (fopen("CHIAMOD.INP", "r")) {
        freopen("CHIAMOD.INP", "r", stdin);
        freopen("CHIAMOD.OUT", "w", stdout);
    }
    int q;
    ll p;
    scanf("%d %lld", &q, &p);
    while (q--) {
        ll a, b;
        scanf("%lld %lld", &a, &b);
        b %= p;
        ll inv = 0;
        for (ll x = 1; x < p; x++)          // thử mọi x = 1, 2, ..., p - 1
            if (b * x % p == 1) { inv = x; break; }
        printf("%lld\n", a % p * inv % p);
    }
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Mỗi truy vấn thử tới $p - 1$ giá trị x: tổng cộng $O(Q \cdot p)$. Với $Q = 10^5$ và $p \approx 10^9$, đó là khoảng 10^{14} phép thử: ở tốc độ 10^8 phép tính mỗi giây, mất **hơn mười ngày**.

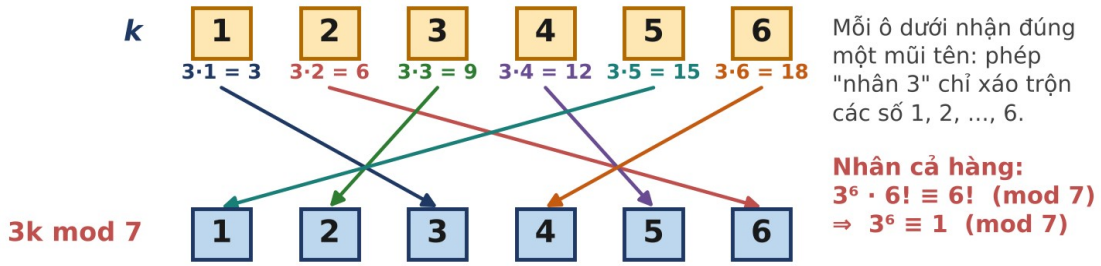
BẾ TẮC

Thử từng x là tìm kiếm mù: nó không dùng chút cấu trúc nào của phép nhân theo mô-đun nguyên tố. Liệu có một **công thức** cho thẳng b^{-1}, chỉ gồm những phép tính mà ta đã biết làm nhanh, chẳng hạn lũy thừa?

4. Phá bế tắc: Định lí Fermat nhỏ

Bước 1 Một hoán vị bí ẩn

Lấy p = 7, nhân mọi số 1, 2, ..., 6 với 3 rồi lấy dư: được 3, 6, 2, 5, 1, 4. Lại đúng là các số 1, 2, ..., 6, chỉ bị xáo trộn thứ tự! Thử nhân với 2: được 2, 4, 6, 1, 3, 5. Cũng thế. (Đó chính là các hàng của bảng nhân mô-đun 7 ở Hình 10.1.)



Hình 10.3. *Nhân các số 1..6 với 3 theo mô-đun 7 chỉ đổi chỗ chúng cho nhau.*

HÃY HÌNH DUNG

Sáu bạn ngồi trên sáu chiếc ghế đánh số 1..6. Lệnh "nhân 3" bảo bạn ở ghế k chuyển sang ghế 3k mod 7. Không có hai bạn tranh cùng một ghế, cũng không có ghế nào bỏ trống: mọi người chỉ đổi chỗ cho nhau. Vì thế **tích các số ghế** trước và sau khi đổi chỗ bằng nhau.

Vì sao không có hai bạn tranh cùng ghế? Nếu $3k \equiv 3k' \pmod 7$ thì nhân hai vế với 3^{-1} (tồn tại vì gcd(3, 7) = 1) được $k \equiv k'$. Bây giờ nhân tất cả các số ở mỗi hàng của Hình 10.3:

$$\mathbf{(3 \cdot 1) \cdot (3 \cdot 2) \cdot \ldots \cdot (3 \cdot 6) \equiv 1 \cdot 2 \cdot \ldots \cdot 6 \pmod 7 \quad \Rightarrow \quad 3^6 \cdot 6! \equiv 6! \pmod 7}$$

Số 6! = 720 không chia hết cho 7 nên có nghịch đảo; nhân hai vế với nó được $3^6 \equiv 1 \pmod 7$. Kiểm tra: $3^6 = 729 = 7 \cdot 104 + 1$. Đúng!

Bước 2 Định lí Fermat nhỏ

ĐỊNH LÍ (FERMAT NHỎ)

Nếu p là số nguyên tố và b không chia hết cho p thì $b^{p-1} \equiv 1 \pmod p$.

VÌ SAO ĐÚNG?

Lặp lại lập luận trên, thay 7 bằng p và 3 bằng b. Các số $b \cdot 1, b \cdot 2, \ldots, b \cdot (p-1)$ đều không chia hết cho p (tích của hai số không chia hết cho số nguyên tố p thì không chia hết cho p) và đôi một khác nhau theo mô-đun p (nếu $b \cdot k \equiv b \cdot k' \pmod p$ thì nhân với b^{-1} được $k \equiv k'$). Vậy số dư của chúng là một hoán vị của $1, 2, \ldots, p-1$.

Nhân tất cả lại: $b^{p-1} \cdot (p-1)! \equiv (p-1)! \pmod p$. Vì $(p-1)!$ không chia hết cho p nên nó có nghịch đảo; nhân hai vế với nghịch đảo đó được $b^{p-1} \equiv 1 \pmod p$.

Bước 3 Nghịch đảo bằng lũy thừa nhanh

Viết $b^{p-1} = b \cdot b^{p-2}$. Định lí Fermat nói $b \cdot b^{p-2} \equiv 1$, tức là b^{p-2} chính là nghịch đảo của b:

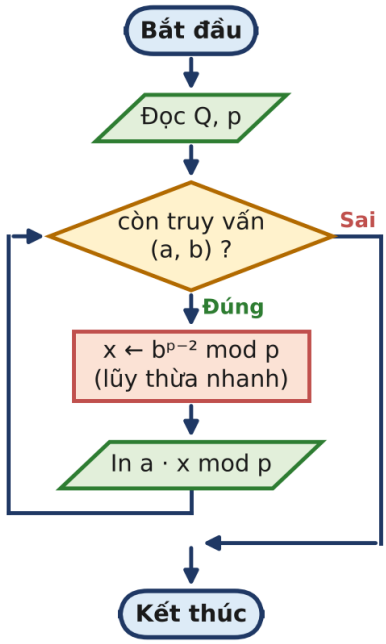
$$\mathbf{b^{-1} \equiv b^{p-2} \pmod p}$$

Một lũy thừa! Và Bài 9 đã dạy ta tính lũy thừa trong $O(\log p)$ phép nhân. Chạy tay truy vấn đầu của ví dụ 1: $3^{-1} \equiv 3^5 \pmod 7$, với $5 = 101_2$:

Lượt	b (nhị phân)	Bit cuối	r sau lượt	a sau lượt
1	5 (101)	1	$1 \cdot 3 = 3$	$3 \cdot 3 = 9 \equiv 2$
2	2 (10)	0	3 (giữ nguyên)	$2 \cdot 2 = 4$
3	1 (1)	1	$3 \cdot 4 = 12 \equiv \mathbf{5}$	$4 \cdot 4 = 16 \equiv 2$

Được $3^{-1} \equiv 5 \pmod 7$, khớp với bảng nhân. Truy vấn thứ hai: $2/3 \equiv 2 \cdot 5 = 10 \equiv 3$. Truy vấn thứ ba: $4^{-1} \equiv 4^5 = 1024 \equiv 2 \pmod 7$, nên $6/4 \equiv 6 \cdot 2 = 12 \equiv 5$.

Bước 4 Chương trình



Hình 10.4. *Sơ đồ khối của cách dùng định lí Fermat.*

CHIAMOD.CPP: nghịch đảo bằng định lí Fermat và lũy thừa nhanh

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

ll power(ll a, ll b, ll m) {        // lũy thừa nhanh (Bài 9)
    ll r = 1 % m;
    a %= m;
    while (b > 0) {
        if (b & 1) r = r * a % m;   // p ≤ 2 · 10^9: tích < 4 · 10^18
        a = a * a % m;
        b >>= 1;
    }
    return r;
}

int main() {
    if (fopen("CHIAMOD.INP", "r")) {
        freopen("CHIAMOD.INP", "r", stdin);
        freopen("CHIAMOD.OUT", "w", stdout);
    }
    int q;
    ll p;
    scanf("%d %lld", &q, &p);
    while (q--) {
        ll a, b;
        scanf("%lld %lld", &a, &b);
        ll inv = power(b % p, p - 2, p);    // Fermat: b^(p-2) = 1/b
        printf("%lld\n", a % p * inv % p);
    }
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Mỗi truy vấn là một lần lũy thừa nhanh với số mũ $p - 2 < 2^{31}$: không quá 31 lượt, khoảng 60 phép nhân. Cả $Q = 10^5$ truy vấn tốn chừng $6 \cdot 10^6$ phép nhân, chạy trong vài phần trăm giây. Vì $p \le 2 \cdot 10^9$ nên tích hai số dư nhỏ hơn $4 \cdot 10^{18}$, vừa `long long`: không cần `__int128`.

Độ phức tạp: $O(Q \cdot \log p)$.

5. Khi mô-đun không nguyên tố: Euclid mở rộng

Định lí Fermat cần mô-đun nguyên tố. Với mô-đun m bất kì (chẳng hạn m = 60), ta quay về chứng minh của định lí tồn tại: nghịch đảo của b chính là hệ số x trong đẳng thức Bézout $b \cdot x + m \cdot y = 1$, và thuật toán **Euclid mở rộng** (Bài 4) tìm ra nó trong $O(\log m)$ bước.

Chạy tay với b = 17, m = 60. Mỗi hàng gọi đệ quy sang hàng dưới với cặp (b, a mod b); tới hàng cuối (b = 0) thì x = 1, y = 0; rồi các hệ số được tính ngược từ dưới lên theo $x = y'$, $y = x' - \lfloor a/b \rfloor \cdot y'$ (x', y' là hệ số của hàng ngay dưới):

a	b	⌊a / b⌋	x	y	Kiểm tra a · x + b · y
17	60	0	**−7**	2	17 · (−7) + 60 · 2 = 1
60	17	3	2	−7	60 · 2 + 17 · (−7) = 1

a	b	⌊a / b⌋	x	y	Kiểm tra a · x + b · y
17	9	1	−1	2	17 · (−1) + 9 · 2 = 1
9	8	1	1	−1	9 · 1 + 8 · (−1) = 1
8	1	8	0	1	8 · 0 + 1 · 1 = 1
1	0	–	1	0	1 · 1 + 0 · 0 = 1

Vậy 17 · (−7) ≡ 1 (mod 60): nghịch đảo là −7 + 60 = **53**. Kiểm tra: 17 · 53 = 901 = 15 · 60 + 1. Nếu gcd(b, m) ≠ 1, hàm trả về g ≠ 1 và ta kết luận không có nghịch đảo.

CHIAMOD.CPP (Euclid mở rộng): dùng được cho mọi mô-đun m

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

// Euclid mở rộng (Bài 4): trả về g = gcd(a, b) và x, y với a·x + b·y = g
ll extgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1;
    ll g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

// nghịch đảo của b theo mô-đun m bất kì; -1 nếu không tồn tại
ll inverse(ll b, ll m) {
    ll x, y;
    ll g = extgcd(b % m, m, x, y);
    if (g != 1) return -1;              // gcd(b, m) > 1: không có
    return (x % m + m) % m;             // x có thể âm
}

int main() {
    if (fopen("CHIAMOD.INP", "r")) {
        freopen("CHIAMOD.INP", "r", stdin);
        freopen("CHIAMOD.OUT", "w", stdout);
    }
    int q;
    ll m;
    scanf("%d %lld", &q, &m);
    while (q--) {
        ll a, b;
        scanf("%lld %lld", &a, &b);
        ll inv = inverse(b, m);
        if (inv < 0) printf("-1\n");
        else printf("%lld\n", a % m * inv % m);
    }
    return 0;
}
```

SO SÁNH HAI CÁCH TÍNH NGHỊCH ĐẢO

	Fermat: $b^{p-2} \bmod p$	Euclid mở rộng
Mô-đun	chỉ số nguyên tố p	mọi m, miễn gcd(b, m) = 1
Độ phức tạp	O(log p)	O(log m)
Cài đặt	dùng lại hàm lũy thừa nhanh	cần thêm hàm Euclid mở rộng
Khi không có nghịch đảo	in kết quả sai, không báo	phát hiện được (g ≠ 1)

6. Mở rộng

Mở rộng 1: Nghịch đảo của mọi số 1..n trong O(n)

Khi cần nghịch đảo của rất nhiều số (chẳng hạn 1, 2, ..., 10^6 trong các bài tổ hợp ở Bài 13), gọi lũy thừa nhanh cho từng số tốn O(n log p). Có một công thức truy hồi đẹp hơn. Chia p cho i: $p = q \cdot i + r$ với $q = \lfloor p / i \rfloor$ và $r = p \bmod i < i$. Lấy đồng dư theo p: $q \cdot i + r \equiv 0$, tức $r \equiv -q \cdot i$. Nhân hai vế với $i^{-1} \cdot r^{-1}$:

$$\mathbf{i^{-1} \equiv -q \cdot r^{-1} \equiv (p - \lfloor p / i \rfloor) \cdot (p \bmod i)^{-1} \pmod p}$$

Vì r < i, nghịch đảo của r đã được tính trước đó. Bảng sau chạy công thức với p = 11:

i	2	3	4	5	6	7	8	9	10
$q = \lfloor 11 / i \rfloor$	5	3	2	2	1	1	1	1	1
$r = 11 \bmod i$	1	2	3	1	5	4	3	2	1
inv[r]	1	6	4	1	9	3	4	6	1
$\text{inv}[i] = (11 - q) \cdot \text{inv}[r]$	**6**	**4**	**3**	**9**	**2**	**8**	**7**	**5**	**10**

Ví dụ cột i = 7: $(11 - 1) \cdot \text{inv}[4] = 10 \cdot 3 = 30 \equiv 8$, và đúng là $7 \cdot 8 = 56 = 5 \cdot 11 + 1$.

NGHICHDAO.CPP: nghịch đảo của 1..n theo mô-đun nguyên tố p > n

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

int main() {
    if (fopen("NGHICHDAO.INP", "r")) {
        freopen("NGHICHDAO.INP", "r", stdin);
        freopen("NGHICHDAO.OUT", "w", stdout);
    }
    int n;
    ll p;                                   // p nguyên tố, p > n
    scanf("%d %lld", &n, &p);
    vector<ll> inv(n + 1);
    inv[1] = 1;
    for (int i = 2; i <= n; i++)            // p = (p/i)·i + p%i, với p%i < i
        inv[i] = (p - p / i) * inv[p % i] % p;
    for (int i = 1; i <= n; i++) printf("%lld ", inv[i]);
    printf("\n");
    return 0;
}
```

Mở rộng 2: Nghịch đảo của giai thừa

Bài 13 cần các giá trị 1 / k! mod p với mọi k ≤ n. Chỉ cần **một** lần lũy thừa nhanh: tính `invf[n] = power(fact[n], p - 2, p)`, rồi đi ngược `invf[k - 1] = invf[k] * k % p`, vì 1 / (k − 1)! = k / k!. Tổng cộng O(n + log p).

Mở rộng 3: Phương trình đồng dư bậc nhất a · x ≡ c (mod m)

Đặt g = gcd(a, m). Phương trình có nghiệm khi và chỉ khi g | c (lập luận giống hệt định lí tồn tại nghịch đảo). Khi đó chia cả a, c, m cho g được a' · x ≡ c' (mod m') với gcd(a', m') = 1, có nghiệm duy nhất $x_0 \equiv c' \cdot a'^{-1} \pmod{m'}$. Theo mô-đun m có đúng g nghiệm: x_0, $x_0 + m'$, ..., $x_0 + (g - 1) \cdot m'$.

Ví dụ 6x ≡ 4 (mod 10): g = 2 chia hết 4; chia đôi được 3x ≡ 2 (mod 5); $3^{-1} \equiv 2$ nên x ≡ 4 (mod 5). Theo mô-đun 10 có hai nghiệm x = 4 và x = 9 (6 · 4 = 24 ≡ 4, 6 · 9 = 54 ≡ 4). Còn 6x ≡ 3 (mod 10) vô nghiệm vì 2 ∤ 3. Kĩ thuật này là chìa khóa của Bài 12.

BẪY THƯỜNG GẶP

- **Chia thẳng bằng `/`:** `(a / b) % p` với phép chia nguyên của C++ chỉ đúng khi b | a; chẳng hạn 7 / 2 cho 3, còn $7 \cdot 2^{-1} \bmod 11 = 9$.
- **Dùng Fermat khi mô-đun không nguyên tố:** $3^{10-2} = 3^8 \equiv 1 \pmod{10}$, nhưng nghịch đảo thật của 3 theo mô-đun 10 là 7. Mô-đun hợp số phải dùng Euclid mở rộng.
- **Quên điều kiện gcd(b, m) = 1:** nếu p | b thì $b^{p-2} \equiv 0$ và chương trình in 0 mà không báo lỗi. Với Euclid mở rộng, phải kiểm tra g = 1.
- **Hệ số x âm:** Euclid mở rộng có thể trả về x âm (như −7 ở trên). Đưa về [0, m) bằng `(x % m + m) % m`.
- **Quên lấy dư b trước khi nhân:** b tới 10^{18} thì `b * b` tràn ngay lượt đầu. Hàm lũy thừa phải có dòng `a %= m`.

GHI NHỚ

- $b^{-1} \bmod m$ là số x với $b \cdot x \equiv 1 \pmod{m}$; tồn tại và duy nhất khi và chỉ khi gcd(b, m) = 1.
- "Chia cho b" theo mô-đun nghĩa là nhân với b^{-1}.
- Fermat nhỏ: $b^{p-1} \equiv 1 \pmod{p}$ với p nguyên tố, $p \nmid b$; do đó $b^{-1} \equiv b^{p-2}$, tính bằng lũy thừa nhanh.
- Mô-đun bất kì: dùng Euclid mở rộng, nghịch đảo là hệ số x trong $b \cdot x + m \cdot y = 1$.
- Nghịch đảo của 1..n: $inv[i] = (p - \lfloor p/i \rfloor) \cdot inv[p \bmod i] \bmod p$, O(n).

Luyện tập

Bài	Tên	Gợi ý liên hệ
10.1	**XÁC SUẤT**	In xác suất P / Q dưới dạng $P \cdot Q^{-1} \bmod (10^9 + 7)$: nghịch đảo bằng định lí Fermat.
10.2	**NGHỊCH ĐẢO DÃY**	Nghịch đảo của 1..n ($n \le 10^7$) theo mô-đun nguyên tố: công thức truy hồi O(n) của Mở rộng 1.
10.3	**PHƯƠNG TRÌNH ĐỒNG DƯ**	Đếm và liệt kê nghiệm của $a \cdot x \equiv c \pmod{m}$, $m \le 10^{18}$: gcd và Euclid mở rộng (Mở rộng 3).

Tự kiểm tra nhanh

1) Tìm nghịch đảo của 4 theo mô-đun 11. *(Đáp số: 3, vì $4 \cdot 3 = 12 \equiv 1$)*

2) Số 6 có nghịch đảo theo mô-đun 15 không? Vì sao? *(Đáp số: không, vì $\gcd(6, 15) = 3 \neq 1$)*

3) Tính $3^{-1} \bmod (10^9 + 7)$. *(Đáp số: 333 333 336)*

4) Dùng định lí Fermat, tính nhanh $5^{100} \bmod 7$. *(Đáp số: $100 = 6 \cdot 16 + 4$ nên $5^{100} \equiv 5^4 = 625 \equiv 2$)*

BÀI 11 Hàm phi Euler và định lí Euler

Học xong bài này, em sẽ:

- ✓ Hiểu hàm phi Euler $\varphi(n)$ và các tính chất: $\varphi(p) = p - 1$, $\varphi(p^k) = p^k - p^{k-1}$, **công thức tích**.
- ✓ Tính $\varphi(n)$ với n tới 10^{12} trong $O(\sqrt{n})$, và φ của mọi số 1..N bằng sàng.
- ✓ Phát biểu và chứng minh **định lí Euler**, mở rộng của định lí Fermat nhỏ cho mọi mô-đun.
- ✓ Rút gọn số mũ rất lớn khi tính lũy thừa theo mô-đun, và tránh cái bẫy khi $\gcd(a, m) \neq 1$.

KHỞI ĐỘNG

Mặt đồng hồ có 12 vạch, đánh số 0..11. Một chiếc kim kì lạ mỗi lần nhảy 5 vạch: 0 → 5 → 10 → 3 → 8 → 1 → ... Nó có đi qua đủ 12 vạch không? Còn nếu mỗi lần nhảy 4 vạch thì sao? Em hãy thử trước khi đọc tiếp.

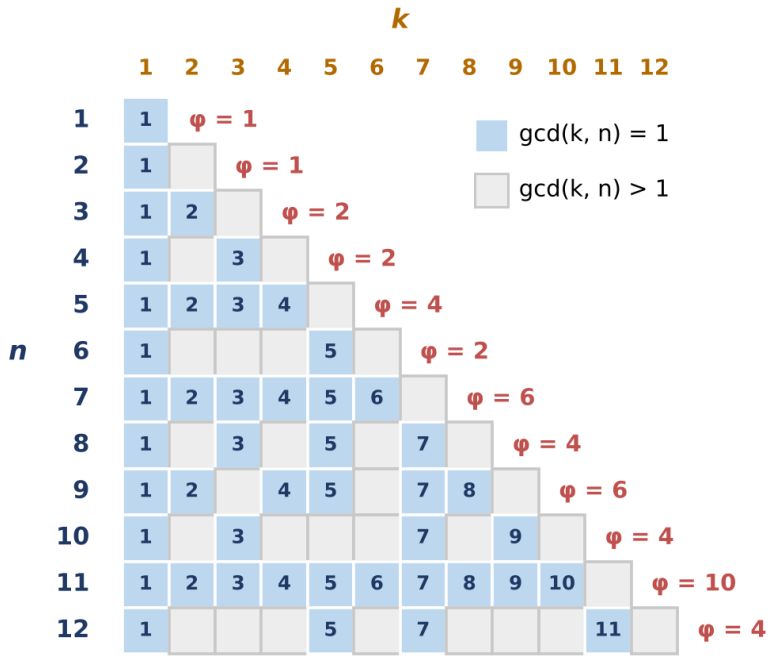
Kim nhảy 5 vạch đi qua đủ 12 vạch, còn kim nhảy 4 vạch chỉ đi qua 0, 4, 8 rồi quay lại. Kim nhảy k vạch đi hết vòng khi và chỉ khi k và 12 không có ước chung nào ngoài 1. Trong 1..12 chỉ có 1, 5, 7, 11 như vậy. Con số 4 ấy có tên riêng: **$\varphi(12)$**. Bài học này dạy em tính $\varphi(n)$ cho n tới 10^{12} và dùng nó để mở rộng định lí Fermat sang mọi mô-đun.

1. Số nguyên tố cùng nhau và hàm φ

ĐỊNH NGHĨA (HÀM PHI EULER)

Hai số nguyên a, b gọi là **nguyên tố cùng nhau** nếu $\gcd(a, b) = 1$. Với số nguyên dương n, **hàm phi Euler** $\varphi(n)$ là số các số k trong 1, 2, ..., n nguyên tố cùng nhau với n.

Ví dụ $\varphi(12) = 4$ (các số 1, 5, 7, 11), $\varphi(7) = 6$ (mọi số từ 1 tới 6), $\varphi(1) = 1$ (vì $\gcd(1, 1) = 1$). Hình 11.1 tô màu mọi cặp (n, k) với $k \le n \le 12$ nguyên tố cùng nhau. Hãy dừng lại vài giây để tìm quy luật trước khi đọc tiếp.



Hình 11.1. *Ô xanh: gcd(k, n) = 1. Số ô xanh trên hàng n chính là φ(n).*

Vài điều nhìn thấy ngay từ hình:

- Hàng của số nguyên tố p (2, 3, 5, 7, 11) xanh gần hết: **φ(p) = p − 1**, vì chỉ có k = p là chia hết cho p.
- Hàng 8 = 2^3 xanh đúng ở các số lẻ: φ(8) = 4. Tổng quát $\varphi(p^k) = p^k - p^{k-1}$, vì trong 1..$p^k$ chỉ các bội của p (có p^{k-1} số) là có ước chung với p^k.
- Hàng 12 = 4 · 3 có φ(12) = 4 = φ(4) · φ(3); hàng 6 = 2 · 3 có φ(6) = 2 = φ(2) · φ(3). Có vẻ φ "nhân" được khi tách n thành các phần nguyên tố cùng nhau. Bài 12 sẽ chứng minh điều này.

2. Bài toán PHIEULER

BÀI TOÁN: PHIEULER

Cho số nguyên dương n. Hãy tính φ(n), số các số nguyên k với 1 ≤ k ≤ n và gcd(k, n) = 1.

Dữ liệu vào: Một số nguyên n (1 ≤ n ≤ 10^{12}).

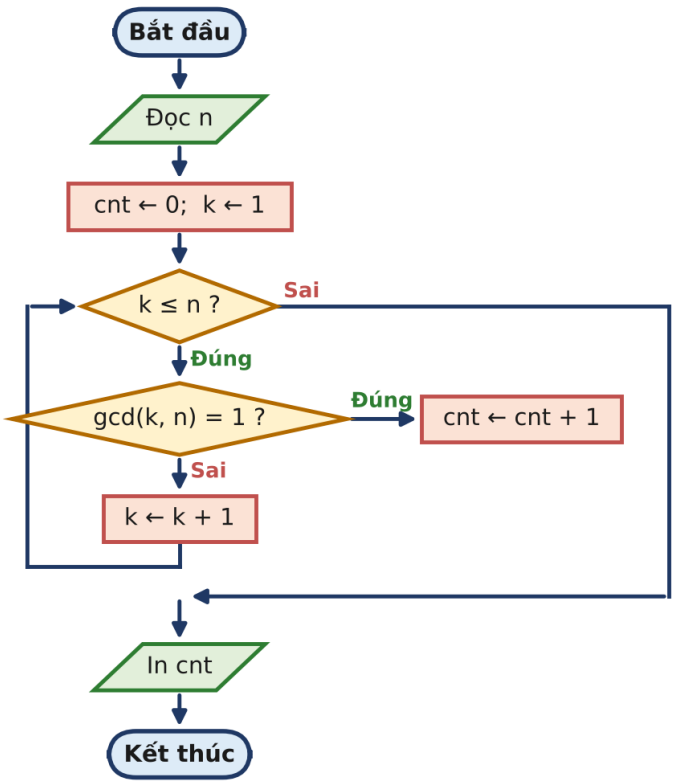
Kết quả: Một số nguyên là φ(n).

PHIEULER.INP	PHIEULER.OUT	Giải thích
`360`	`96`	360 = $2^3 \cdot 3^2 \cdot 5$, xem phần chạy tay.
`1`	`1`	φ(1) = 1.
`1000000007`	`1000000006`	$10^9 + 7$ là số nguyên tố nên φ = $10^9 + 6$.

PHIEULER.INP	PHIEULER.OUT	Giải thích
1000000000000	400000000000	$10^{12} = 2^{12} \cdot 5^{12}$: $\varphi = 10^{12} \cdot (1/2) \cdot (4/5) = 4 \cdot 10^{11}$.

3. Ý tưởng ngây thơ: Đếm theo định nghĩa

Duyệt k = 1, 2, ..., n và đếm những k có gcd(k, n) = 1, đúng như cách ta tô màu Hình 11.1.



Hình 11.2. *Sơ đồ khối của cách đếm theo định nghĩa.*

PHIEULER.CPP (ngây thơ): đếm k có gcd(k, n) = 1

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

int main() {
    if (fopen("PHIEULER.INP", "r")) {
        freopen("PHIEULER.INP", "r", stdin);
        freopen("PHIEULER.OUT", "w", stdout);
    }
    ll n;
    cin >> n;
    ll cnt = 0;
    for (ll k = 1; k <= n; k++)             // xét từng k = 1, 2, ..., n
        if (__gcd(k, n) == 1) cnt++;        // k nguyên tố cùng nhau với n
    cout << cnt << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

n lần gọi gcd, mỗi lần O(log n) bước: O(n log n). Với $n = 10^6$ chương trình vẫn chạy rất nhanh.

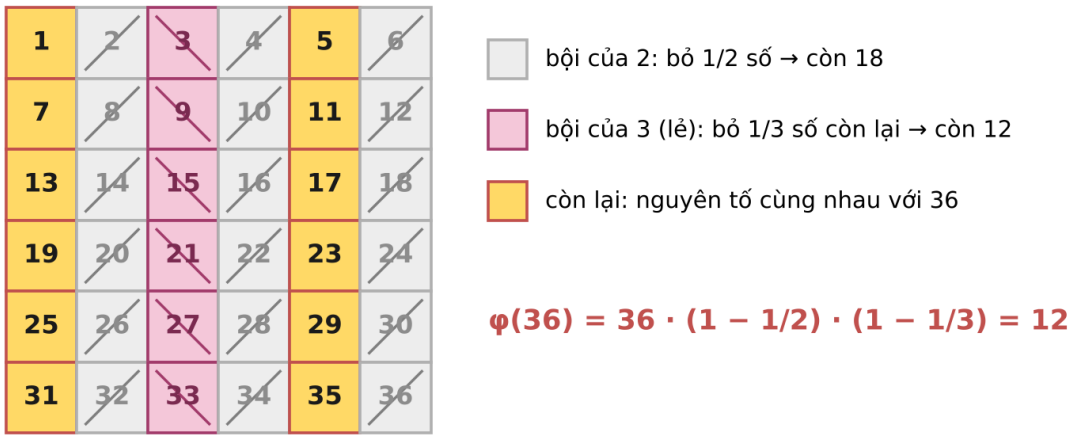
BẾ TẮC

Với $n = 10^{12}$, riêng vòng lặp 10^{12} lượt đã cần khoảng 10^4 giây, gần 3 giờ, chưa kể các phép gcd. Ta đang đếm từng số một. Có cách nào **đếm cả nhóm**, như đếm số bội của 2 trong 1..n chỉ bằng một phép chia n / 2 không?

4. Phá bế tắc: Công thức tích của φ

Bước 1 **Gạch bỏ bội của từng ước nguyên tố**

Một số k có ước chung lớn hơn 1 với n khi và chỉ khi k và n có chung một **ước nguyên tố** p nào đó. Vậy φ(n) là số lượng số còn lại trong 1..n sau khi gạch bỏ bội của mọi ước nguyên tố của n, giống sàng Eratosthenes ở Bài 6. Với $n = 36 = 2^2 \cdot 3^2$:



Hình 11.3. *Gạch bỏ bội của 2 và bội của 3 trong 1..36: còn lại φ(36) = 12 số.*

Bội của 2 chiếm đúng một nửa: còn $36 \cdot (1 - 1/2) = 18$ số lẻ. Trong 18 số lẻ ấy, các bội của 3 (là 3, 9, 15, 21, 27, 33) chiếm đúng một phần ba, vì trong mỗi 6 số liên tiếp có đúng 3 số lẻ và đúng một trong chúng chia hết cho 3: còn $18 \cdot (1 - 1/3) = 12$. Tổng quát:

ĐỊNH LÍ (CÔNG THỨC TÍCH)

Nếu $p_1, p_2, ..., p_k$ là các ước nguyên tố **khác nhau** của n thì

$$\varphi(n) = n \cdot (1 - 1/p_1) \cdot (1 - 1/p_2) \cdot ... \cdot (1 - 1/p_k)$$

VÌ SAO ĐÚNG?

Xét n có đúng hai ước nguyên tố p, q (trường hợp tổng quát làm hoàn toàn tương tự bằng nguyên lí bù trừ ở Bài 14). Trong 1..n có n/p bội của p, n/q bội của q, và n/(pq) số chia hết cho cả hai (chính là các bội của pq). Khi gạch bội của p rồi bội của q, các bội của pq bị tính hai lần, nên số bị gạch là n/p + n/q − n/(pq). Do đó:

$$\varphi(n) = n - n/p - n/q + n/(pq) = n \cdot (1 - 1/p) \cdot (1 - 1/q)$$

Khai triển tích ở vế phải sẽ thấy hai vế bằng nhau. Các phép chia ở đây đều là chia hết vì p, q, pq đều là ước của n.

Bước 2 Ghép với phân tích thừa số

Ta cần các ước nguyên tố của n: thử chia cho p = 2, 3, 4, ... tới √n như ở Bài 5 và Bài 7. Mỗi khi gặp một ước nguyên tố p, chia n cho p tới hết (để không gặp lại p), và nhân kết quả với (1 − 1/p). Phép nhân với phân số này làm được hoàn toàn bằng số nguyên:

$$\text{res} \leftarrow \text{res} - \text{res} / p$$

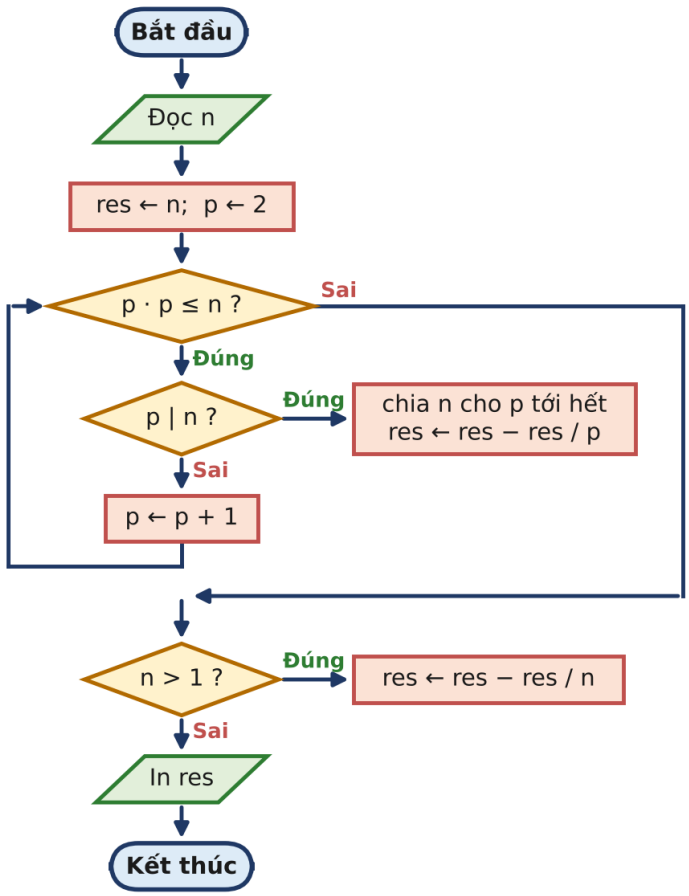
Phép chia luôn chia hết: res là n nhân với các thừa số (1 − 1/q) của những số nguyên tố q khác p, nên vẫn là bội của p. Những p hợp số như 4, 6 không bao giờ chia hết n vào lúc được thử, vì các ước nguyên tố của chúng đã bị chia hết khỏi n từ trước. Cuối cùng, nếu n còn lại lớn hơn 1 thì phần còn lại là **một số nguyên tố lớn hơn √n** (một số không thể có hai ước nguyên tố cùng lớn hơn căn bậc hai của nó), nên nhân thêm (1 − 1/n).

Bước 3 Chạy tay với n = 360

p	n lúc thử	p | n ?	n sau khi chia hết cho p	res
2	360	có	360 → 180 → 90 → 45	360 − 360/2 = 180
3	45	có	45 → 15 → 5	180 − 180/3 = 120
4	5	4 · 4 > 5: dừng	5	120
(còn lại)	5	n = 5 > 1	số nguyên tố 5	120 − 120/5 = **96**

Kiểm tra bằng công thức tích: 360 · (1/2) · (2/3) · (4/5) = 96.

Bước 4 Chương trình



Hình 11.4. *Sơ đồ khối của cách tính φ(n) bằng thử chia tới √n.*

PHIEULER.CPP: công thức tích, thử chia tới √n

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

// φ(n) = n · (1 - 1/p1) · (1 - 1/p2) · ... với p1, p2, ... là các
// ước nguyên tố khác nhau của n, tìm bằng cách thử chia tới √n
ll phi(ll n) {
    ll res = n;
    for (ll p = 2; p * p <= n; p++)
        if (n % p == 0) {                   // p là ước nguyên tố của n
            while (n % p == 0) n /= p;      // chia hết mọi thừa số p
            res -= res / p;                 // res = res · (1 - 1/p)
        }
    if (n > 1) res -= res / n;      // còn lại một số nguyên tố > √n
    return res;
}

int main() {
    if (fopen("PHIEULER.INP", "r")) {
        freopen("PHIEULER.INP", "r", stdin);
        freopen("PHIEULER.OUT", "w", stdout);
    }
    ll n;
    cin >> n;
    cout << phi(n) << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Vòng `for` chạy tới $\sqrt{n} \le 10^6$ lượt; mỗi lần chia hết làm n giảm ít nhất một nửa nên các vòng `while` bên trong tổng cộng không quá $\log_2 n \approx 40$ lần. Tổng cộng $O(\sqrt{n})$: khoảng một phần trăm giây với $n = 10^{12}$, so với gần 3 giờ của cách ngây thơ.

Độ phức tạp: $O(\sqrt{n})$.

5. Sàng φ cho mọi số 1..N

Nhiều bài cần φ(1), φ(2), ..., φ(N) cùng lúc, N tới 10^7. Gọi hàm trên N lần tốn $O(N\sqrt{N})$, quá chậm. Hãy làm như sàng Eratosthenes: thay vì xét từng số n để tìm ước nguyên tố của nó, ta xét từng số nguyên tố p và **nhân (1 − 1/p) vào mọi bội của p**.

THUẬT TOÁN (SÀNG φ)

1) phi[i] ← i với mọi i = 1..N.

2) Duyệt p = 2, 3, ..., N. Nếu phi[p] vẫn bằng p thì chưa số nguyên tố nào nhỏ hơn p chia hết p, tức p là số nguyên tố: với mọi bội j của p, gán phi[j] ← phi[j] − phi[j] / p.

Bảng dưới chạy sàng với N = 12; ô in đậm là ô vừa bị sửa ở lượt đó. Hàng cuối chính là φ(1), ..., φ(12) trong Hình 11.1.

Sau	1	2	3	4	5	6	7	8	9	10	11	12
ban đầu	1	2	3	4	5	6	7	8	9	10	11	12
p = 2	1	**1**	3	**2**	5	**3**	7	**4**	9	**5**	11	**6**
p = 3	1	1	**2**	2	5	**2**	7	4	**6**	5	11	**4**
p = 5	1	1	2	2	**4**	2	7	4	6	**4**	11	4
p = 7	1	1	2	2	4	2	**6**	4	6	4	11	4
p = 11	1	1	2	2	4	2	6	4	6	4	**10**	4

SANGPHI.CPP: φ(1), φ(2), ..., φ(N) bằng sàng

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("SANGPHI.INP", "r")) {
        freopen("SANGPHI.INP", "r", stdin);
        freopen("SANGPHI.OUT", "w", stdout);
    }
    int N;
    scanf("%d", &N);
    vector<int> phi(N + 1);
    for (int i = 0; i <= N; i++) phi[i] = i;
    for (int p = 2; p <= N; p++)
        if (phi[p] == p)                    // chưa bị sửa: p nguyên tố
            for (int j = p; j <= N; j += p)
                phi[j] -= phi[j] / p;       // nhân φ(j) với (1 - 1/p)
    for (int i = 1; i <= N; i++) printf("%d ", phi[i]);
    printf("\n");
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Mỗi số nguyên tố p duyệt N/p bội của nó: tổng cộng $N \cdot (1/2 + 1/3 + 1/5 + 1/7 + \ldots) = O(N \log \log N)$, giống sàng Eratosthenes. Với $N = 10^7$ chạy chưa tới một giây.

6. Định lí Euler

Định lí Fermat nói $b^{p-1} \equiv 1 \pmod p$. Để ý rằng $p - 1$ chính là $\varphi(p)$. Euler phát hiện ra định lí còn đúng với **mọi** mô-đun m, nếu thay $p - 1$ bằng $\varphi(m)$.

ĐỊNH LÍ (EULER)

Nếu $\gcd(a, m) = 1$ thì $a^{\varphi(m)} \equiv 1 \pmod m$.

VÌ SAO ĐÚNG?

Lặp lại chứng minh hoán vị của Bài 10, nhưng chỉ với φ(m) số $r_1, r_2, ..., r_{\varphi(m)}$ trong 1..m nguyên tố cùng nhau với m. Các số $a \cdot r_i \bmod m$ vẫn nguyên tố cùng nhau với m và đôi một khác nhau (vì a có nghịch đảo), nên chúng là một hoán vị của các r_i. Nhân tất cả lại: $a^{\varphi(m)} \cdot R \equiv R \pmod m$ với $R = r_1 \cdot r_2 \cdot ... \cdot r_{\varphi(m)}$. Vì R nguyên tố cùng nhau với m nên có nghịch đảo, nhân hai vế với R^{-1} được điều phải chứng minh.

Ví dụ m = 12, a = 5: nhân các số 1, 5, 7, 11 với 5 được 5, 25, 35, 55 ≡ 5, 1, 11, 7: đúng là đổi chỗ. Và $5^4 = 625 = 52 \cdot 12 + 1$.

Hệ quả quan trọng nhất cho lập trình: khi gcd(a, m) = 1, **số mũ được phép lấy dư theo φ(m)**. Thật vậy, nếu $b = q \cdot \varphi(m) + r$ thì $a^b = (a^{\varphi(m)})^q \cdot a^r \equiv a^r \pmod m$. Với m = p nguyên tố, số mũ lấy dư theo p − 1.

Ứng dụng: tính a mũ (b^c) theo mô-đun nguyên tố p

Với b, c tới 10^{18}, số mũ b^c có thể có hàng tỉ tỉ chữ số, không thể tính thẳng. Nhưng nếu $p \nmid a$, ta chỉ cần $b^c \bmod (p - 1)$: lại là một lũy thừa nhanh! Ví dụ:

- 2 mũ (3^{100}) theo mô-đun 7: $3^{100} \bmod 6 = 3$, nên đáp số là $2^3 = 8 \equiv$ **1** (mod 7).
- 3 mũ (10^{18}) theo mô-đun $10^9 + 7$: $10^{18} \bmod (10^9 + 6) = 36$, nên đáp số là $3^{36} \bmod (10^9 + 7)$ = **246 336 683**.

Cẩn thận trường hợp p | a: khi đó a ≡ 0 nên đáp số là 0 (số mũ dương), và **không** được rút gọn số mũ: nếu số mũ rút gọn bằng 0 thì chương trình sẽ in $a^0 = 1$, sai.

THAPLT.CPP: a mũ (b mũ c) theo mô-đun nguyên tố p, với b, c ≥ 1

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

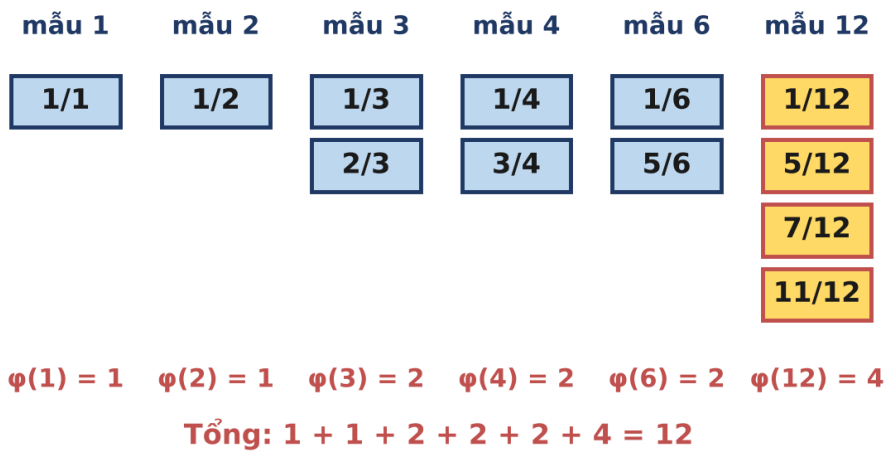
ll power(ll a, ll b, ll m) {
    ll r = 1 % m;
    a %= m;
    while (b > 0) {
        if (b & 1) r = r * a % m;
        a = a * a % m;
        b >>= 1;
    }
    return r;
}

int main() {                // tính a^(b^c) mod p, p nguyên tố, b, c ≥ 1
    if (fopen("THAPLT.INP", "r")) {
        freopen("THAPLT.INP", "r", stdin);
        freopen("THAPLT.OUT", "w", stdout);
    }
    ll a, b, c, p;
    cin >> a >> b >> c >> p;
    if (a % p == 0) { cout << 0 << "\n"; return 0; }   // b^c ≥ 1
    ll e = power(b, c, p - 1);      // Fermat: số mũ chỉ cần mod p - 1
    cout << power(a, e, p) << "\n";
    return 0;
}
```

7. Mở rộng

Mở rộng 1: Tổng φ trên các ước của n bằng n

Viết 12 phân số 1/12, 2/12, ..., 12/12 rồi rút gọn. Mỗi phân số tối giản thu được có mẫu d là một ước của 12, còn tử số là một số trong 1..d nguyên tố cùng nhau với d; ngược lại mọi phân số như vậy đều xuất hiện. Vậy có đúng φ(d) phân số mẫu d:



Hình 11.5. Rút gọn k/12 và xếp theo mẫu số: cột mẫu d có đúng φ(d) phân số.

$$\Sigma_{d \mid n}\, \varphi(d) = n$$

Với n = 12: φ(1) + φ(2) + φ(3) + φ(4) + φ(6) + φ(12) = 1 + 1 + 2 + 2 + 2 + 4 = 12.

Mở rộng 2: Đếm phân số tối giản

Có bao nhiêu phân số tối giản a/b với 0 < a < b ≤ N? Với mỗi mẫu b có đúng φ(b) tử số, nên đáp số là φ(2) + φ(3) + ... + φ(N): dùng sàng φ rồi cộng dồn, O(N log log N). Với N = 8 có 1 + 2 + 2 + 4 + 2 + 6 + 4 = 21 phân số. Xếp các phân số này tăng dần (thêm 0/1 và 1/1 ở hai đầu) ta được **dãy Farey** bậc N.

Mở rộng 3: Định lí Euler mở rộng

Nếu a và m không nguyên tố cùng nhau thì không được lấy b mod φ(m). Ví dụ $2^{100} \bmod 12 = 4$, nhưng $100 \bmod \varphi(12) = 100 \bmod 4 = 0$ và $2^0 = 1$: sai. Người ta chứng minh được rằng với **mọi** a, khi b ≥ φ(m) thì:

$$\mathbf{a^b \equiv a^{(b \bmod \varphi(m)) + \varphi(m)} \pmod m}$$

Thử lại: $2^{0+4} = 16 \equiv 4 \pmod{12}$. Đúng. Công thức này cho phép tính tháp lũy thừa nhiều tầng theo mô-đun bất kì bằng đệ quy theo dãy m, φ(m), φ(φ(m)), ..., dãy này giảm về 1 rất nhanh (chỉ sau O(log m) bước).

BẪY THƯỜNG GẶP

- **Nhân với (1 − 1/p) bằng số thực:** `res *= (1 - 1.0 / p)` sai số làm tròn với n cỡ 10^{12}. Dùng `res -= res / p`.
- **Quên ước nguyên tố lớn còn lại:** sau vòng lặp, nếu n > 1 thì n là một số nguyên tố lớn hơn √n. Thiếu dòng này, chương trình in φ(2 · 999 983) = 999 983 thay vì 999 982.
- **Tính trùng ước nguyên tố:** công thức chỉ dùng các ước nguyên tố khác nhau: φ(8) = 8 · (1 − 1/2) = 4, không phải 8 · $(1/2)^3$ = 1.
- **Rút gọn số mũ sai cách:** chỉ được lấy b mod φ(m) khi a, m nguyên tố cùng nhau (xem Mở rộng 3); với mô-đun nguyên tố p thì lấy theo p − 1 chứ không phải theo p, và nhớ xét riêng trường hợp p | a.
- **Khai báo p là `int`:** với n tới 10^{12}, p · p trong điều kiện `p * p <= n` cần `long long`.

GHI NHỚ

- φ(n) là số các k ∈ [1, n] có gcd(k, n) = 1; φ(p) = p − 1 và $\varphi(p^k) = p^k - p^{k-1}$.
- $\varphi(n) = n \cdot \Pi(1 - 1/p)$ qua các ước nguyên tố khác nhau p của n; tính trong O(√n) bằng res ← res − res / p.
- Sàng φ: phi[j] ← phi[j] − phi[j] / p với mọi bội j của số nguyên tố p; O(N log log N).
- Định lí Euler: gcd(a, m) = 1 ⇒ $a^{\varphi(m)} \equiv 1 \pmod m$; số mũ được lấy dư theo φ(m), theo p − 1 khi m = p nguyên tố.

Luyện tập

Bài	Tên	Gợi ý liên hệ
11.1	**PHI**	Q ≤ 100 truy vấn φ(n) với n ≤ 10^{12}: thử chia tới √n.
11.2	**PHÂN SỐ TỐI GIẢN**	Đếm phân số tối giản có mẫu ≤ N, nhiều truy vấn N ≤ 10^7: sàng φ và tổng tiền tố (Mở rộng 2).
11.3	**THÁP LŨY THỪA**	a mũ (b mũ c) theo mô-đun m bất kì: rút gọn số mũ theo φ(m) và định lí Euler mở rộng (Mở rộng 3).

Tự kiểm tra nhanh

1) Tính φ(100). *(Đáp số: 100 · (1/2) · (4/5) = 40)*

2) Tính φ(2026), biết 2026 = 2 · 1013 với 1013 là số nguyên tố. *(Đáp số: φ(2) · φ(1013) = 1 · 1012 = 1012)*

3) Tìm chữ số tận cùng của 3^{2026} bằng định lí Euler. *(Đáp số: φ(10) = 4, 2026 mod 4 = 2, nên $3^{2026} \equiv 3^2 = 9 \pmod{10}$)*

4) Có bao nhiêu phân số tối giản dạng k/12 với 1 ≤ k ≤ 12? *(Đáp số: φ(12) = 4, đó là 1/12, 5/12, 7/12, 11/12)*

BÀI 12 Định lí số dư Trung Hoa

Học xong bài này, em sẽ:

- ✓ Phát biểu và chứng minh **định lí số dư Trung Hoa**, hiểu nó qua bảng số dư và hai vòng Can Chi.
- ✓ Biết khi nào hệ đồng dư có nghiệm khi các mô-đun **có ước chung**.
- ✓ Giải hệ k phương trình bằng cách **gộp từng cặp** với Euclid mở rộng, không tràn số.
- ✓ Biết công thức cổ điển của Tôn Tử (bài toán Hàn Tín điểm binh) và mối liên hệ với hàm φ.

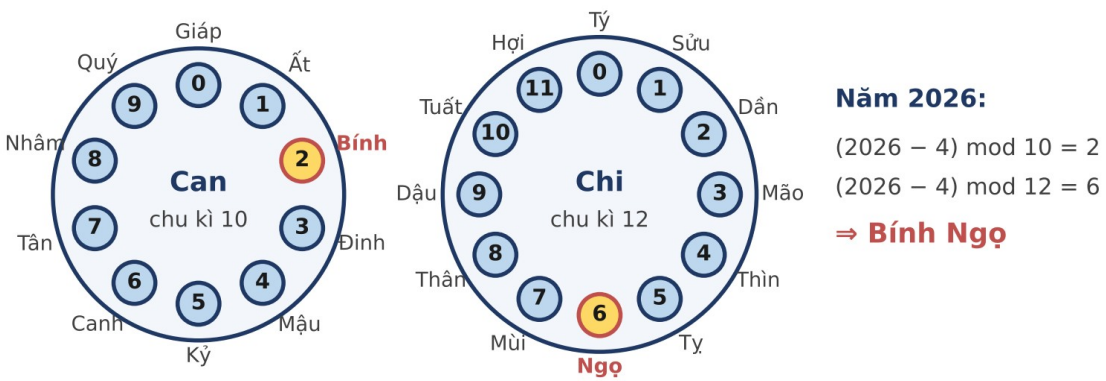
KHỞI ĐỘNG

Tương truyền danh tướng Hàn Tín điểm quân rất lạ: ông không đếm từng người mà cho quân xếp hàng 3, rồi hàng 5, rồi hàng 7, mỗi lần chỉ hỏi số người lẻ ra ở hàng cuối. Một đội quân nhỏ xếp hàng 3 thừa 2 người, hàng 5 thừa 3 người, hàng 7 thừa 2 người. Đội có bao nhiêu người (biết rằng không quá 100)?

Em có thể thử 2, 5, 8, 11, ... (các số chia 3 dư 2) cho tới khi gặp số đúng cả ba điều kiện: đó là **23**. Nhưng nếu xếp hàng 999 999 937 và hàng 10^9 thì sao? Bài học này cho em cách ghép từng điều kiện một, nhanh và chắc chắn, kể cả khi các số hàng có ước chung và bài toán có thể vô nghiệm.

1. Hệ đồng dư và hai vòng Can Chi

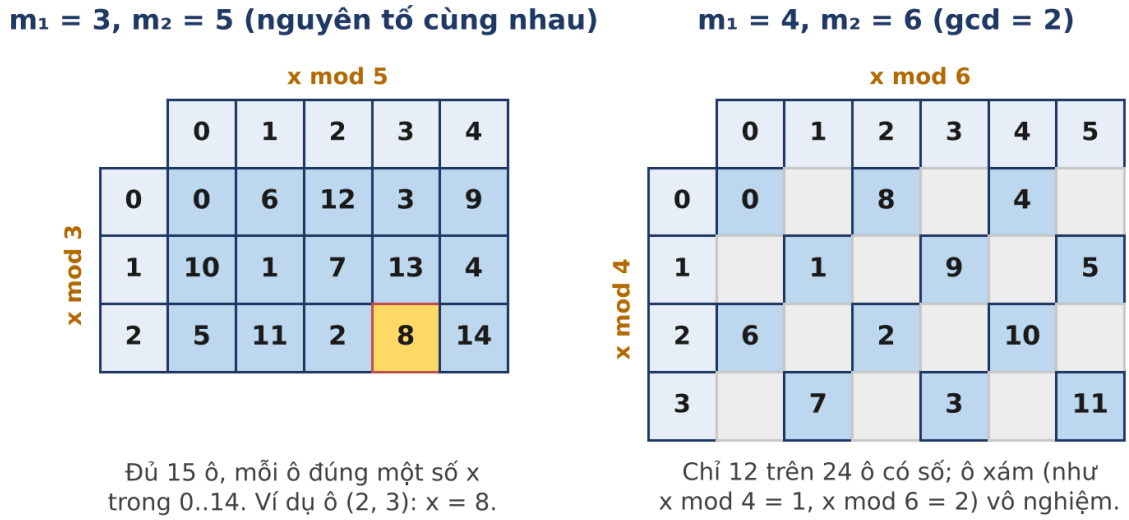
Người Việt gọi tên năm theo **Can Chi**: Can có chu kì 10 (Giáp, Ất, Bính, Đinh, Mậu, Kỷ, Canh, Tân, Nhâm, Quý), Chi có chu kì 12 (Tý, Sửu, Dần, ..., Hợi). Đánh số Can và Chi từ 0, năm Y có số Can là (Y − 4) mod 10 và số Chi là (Y − 4) mod 12. Năm 1984 là năm Giáp Tý (0, 0), còn năm 2026 là năm Bính Ngọ (2, 6).



Hình 12.1. *Hai vòng Can (chu kì 10) và Chi (chu kì 12) cùng quay, mỗi năm một nấc.*

Câu hỏi "những năm nào là năm Bính Ngọ?" chính là giải **hệ đồng dư** $x \equiv 2 \pmod{10}$, $x \equiv 6 \pmod{12}$ với $x = Y - 4$. Có một điều lạ: hai vòng tạo ra $10 \cdot 12 = 120$ cặp tên, nhưng tên năm lặp lại sau mỗi **60** năm, và chưa từng có năm "Giáp Sửu". Vì sao? Giáp Sửu cần $x \equiv 0 \pmod{10}$, tức x chẵn, và $x \equiv 1 \pmod{12}$, tức x lẻ: mâu thuẫn. Ước chung 2 của 10 và 12 ràng buộc hai vòng: Can số chẵn chỉ đi với Chi số chẵn.

Để hiểu trọn vẹn, xếp các số $x = 0, 1, 2, \ldots$ vào một bảng theo cặp số dư $(x \bmod m_1, x \bmod m_2)$:



Hình 12.2. *Bên trái: $m_1 = 3$, $m_2 = 5$, mỗi cặp số dư ứng với đúng một x trong 0..14. Bên phải: $m_1 = 4$, $m_2 = 6$, chỉ một nửa số cặp có nghiệm.*

ĐỊNH LÍ (SỐ DƯ TRUNG HOA)

Nếu m_1, m_2 nguyên tố cùng nhau thì với mọi a_1, a_2, hệ $x \equiv a_1 \pmod{m_1}$, $x \equiv a_2 \pmod{m_2}$ có **đúng một nghiệm** x trong khoảng $[0, m_1 \cdot m_2)$.

Tổng quát, với $g = \gcd(m_1, m_2)$: hệ có nghiệm khi và chỉ khi $a_1 \equiv a_2 \pmod{g}$, và khi đó nghiệm là duy nhất theo mô-đun $\operatorname{lcm}(m_1, m_2)$.

VÌ SAO ĐÚNG?

Trường hợp nguyên tố cùng nhau: xét $m_1 m_2$ số $x = 0, 1, \ldots, m_1 m_2 - 1$ và $m_1 m_2$ ô của bảng. Hai số $x < y$ không thể rơi vào cùng một ô: nếu thế thì $m_1 \mid y - x$ và $m_2 \mid y - x$, mà m_1, m_2 nguyên tố cùng nhau nên $m_1 m_2 \mid y - x$, vô lí vì $0 < y - x < m_1 m_2$. Vậy $m_1 m_2$ số rơi vào $m_1 m_2$ ô khác nhau: mỗi ô có đúng một số.

Điều kiện cần trong trường hợp tổng quát: g chia hết cả m_1 và m_2 nên $x \equiv a_1 \pmod{g}$ và $x \equiv a_2 \pmod{g}$, do đó $a_1 \equiv a_2 \pmod{g}$. Điều kiện đủ và tính duy nhất theo lcm sẽ được chứng minh cùng thuật toán ở Bước 1 dưới đây.

2. Bài toán CRT

BÀI TOÁN: CRT

Cho hệ k phương trình đồng dư $x \equiv a_i \pmod{m_i}$, $i = 1, 2, ..., k$. Các m_i **không nhất thiết** nguyên tố cùng nhau. Hãy tìm số nguyên không âm x nhỏ nhất thỏa mọi phương trình, hoặc in −1 nếu hệ vô nghiệm.

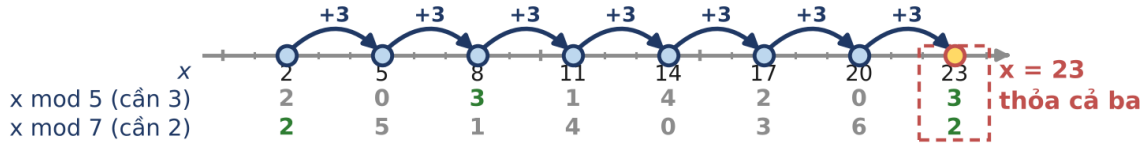
Dữ liệu vào: Dòng đầu chứa k ($1 \le k \le 10$). Dòng thứ i trong k dòng tiếp theo chứa a_i và m_i ($1 \le m_i \le 10^9$, $0 \le a_i < m_i$). Bảo đảm bội chung nhỏ nhất của mọi m_i không vượt quá 10^{18}.

Kết quả: Số x nhỏ nhất tìm được, hoặc −1 nếu hệ vô nghiệm.

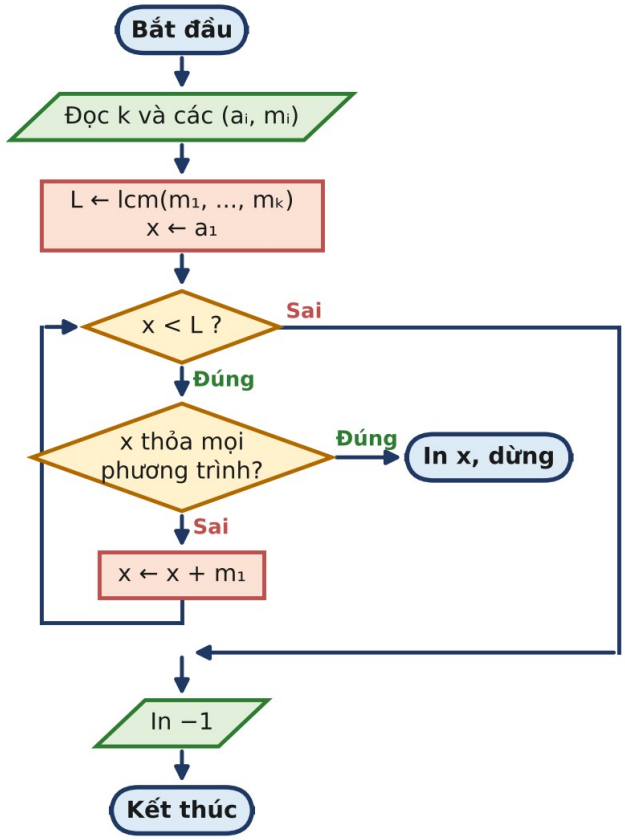
CRT.INP	CRT.OUT	Giải thích
3 2 3 3 5 2 7	23	Bài toán của Hàn Tín: $23 = 3 \cdot 7 + 2 = 5 \cdot 4 + 3 = 7 \cdot 3 + 2$.
3 1 4 3 6 3 9	21	$21 \bmod 4 = 1$, $21 \bmod 6 = 3$, $21 \bmod 9 = 3$; các mô-đun có ước chung, nghiệm lặp lại sau lcm = 36.
2 1 4 2 6	-1	$x \equiv 1 \pmod 4$ buộc x lẻ, $x \equiv 2 \pmod 6$ buộc x chẵn: vô nghiệm.
3 987654321 1000000000 111111111 999999999 487654321 500000000	123456789987654321	Các mô-đun cỡ 10^9, lcm = 999 999 999 000 000 000.

3. Ý tưởng ngây thơ: Nhảy theo bước m_1

Như cách thử trong phần Khởi động: nghiệm phải có dạng $a_1 + m_1 \cdot t$, nên chỉ cần thử các số a_1, $a_1 + m_1$, $a_1 + 2m_1$, ... và kiểm tra các phương trình còn lại. Nếu x là nghiệm thì x + L cũng là nghiệm, với $L = \text{lcm}(m_1, ..., m_k)$, nên nếu có nghiệm thì nghiệm nhỏ nhất bé hơn L. Thử tới L mà chưa thấy thì in −1.



Hình 12.3. Ví dụ 1: nhảy từ 2 theo bước 3, kiểm tra số dư khi chia cho 5 và 7; tới lần thử thứ 8 thì gặp 23.



Hình 12.4. *Sơ đồ khối của cách nhảy theo bước m_1.*

CRT.CPP (ngây thơ): thử các x ≡ a1 (mod m1) nhỏ hơn L

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

int main() {
    if (fopen("CRT.INP", "r")) {
        freopen("CRT.INP", "r", stdin);
        freopen("CRT.OUT", "w", stdout);
    }
    int k;
    cin >> k;
    vector<ll> a(k), m(k);
    ll L = 1;                                  // L = lcm(m1, ..., mk)
    for (int i = 0; i < k; i++) {
        cin >> a[i] >> m[i];
        L = L / __gcd(L, m[i]) * m[i];
    }
    // nghiệm nhỏ nhất (nếu có) nhỏ hơn L; chỉ thử các x ≡ a1 (mod m1)
    for (ll x = a[0]; x < L; x += m[0]) {
        bool good = true;
        for (int i = 1; i < k && good; i++)
            if (x % m[i] != a[i]) good = false;
        if (good) { cout << x << "\n"; return 0; }
    }
    cout << -1 << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Số lần thử tới L / m_1, mỗi lần kiểm tra k phương trình: O(k · L / m_1). Ví dụ 1 chỉ cần 8 lần thử.

BẾ TẮC

L có thể tới 10^{18} còn m_1 có thể rất nhỏ (thậm chí $m_1 = 1$): tới 10^{18} lần thử. Ngay ở ví dụ 4, với $m_1 = 10^9$ lớn nhất có thể, vẫn cần gần 10^9 lần thử. Nhảy từng bước m_1 vẫn là thử mù. Liệu có thể **tính thẳng** số bước nhảy t cần thiết, thay vì thử từng bước?

4. Phá bế tắc: Gộp từng cặp phương trình

Bước 1 Gộp hai phương trình thành một

Giả sử ta đã biết tập nghiệm của vài phương trình đầu là x ≡ r (mod M). Thêm phương trình x ≡ a (mod m). Mọi x thỏa các phương trình cũ có dạng x = r + M · t, nên ta cần tìm t sao cho:

$$\mathbf{r + M \cdot t \equiv a \pmod m} \quad \Leftrightarrow \quad \mathbf{M \cdot t \equiv a - r \pmod m}$$

Đây là phương trình đồng dư bậc nhất ẩn t, đúng như Mở rộng 3 của Bài 10. Đặt g = gcd(M, m):

- Nếu g ∤ (a − r): **vô nghiệm**. Đây chính là điều kiện a ≡ r (mod g) trong định lí.
- Nếu g | (a − r): chia cả hai vế và mô-đun cho g được (M/g) · t ≡ (a − r)/g (mod m/g). Lúc này gcd(M/g, m/g) = 1 nên M/g có nghịch đảo, và t ≡ (a − r)/g · $(M/g)^{-1}$ (mod m/g).

Thay t vào được nghiệm mới $x = r + M \cdot t$. Hai phương trình đã thành một!

VÌ SAO ĐÚNG?

Nghiệm t của $(M/g) \cdot t \equiv (a - r)/g \pmod{m/g}$ là duy nhất theo mô-đun m/g. Tăng t thêm m/g thì x tăng thêm $M \cdot (m/g) = \text{lcm}(M, m)$. Vậy tập nghiệm của hệ gộp đúng là một lớp đồng dư $x \equiv r + M \cdot t \pmod{\text{lcm}(M, m)}$. Áp dụng cho hai phương trình (bắt đầu từ $r = a_1$, $M = m_1$), ta được phần còn lại của định lí số dư Trung Hoa: có nghiệm khi $g \mid (a_2 - a_1)$, và nghiệm duy nhất theo mô-đun $\text{lcm}(m_1, m_2)$.

Bước 2 Gộp lần lượt k phương trình

Bắt đầu với "hệ rỗng" $x \equiv 0 \pmod 1$: mọi số nguyên đều thỏa. Rồi gộp lần lượt từng phương trình vào. Nghịch đảo $(M/g)^{-1} \bmod (m/g)$ lấy từ Euclid mở rộng: gọi extgcd(M mod m, m) được x, y với $(M \bmod m) \cdot x + m \cdot y = g$; chia hai vế cho g thấy ngay x chính là nghịch đảo cần tìm.

Bước 3 Chạy tay ví dụ 1 và ví dụ 2

Trong hai bảng dưới, $d = (a - r) \bmod m$ và $t = (d / g) \cdot (M/g)^{-1} \bmod (m/g)$. Ví dụ 1, các mô-đun đôi một nguyên tố cùng nhau:

Phương trình thêm	(r, M) cũ	g	d	$(M/g)^{-1} \bmod (m/g)$	t	(r, M) mới
$x \equiv 2 \pmod 3$	(0, 1)	1	2	1	2	(2, 3)
$x \equiv 3 \pmod 5$	(2, 3)	1	1	2	2	(8, 15)
$x \equiv 2 \pmod 7$	(8, 15)	1	1	1	1	(**23**, 105)

Ví dụ 2, các mô-đun có ước chung:

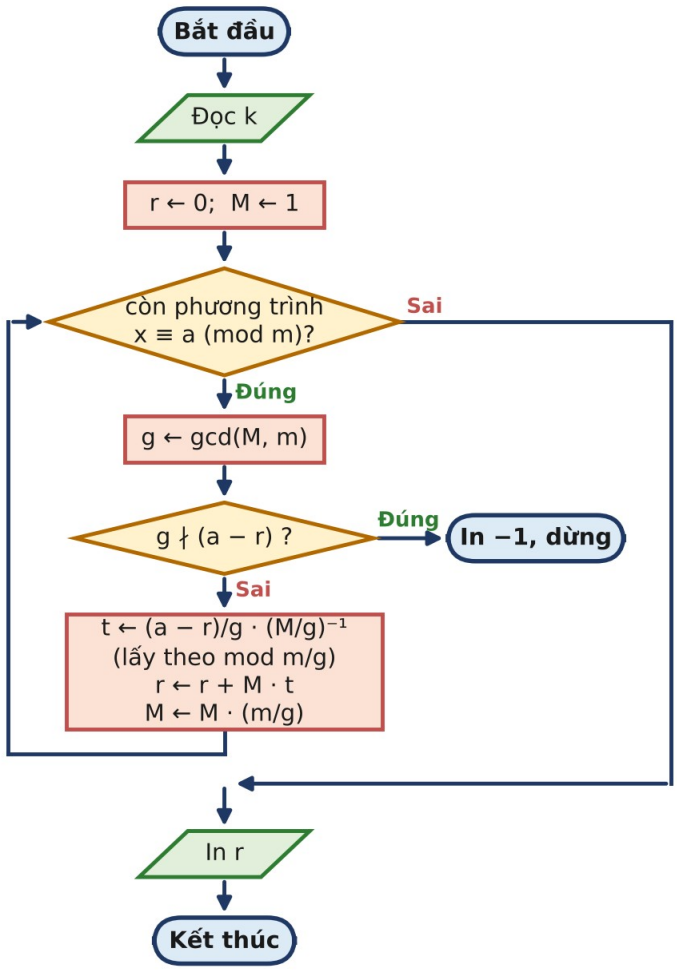
Phương trình thêm	(r, M) cũ	g	d	$(M/g)^{-1} \bmod (m/g)$	t	(r, M) mới
$x \equiv 1 \pmod 4$	(0, 1)	1	1	1	1	(1, 4)
$x \equiv 3 \pmod 6$	(1, 4)	2	2	$2^{-1} \bmod 3 = 2$	2	(9, 12)
$x \equiv 3 \pmod 9$	(9, 12)	3	3	$4^{-1} \bmod 3 = 1$	1	(**21**, 36)

Chẳng hạn ở hàng cuối: $d = (3 - 9) \bmod 9 = 3$ chia hết cho $g = 3$; $t = (3/3) \cdot 1 \bmod 3 = 1$; $x = 9 + 12 \cdot 1 = 21$, mô-đun mới $12 \cdot 9/3 = 36$. Còn ví dụ 3: sau phương trình đầu có (r, M) = (1, 4); thêm $x \equiv 2 \pmod 6$ thì $g = 2$ nhưng $d = (2 - 1) \bmod 6 = 1$ không chia hết cho 2, kết luận −1.

Bước 4 Tránh tràn số

Mọi giá trị đều vừa `long long`. Thật vậy, d/g và nghịch đảo đều nhỏ hơn $m/g \le 10^9$ nên tích của chúng nhỏ hơn 10^{18}. Còn $r + M \cdot t < M \cdot (m/g) = \text{lcm}$ của các mô-đun đã gộp, là ước của lcm của mọi m_i, mà đề bài bảo đảm số này không vượt quá 10^{18}. Nếu đề không cho bảo đảm ấy, phải nhân bằng `__int128` và kiểm tra M có vượt giới hạn không.

Bước 5 Chương trình



Hình 12.5. *Sơ đồ khối của cách gộp từng cặp phương trình.*

CRT.CPP: gộp từng cặp bằng Euclid mở rộng

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

ll extgcd(ll a, ll b, ll &x, ll &y) {   // a·x + b·y = gcd(a, b)
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1;
    ll g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

int main() {
    if (fopen("CRT.INP", "r")) {
        freopen("CRT.INP", "r", stdin);
        freopen("CRT.OUT", "w", stdout);
    }
    int k;
    cin >> k;
    ll r = 0, M = 1;            // các phương trình đã gộp: x ≡ r (mod M)
    bool ok = true;
    for (int i = 0; i < k; i++) {
        ll a, m;
        cin >> a >> m;
        if (!ok) continue;
        // x = r + M·t, cần M·t ≡ a - r (mod m)
        ll x, y;
        ll g = extgcd(M % m, m, x, y);      // g = gcd(M, m)
        ll d = ((a - r) % m + m) % m;
        if (d % g != 0) { ok = false; continue; }   // vô nghiệm
        ll m2 = m / g;                      // t xác định theo mod m/g
        ll t = (d / g) % m2 * ((x % m2 + m2) % m2) % m2;
        r = r + M * t;                      // r mới < M·m2 = lcm(M, m)
        M = M * m2;
    }
    cout << (ok ? r : -1) << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Mỗi phương trình tốn một lần Euclid mở rộng O(log m) cùng vài phép nhân, chia: tổng cộng $O(k \cdot \log \max m_i)$, chỉ vài trăm phép tính, so với tới 10^{18} lần thử của cách ngây thơ.

Độ phức tạp: O(k log m).

5. Mở rộng

Mở rộng 1: Hàn Tín điểm binh và các số 70, 21, 15

Sách cổ "Tôn Tử toán kinh" (Trung Hoa, khoảng thế kỉ 3–5) giải bài toán ở phần Khởi động theo cách khác, được truyền lại thành một bài thơ mà đại ý là: số dư khi chia cho 3 thì nhân 70, chia cho 5 thì nhân 21, chia cho 7 thì nhân 15, cộng lại rồi bớt dần 105. Vì sao lại là 70, 21, 15? Hãy xem số dư của chúng:

Số	mod 3	mod 5	mod 7
70	**1**	0	0
21	0	**1**	0
15	0	0	**1**

Mỗi số "bật" đúng một phương trình và "tắt" hai phương trình kia. Vậy $x = 70 \cdot a_1 + 21 \cdot a_2 + 15 \cdot a_3$ có số dư a_1, a_2, a_3 khi chia cho 3, 5, 7. Với (2, 3, 2): $140 + 63 + 30 = 233 = 2 \cdot 105 + 23$. Tổng quát, với các m_i đôi một nguyên tố cùng nhau, $M = m_1 \cdot m_2 \cdot \ldots \cdot m_k$ và $M_i = M / m_i$:

$$\mathbf{x \equiv \Sigma\, a_i \cdot M_i \cdot y_i \pmod{M}, \quad \text{với } y_i = M_i^{-1} \bmod m_i}$$

Ở đây $70 = 35 \cdot 2$ (vì $35^{-1} \equiv 2 \bmod 3$), $21 = 21 \cdot 1$ và $15 = 15 \cdot 1$. Công thức đẹp, nhưng các số hạng $a_i \cdot M_i \cdot y_i$ có thể tới 10^{27} nên phải nhân theo mô-đun M bằng `__int128`, và nó chỉ dùng được khi các m_i đôi một nguyên tố cùng nhau. Cách gộp từng cặp tổng quát hơn.

CRT.CPP (công thức Tôn Tử): các mô-đun đôi một nguyên tố cùng nhau

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

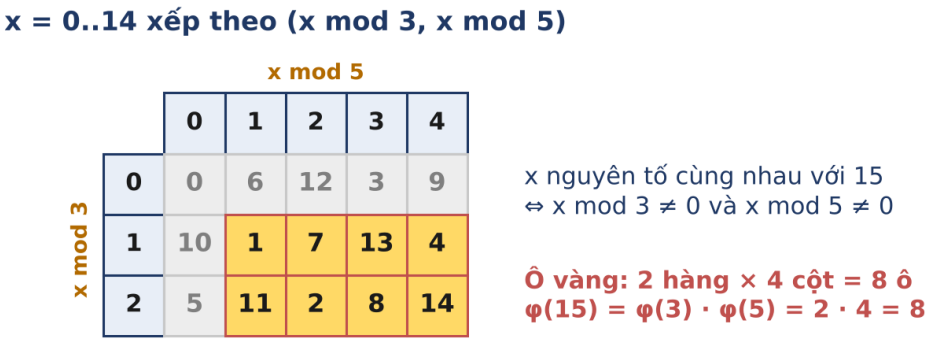
ll mulmod(ll a, ll b, ll m) { return (__int128)a * b % m; }

ll extgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1;
    ll g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

// Công thức Tôn Tử, chỉ dùng khi các m_i đôi một nguyên tố cùng nhau
int main() {
    if (fopen("CRT.INP", "r")) {
        freopen("CRT.INP", "r", stdin);
        freopen("CRT.OUT", "w", stdout);
    }
    int k;
    cin >> k;
    vector<ll> a(k), m(k);
    ll M = 1;
    for (int i = 0; i < k; i++) { cin >> a[i] >> m[i]; M *= m[i]; }
    ll x = 0;
    for (int i = 0; i < k; i++) {
        ll Mi = M / m[i], y, z;
        extgcd(Mi % m[i], m[i], y, z);      // y = Mi^(-1) mod m[i]
        y = (y % m[i] + m[i]) % m[i];
        // số hạng a_i·M_i·y_i: ≡ a_i (mod m_i), ≡ 0 (mod m_j), j ≠ i
        x = (x + mulmod(mulmod(a[i], Mi, M), y, M)) % M;
    }
    cout << x << "\n";
    return 0;
}
```

Mở rộng 2: Định lí số dư Trung Hoa và hàm φ

Bảng 3 × 5 ở Hình 12.2 còn cho một chứng minh đẹp của tính chất mà Bài 11 đã hứa: $\varphi(m_1 m_2) = \varphi(m_1) \cdot \varphi(m_2)$ khi $\gcd(m_1, m_2) = 1$. Số x nguyên tố cùng nhau với $m_1 m_2$ khi và chỉ khi x mod m_1 nguyên tố cùng nhau với m_1 và x mod m_2 nguyên tố cùng nhau với m_2. Trong bảng, đó là các ô nằm trên một trong $\varphi(m_1)$ hàng "tốt" và một trong $\varphi(m_2)$ cột "tốt":



Hình 12.6. *Các số trong 0..14 nguyên tố cùng nhau với 15 nằm đúng ở 2 hàng × 4 cột.*

Kết hợp với $\varphi(p^k) = p^k - p^{k-1}$, ta được ngay công thức tích của Bài 11 cho mọi n.

Mở rộng 3: Tính theo mô-đun hợp số bằng cách tách

Khi đề bài yêu cầu kết quả theo một mô-đun hợp số, chẳng hạn $m = 1001 = 7 \cdot 11 \cdot 13$, mà việc tính trực tiếp gặp khó (cần chia, cần định lí Fermat...), ta có thể tính kết quả theo từng mô-đun nguyên tố 7, 11, 13, nơi mọi số khác 0 đều có nghịch đảo, rồi dùng định lí số dư Trung Hoa ghép ba số dư lại thành số dư theo 1001. Em sẽ gặp lại kĩ thuật này khi tính tổ hợp C(n, k) theo mô-đun hợp số.

BẪY THƯỜNG GẶP

- **Tưởng hệ luôn có nghiệm:** chỉ đúng khi các mô-đun đôi một nguyên tố cùng nhau. Hệ $x \equiv 1 \pmod 4$, $x \equiv 2 \pmod 6$ vô nghiệm.
- **Nhân M với m thay vì m/g:** mô-đun mới là $\text{lcm}(M, m) = M \cdot (m/g)$, không phải $M \cdot m$. Dùng $M \cdot m$ thì các bước gộp sau tính sai và số dễ tràn.
- **Hiệu a − r âm:** r có thể lớn hơn a, và C++ cho số dư âm. Dùng `((a - r) % m + m) % m`.
- **Lấy nghịch đảo bằng định lí Fermat:** m/g thường không phải số nguyên tố; phải dùng Euclid mở rộng.
- **Tràn số trong công thức Tôn Tử:** $a_i \cdot M_i \cdot y_i$ vượt xa 10^{18}; phải nhân theo mô-đun M bằng `__int128`.

GHI NHỚ

- m_1, m_2 nguyên tố cùng nhau: mỗi cặp số dư (a_1, a_2) ứng với đúng một x theo mô-đun m_1m_2.
- Tổng quát: hệ có nghiệm $\Leftrightarrow$ $a_1 \equiv a_2 \pmod{\gcd(m_1, m_2)}$; nghiệm duy nhất theo mô-đun $\operatorname{lcm}(m_1, m_2)$.
- Gộp $x \equiv r \pmod M$ với $x \equiv a \pmod m$: giải $M \cdot t \equiv a - r \pmod m$ bằng gcd và Euclid mở rộng; mô-đun mới là $\operatorname{lcm}(M, m)$.
- Bắt đầu từ $(r, M) = (0, 1)$; gặp $g \nmid (a - r)$ thì kết luận vô nghiệm.

Luyện tập

Bài	Tên	Gợi ý liên hệ
12.1	**ĐIỂM BINH**	Hệ với các mô-đun đôi một nguyên tố cùng nhau, tích $\le 10^{18}$: công thức Tôn Tử hoặc gộp từng cặp.
12.2	**HAI NGỌN ĐÈN**	Hai đèn nháy theo chu kì và độ lệch khác nhau, hỏi lần đầu cùng sáng: hệ hai phương trình có thể vô nghiệm khi chu kì có ước chung.
12.3	**LỊCH CAN CHI**	Từ tên Can Chi tìm các năm trong một khoảng; đếm số tổ hợp tên có thể xuất hiện với chu kì tùy ý: điều kiện gcd và lcm.

Tự kiểm tra nhanh

1) Giải hệ $x \equiv 1 \pmod 2$, $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$. *(Đáp số: $x \equiv 23 \pmod{30}$)*

2) Giải hệ $x \equiv 3 \pmod 6$, $x \equiv 5 \pmod 8$. *(Đáp số: $g = 2$ chia hết $5 - 3$, nên có nghiệm: $x \equiv 21 \pmod{24}$)*

3) Hệ $x \equiv 2 \pmod 6$, $x \equiv 3 \pmod 9$ có nghiệm không? *(Đáp số: không, vì $\gcd(6, 9) = 3$, mà 2 và 3 không đồng dư theo mô-đun 3)*

4) Năm 2026 là năm Bính Ngọ. Năm Giáp Tý đầu tiên sau năm 2026 là năm nào? *(Đáp số: 2044, vì $2044 - 4 = 2040$ chia hết cho cả 10 và 12)*

CHƯƠNG 4. TỔ HỢP VÀ ĐẾM

Có bao nhiêu cách chọn đội tuyển? Bao nhiêu số nhỏ hơn 10^{18} chia hết cho ít nhất một trong hai mươi số cho trước? Số Fibonacci thứ 10^{18} chia cho 10^9 dư bao nhiêu? Tổng số ước của mọi số từ 1 đến một nghìn tỉ là bao nhiêu? Những câu hỏi "đếm" như vậy có đáp số khổng lồ, không thể liệt kê từng trường hợp, nhưng lại trả lời được trong chưa tới một giây nếu ta **đếm thông minh**.

Chương này trình bày bốn công cụ đếm cơ bản của lập trình thi đấu: hệ số nhị thức theo mô-đun, nguyên lí bù trừ, lũy thừa ma trận cho dãy truy hồi và kĩ thuật chia khối $\lfloor n / i \rfloor$. Mỗi công cụ dựa trên kiến thức của ba chương trước: nghịch đảo mô-đun, ước chung lớn nhất, lũy thừa nhanh, phép chia có dư, và là nền móng cho rất nhiều bài toán ở Tập 2.

TRONG CHƯƠNG NÀY

Bài 13. Tổ hợp và hệ số nhị thức: đếm số cách chọn và tính C(n, k) mod p thật nhanh

Bài 14. Nguyên lí bù trừ: đếm hợp của nhiều tập hợp: cộng, trừ, cộng xen kẽ

Bài 15. Dãy truy hồi và lũy thừa ma trận: số Fibonacci thứ 10^{18} trong 84 phép nhân ma trận

Bài 16. Kĩ thuật chia khối $\lfloor n / i \rfloor$: tổng số ước của 1..n trong $O(\sqrt{n})$

BÀI 13 Tổ hợp và hệ số nhị thức

Học xong bài này, em sẽ:

- ✓ Nắm **quy tắc nhân**, hoán vị, chỉnh hợp, tổ hợp và hiểu vì sao C(n, k) = n! / (k! · (n − k)!).
- ✓ Chứng minh được **hằng đẳng thức Pascal** C(n, k) = C(n − 1, k − 1) + C(n − 1, k) bằng một lập luận đếm.
- ✓ Biết vì sao không thể chia thẳng giai thừa theo mô-đun, và dùng **nghịch đảo giai thừa** để trả lời mỗi truy vấn C(n, k) mod p trong O(1).
- ✓ Chọn đúng công cụ cho từng giới hạn: tam giác Pascal, giai thừa, nhân dần, định lí Lucas.

KHỞI ĐỘNG

Lớp em có 30 bạn. Cô giáo cần chọn **3 bạn đi thi Tin học**. Bạn Minh nhẩm: "Bạn thứ nhất có 30 cách chọn, bạn thứ hai 29, bạn thứ ba 28, vậy 30 · 29 · 28 = 24 360 cách." Bạn Lan cãi: "Chọn An, Bình, Chi hay chọn Chi, An, Bình thì vẫn là một đội thôi!" Ai đúng?

Cả hai đều có lí, chỉ là hai câu hỏi khác nhau: Minh đếm **có thứ tự** (thi đấu theo lượt 1, 2, 3), Lan đếm **không thứ tự** (chỉ cần biết đội gồm những ai). Bài học này phân biệt rõ hai cách đếm ấy, rồi trả lời một câu hỏi rất "Tin học": làm sao tính số cách chọn k trong n vật khi n tới một triệu, cho một trăm nghìn câu hỏi, mỗi câu trong nháy mắt?

1. Quy tắc nhân, hoán vị và giai thừa

Mọi phép đếm trong bài này dựa trên một quy tắc rất đơn giản: nếu một việc gồm **hai bước liên tiếp**, bước đầu có a cách làm và **với mỗi cách** của bước đầu, bước sau có b cách làm, thì cả việc có **a · b** cách. Quy tắc mở rộng cho nhiều bước: số cách bằng tích số cách của từng bước.

Bạn Minh trong phần khởi động đã dùng đúng quy tắc này: chọn người thi lượt 1 (30 cách), rồi lượt 2 (còn 29 cách), rồi lượt 3 (còn 28 cách), được 30 · 29 · 28 = 24 360. Cách đếm có thứ tự như vậy có tên riêng.

ĐỊNH NGHĨA (HOÁN VỊ, CHỈNH HỢP)

- Một **hoán vị** của n vật là một cách xếp cả n vật thành một hàng. Số hoán vị là n! = 1 · 2 · ... · n (đọc là "n giai thừa"); quy ước 0! = 1.
- Một **chỉnh hợp chập k** của n vật là một cách chọn k vật **có thứ tự**. Số chỉnh hợp là A(n, k) = n · (n − 1) · ... · (n − k + 1) = n! / (n − k)!.

Giai thừa tăng **cực nhanh**. Bảng dưới cho thấy 20! vẫn vừa kiểu `long long` (giới hạn khoảng $9{,}2 \cdot 10^{18}$), nhưng 21! đã vượt cả `unsigned long long`. Đây là lí do mọi bài đếm trong thi đấu đều yêu cầu in kết quả **theo mô-đun** một số nguyên tố, thường là $p = 10^9 + 7$.

n	n!	n	n!
0	1	6	720
1	1	7	5 040
2	2	8	40 320
3	6	9	362 880
4	24	10	3 628 800
5	120	20	2 432 902 008 176 640 000 (vừa `long long`)
		21	51 090 942 171 709 440 000 (**tràn** 2^{64})

2. Tổ hợp và tam giác Pascal

Bây giờ đến câu hỏi của bạn Lan: chỉ quan tâm **đội gồm những ai**, không quan tâm thứ tự.

ĐỊNH NGHĨA (TỔ HỢP)

Một **tổ hợp chập k** của n vật là một tập con gồm k vật (không có thứ tự). Số tổ hợp kí hiệu là C(n, k) (sách giáo khoa viết C_n^k, đọc là "n chọn k"), còn gọi là **hệ số nhị thức**. Quy ước C(n, k) = 0 khi k < 0 hoặc k > n.

$$\mathbf{C(n, k) = n! / (k! \cdot (n - k)!)}$$

VÌ SAO CÔNG THỨC ĐÚNG?

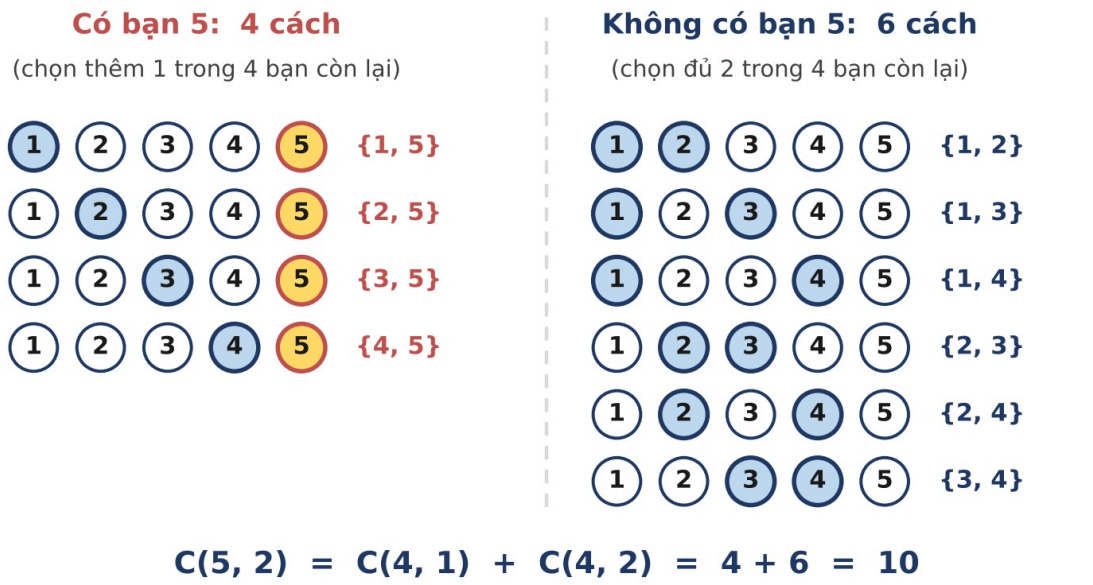
Đếm số chỉnh hợp A(n, k) theo **hai bước**: bước 1 chọn tập k vật (C(n, k) cách), bước 2 xếp thứ tự k vật đó (k! cách). Theo quy tắc nhân, A(n, k) = C(n, k) · k!, nên C(n, k) = A(n, k) / k! = n! / (k! · (n − k)!).

Với lớp 30 bạn: mỗi đội 3 bạn được Minh đếm đúng 3! = 6 lần (6 thứ tự), nên số đội là 24 360 / 6 = **4 060**. Cả Minh và Lan đều đúng với câu hỏi của mình.

VÍ DỤ: XỔ SỐ 6/45

Một vé xổ số chọn 6 số khác nhau trong các số 1..45, thứ tự không quan trọng. Số vé khác nhau là C(45, 6) = 45 · 44 · 43 · 42 · 41 · 40 / 720 = **8 145 060**. Mua một vé, khả năng trúng giải đặc biệt chỉ là 1 phần 8 triệu.

Công thức giai thừa đẹp nhưng chưa cho ta thấy **cấu trúc** của các số C(n, k). Hãy xét một ví dụ nhỏ: chọn 2 bạn trong 5 bạn đánh số 1..5. Ta chia các cách chọn thành hai nhóm theo câu hỏi "bạn số 5 có được chọn không?".



Hình 13.1. *Mười cách chọn 2 trong 5 bạn, chia theo bạn số 5: có (4 cách) hoặc không (6 cách).*

Nếu bạn 5 được chọn, ta chỉ còn chọn thêm 1 bạn trong 4 bạn kia: C(4, 1) = 4 cách. Nếu bạn 5 không được chọn, ta phải chọn đủ 2 bạn trong 4 bạn kia: C(4, 2) = 6 cách. Hai nhóm không giao nhau và phủ hết mọi cách, nên C(5, 2) = 4 + 6 = 10. Lập luận này đúng với mọi n và k.

ĐỊNH LÍ (HẰNG ĐẲNG THỨC PASCAL)

Với mọi $n \geq 1$ và $1 \leq k \leq n - 1$:

$$C(n, k) = C(n - 1, k - 1) + C(n - 1, k)$$

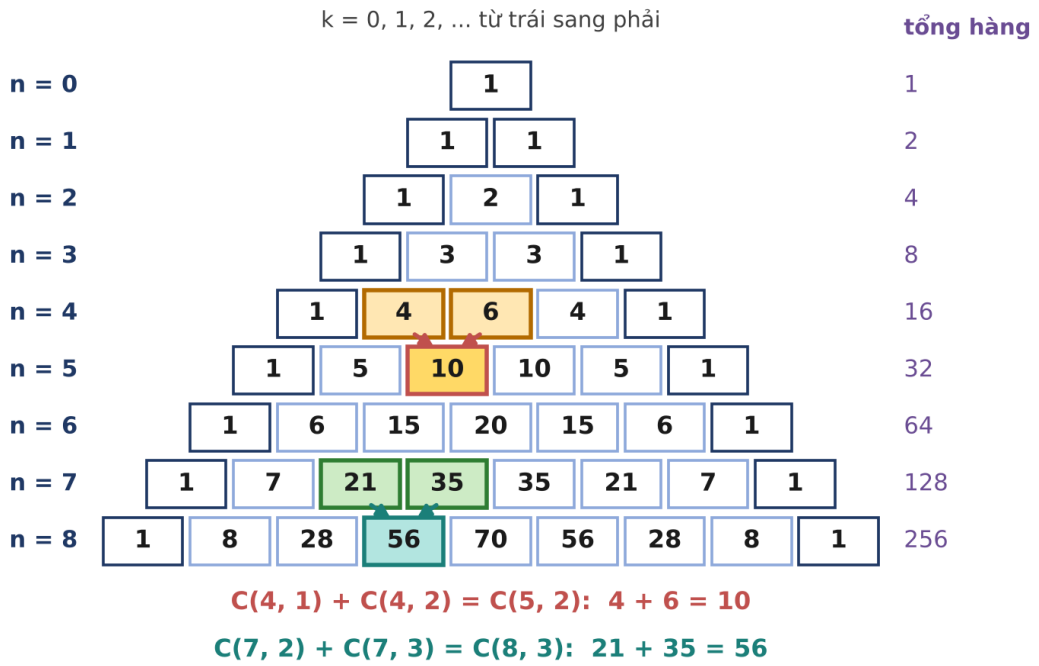
cùng với hai "đầu mút" C(n, 0) = C(n, n) = 1 và tính đối xứng C(n, k) = C(n, n − k).

VÌ SAO ĐÚNG?

Xét một vật cố định, chẳng hạn vật thứ n. Mỗi tập con k phần tử **hoặc chứa** vật n (khi đó k − 1 phần tử còn lại chọn trong n − 1 vật: C(n − 1, k − 1) cách), **hoặc không chứa** vật n (cả k phần tử chọn trong n − 1 vật: C(n − 1, k) cách). Hai trường hợp loại trừ nhau, nên cộng lại.

Đối xứng: chọn k vật để **lấy** cũng chính là chọn n − k vật để **bỏ lại**, hai việc tương ứng một-một.

Xếp các số C(n, k) thành hàng thứ n (k = 0, 1, ..., n), ta được **tam giác Pascal**: mỗi số bên trong bằng tổng hai số đứng ngay trên nó.



Hình 13.2. *Tam giác Pascal tới hàng 8. Mỗi ô bên trong bằng tổng hai ô phía trên; cột phải là tổng mỗi hàng.*

NHỊ THỨC NEWTON VÀ TỔNG MỖI HÀNG

Khai triển $(a + b)^n = (a + b)(a + b)...(a + b)$: mỗi số hạng $a^k b^{n-k}$ xuất hiện đúng bằng số cách chọn k thừa số đóng góp a, tức C(n, k) lần. Vậy $(a + b)^n = \Sigma\, C(n, k) \cdot a^k \cdot b^{n-k}$, đó là lí do C(n, k) được gọi là **hệ số nhị thức**. Thay a = b = 1: tổng hàng thứ n bằng $\mathbf{2^n}$ (số tập con của tập n phần tử). Thay a = 1, b = −1: tổng đan dấu của mỗi hàng (n ≥ 1) bằng **0**; em sẽ dùng điều này ở Bài 14.

3. Bài toán TOHOP

BÀI TOÁN: TOHOP

Cho Q truy vấn, mỗi truy vấn gồm hai số nguyên n, k. Với mỗi truy vấn, hãy in C(n, k) mod $(10^9 + 7)$; nếu k > n thì in 0.

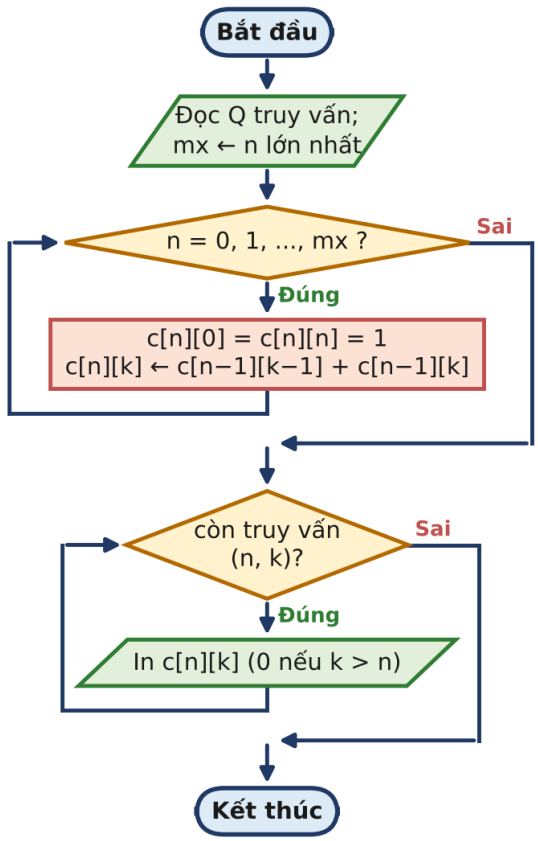
Dữ liệu vào: Dòng đầu chứa Q ($1 \le Q \le 10^5$). Mỗi dòng trong Q dòng sau chứa n, k ($0 \le n, k \le 10^6$).

Kết quả: Q dòng, mỗi dòng là đáp số của một truy vấn.

TOHOP.INP	TOHOP.OUT	Giải thích
4 5 2 6 3 10 0 3 5	10 20 1 0	C(5, 2) = 10, C(6, 3) = 20, C(10, 0) = 1; k > n nên C(3, 5) = 0.
3 1000000 500000 1000000 1 1000000 999999	996692777 1000000 1000000	$C(10^6, 5 \cdot 10^5)$ có tới 301 027 chữ số, ta chỉ in số dư. C(n, 1) = C(n, n − 1) = n.
2 45 6 0 0	8145060 1	Số vé xổ số 6/45. C(0, 0) = 1: chỉ có một cách chọn "không gì cả" từ tập rỗng.

4. Ý tưởng ngây thơ: Dựng cả tam giác Pascal

Hằng đẳng thức Pascal chỉ dùng **phép cộng**, mà phép cộng thì lấy dư thoải mái (Bài 1). Vậy ta dựng tam giác Pascal theo mô-đun p, từ hàng 0 đến hàng mx là n lớn nhất trong các truy vấn, rồi tra bảng. Không có phép chia nào, không cần biết p có nguyên tố hay không.



Hình 13.3. Sơ đồ khối của cách dựng tam giác Pascal.

TOHOP.CPP (ngây thơ): tam giác Pascal theo mô-đun

```cpp
#include <bits/stdc++.h>
using namespace std;
const int MOD = 1000000007;

int main() {
    if (fopen("TOHOP.INP", "r")) {
        freopen("TOHOP.INP", "r", stdin);
        freopen("TOHOP.OUT", "w", stdout);
    }
    int q;
    scanf("%d", &q);
    vector<int> qn(q), qk(q);
    int mx = 0;
    for (int i = 0; i < q; i++) {
        scanf("%d %d", &qn[i], &qk[i]);
        mx = max(mx, qn[i]);
    }
    // c[n][k] = C(n, k) mod MOD: dựng cả tam giác Pascal tới hàng mx
    vector<vector<int>> c(mx + 1);
    for (int n = 0; n <= mx; n++) {
        c[n].assign(n + 1, 1);                  // hai đầu hàng bằng 1
        for (int k = 1; k < n; k++)
            c[n][k] = (c[n - 1][k - 1] + c[n - 1][k]) % MOD;
    }
    for (int i = 0; i < q; i++) {
        int n = qn[i], k = qk[i];
        printf("%d\n", (k < 0 || k > n) ? 0 : c[n][k]);
    }
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Hàng n có n + 1 ô, mỗi ô một phép cộng, nên dựng tới hàng mx tốn khoảng $mx^2 / 2$ phép cộng và **cùng chừng ấy ô nhớ**. Với mx = 5000: 12,5 triệu ô, chạy tốt. Với mx = 10^6: khoảng $5 \cdot 10^{11}$ ô, tức **2 TB** bộ nhớ nếu mỗi ô 4 byte, và hàng trăm giây tính toán.

BẾ TẮC

Tam giác Pascal tính **tất cả** các C(n, k) trong khi ta chỉ cần 10^5 giá trị. Công thức n! / (k! · (n − k)!) thì chỉ cần ba giai thừa, và mọi giai thừa tới 10^6 có thể tính sẵn trong O(n). Nhưng công thức có **phép chia**, mà Bài 1 đã cảnh báo: đồng dư không giữ qua phép chia. 720 mod 7 = 6 và 6 · 6 mod 7 = 1, ta không thể "chia 6 cho 1" để ra C(6, 3) = 20 được.

5. Phá bế tắc: Giai thừa và nghịch đảo giai thừa

Bước 1 Đổi phép chia thành phép nhân với nghịch đảo

Ở Bài 10 em đã biết: khi p nguyên tố và b không chia hết cho p, tồn tại duy nhất **nghịch đảo** b^{-1} (mod p) thỏa $b \cdot b^{-1} \equiv 1$, và theo định lí Fermat nhỏ $b^{-1} \equiv b^{p-2}$ (mod p). Chia cho b theo mô-đun p nghĩa là nhân với b^{-1}. Ví dụ $2^{-1} \equiv 500\,000\,004$ vì $2 \cdot 500\,000\,004 = 1\,000\,000\,008 \equiv 1$ (mod $10^9 + 7$).

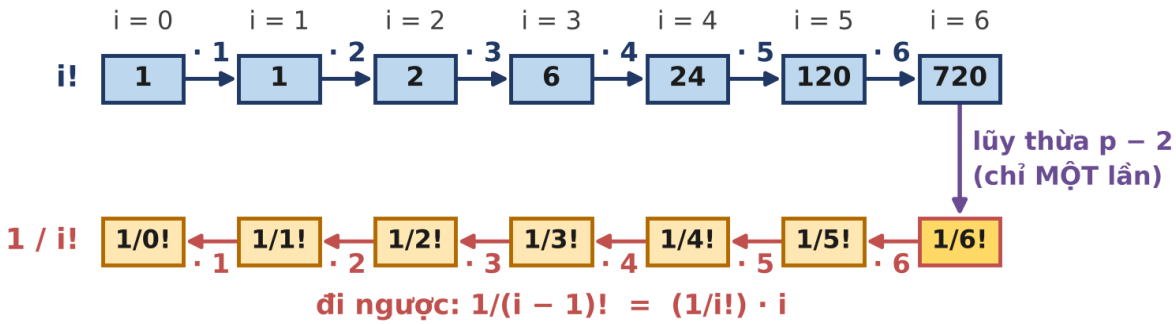
Với $p = 10^9 + 7 > 10^6 \geq n$, các thừa số 1, 2, ..., n đều nhỏ hơn p nên **không chia hết cho p**, và k!, (n − k)! có nghịch đảo. Đặt fact[i] = i! mod p và ifact[i] = $(i!)^{-1}$ mod p, ta có:

$$\mathbf{C(n, k) \equiv fact[n] \cdot ifact[k] \cdot ifact[n - k] \pmod p}$$

Bước 2 Tính giai thừa đi xuôi, nghịch đảo đi ngược

fact[i] = fact[i − 1] · i mod p: dễ. Còn ifact[i], nếu dùng lũy thừa nhanh cho từng i thì tốn N · log p, khoảng $3 \cdot 10^7$ phép nhân, cũng được, nhưng có một mẹo đẹp hơn nhiều: chỉ cần **một** lần lũy thừa cho ifact[N], rồi **đi ngược** về 0.

$$\mathbf{ifact[i - 1] = ifact[i] \cdot i \pmod p}$$



Hình 13.4. *Giai thừa đi xuôi (nhân thêm i); nghịch đảo giai thừa đi ngược (cũng nhân với i), chỉ một lần lũy thừa.*

VÌ SAO ĐÚNG?

Vì i! = (i − 1)! · i, nên 1 / (i − 1)! = i / i! = (1 / i!) · i. Viết theo mô-đun: ifact[i] · i · fact[i − 1] ≡ ifact[i] · fact[i] ≡ 1 (mod p), tức ifact[i] · i đúng là nghịch đảo của fact[i − 1]. Theo tính duy nhất của nghịch đảo, ifact[i − 1] = ifact[i] · i mod p.

Bước 3 Chạy tay với $p = 10^9 + 7$

Bảng dưới được sinh bằng chính thuật toán: fact đi xuôi từ 0 đến 6; ifact[6] = 720^{p-2} mod p = 301 388 891, rồi ifact đi ngược. Cột cuối kiểm tra fact[i] · ifact[i] ≡ 1.

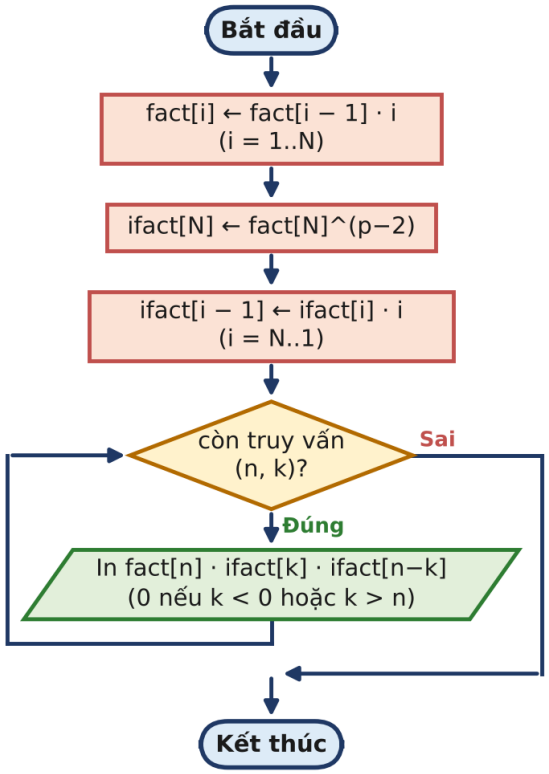
i	fact[i]	ifact[i]	fact[i] · ifact[i] mod p
0	1	1	1
1	1	1	1
2	2	500 000 004	1
3	6	166 666 668	1
4	24	41 666 667	1
5	120	808 333 339	1
6	720	**301 388 891** (lũy thừa)	1

Trả lời các truy vấn của ví dụ 1:

(n, k)	fact[n] · ifact[k] mod p	· ifact[n − k] mod p	Đáp số
(5, 2)	120 · 500 000 004 → 60	60 · 166 666 668 → 10	**10**
(6, 3)	720 · 166 666 668 → 120	120 · 166 666 668 → 20	**20**
(10, 0)	3 628 800 · 1 → 3 628 800	· ifact[10] → 1	**1**
(3, 5)	k > n		**0**

Để ý dòng đầu: nhân 120 với nghịch đảo của 2 ra 60, đúng như 120 / 2; nhân tiếp với nghịch đảo của 6 ra 10, đúng như 60 / 6. Các con số nghịch đảo trông "kì quặc" nhưng chúng cư xử y như phân số.

Bước 4 Chương trình



Hình 13.5. *Sơ đồ khối của cách dùng giai thừa và nghịch đảo giai thừa.*

TOHOP.CPP: giai thừa và nghịch đảo giai thừa

```cpp
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007;
const int N = 1000000;
long long fact[N + 1], ifact[N + 1];

long long power(long long a, long long b) {   // a^b mod MOD (Bài 9)
    long long r = 1;
    a %= MOD;
    while (b > 0) {
        if (b & 1) r = r * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return r;
}

long long C(int n, int k) {
    if (k < 0 || k > n) return 0;
    return fact[n] * ifact[k] % MOD * ifact[n - k] % MOD;
}

int main() {
    if (fopen("TOHOP.INP", "r")) {
        freopen("TOHOP.INP", "r", stdin);
        freopen("TOHOP.OUT", "w", stdout);
    }
    fact[0] = 1;
    for (int i = 1; i <= N; i++) fact[i] = fact[i - 1] * i % MOD;
    ifact[N] = power(fact[N], MOD - 2);        // Fermat nhỏ (Bài 10)
    for (int i = N; i >= 1; i--)               // đi ngược: 1/(i-1)!
        ifact[i - 1] = ifact[i] * i % MOD;     //   = 1/i! * i
    int q;
    scanf("%d", &q);
    while (q--) {
        int n, k;
        scanf("%d %d", &n, &k);
        printf("%lld\n", C(n, k));
    }
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

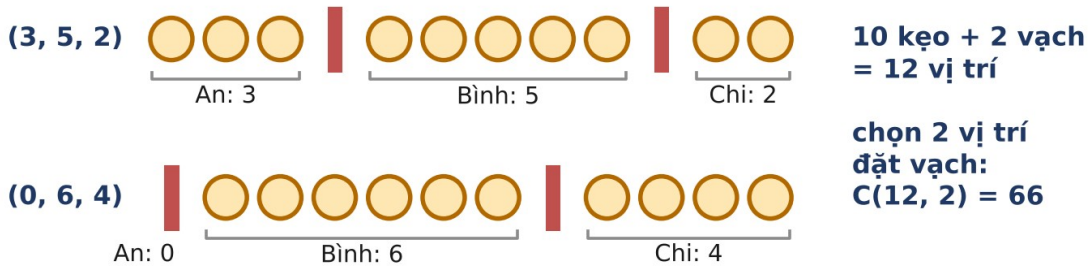
Chuẩn bị: N phép nhân cho fact, một lần lũy thừa (khoảng $2 \cdot 30$ phép nhân), N phép nhân cho ifact: tổng $O(N + \log p)$ với $N = 10^6$. Mỗi truy vấn: 2 phép nhân, O(1). Cả chương trình khoảng $2 \cdot 10^6 + 2 \cdot 10^5$ phép nhân, chạy dưới 0,1 giây; bộ nhớ hai mảng 10^6 số `long long`, 16 MB.

	Tam giác Pascal	Giai thừa + nghịch đảo
Thời gian chuẩn bị	$O(n^2)$: $5 \cdot 10^{11}$	$O(n + \log p)$: $2 \cdot 10^6$
Bộ nhớ	$O(n^2)$: 2 TB	O(n): 16 MB
Mỗi truy vấn	O(1)	O(1)
Yêu cầu mô-đun	bất kì	p nguyên tố, p > n

6. Mở rộng

Mở rộng 1: Bài toán chia kẹo của Euler

Có bao nhiêu cách chia 10 cái kẹo giống nhau cho 3 bạn An, Bình, Chi (bạn nào cũng có thể không được cái nào)? Mẹo của Euler: xếp 10 cái kẹo thành hàng và đặt thêm **2 vạch ngăn**. Kẹo trước vạch thứ nhất là của An, giữa hai vạch là của Bình, sau vạch thứ hai là của Chi. Mỗi cách chia ứng với đúng một cách chọn 2 vị trí làm vạch trong 12 vị trí.



Hình 13.6. *Mỗi cách chia kẹo là một cách đặt 2 vạch giữa 10 viên kẹo.*

ĐỊNH LÍ (CHIA KẸO)

Số nghiệm nguyên **không âm** của $x_1 + x_2 + ... + x_k = n$ là $C(n + k - 1, k - 1)$. Số nghiệm nguyên **dương** (mỗi bạn ít nhất một kẹo) là $C(n - 1, k - 1)$: phát trước mỗi bạn một cái rồi chia $n - k$ cái còn lại.

Với 10 kẹo, 3 bạn: $C(12, 2) = 66$ cách; nếu mỗi bạn ít nhất một cái: $C(9, 2) = 36$ cách. Cả hai đều trả lời được bằng TOHOP.

Mở rộng 2: n rất lớn, k nhỏ: nhân dần

Nếu n tới 10^{18} nhưng k chỉ tới 10^6, ta không thể lập bảng giai thừa tới n. Khi đó dùng trực tiếp $C(n, k) = n \cdot (n - 1) \cdot ... \cdot (n - k + 1) / k!$: tử số là tích k thừa số (lấy dư từng thừa số), mẫu số k! có nghịch đảo vì $k < p$. Công thức đúng cả khi $n \geq p$, vì đẳng thức $C(n, k) \cdot k! = n(n - 1)...(n - k + 1)$ là đẳng thức số nguyên. Mỗi truy vấn tốn $O(k + \log p)$.

TOHOP_NHANDAN.CPP: C(n, k) bằng tích k thừa số

```cpp
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007;

long long power(long long a, long long b) {
    long long r = 1;
    a %= MOD;
    while (b > 0) {
        if (b & 1) r = r * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return r;
}

// C(n, k) = n(n-1)...(n-k+1) / k!, mỗi truy vấn O(k + log MOD)
// dùng được cả khi n rất lớn (tới 10^18), miễn là k < MOD
long long C(long long n, long long k) {
    if (k < 0 || k > n) return 0;
    long long tu = 1, mau = 1;
    for (long long i = 0; i < k; i++) {
        tu = tu * ((n - i) % MOD) % MOD;
        mau = mau * ((i + 1) % MOD) % MOD;
    }
    return tu * power(mau, MOD - 2) % MOD;
}

int main() {
    if (fopen("TOHOP.INP", "r")) {
        freopen("TOHOP.INP", "r", stdin);
        freopen("TOHOP.OUT", "w", stdout);
    }
    int q;
    scanf("%d", &q);
    while (q--) {
        long long n, k;
        scanf("%lld %lld", &n, &k);
        if (k >= 0 && k <= n) k = min(k, n - k);   // C(n,k) = C(n,n-k)
        printf("%lld\n", C(n, k));
    }
    return 0;
}
```

Mở rộng 3: Định lí Lucas khi p nhỏ

Khi mô-đun p là số nguyên tố **nhỏ** (ví dụ p = 7 hay p = 10 007) mà n lớn hơn p, cách giai thừa hỏng vì n! chia hết cho p. Định lí Lucas cứu ta: viết n và k trong **hệ cơ số p**.

ĐỊNH LÍ (LUCAS)

Nếu $n = (n_t \dots n_1\, n_0)_p$ và $k = (k_t \dots k_1\, k_0)_p$ là biểu diễn cơ số p (p nguyên tố) thì:

$$\mathbf{C(n, k) \equiv C(n_t, k_t) \cdot \dots \cdot C(n_1, k_1) \cdot C(n_0, k_0) \pmod p}$$

trong đó C(a, b) = 0 nếu b > a. Mỗi thừa số có a, b < p, tính bằng giai thừa tới p − 1.

Ví dụ: C(1000, 500) mod 7. Ta có 1000 = $(2626)_7$, 500 = $(1313)_7$, nên C(1000, 500) ≡ C(2, 1) · C(6, 3) · C(2, 1) · C(6, 3) = 2 · 20 · 2 · 20 = 1600 ≡ **4** (mod 7). Với n = 10^{18} chỉ có khoảng $\log_p n$ chữ số, nên mỗi truy vấn rất nhanh sau O(p) chuẩn bị.

TOHOP_LUCAS.CPP: định lí Lucas, đọc n, k, p

```cpp
#include <bits/stdc++.h>
using namespace std;
long long p;
vector<long long> f, inf;   // giai thừa và nghịch đảo, chỉ tới p - 1

long long power(long long a, long long b) {
    long long r = 1;
    a %= p;
    while (b > 0) {
        if (b & 1) r = r * a % p;
        a = a * a % p;
        b >>= 1;
    }
    return r;
}

long long Csmall(long long n, long long k) {   // 0 <= n < p
    if (k > n) return 0;
    return f[n] * inf[k] % p * inf[n - k] % p;
}

int main() {
    if (fopen("TOHOP.INP", "r")) {
        freopen("TOHOP.INP", "r", stdin);
        freopen("TOHOP.OUT", "w", stdout);
    }
    long long n, k;
    cin >> n >> k >> p;          // n, k <= 10^18; p nguyên tố <= 10^6
    f.assign(p, 1);
    inf.assign(p, 1);
    for (long long i = 1; i < p; i++) f[i] = f[i - 1] * i % p;
    inf[p - 1] = power(f[p - 1], p - 2);
    for (long long i = p - 1; i >= 1; i--) inf[i - 1] = inf[i] * i % p;
    long long r = (k <= n) ? 1 : 0;
    while (r && (n > 0 || k > 0)) {  // tách chữ số cơ số p của n, k
        r = r * Csmall(n % p, k % p) % p;
        n /= p;
        k /= p;
    }
    cout << r << "\n";
    return 0;
}
```

Cách tính C(n, k) mod m	Dùng khi	Độ phức tạp
Tam giác Pascal	n ≤ khoảng 5000, m bất kì	O(n^2) chuẩn bị, O(1) mỗi truy vấn
Giai thừa + nghịch đảo	n ≤ 10^7, m = p nguyên tố > n	O(n + log p) chuẩn bị, O(1)
Nhân dần	n tới 10^{18}, k nhỏ, p nguyên tố > k	O(k + log p) mỗi truy vấn
Định lí Lucas	p nguyên tố nhỏ, n tới 10^{18}	O(p) chuẩn bị, O($\log_p$ n)

Mở rộng 4: Số Catalan

Có bao nhiêu dãy gồm n cặp ngoặc "đúng" (mỗi tiền tố có số ngoặc mở không ít hơn số ngoặc đóng)? Với n = 3 có 5 dãy: ((())), (()()), (())(), ()(()), ()()(). Đáp số tổng quát là **số Catalan** $C_n = C(2n, n) / (n + 1) = C(2n, n) − C(2n, n + 1)$; dãy bắt đầu 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862. Số Catalan cũng đếm số cây nhị phân n nút, số cách chia đa giác lồi n + 2 đỉnh thành tam giác... Với TOHOP em tính được C_n mod p trong O(1): fact[2n] · ifact[n] · ifact[n + 1].

BẪY THƯỜNG GẶP

- **Chia thẳng theo mô-đun:** `fact[n] / (fact[k] * fact[n-k]) % p` cho kết quả vô nghĩa, vì fact[] đã là số dư. Phải nhân với nghịch đảo.
- **Quên kiểm tra k > n hoặc k < 0:** truy cập ifact[n − k] với chỉ số âm gây lỗi bộ nhớ. Luôn trả 0 trước.
- **Tràn khi nhân ba số:** `fact[n] * ifact[k] * ifact[n-k] % p` tràn `long long` vì tích ba số cỡ 10^9 tới 10^{27}. Lấy dư sau **mỗi** phép nhân.
- **Mô-đun không nguyên tố hoặc p ≤ n:** khi đó k! có thể không có nghịch đảo. Dùng tam giác Pascal (n nhỏ) hoặc Lucas (p nhỏ).
- **Nhầm có thứ tự và không thứ tự:** chọn đội thì dùng C(n, k), xếp lượt thi thì dùng A(n, k) = C(n, k) · k!.

GHI NHỚ

- C(n, k) = n! / (k! · (n − k)!) = số tập con k phần tử của tập n phần tử; C(n, k) = 0 khi k < 0 hoặc k > n.
- Hằng đẳng thức Pascal C(n, k) = C(n − 1, k − 1) + C(n − 1, k): chứng minh bằng cách xét một phần tử có được chọn hay không.
- Với p nguyên tố > n: C(n, k) ≡ fact[n] · ifact[k] · ifact[n − k]; ifact[N] = fact[N]^{p-2}, ifact[i − 1] = ifact[i] · i.
- Chia kẹo: số nghiệm không âm của $x_1 + ... + x_k = n$ là $C(n + k − 1, k − 1)$.
- p nhỏ, n lớn: định lí Lucas, tách n, k theo cơ số p.

Luyện tập

Bài	Tên	Gợi ý liên hệ
13.1	CHIA KẸO	Đếm số cách chia n kẹo cho k bạn (có/không yêu cầu mỗi bạn ít nhất một cái), mod $10^9 + 7$: chia kẹo Euler + TOHOP.
13.2	ĐƯỜNG ĐI TRÊN LƯỚI	Số đường đi từ (0, 0) tới (m, n) chỉ đi phải/lên là C(m + n, n); có một ô cấm thì trừ số đường đi qua ô đó (quy tắc nhân).
13.3	TỔ HỢP KHỔNG LỒ	C(n, k) mod p với n, k ≤ 10^{18}, p nguyên tố ≤ 10^5, nhiều truy vấn: định lí Lucas.

Tự kiểm tra nhanh

1) Có bao nhiêu cách chọn 3 bạn trong 30 bạn làm đội tuyển? Bao nhiêu cách nếu 3 bạn thi theo lượt 1, 2, 3? *(Đáp số: $C(30, 3) = 4060$ và $A(30, 3) = 24\,360$)*

2) Từ hàng 7 của tam giác Pascal (1, 7, 21, 35, 35, 21, 7, 1), hãy tính $C(8, 3)$ và tổng các số ở hàng 8. *(Đáp số: $C(8, 3) = 21 + 35 = 56$; tổng hàng 8 là $2^8 = 256$)*

3) Chia 10 cái kẹo giống nhau cho 3 bạn sao cho bạn nào cũng có ít nhất 1 cái. Có bao nhiêu cách? *(Đáp số: $C(9, 2) = 36$)*

4) Dùng định lí Lucas, tính $C(1000, 500) \bmod 7$ (gợi ý $1000 = (2626)_7$, $500 = (1313)_7$). *(Đáp số: $2 \cdot 20 \cdot 2 \cdot 20 = 1600 \equiv 4 \pmod 7$)*

BÀI 14 Nguyên lí bù trừ

Học xong bài này, em sẽ:

- ✓ Phát biểu và chứng minh **nguyên lí bù trừ** cho 2, 3 và k tập hợp.
- ✓ Đếm số bội số trong [1, N] bằng ⌊N / a⌋, biết vì sao bội chung của a, b là bội của **lcm(a, b)** chứ không phải của a · b.
- ✓ Duyệt 2^k tập con bằng **mặt nạ bit**, tính lcm của mỗi tập con chỉ bằng một phép gcd, và **chặn tràn số** khi lcm vượt N.
- ✓ Áp dụng bù trừ để đếm số nguyên tố cùng nhau với n và số hoán vị không điểm cố định.

KHỞI ĐỘNG

Lớp 10 Tin có 100 học sinh. Thống kê cho thấy 60 em thích Toán, 50 em thích Tin, và 30 em thích cả hai. Hỏi có bao nhiêu em thích ít nhất một trong hai môn? Cộng 60 + 50 = 110 thì vượt cả sĩ số! Lỗi ở đâu? Ba mươi em thích cả hai đã bị **đếm hai lần**: đáp số đúng là 60 + 50 − 30 = 80.

Ý tưởng "cộng rồi trừ phần đếm trùng" nghe đơn giản, nhưng với ba, bốn, ... hai mươi tập hợp thì phần trùng chồng lên nhau rất rối. Bài này cho em quy luật chung, rồi dùng nó trả lời câu hỏi: trong các số từ 1 đến 10^{18}, có bao nhiêu số chia hết cho ít nhất một trong 20 số cho trước?

1. Đếm bội số trong một đoạn

Trong các số 1, 2, ..., N, các bội của a là a, 2a, 3a, ..., q · a với q · a ≤ N < (q + 1) · a. Theo định nghĩa phép chia có dư (Bài 1), q chính là thương ⌊N / a⌋. Vậy:

số bội của a trong [1, N] = ⌊N / a⌋

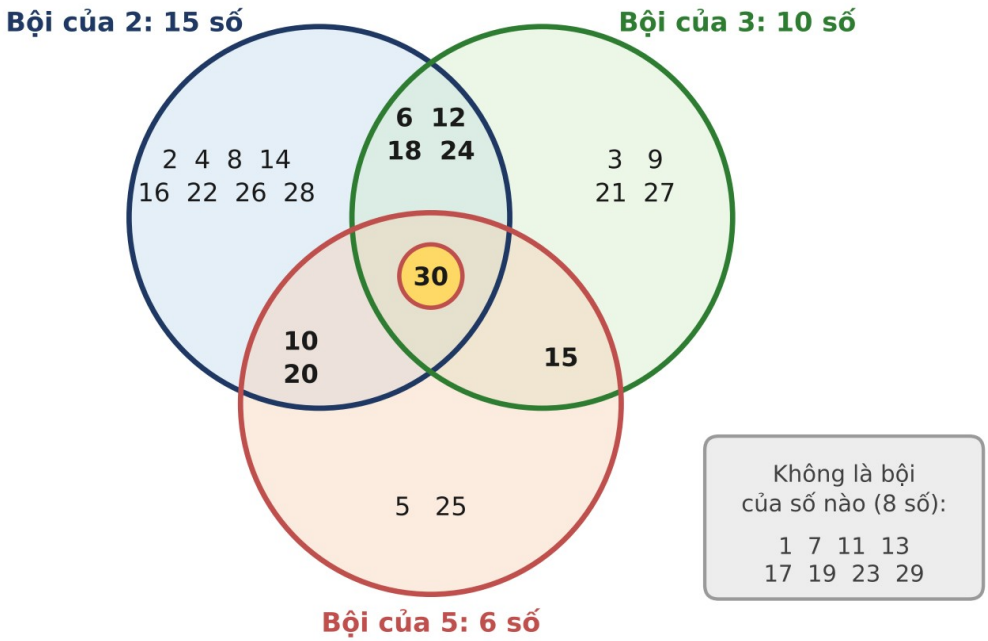
Ví dụ trong [1, 30] có ⌊30 / 2⌋ = 15 số chẵn, ⌊30 / 3⌋ = 10 bội của 3, ⌊30 / 5⌋ = 6 bội của 5. Còn số chia hết cho **cả** 2 và 3? Một số chia hết cho cả a và b khi và chỉ khi nó là bội chung của a và b, tức là bội của **lcm(a, b)** (Bài 3). Với 2 và 3, lcm = 6, nên có ⌊30 / 6⌋ = 5 số: 6, 12, 18, 24, 30.

LCM, KHÔNG PHẢI TÍCH

Chỉ khi a, b nguyên tố cùng nhau thì lcm(a, b) = a · b. Số chia hết cho cả 4 và 6 là bội của lcm(4, 6) = 12 chứ không phải của 24: trong [1, 100] có ⌊100 / 12⌋ = **8** số như vậy (12, 24, ..., 96), không phải ⌊100 / 24⌋ = 4.

2. Nguyên lí bù trừ

Kí hiệu |A| là số phần tử của tập A. Câu chuyện lớp 10 Tin chính là công thức cho **hai tập**: |A ∪ B| = |A| + |B| − |A ∩ B|. Với **ba tập**, hãy nhìn vào hình dưới: ba hình tròn là các bội của 2, 3 và 5 trong [1, 30], và mỗi số được viết đúng vào vùng của nó.



Hình 14.1. *Các số 1..30 xếp theo chúng chia hết cho 2, 3, 5 hay không. Có 22 số nằm trong ít nhất một hình tròn.*

Cộng ba tập: 15 + 10 + 6 = 31. Nhưng các số ở vùng giao hai tập (6, 12, 10, 15...) bị đếm hai lần, còn số 30 ở giữa bị đếm **ba** lần. Trừ các giao từng cặp: 31 − 5 − 3 − 2 = 21. Lúc này số 30 đã bị cộng 3 lần rồi trừ 3 lần, tức **không được đếm lần nào**! Cộng lại giao của cả ba: 21 + 1 = **22**. Đếm trực tiếp trên hình: 8 + 4 + 2 + 4 + 2 + 1 + 1 = 22, khớp.

ĐỊNH LÍ (NGUYÊN LÍ BÙ TRỪ)

Với các tập hữu hạn $A_1, A_2, ..., A_k$:

$$|A_1 \cup ... \cup A_k| = \sum |A_i| - \sum |A_i \cap A_j| + \sum |A_i \cap A_j \cap A_l| - ...$$

Nói gọn: lấy tổng theo mọi tập con khác rỗng S của {1, ..., k} của số phần tử của giao các A_i ($i \in S$), mang dấu + khi S có lẻ phần tử, dấu − khi S có chẵn phần tử.

VÌ SAO ĐÚNG?

Xét một phần tử x nằm trong đúng t ≥ 1 tập. Trong tổng trên, x được cộng ở mọi tập con S gồm các tập chứa x: có C(t, 1) tập con 1 phần tử (dấu +), C(t, 2) tập con 2 phần tử (dấu −), ... Tổng số lần x được đếm là:

$$C(t, 1) - C(t, 2) + C(t, 3) - \ldots = 1 - (1 - 1)^t = 1$$

Đẳng thức cuối là nhị thức Newton với a = 1, b = −1 (Bài 13): C(t, 0) − C(t, 1) + C(t, 2) − ... = 0. Vậy mỗi phần tử của hợp được đếm **đúng một lần**, còn phần tử nằm ngoài mọi tập (t = 0) không được đếm lần nào.

Bảng dưới kiểm tra điều đó với ba tập: dù một số nằm trong 1, 2 hay 3 tập, tổng số lần đếm luôn là 1.

Số nằm trong t tập	+ C(t, 1)	− C(t, 2)	+ C(t, 3)	Tổng số lần đếm
t = 1 (ví dụ 4)	1	0	0	**1**
t = 2 (ví dụ 6)	2	−1	0	**1**
t = 3 (ví dụ 30)	3	−3	1	**1**

3. Bài toán BUTRU

BÀI TOÁN: BUTRU

Cho số nguyên dương N và k số nguyên dương $a_1, a_2, \ldots, a_k$. Hãy đếm số các số nguyên x trong đoạn [1, N] chia hết cho **ít nhất một** trong các số a_i.

Dữ liệu vào: Dòng đầu chứa N và k ($1 \le N \le 10^{18}$, $1 \le k \le 20$). Dòng thứ hai chứa $a_1, \ldots, a_k$ ($1 \le a_i \le 10^{18}$, có thể trùng nhau).

Kết quả: Một số nguyên là số lượng cần đếm.

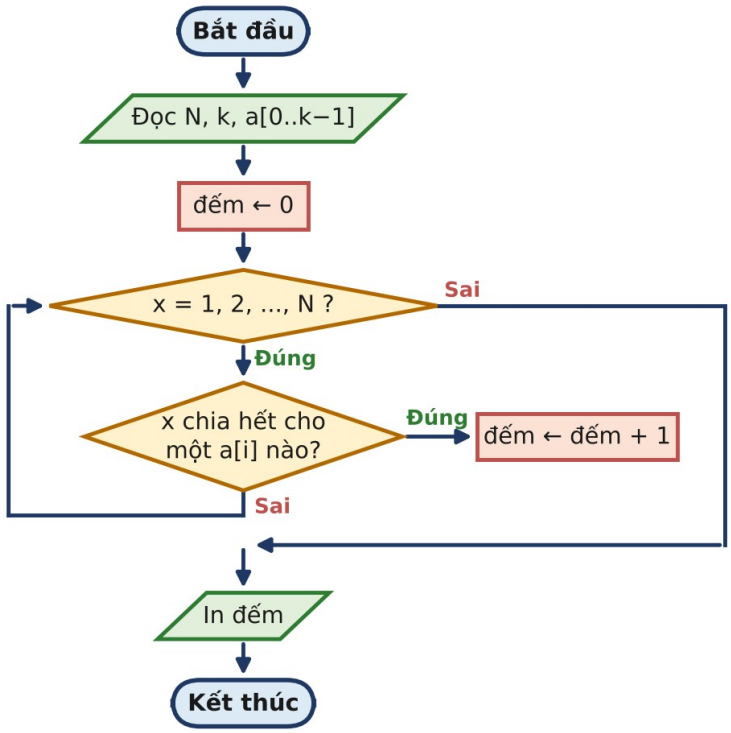
BUTRU.INP	BUTRU.OUT	Giải thích
`30 3` `2 3 5`	`22`	Đúng như Hình 14.1: 15 + 10 + 6 − 5 − 3 − 2 + 1 = 22.

`1000000000000000000`
`3`
`1000000007`
`1000000009`
`998244353`
	`3001758716`	lcm($10^9 + 7$, $10^9 + 9$) đã vượt 10^{18} nên mọi giao chứa cặp này bằng 0; lcm của cả ba số cỡ 10^{27}, nhân thẳng thì tràn số.

`100 3` `4 6 10`	`36`	25 + 16 + 10 − 8 − 5 − 3 + 1 = 36; giao của bội 4 và bội 6 là bội của 12.

4. Ý tưởng ngây thơ: Xét từng số

Cách tự nhiên nhất: đi qua từng x từ 1 đến N, thử chia cho từng a_i; gặp một a_i chia hết x thì đếm x và dừng thử (để mỗi x chỉ được đếm một lần).



Hình 14.2. Sơ đồ khối của cách xét từng số.

BUTRU.CPP (ngây thơ): xét từng x trong [1, N]

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("BUTRU.INP", "r")) {
        freopen("BUTRU.INP", "r", stdin);
        freopen("BUTRU.OUT", "w", stdout);
    }
    long long n;
    int k;
    cin >> n >> k;
    vector<long long> a(k);
    for (auto &x : a) cin >> x;
    long long ans = 0;
    for (long long x = 1; x <= n; x++)      // xét từng số trong [1, n]
        for (int i = 0; i < k; i++)
            if (x % a[i] == 0) {            // x là bội của a[i]
                ans++;
                break;                      // đếm x đúng một lần
            }
    cout << ans << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Tối đa $N \cdot k$ phép chia lấy dư. Với $N = 10^6$, $k = 20$: $2 \cdot 10^7$ phép, chạy tức thì. Với $N = 10^{18}$: $2 \cdot 10^{19}$ phép, máy tính 10^9 phép mỗi giây cần khoảng **600 năm**.

BẾ TẮC

Ta không thể nhìn từng số một. Nhưng câu hỏi "có bao nhiêu bội của L trong [1, N]" thì trả lời được trong **một phép chia**: $\lfloor N / L \rfloor$. Nguyên lí bù trừ biến bài toán về đúng những câu hỏi như vậy, mỗi câu cho một tập con của $\{a_1, ..., a_k\}$. Với $k = 20$ có $2^{20} - 1 = 1\,048\,575$ tập con khác rỗng: nhỏ hơn 10^{18} rất nhiều.

5. Phá bế tắc: Bù trừ trên 2^k tập con

Bước 1 Mỗi tập con cho một số hạng

Gọi A_i là tập các bội của a_i trong [1, N]. Giao các A_i với i thuộc tập con S là tập các bội của L(S) = lcm của các a_i, $i \in S$, nên có $\lfloor N / L(S) \rfloor$ phần tử. Nguyên lí bù trừ cho ngay:

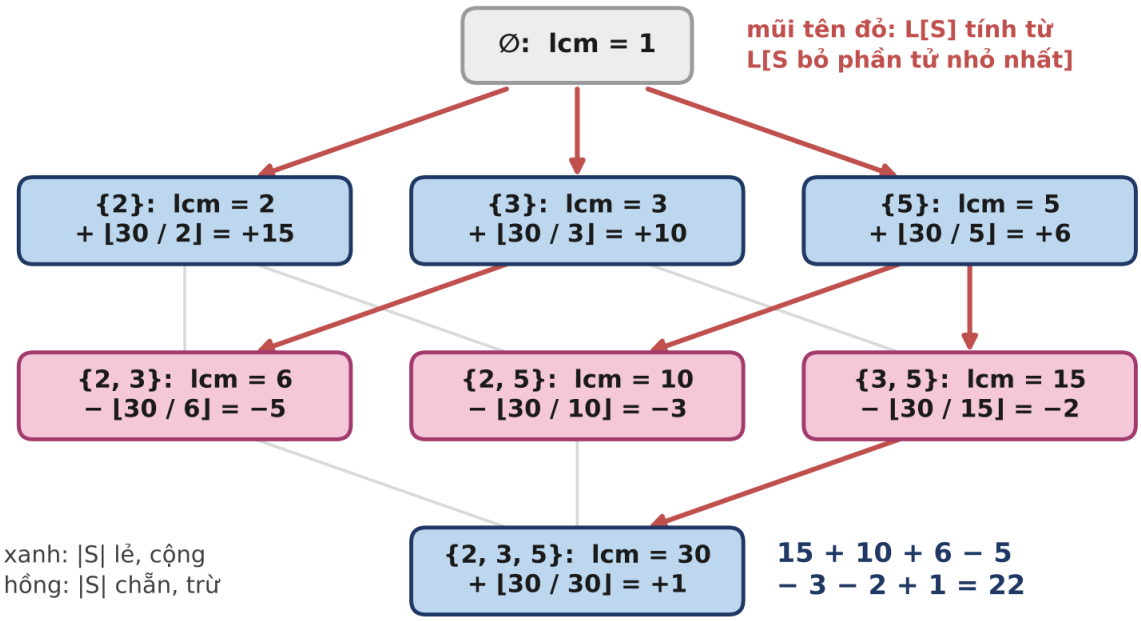
$$\textbf{đáp số} = \Sigma_{S \neq \varnothing} (-1)^{|S|+1} \cdot \lfloor N / L(S) \rfloor$$

Để duyệt mọi tập con, ta dùng **mặt nạ bit**: số nguyên S từ 1 đến $2^k - 1$, bit thứ i của S bằng 1 nghĩa là a_i thuộc tập con (đánh số $a_0, ..., a_{k-1}$ như trong chương trình). Số phần tử |S| là số bit 1, hàm `__builtin_popcount(S)` cho ngay.

Bước 2 Tính lcm của mỗi tập con chỉ bằng một phép gcd

Nếu với mỗi S ta lại tính lcm của cả |S| số thì tốn k phép gcd mỗi tập. Mẹo: gọi i là **bit 1 thấp nhất** của S (`__builtin_ctz(S)`), thì S bỏ đi i là `S & (S - 1)`, một số **nhỏ hơn** S nên đã được tính trước. Vì lcm có tính kết hợp:

$$L[S] = \text{lcm}(L[S \,\&\, (S-1)], a_i)$$



Hình 14.3. *Tám tập con của {2, 3, 5}. Mũi tên đỏ: L[S] được tính từ tập con bỏ phần tử nhỏ nhất. Xanh cộng, hồng trừ.*

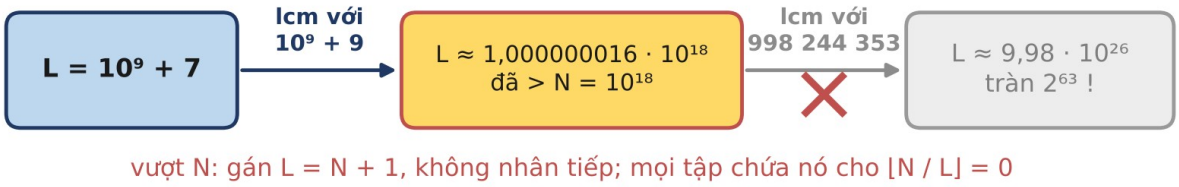
Bước 3 Chạy tay với N = 30, a = (2, 3, 5)

Bảng dưới là kết quả chạy đúng vòng lặp của chương trình (mặt nạ viết bit a_0 ở bên phải):

S	Tập con	Bit i	L[S bỏ i]	L[S]	Dấu	⌊30 / L⌋	ans
001	{2}	0	1	2	+	15	15
010	{3}	1	1	3	+	10	25
011	{2, 3}	0	3	6	−	5	20
100	{5}	2	1	5	+	6	26
101	{2, 5}	0	5	10	−	3	23
110	{3, 5}	1	5	15	−	2	21
111	{2, 3, 5}	0	15	30	+	1	**22**

Bước 4 Chặn tràn số khi lcm vượt N

Ví dụ 2 cho thấy nguy hiểm: lcm của ba số nguyên tố cỡ 10^9 là khoảng 10^{27}, vượt xa 2^{63}. Nhân thẳng thì `long long` tràn và ⌊N / L⌋ ra rác. Nhưng để ý: **một khi L > N thì ⌊N / L⌋ = 0**, và mọi tập lớn hơn có lcm còn lớn hơn (là bội của L), cũng cho 0. Vậy ta chỉ cần biết "L có vượt N không", không cần giá trị thật của L.



Hình 14.4. *Gặp lcm vượt N thì ghi L = N + 1 và không nhân thêm nữa, nên không bao giờ tràn số.*

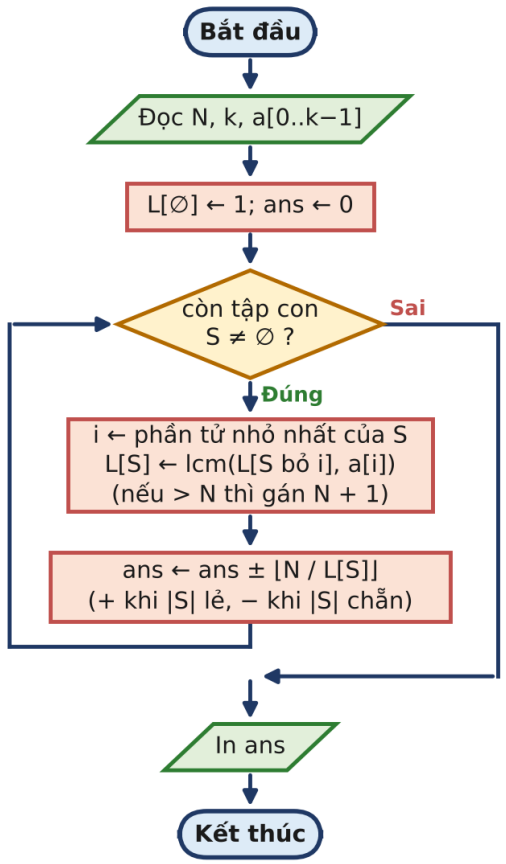
Tính lcm(x, a) = (x / gcd(x, a)) · a. Đặt y = x / gcd(x, a); để biết y · a > N mà **không nhân**, ta so sánh y > ⌊N / a⌋.

VÌ SAO PHÉP SO SÁNH ĐÚNG?

Đặt q = ⌊N / a⌋, tức q · a ≤ N < (q + 1) · a. Nếu y ≤ q thì y · a ≤ q · a ≤ N. Nếu y ≥ q + 1 thì y · a ≥ (q + 1) · a > N. Vậy y · a > N khi và chỉ khi y > q. Mọi phép tính ở đây đều có giá trị không vượt quá N ≤ 10^{18}, an toàn trong `long long`; và khi y · a ≤ N thì phép nhân cũng không tràn.

Còn giá trị gán N + 1 ≤ 10^{18} + 1 vẫn vừa `long long`; mọi tập chứa S kế thừa giá trị này qua dòng `if (x > n)`.

Bước 5 Chương trình



Hình 14.5. *Sơ đồ khối của cách bù trừ trên các tập con.*

BUTRU.CPP: bù trừ trên mọi tập con, chặn tràn lcm

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("BUTRU.INP", "r")) {
        freopen("BUTRU.INP", "r", stdin);
        freopen("BUTRU.OUT", "w", stdout);
    }
    long long n;
    int k;
    cin >> n >> k;
    vector<long long> a(k);
    for (auto &x : a) cin >> x;
    // L[S] = lcm các a[i] với i thuộc tập con S, hoặc n + 1 nếu > n
    vector<long long> L(1 << k);
    L[0] = 1;
    long long ans = 0;
    for (int S = 1; S < (1 << k); S++) {
        int i = __builtin_ctz(S);          // phần tử nhỏ nhất của S
        long long x = L[S & (S - 1)];      // lcm của S khi bỏ i đi
        if (x > n) L[S] = n + 1;           // đã vượt n thì mãi vượt n
        else {
            long long y = x / __gcd(x, a[i]);
            // y * a[i] > n  <=>  y > n / a[i]: so sánh trước khi nhân
            L[S] = (y > n / a[i]) ? n + 1 : y * a[i];
        }
        long long cnt = n / L[S];          // số bội của L[S] trong [1, n]
        if (__builtin_popcount(S) % 2 == 1) ans += cnt;
        else ans -= cnt;
    }
    cout << ans << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Mỗi tập con: một phép gcd (không quá khoảng 90 bước với số 64 bit, theo định lí Lamé ở Bài 3), một phép chia và một phép so sánh. Tổng $O(2^k \cdot \log N)$: với k = 20 khoảng 10^6 tập con, chạy trong vài phần trăm giây. Mảng L có 2^{20} phần tử `long long`, 8 MB. So với cách ngây thơ, ta đã đổi sự phụ thuộc vào **N** (10^{18}) thành phụ thuộc vào **k** ($2^{20} \approx 10^6$).

6. Mở rộng

Mở rộng 1: Cắt nhánh bằng đệ quy

Khi các a_i khá lớn, phần lớn tập con có lcm vượt N. Thay vì duyệt cả 2^k mặt nạ, ta xây tập con bằng **đệ quy**: thêm dần a_j với chỉ số tăng dần, và khi lcm vượt N thì **bỏ cả nhánh** (mọi tập chứa nó đều cho 0). Ta cũng bỏ trước các số trùng nhau, vì chúng không làm thay đổi tập các bội. Trường hợp xấu nhất vẫn là 2^k, nhưng thường nhanh hơn rất nhiều và không cần mảng 2^k phần tử.

BUTRU_DFS.CPP: bù trừ bằng đệ quy có cắt nhánh

```cpp
#include <bits/stdc++.h>
using namespace std;
long long n, ans = 0;
int k;
vector<long long> a;

// đã chọn một tập con có lcm = l, dấu sg; thử thêm a[j] với j >= i
void dfs(int i, long long l, int sg) {
    for (int j = i; j < k; j++) {
        long long y = l / __gcd(l, a[j]);
        if (y > n / a[j]) continue;        // lcm > n: cắt cả nhánh
        long long nl = y * a[j];
        ans += sg * (n / nl);
        dfs(j + 1, nl, -sg);
    }
}

int main() {
    if (fopen("BUTRU.INP", "r")) {
        freopen("BUTRU.INP", "r", stdin);
        freopen("BUTRU.OUT", "w", stdout);
    }
    cin >> n >> k;
    a.resize(k);
    for (auto &x : a) cin >> x;
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());  // bỏ số trùng
    k = a.size();
    dfs(0, 1, 1);
    cout << ans << "\n";
    return 0;
}
```

Mở rộng 2: Đếm số nguyên tố cùng nhau với n

Có bao nhiêu x trong [1, N] nguyên tố cùng nhau với n? Phân tích n ra thừa số nguyên tố (Bài 7) được các ước nguyên tố **phân biệt** $p_1, ..., p_k$. Số x **không** nguyên tố cùng nhau với n là số x chia hết cho ít nhất một p_i: chính là BUTRU với a = $(p_1, ..., p_k)$. Đáp số là N trừ đi số đó. Vì tích 15 số nguyên tố đầu tiên đã là 614 889 782 588 491 410 ≈ $6{,}1 \cdot 10^{17}$, mọi $n \le 10^{18}$ có **không quá 15** ước nguyên tố phân biệt: tối đa 2^{15} tập con.

Ví dụ n = 30, N = 100: số bội của 2, 3 hoặc 5 là 50 + 33 + 20 − 16 − 10 − 6 + 3 = 74, nên có 100 − 74 = **26** số nguyên tố cùng nhau với 30. Khi N = n, đáp số chính là φ(n) của Bài 11: với n = 30 là 30 − 22 = 8 (đúng tám số nằm ngoài ba hình tròn ở Hình 14.1). Công thức tích $\varphi(n) = n \cdot \Pi(1 - 1/p)$ thực ra là nguyên lí bù trừ viết gọn.

Mở rộng 3: Hoán vị không điểm cố định

Cuối năm, n bạn bỏ quà vào một hộp rồi mỗi bạn rút ngẫu nhiên một món. Có bao nhiêu cách rút để **không bạn nào nhận lại quà của mình**? Đó là số hoán vị không có điểm cố định, kí hiệu D(n). Gọi A_i là tập các hoán vị giữ nguyên vị trí i; giao của j tập A_i có (n − j)! phần tử, và có C(n, j) cách chọn j vị trí. Bù trừ cho:

$$\mathbf{D(n) = \Sigma_{j=0..n}\,(-1)^j \cdot C(n, j) \cdot (n - j)! = n! \cdot (1 - 1/1! + 1/2! - ... \pm 1/n!)}$$

Ví dụ D(4) = 24 − 24 + 12 − 4 + 1 = 9. Khi lập trình, dùng hệ thức truy hồi D(n) = (n − 1) · (D(n − 1) + D(n − 2)) với D(0) = 1, D(1) = 0 để tính mọi D(n) mod p trong O(n). Tỉ lệ D(n) / n! tiến rất nhanh về 1/e ≈ 0,3679: dù có bao nhiêu bạn, xác suất không ai nhận lại quà của mình luôn xấp xỉ 37%.

n	1	2	3	4	5	6	7
D(n)	0	1	2	9	44	265	1854
n!	1	2	6	24	120	720	5040
D(n) / n!	0	0,5	0,333	0,375	0,3667	0,36806	0,36786

BẪY THƯỜNG GẶP

- **Giao là bội của tích:** bội chung của a, b là bội của lcm(a, b), không phải a · b. Chỉ bằng nhau khi gcd(a, b) = 1.
- **Tràn số khi tính lcm:** tích (x / g) · a có thể vượt 2^{63}. Luôn so sánh x / g > N / a trước khi nhân, gặp lcm > N thì gán N + 1.
- **Nhầm dấu:** tập con có lẻ phần tử mang dấu +, chẵn phần tử mang dấu −. Đổi ngược sẽ ra số âm.
- **Dùng `1 << k` với k lớn:** với k = 20 thì 2^{20} ổn, nhưng nếu k tới 60 thì bù trừ thuần túy không khả thi; cần cắt nhánh hoặc ý tưởng khác.
- **Ngây thơ đếm trùng:** quên `break` sau khi gặp một a_i chia hết x thì x được đếm nhiều lần.

GHI NHỚ

- Số bội của a trong [1, N] là ⌊N / a⌋; bội chung của các a_i là bội của lcm của chúng.
- |A ∪ B ∪ C| = |A| + |B| + |C| − |A ∩ B| − |A ∩ C| − |B ∩ C| + |A ∩ B ∩ C|; tổng quát: dấu $(-1)^{|S|+1}$ theo mọi tập con S.
- Chứng minh: phần tử thuộc t tập được đếm C(t, 1) − C(t, 2) + ... = 1 lần.
- Duyệt tập con bằng mặt nạ bit; L[S] = lcm(L[S & (S − 1)], a[ctz(S)]); lcm vượt N thì chặn ở N + 1.

Luyện tập

Bài	Tên	Gợi ý liên hệ
14.1	**BỘI SỐ**	Đếm số trong [L, R] chia hết cho a hoặc b nhưng không chia hết cho c: đếm trên [1, R] trừ [1, L − 1], bù trừ với 3 tập.
14.2	**NGUYÊN TỐ CÙNG NHAU**	Q truy vấn: đếm x ≤ N nguyên tố cùng nhau với n (n, N ≤ 10^{12}): phân tích n, bù trừ trên các ước nguyên tố.
14.3	**TẶNG QUÀ**	Số hoán vị của n phần tử có đúng m điểm cố định, mod $10^9 + 7$: C(n, m) · D(n − m), dùng TOHOP và truy hồi D(n).

Tự kiểm tra nhanh

1) Trong các số từ 1 đến 1000 có bao nhiêu số chia hết cho ít nhất một trong các số 3, 5, 7? *(Đáp số: 333 + 200 + 142 − 66 − 47 − 28 + 9 = 543)*

2) Có bao nhiêu số trong [1, 100] nguyên tố cùng nhau với 30? *(Đáp số: 100 − 74 = 26)*

3) Năm bạn đổi quà sao cho không ai nhận lại quà của mình. Có bao nhiêu cách? *(Đáp số: D(5) = 120 − 120 + 60 − 20 + 5 − 1 = 44)*

4) Với k = 20, chương trình BUTRU duyệt bao nhiêu tập con khác rỗng? Vì sao trong [1, 100] có 8 số chia hết cho cả 4 và 6? *(Đáp số: $2^{20} − 1 = 1\,048\,575$; vì chúng là bội của lcm(4, 6) = 12)*

BÀI 15 Dãy truy hồi và lũy thừa ma trận

Học xong bài này, em sẽ:

- ✓ Hiểu **dãy truy hồi** qua dãy Fibonacci và bài toán lát gạch; biết vì sao F_n tăng theo cấp số nhân.
- ✓ Biết nhân hai ma trận và viết một bước truy hồi thành **phép nhân ma trận** với một vectơ trạng thái.
- ✓ Chứng minh $Q^n = [[F_{n+1}, F_n], [F_n, F_{n-1}]]$ và dùng **lũy thừa nhanh** (Bài 9) để tính $F_n \bmod m$ với $n \le 10^{18}$ trong $O(\log n)$.
- ✓ Mở rộng cho truy hồi tuyến tính bậc k và đếm đường đi trên đồ thị.

KHỞI ĐỘNG

Hành lang trường em dài n mét, rộng 1 mét. Bác bảo vệ lát gạch bằng hai loại: viên vuông 1 × 1 và viên dài 1 × 2. Hành lang 1 mét có 1 cách lát, 2 mét có 2 cách (hai viên vuông hoặc một viên dài), 3 mét có 3 cách, 4 mét có 5 cách... Hành lang 10^{18} mét có bao nhiêu cách? (Lấy dư cho 10^9 cho gọn.)

Các số 1, 2, 3, 5, 8, 13... chính là **dãy Fibonacci** nổi tiếng. Tính số hạng thứ một trăm thì dễ, chỉ cần một vòng lặp. Nhưng số hạng thứ 10^{18} thì vòng lặp chạy hơn 30 năm. Bài này cho em một công cụ rất mạnh: biến "một bước của dãy" thành **một phép nhân ma trận**, rồi nhảy 10^{18} bước chỉ bằng vài chục phép nhân.

1. Dãy Fibonacci và bài toán lát gạch

ĐỊNH NGHĨA (DÃY FIBONACCI)

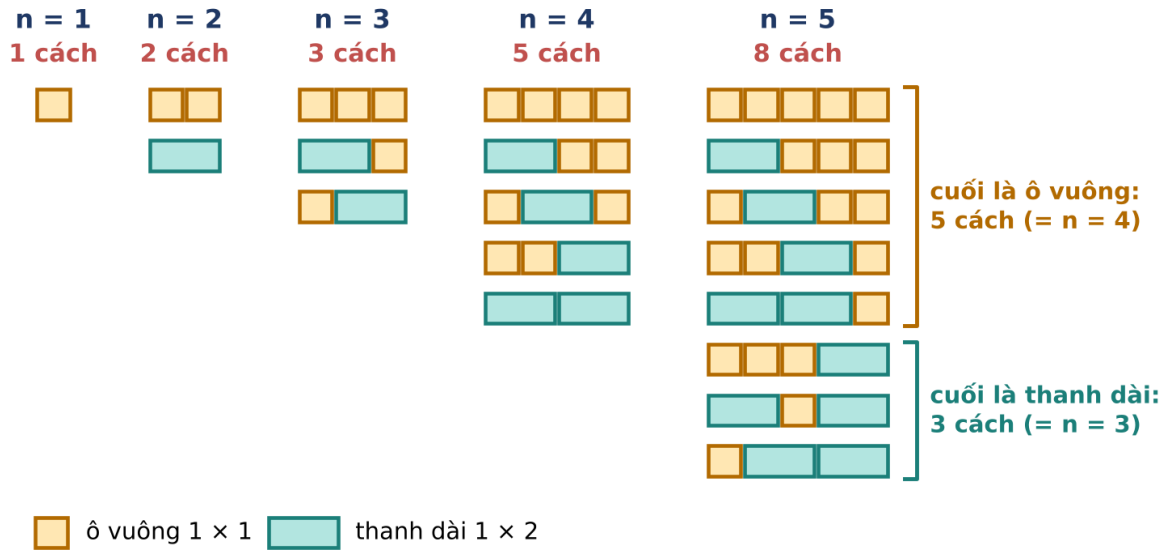
$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \ (n \ge 2)$$

Một dãy mà mỗi số hạng được tính từ một số số hạng đứng trước theo một quy tắc cố định gọi là **dãy truy hồi**. Các giá trị đầu (ở đây F_0, F_1) gọi là **giá trị khởi đầu**.

n	0	1	2	3	4	5	6	7	8	9
F_n	0	1	1	2	3	5	8	13	21	34
n	10	11	12	13	14	15	16	17	18	19
F_n	55	89	144	233	377	610	987	1597	2584	4181

Vì sao hành lang lại dẫn tới Fibonacci? Gọi T(n) là số cách lát hành lang dài n. Nhìn vào **viên gạch cuối cùng**: nếu là viên vuông, phần còn lại dài n − 1, có T(n − 1) cách; nếu là viên dài,

phần còn lại dài n − 2, có T(n − 2) cách. Hai trường hợp tách bạch, nên T(n) = T(n − 1) + T(n − 2), với T(1) = 1, T(2) = 2. Đây đúng là dãy Fibonacci dịch đi một chỉ số: **T(n) = F_{n+1}**.



Hình 15.1. *Mọi cách lát hành lang dài 1..5. Với n = 5: viên cuối là ô vuông (5 cách) hoặc thanh dài (3 cách).*

Dãy Fibonacci tăng rất nhanh: mỗi số hạng gấp khoảng 1,618 lần số trước (con số này là **tỉ lệ vàng** (1 + √5) / 2). F_{92} = 7 540 113 804 746 346 429 là số Fibonacci lớn nhất còn vừa `long long`, F_{93} đã tràn. Vì vậy đề bài luôn hỏi F_n **mod m**.

2. Ma trận và bước truy hồi

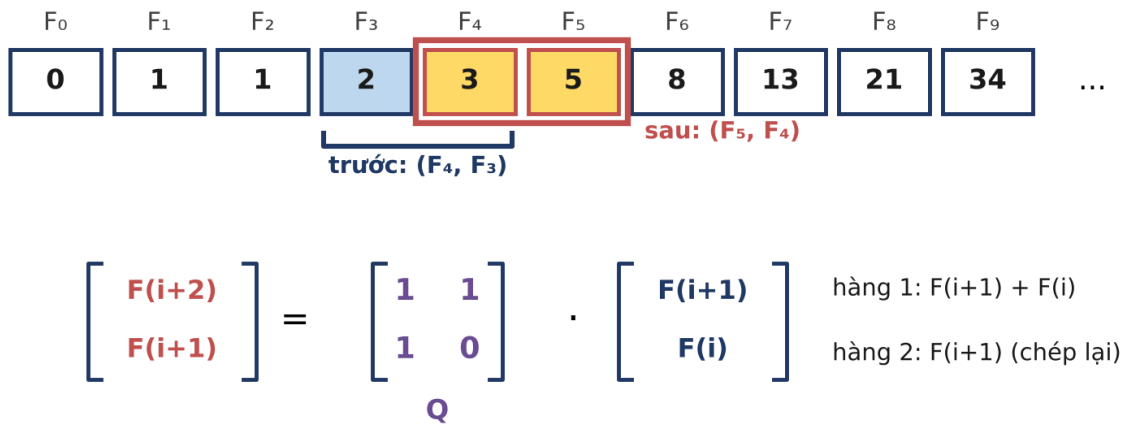
Để tính F_{i+2} ta cần **hai** số liền trước F_{i+1} và F_i. Hãy gói chúng thành một cặp (F_{i+1}, F_i), gọi là **trạng thái**. Một bước của dãy biến trạng thái (F_{i+1}, F_i) thành (F_{i+2}, F_{i+1}) = (F_{i+1} + F_i, F_{i+1}): giống như một cửa sổ rộng hai ô trượt sang phải một ô. Mỗi thành phần mới là một **tổ hợp tuyến tính** của các thành phần cũ, và toán học có sẵn một công cụ để viết những phép biến đổi như vậy: ma trận.

ĐỊNH NGHĨA (NHÂN MA TRẬN)

Ma trận cỡ r × c là một bảng số có r hàng, c cột; X[i][j] là phần tử ở hàng i, cột j (đánh số từ 0). Tích của ma trận A cỡ r × t và B cỡ t × c là ma trận C = A · B cỡ r × c với:

$$C[i][j] = A[i][0] \cdot B[0][j] + A[i][1] \cdot B[1][j] + \ldots + A[i][t-1] \cdot B[t-1][j]$$

tức "hàng i của A nhân lần lượt với cột j của B rồi cộng lại". Ma trận đơn vị I có 1 trên đường chéo, 0 ở chỗ khác, thỏa A · I = I · A = A.



Hình 15.2. *Một bước Fibonacci là một phép nhân với ma trận Q: cửa sổ (F_{i+1}, F_i) trượt sang phải một ô.*

Nhân Q = [[1, 1], [1, 0]] (viết theo hàng) với cột (F_{i+1}, F_i): hàng đầu cho $1 \cdot F_{i+1} + 1 \cdot F_i = F_{i+2}$, hàng sau cho $1 \cdot F_{i+1} + 0 \cdot F_i = F_{i+1}$. Vậy **mỗi bước của dãy là một lần nhân với Q**, và đi n bước là nhân với Q n lần, tức nhân với Q^n.

ĐỊNH LÍ (LŨY THỪA CỦA Q)

Với mọi $n \geq 1$:

$$\mathbf{Q^n = [[F_{n+1}, F_n], [F_n, F_{n-1}]]}$$

Nói riêng, F_n chính là phần tử $Q^n[0][1]$. Quy ước $Q^0 = I$ thì công thức cũng đúng với n = 0 nếu đặt $F_{-1} = 1$.

VÌ SAO ĐÚNG?

Quy nạp theo n. Với n = 1: Q = [[1, 1], [1, 0]] = [[F_2, F_1], [F_1, F_0]], đúng. Giả sử đúng với n, ta có $Q^{n+1} = Q^n \cdot Q = [[F_{n+1} + F_n, F_{n+1}], [F_n + F_{n-1}, F_n]] = [[F_{n+2}, F_{n+1}], [F_{n+1}, F_n]]$, đúng với n + 1.

TÍNH KẾT HỢP: CHÌA KHÓA CỦA LŨY THỪA NHANH

Phép nhân ma trận **không giao hoán** (thường $A \cdot B \neq B \cdot A$) nhưng **có tính kết hợp**: $(A \cdot B) \cdot C = A \cdot (B \cdot C)$. Lũy thừa nhanh ở Bài 9 chỉ cần tính kết hợp, vì nó gom các thừa số Q thành từng nhóm Q^2, Q^4, Q^8... Mọi lũy thừa của cùng một ma trận Q lại giao hoán với nhau, nên thứ tự nhân trong chương trình không quan trọng.

3. Bài toán FIBO

BÀI TOÁN: FIBO

Cho hai số nguyên n và m. Hãy tính F_n mod m, với $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$.

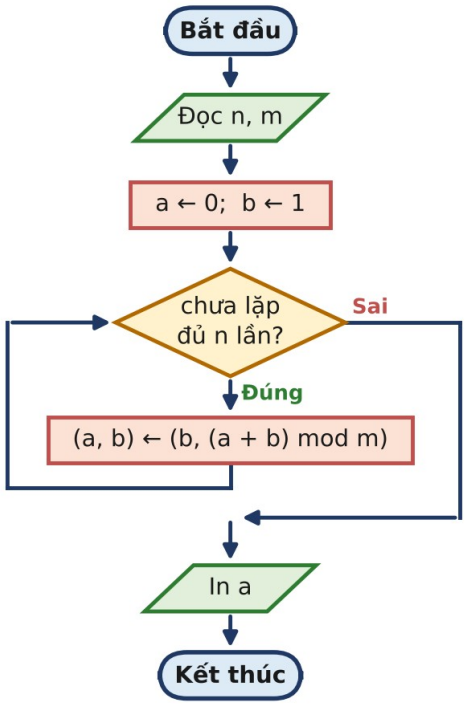
Dữ liệu vào: Một dòng chứa n và m ($0 \le n \le 10^{18}$, $1 \le m \le 10^9$).

Kết quả: Một số nguyên là F_n mod m.

FIBO.INP	FIBO.OUT	Giải thích
`10 1000`	`55`	$F_{10} = 55$; cũng là số cách lát hành lang dài 9.
`1000000000000000000 1000000000`	`560546875`	$n = 10^{18}$: vòng lặp n bước chạy hơn 30 năm.
`0 5`	`0`	$F_0 = 0$. Các trường hợp n = 0 và m = 1 (mọi số dư đều 0) là bẫy hay gặp.

4. Ý tưởng ngây thơ: Lặp n bước

Giữ hai biến $a = F_i$, $b = F_{i+1}$ (đã lấy dư cho m) và tịnh tiến n lần: $(a, b) \leftarrow (b, (a + b) \bmod m)$. Sau n lần, $a = F_n$ mod m. Lưu ý b khởi tạo bằng `1 % m` để đúng cả khi m = 1.



Hình 15.3. Sơ đồ khối của cách lặp n bước.

FIBO.CPP (ngây thơ): lặp n bước

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("FIBO.INP", "r")) {
        freopen("FIBO.INP", "r", stdin);
        freopen("FIBO.OUT", "w", stdout);
    }
    long long n, m;
    cin >> n >> m;
    long long a = 0, b = 1 % m;          // a = F(i), b = F(i + 1), i = 0
    for (long long i = 0; i < n; i++) {  // n lần: tịnh tiến i -> i + 1
        long long c = (a + b) % m;
        a = b;
        b = c;
    }
    cout << a << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Đúng n vòng lặp, mỗi vòng một phép cộng và một phép lấy dư: O(n) thời gian, O(1) bộ nhớ. Với $n = 10^9$, chương trình chạy khoảng 6 giây; với $n = 10^{18}$ thì cần khoảng 10^9 lần như thế, tức **hơn 30 năm**.

BẾ TẮC

Mỗi vòng lặp chỉ đi được **một** bước của dãy. Muốn nhanh, ta phải nhảy nhiều bước một lúc: biết F_{1000} và F_{1001} thì có tính thẳng F_{2000} được không? Định lí lũy thừa của Q trả lời: nhảy n bước là nhân với Q^n, và Q^n thì tính được bằng bình phương liên tiếp như a^b ở Bài 9.

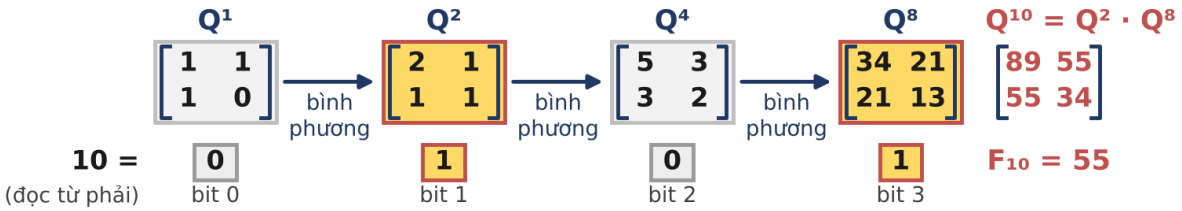
5. Phá bế tắc: Lũy thừa nhanh của ma trận

Bước 1 Đưa bài toán về tính Q^n

Theo định lí, $F_n \bmod m = Q^n[0][1] \bmod m$. Vậy chỉ cần tính Q^n với mọi phần tử lấy dư cho m. Phép lấy dư được phép làm sau mỗi phép nhân, cộng (Bài 1), vì mỗi phần tử của tích chỉ dùng nhân và cộng.

Bước 2 Bình phương liên tiếp

Viết n ở hệ nhị phân. Ví dụ $10 = (1010)_2 = 8 + 2$, nên $Q^{10} = Q^8 \cdot Q^2$. Các lũy thừa Q, Q^2, Q^4, Q^8 có được bằng cách **bình phương** liên tiếp; ta nhân vào kết quả những lũy thừa ứng với **bit 1** của n.



Hình 15.4. $Q^{10} = Q^2 \cdot Q^8$: *chỉ nhân các lũy thừa ứng với bit 1 của* $10 = (1010)_2$.

Bước 3 Chạy tay với n = 10, m = 1000

Bảng dưới là các giá trị thật của vòng lặp trong chương trình (ma trận viết theo hàng, hàng cách nhau bởi dấu chấm phẩy). Ban đầu R = I = [1 0; 0 1], M = Q.

Lượt	n còn lại	Bit cuối	M	R sau lượt	n mới
0	10	0	Q = [1 1; 1 0]	[1 0; 0 1] (giữ nguyên)	5
1	5	1	Q^2 = [2 1; 1 1]	R · M = [2 1; 1 1]	2
2	2	0	Q^4 = [5 3; 3 2]	[2 1; 1 1] (giữ nguyên)	1
3	1	1	Q^8 = [34 21; 21 13]	R · M = **[89 55; 55 34]**	0

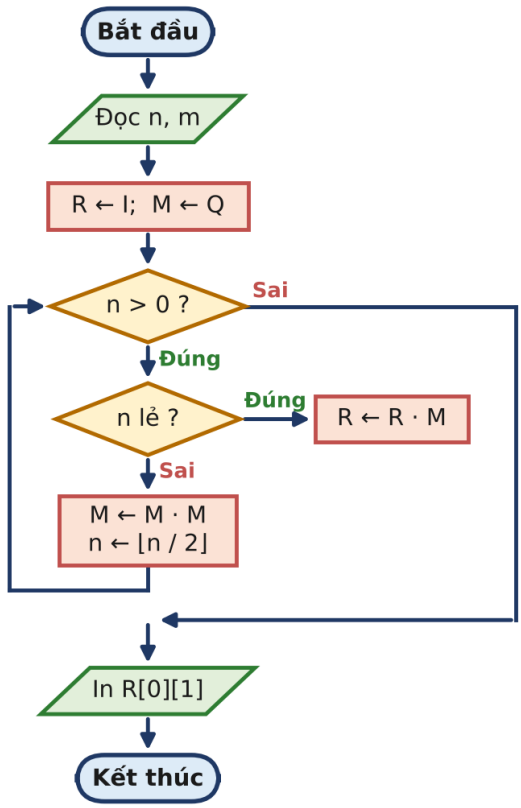
Kết thúc: $R = Q^{10} = [89\ 55;\ 55\ 34] = [F_{11}\ F_{10};\ F_{10}\ F_9]$, đáp số R[0][1] = **55**. Bốn lượt thay cho mười bước của cách ngây thơ; với $n = 10^{18}$ là 60 lượt thay cho 10^{18} bước.

> **VÌ SAO VÒNG LẶP ĐÚNG?**
>
> Gọi N là giá trị ban đầu của n. **Bất biến:** ở đầu mỗi lượt, $R \cdot M^n = Q^N$. Ban đầu R = I, M = Q, n = N: đúng. Nếu n chẵn, $M^n = (M \cdot M)^{n/2}$, nên thay $M \leftarrow M \cdot M$, $n \leftarrow n / 2$ giữ nguyên bất biến. Nếu n lẻ, $M^n = M \cdot (M \cdot M)^{(n-1)/2}$, nên $R \leftarrow R \cdot M$ rồi bình phương M và $n \leftarrow (n - 1) / 2$ cũng giữ bất biến. Khi n = 0: $R \cdot I = R = Q^N$.

Bước 4 Tràn số và trường hợp m = 1

Mọi phần tử luôn nhỏ hơn $m \le 10^9$, nên mỗi tích nhỏ hơn 10^{18} và tổng hai tích nhỏ hơn $2 \cdot 10^{18}$, vẫn dưới giới hạn $9{,}2 \cdot 10^{18}$ của `long long`. Lấy dư một lần sau khi cộng là đủ. Ma trận đơn vị ban đầu viết là `1 % m` để khi m = 1 mọi phần tử đều bằng 0.

Bước 5 Chương trình



Hình 15.5. *Sơ đồ khối của lũy thừa nhanh ma trận.*

FIBO.CPP: lũy thừa nhanh ma trận 2 x 2

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef array<array<ll, 2>, 2> Mat;      // ma trận 2 x 2
ll m;

Mat mul(const Mat &A, const Mat &B) {    // A * B, mọi phần tử mod m
    Mat C;
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)      // mỗi tích < 10^18, tổng < 2*10^18
            C[i][j] = (A[i][0] * B[0][j] + A[i][1] * B[1][j]) % m;
    return C;
}

int main() {
    if (fopen("FIBO.INP", "r")) {
        freopen("FIBO.INP", "r", stdin);
        freopen("FIBO.OUT", "w", stdout);
    }
    ll n;
    cin >> n >> m;
    Mat R = {{{1 % m, 0}, {0, 1 % m}}};  // R = I (ma trận đơn vị)
    Mat M = {{{1, 1}, {1, 0}}};          // M = Q^(2^t) ở lượt thứ t
    while (n > 0) {                      // lũy thừa nhanh như Bài 9
        if (n & 1) R = mul(R, M);
        M = mul(M, M);
        n >>= 1;
    }
    cout << R[0][1] << "\n";             // Q^n = [[F(n+1), F(n)], ...]
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Số lượt bằng số bit của n, tối đa 60 với $n \le 10^{18}$. Mỗi lượt một phép bình phương và có thể thêm một phép nhân $R \cdot M$: tối đa 120 phép nhân ma trận, mỗi phép 8 phép nhân số. Với $n = 10^{18}$ (60 bit, 24 bit 1): đúng 60 + 24 = **84** phép nhân ma trận, khoảng 700 phép nhân số. Độ phức tạp O(log n), so với O(n) của cách lặp.

n	Lặp n bước	Lũy thừa ma trận
10	10 bước	4 lượt
10^9	10^9 bước (≈ 6 giây)	30 lượt
10^{18}	10^{18} bước (≈ 30 năm)	60 lượt (≈ 1 micro giây)

6. Mở rộng

Mở rộng 1: Truy hồi tuyến tính bậc k

Cùng ý tưởng áp dụng cho mọi dãy $f_n = c_1 f_{n-1} + c_2 f_{n-2} + \ldots + c_k f_{n-k}$. Trạng thái là k số hạng liên tiếp $(f_{i+k-1}, \ldots, f_i)$; ma trận chuyển T cỡ $k \times k$ có hàng đầu là $c_1, \ldots, c_k$, các hàng sau "chép" thành phần lên một vị trí ($T[i][i-1] = 1$). Ma trận này gọi là **ma trận đồng hành**. Mỗi phép nhân ma trận $k \times k$ tốn k^3 phép nhân, nên cả thuật toán là $O(k^3 \log n)$.

Ví dụ dãy **Tribonacci** $f_n = f_{n-1} + f_{n-2} + f_{n-3}$ với $f_0 = f_1 = 0$, $f_2 = 1$ cho 0, 0, 1, 1, 2, 4, 7, 13, 24, 44, 81, 149, 274...; nó đếm số cách đi lên cầu thang khi mỗi bước được bước 1, 2 hoặc 3 bậc.

FIBO_TONGQUAT.CPP: truy hồi bậc k, đọc k n m, các c và f(0..k-1)

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef vector<vector<ll>> Mat;
ll m;

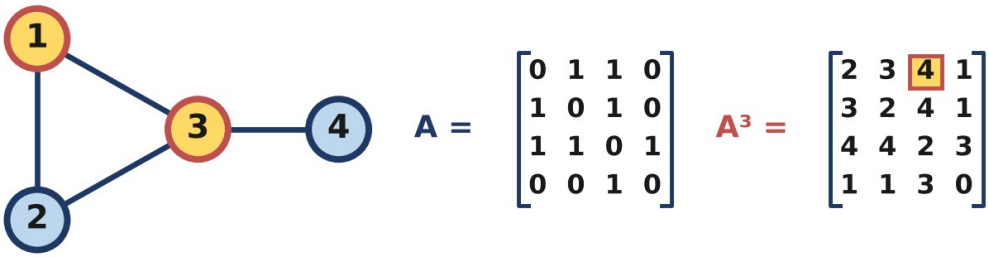
Mat mul(const Mat &A, const Mat &B) {    // nhân ma trận bất kì, mod m
    int r = A.size(), t = B.size(), c = B[0].size();
    Mat C(r, vector<ll>(c, 0));
    for (int i = 0; i < r; i++)
        for (int l = 0; l < t; l++)
            for (int j = 0; j < c; j++)
                C[i][j] = (C[i][j] + A[i][l] * B[l][j]) % m;
    return C;
}

Mat power(Mat A, ll e) {                 // A^e, A vuông
    int k = A.size();
    Mat R(k, vector<ll>(k, 0));
    for (int i = 0; i < k; i++) R[i][i] = 1 % m;
    while (e > 0) {
        if (e & 1) R = mul(R, A);
        A = mul(A, A);
        e >>= 1;
    }
    return R;
}

int main() {
    if (fopen("FIBO.INP", "r")) {
        freopen("FIBO.INP", "r", stdin);
        freopen("FIBO.OUT", "w", stdout);
    }
    int k;
    ll n;
    cin >> k >> n >> m;
    // f(i) = c1 f(i-1) + c2 f(i-2) + ... + ck f(i-k)
    vector<ll> c(k), f(k);
    for (auto &x : c) cin >> x;
    for (auto &x : f) cin >> x;          // f(0), f(1), ..., f(k-1)
    Mat T(k, vector<ll>(k, 0));          // ma trận đồng hành
    for (int j = 0; j < k; j++) T[0][j] = c[j] % m;
    for (int i = 1; i < k; i++) T[i][i - 1] = 1 % m;
    Mat v(k, vector<ll>(1));             // v = (f(k-1), ..., f(1), f(0))
    for (int i = 0; i < k; i++) v[i][0] = f[k - 1 - i] % m;
    v = mul(power(T, n), v);             // v = (f(n+k-1), ..., f(n))
    cout << v[k - 1][0] << "\n";
    return 0;
}
```

Mở rộng 2: Đếm đường đi độ dài n trên đồ thị

Cho đồ thị có ma trận kề A (A[u][v] = 1 nếu có cạnh u − v). Khi đó **A^n[u][v] là số đường đi độ dài đúng n** (được đi lại đỉnh, cạnh cũ) từ u tới v. Lí do giống hệt quy tắc nhân: một đường đi độ dài n + 1 từ u tới v là đường đi độ dài n từ u tới một đỉnh w nào đó, rồi thêm cạnh w − v; tổng theo w chính là công thức nhân ma trận.



Hình 15.6. *$A^3[1][3] = 4$: có bốn đường đi độ dài 3 từ đỉnh 1 tới đỉnh 3.*

Bốn đường đi đó là 1 → 2 → 1 → 3, 1 → 3 → 1 → 3, 1 → 3 → 2 → 3 và 1 → 3 → 4 → 3. Với n tới 10^{18} và đồ thị vài chục đỉnh, chỉ cần thay ma trận T trong FIBO_TONGQUAT bằng A.

Mở rộng 3: Công thức nhân đôi

Từ $Q^{2k} = Q^k \cdot Q^k$ và định lí lũy thừa của Q, so sánh các phần tử ta được hai đẳng thức đẹp:

$$\mathbf{F_{2k} = F_k \cdot (2F_{k+1} - F_k), \quad F_{2k+1} = F_k^2 + F_{k+1}^2}$$

Biết cặp (F_k, F_{k+1}) thì tính ngay cặp (F_{2k}, F_{2k+1}), và từ đó (F_{2k+1}, F_{2k+2}). Đệ quy theo n / 2 cho thuật toán O(log n) gọn hơn, ít phép nhân hơn ma trận (khoảng 3 thay vì 8 phép nhân số mỗi bậc).

FIBO_NHANDOI.CPP: công thức nhân đôi

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
ll m;

// trả về cặp (F(n), F(n + 1)) mod m
pair<ll, ll> fib(ll n) {
    if (n == 0) return {0, 1 % m};
    pair<ll, ll> p = fib(n / 2);          // (F(k), F(k + 1)), k = n / 2
    ll a = p.first, b = p.second;
    ll c = a * ((2 * b - a + m) % m) % m;     // F(2k) = F(k)(2F(k+1) - F(k))
    ll d = (a * a + b * b) % m;               // F(2k+1) = F(k)^2 + F(k+1)^2
    if (n % 2 == 0) return {c, d};
    return {d, (c + d) % m};
}

int main() {
    if (fopen("FIBO.INP", "r")) {
        freopen("FIBO.INP", "r", stdin);
        freopen("FIBO.OUT", "w", stdout);
    }
    ll n;
    cin >> n >> m;
    cout << fib(n).first << "\n";
    return 0;
}
```

Mở rộng 4: Chu kì Pisano

Dãy F_n mod m chỉ phụ thuộc vào cặp hai số dư liên tiếp, mà chỉ có m^2 cặp, nên dãy số dư phải **tuần hoàn**; hơn nữa nó lặp lại ngay từ cặp (0, 1). Độ dài chu kì gọi là **chu kì Pisano** π(m). Ví dụ chữ số tận cùng của F_n (m = 10) lặp lại sau 60 số hạng.

m	2	3	5	7	10	100	1000	10^9
π(m)	3	8	20	16	60	300	1500	$1{,}5 \cdot 10^9$

Vì $10^{18} \bmod (1{,}5 \cdot 10^9) = 10^9$, ta có $F(10^{18}) \equiv F(10^9) \pmod{10^9}$: ví dụ 2 của FIBO cũng giải được bằng vòng lặp 10^9 bước. Nhưng chu kì Pisano có thể lớn tới 6m, nên lũy thừa ma trận vẫn là cách an toàn nhất.

BẪY THƯỜNG GẶP

- **Lấy dư quá muộn:** dãy Fibonacci tràn `long long` từ F_{93}. Lấy dư sau mỗi phép cộng, nhân.
- **Cộng hai tích rồi mới lấy dư với m lớn:** với $m \le 10^9$ thì an toàn ($< 2 \cdot 10^{18}$); nếu m tới 10^{18} phải dùng `__int128` (Bài 9).
- **Quên trường hợp m = 1:** khởi tạo ma trận đơn vị bằng 1 thay vì `1 % m` sẽ in 1 khi n = 0, m = 1 thay vì 0.
- **Lệch chỉ số:** $Q^n[0][1] = F_n$ nhưng $Q^n[0][0] = F_{n+1}$. Số cách lát hành lang dài n là F_{n+1}, không phải F_n.
- **Nhân ma trận sai thứ tự chỉ số:** $C[i][j] = \Sigma\, A[i][l] \cdot B[l][j]$; đảo i, j sẽ ra ma trận chuyển vị, sai với ma trận không đối xứng.

GHI NHỚ

- Truy hồi tuyến tính = một phép nhân ma trận lên vectơ trạng thái; n bước = nhân với T^n.
- $Q = [[1, 1], [1, 0]]$, $Q^n = [[F_{n+1}, F_n], [F_n, F_{n-1}]]$.
- Lũy thừa nhanh ma trận: $O(k^3 \log n)$; với Fibonacci (k = 2) chỉ khoảng $2 \cdot \log_2 n$ phép nhân ma trận.
- $A^n[u][v]$ = số đường đi độ dài n từ u tới v.
- Phần tử $< m \le 10^9$ thì tổng hai tích $< 2 \cdot 10^{18}$, an toàn trong `long long`.

Luyện tập

Bài	Tên	Gợi ý liên hệ
15.1	**LÁT GẠCH**	Số cách lát hành lang $2 \times n$ bằng các viên 1×2 (đặt dọc hoặc ngang), $n \le 10^{18}$: lại là Fibonacci, dùng FIBO.
15.2	**TRIBONACCI**	Số cách lên cầu thang n bậc, mỗi bước 1, 2 hoặc 3 bậc, mod $10^9 + 7$, $n \le 10^{18}$: ma trận đồng hành 3×3.
15.3	**ĐƯỜNG ĐI ĐỘ DÀI K**	Đồ thị ≤ 50 đỉnh, đếm đường đi độ dài đúng $K \le 10^{18}$ giữa hai đỉnh: lũy thừa ma trận kề.

Tự kiểm tra nhanh

1) Tính F_{20}. Hành lang dài 10 mét có bao nhiêu cách lát? *(Đáp số: $F_{20} = 6765$; số cách lát là $F_{11} = 89$)*

2) Chữ số tận cùng của F_{2025} là bao nhiêu? (Gợi ý: $\pi(10) = 60$.) *(Đáp số: $2025 \bmod 60 = 45$, $F_{45} = 1\,134\,903\,170$, chữ số tận cùng là 0)*

3) Chương trình FIBO thực hiện bao nhiêu phép nhân ma trận với $n = 10^{18}$ (có 60 chữ số nhị phân, trong đó 24 chữ số 1)? *(Đáp số: 60 phép bình phương + 24 phép nhân vào R = 84)*

4) Với $Q = [[1, 1], [1, 0]]$, hãy tính Q^4 và đọc ra F_4, F_5. *(Đáp số: $Q^4 = [[5, 3], [3, 2]]$, $F_4 = 3$, $F_5 = 5$)*

BÀI 16 Kĩ thuật chia khối ⌊n / i⌋

Học xong bài này, em sẽ:

- ✓ Chứng minh $\Sigma_{x \le n} d(x) = \Sigma_{i \le n} \lfloor n / i \rfloor$ bằng phép **đếm hai lần**.
- ✓ Hiểu vì sao dãy ⌊n / 1⌋, ⌊n / 2⌋, ..., ⌊n / n⌋ chỉ có **không quá 2√n giá trị khác nhau**.
- ✓ Tìm được khối [i, j] lớn nhất có cùng giá trị ⌊n / i⌋ bằng công thức j = ⌊n / ⌊n / i⌋⌋, và tính tổng trong O(√n).
- ✓ Áp dụng cho Σ i · ⌊n / i⌋ (tổng các ước), Σ n mod i và phương pháp hyperbol.

KHỞI ĐỘNG

Thầy giáo viết lên bảng các số 1, 2, ..., 12 rồi hỏi: "Tổng số ước của tất cả các số này là bao nhiêu?" Cả lớp chia nhau đếm: số 1 có 1 ước, số 2 có 2 ước, số 12 có 6 ước... cộng lại được 35. Thầy hỏi tiếp: "Còn từ 1 đến 10^{12} thì sao?"

Đếm ước của từng số là hết hi vọng. Nhưng nếu **đổi góc nhìn**, thay vì hỏi "số x có bao nhiêu ước", ta hỏi "số i là ước của bao nhiêu số", thì bài toán biến thành một tổng các thương ⌊n / i⌋. Và tổng ấy có một bí mật: tuy có n số hạng nhưng chỉ có khoảng 2√n giá trị khác nhau. Với $n = 10^{12}$, đó là hai triệu thay vì một nghìn tỉ.

1. Tổng số ước và phép đếm hai lần

Kí hiệu d(x) là số ước dương của x (Bài 2). Bảng dưới liệt kê d(x) với x từ 1 đến 12 và tổng của chúng.

x	1	2	3	4	5	6	7	8	9	10	11	12	Tổng
d(x)	1	2	2	3	2	4	2	4	3	4	2	6	**35**
⌊12 / x⌋	12	6	4	3	2	2	1	1	1	1	1	1	**35**

Hai hàng có cùng tổng 35 dù từng số hạng khác hẳn nhau. Không phải trùng hợp: hàng dưới đếm cùng một thứ theo một cách khác.

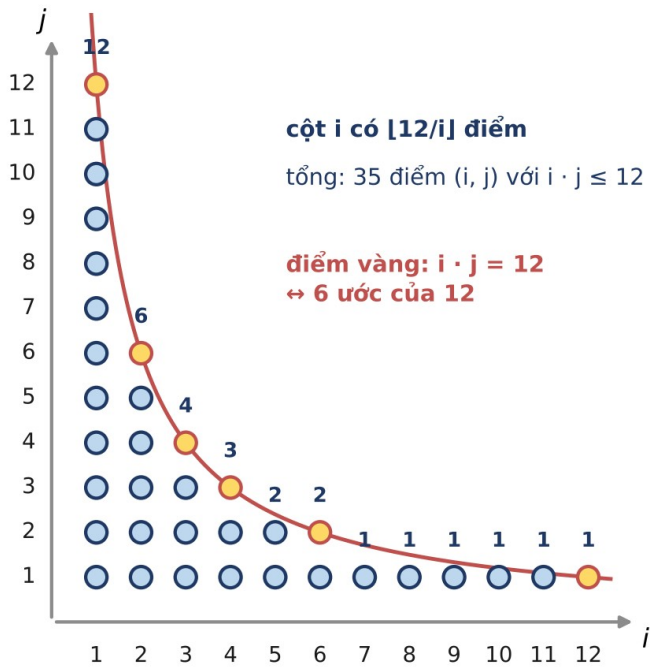
ĐỊNH LÍ (ĐỔI THỨ TỰ ĐẾM)

$$D(n) = d(1) + d(2) + ... + d(n) = \lfloor n / 1 \rfloor + \lfloor n / 2 \rfloor + ... + \lfloor n / n \rfloor$$

VÌ SAO ĐÚNG?

Cả hai vế đều đếm số **cặp (i, x)** với $1 \le x \le n$ và i là ước của x. Đếm theo x trước: với mỗi x có d(x) giá trị i, được vế trái. Đếm theo i trước: với mỗi i, các x nhận i làm ước là các bội của i trong [1, n], có $\lfloor n / i \rfloor$ số (Bài 14), được vế phải.

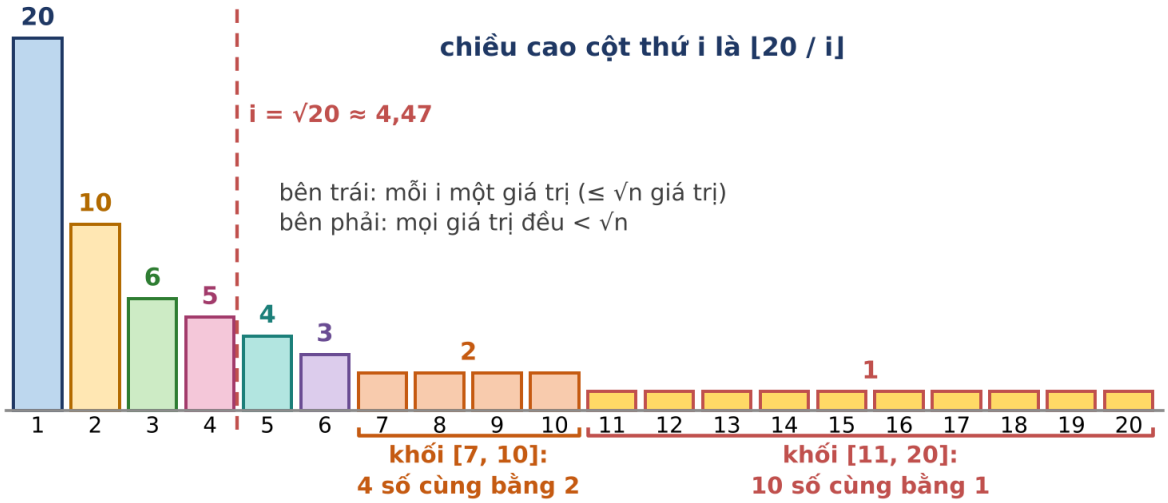
Viết $x = i \cdot j$, ta còn có cách nhìn thứ ba: D(n) là **số điểm nguyên (i, j)** với $i, j \ge 1$ và $i \cdot j \le n$, tức số điểm nguyên nằm dưới đường hypebol $i \cdot j = n$ (Hình 16.1).



Hình 16.1. *Các điểm (i, j) với $i \cdot j \le 12$. Cột i có $\lfloor 12 / i \rfloor$ điểm; các điểm vàng nằm trên đường $i \cdot j = 12$ ứng với 6 ước của 12.*

2. Bậc thang ⌊n / i⌋

Vẽ chiều cao $\lfloor n / i \rfloor$ của từng cột i, ta được một **bậc thang** đi xuống. Phía đầu, mỗi bậc chỉ rộng một cột và tụt rất nhanh (20, 10, 6, 5...); phía sau, bậc thang thoải dần và mỗi bậc rộng ra: 4 cột cùng cao 2, rồi 10 cột cùng cao 1.



Hình 16.2. *Bậc thang ⌊20 / i⌋. Các cột cùng chiều cao tạo thành một khối (cùng màu): chỉ có 8 khối cho 20 cột.*

Chỉ có 8 giá trị khác nhau: 20, 10, 6, 5, 4, 3, 2, 1. Nếu cộng **theo khối** thay vì theo từng cột, ta chỉ làm 8 phép cộng. Hai câu hỏi cần trả lời: có bao nhiêu khối, và làm sao biết một khối kết thúc ở đâu?

ĐỊNH LÍ (SỐ GIÁ TRỊ KHÁC NHAU)

Dãy $\lfloor n / i \rfloor$ với $i = 1, 2, \ldots, n$ có không quá $2\sqrt{n}$ giá trị khác nhau.

VÌ SAO ĐÚNG?

Chia các chỉ số i thành hai phần. **Phần i ≤ √n:** chỉ có không quá √n chỉ số, nên không quá √n giá trị. **Phần i > √n:** khi đó $\lfloor n / i \rfloor \le n / i < \sqrt{n}$, nên giá trị là một số nguyên dương nhỏ hơn √n: cũng không quá √n khả năng. Cộng lại: không quá $2\sqrt{n}$ giá trị.

n	Số khối (số giá trị khác nhau)	$2 \cdot \lfloor \sqrt{n} \rfloor$
20	8	8
100	19	20
10^4	199	200
10^6	1 999	2 000
10^{12}	1 999 999	2 000 000

ĐỊNH LÍ (ĐIỂM CUỐI CỦA KHỐI)

Cho $1 \le i \le n$ và $q = \lfloor n / i \rfloor$. Chỉ số **lớn nhất** j thỏa $\lfloor n / j \rfloor = q$ là:

$$\mathbf{j = \lfloor n / q \rfloor}$$

VÌ SAO ĐÚNG?

$\lfloor n / j \rfloor \geq q$ tương đương $q \cdot j \leq n$, tương đương $j \leq n / q$, tức $j \leq \lfloor n / q \rfloor$. Vậy các chỉ số j có $\lfloor n / j \rfloor \geq q$ là đúng 1, 2, ..., $\lfloor n / q \rfloor$. Vì $\lfloor n / j \rfloor$ không tăng theo j, các chỉ số từ i đến $\lfloor n / q \rfloor$ đều có $\lfloor n / j \rfloor \geq q$ và $\leq \lfloor n / i \rfloor = q$, tức bằng q; còn $j = \lfloor n / q \rfloor + 1$ thì $\lfloor n / j \rfloor < q$. Do đó khối chứa i kết thúc đúng tại $\lfloor n / q \rfloor$.

3. Bài toán TONGUOC

BÀI TOÁN: TONGUOC

Cho số nguyên dương n. Hãy tính $S(n) = \lfloor n / 1 \rfloor + \lfloor n / 2 \rfloor + ... + \lfloor n / n \rfloor$, cũng chính là tổng số ước của các số 1, 2, ..., n.

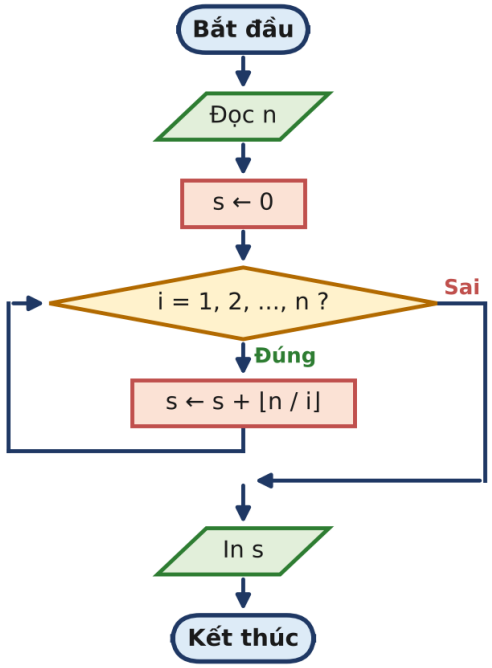
Dữ liệu vào: Một số nguyên n ($1 \leq n \leq 10^{12}$).

Kết quả: Một số nguyên là S(n).

TONGUOC.INP	TONGUOC.OUT	Giải thích
`20`	`66`	20 + 10 + 6 + 5 + 4 + 3 + 2 · 4 + 1 · 10 = 66 (Hình 16.2).
`1000000000000`	`27785452449086`	S(n) xấp xỉ $n \cdot (\ln n + 0{,}1544)$, cỡ $2{,}8 \cdot 10^{13}$: vẫn vừa `long long`.
`1`	`1`	Số 1 có đúng một ước.

4. Ý tưởng ngây thơ: Cộng từng thương

Công thức đã có sẵn, chỉ cần một vòng lặp i từ 1 đến n, cộng dồn n / i.



Hình 16.3. *Sơ đồ khối của cách cộng từng thương.*

TONGUOC.CPP (ngây thơ): cộng từng thương

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("TONGUOC.INP", "r")) {
        freopen("TONGUOC.INP", "r", stdin);
        freopen("TONGUOC.OUT", "w", stdout);
    }
    long long n;
    cin >> n;
    long long s = 0;
    for (long long i = 1; i <= n; i++)
        s += n / i;                // số bội của i trong [1, n]
    cout << s << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Đúng n phép chia: O(n). Phép chia 64 bit khá chậm: với $n = 10^9$ chương trình chạy khoảng 3 giây, nên với $n = 10^{12}$ cần khoảng **một giờ**.

BẾ TẮC

Cách ngây thơ tính $\lfloor n / i \rfloor$ cho cả n chỉ số i, trong khi ta vừa chứng minh chỉ có không quá $2\sqrt{n}$ giá trị khác nhau: với $n = 10^{12}$, hơn 99,9998% phép chia là "tính lại" một giá trị đã biết. Nếu ở mỗi i ta biết ngay khối hiện tại kéo dài tới đâu, ta có thể **nhảy qua cả khối**.

5. Phá bế tắc: Gom các khối cùng giá trị

Bước 1 **Nhảy qua cả khối**

Bắt đầu từ $i = 1$. Ở mỗi lượt: tính $q = \lfloor n / i \rfloor$, điểm cuối khối $j = \lfloor n / q \rfloor$; cả khối $[i, j]$ gồm $j - i + 1$ số hạng cùng bằng q, góp vào tổng $q \cdot (j - i + 1)$. Rồi nhảy sang $i = j + 1$.

$$\mathbf{q = \lfloor n / i \rfloor,\quad j = \lfloor n / q \rfloor,\quad s \leftarrow s + q \cdot (j - i + 1),\quad i \leftarrow j + 1}$$

Bước 2 **Chạy tay với n = 20**

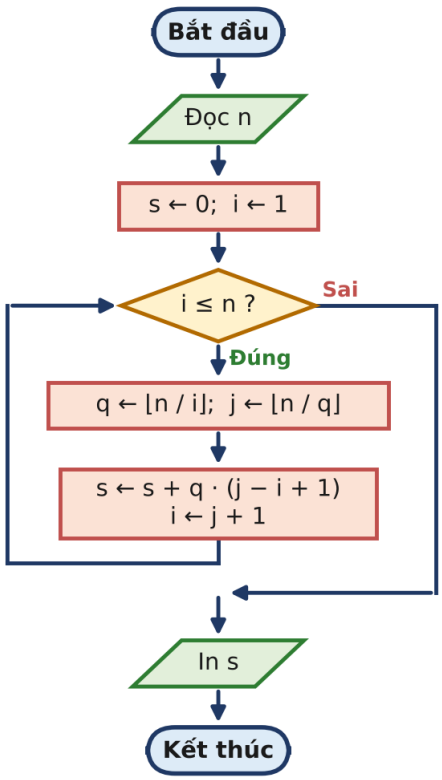
i	$q = \lfloor 20 / i \rfloor$	$j = \lfloor 20 / q \rfloor$	Độ dài khối	Góp $q \cdot (j - i + 1)$	s
1	20	1	1	20	20
2	10	2	1	10	30
3	6	3	1	6	36
4	5	4	1	5	41
5	4	5	1	4	45
6	3	6	1	3	48
7	2	10	4	8	56
11	1	20	10	10	**66**

Tám lượt thay cho hai mươi, đúng bằng số khối trên Hình 16.2. Sáu khối đầu dài 1 (phần $i \le \sqrt{20}$), hai khối cuối dài (phần giá trị nhỏ): đúng như chứng minh về $2\sqrt{n}$.

VÌ SAO THUẬT TOÁN ĐÚNG VÀ DỪNG?

Theo định lí điểm cuối của khối, mọi chỉ số trong $[i, j]$ có cùng giá trị q, nên phần đóng góp $q \cdot (j - i + 1)$ đúng bằng tổng các số hạng của khối. Vì $j \ge i$ (do $\lfloor n / q \rfloor \ge i$ khi $q = \lfloor n / i \rfloor$), mỗi lượt i tăng ít nhất 1, các khối nối tiếp nhau phủ kín $[1, n]$ không chồng lấn, và vòng lặp dừng khi $i > n$. Mỗi lượt kết thúc một khối, nên số lượt bằng số giá trị khác nhau, không quá $2\sqrt{n}$.

Bước 3 Chương trình



Hình 16.4. *Sơ đồ khối của cách gom khối.*

TONGUOC.CPP: gom khối cùng giá trị n / i

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("TONGUOC.INP", "r")) {
        freopen("TONGUOC.INP", "r", stdin);
        freopen("TONGUOC.OUT", "w", stdout);
    }
    long long n;
    cin >> n;
    long long s = 0;
    for (long long i = 1, j; i <= n; i = j + 1) {
        long long q = n / i;        // giá trị chung của cả khối
        j = n / q;                  // chỉ số lớn nhất có n / j == q
        s += q * (j - i + 1);       // khối [i, j] góp q * (độ dài khối)
    }
    cout << s << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

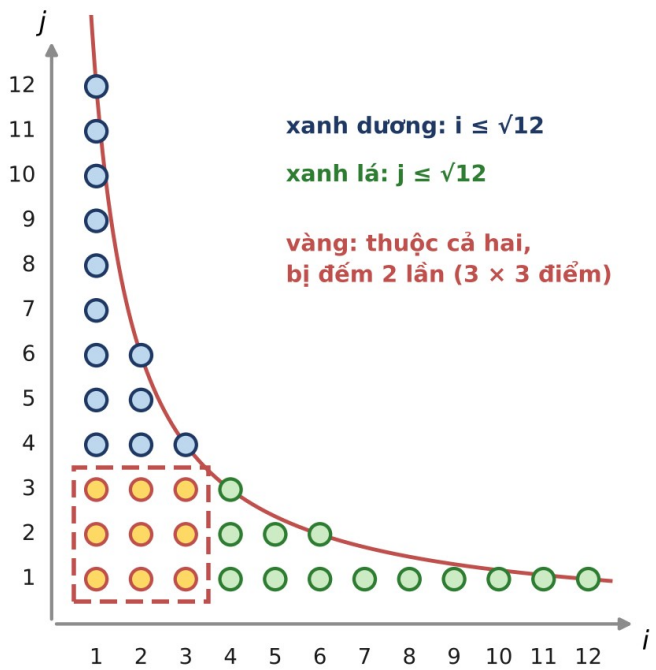
Mỗi lượt hai phép chia, một phép nhân: tổng $O(\sqrt{n})$. Với $n = 10^{12}$: 1 999 999 lượt, khoảng 4 triệu phép chia, chạy trong vài phần trăm giây. Tổng $s \le 2{,}8 \cdot 10^{13}$, tích $q \cdot (j - i + 1) \le n$: không tràn `long long`.

6. Mở rộng

Mở rộng 1: Phương pháp hyperbol

Nhìn lại hình các điểm nguyên dưới hypebol $i \cdot j \le n$. Hình nhận đường chéo $i = j$ làm trục đối xứng. Gọi $r = \lfloor \sqrt{n} \rfloor$. Mỗi điểm có $i \le r$ hoặc $j \le r$ (không thể cả hai đều $> \sqrt{n}$ vì khi đó $i \cdot j > n$). Đếm các điểm có $i \le r$ (cột trái: $\Sigma_{i \le r} \lfloor n / i \rfloor$), cộng các điểm có $j \le r$ (hàng dưới: cùng tổng đó, do đối xứng), thì hình vuông $r \times r$ bị đếm hai lần:

$$S(n) = 2 \cdot (\lfloor n/1 \rfloor + \lfloor n/2 \rfloor + \ldots + \lfloor n/r \rfloor) - r^2$$



Hình 16.5. Phương pháp hyperbol với n = 12, r = 3: S(12) = 2 · (12 + 6 + 4) − 9 = 35.

Thuật toán chỉ có một vòng lặp $r \approx \sqrt{n}$ bước, nhanh gấp đôi cách gom khối. Lưu ý tính $r = \lfloor \sqrt{n} \rfloor$ bằng `sqrtl` rồi **sửa** cho chắc chắn, vì số thực có sai số làm tròn.

TONGUOC_HYPERBOL.CPP: đếm điểm dưới hypebol

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("TONGUOC.INP", "r")) {
        freopen("TONGUOC.INP", "r", stdin);
        freopen("TONGUOC.OUT", "w", stdout);
    }
    long long n;
    cin >> n;
    long long r = sqrtl(n);                // r = phần nguyên của căn n
    while (r * r > n) r--;                 // sửa sai số làm tròn
    while ((r + 1) * (r + 1) <= n) r++;
    long long s = 0;
    for (long long i = 1; i <= r; i++) s += n / i;
    cout << 2 * s - r * r << "\n";         // đếm cặp (i, j), i * j <= n
    return 0;
}
```

Mở rộng 2: Tổng các ước Σ σ(x)

Gọi σ(x) là tổng các ước của x. Đếm hai lần như trên: mỗi i là ước của $\lfloor n / i \rfloor$ số, mỗi lần góp **giá trị i**, nên $\sigma(1) + \ldots + \sigma(n) = \Sigma\, i \cdot \lfloor n / i \rfloor$. Trong một khối [i, j] cùng thương q, phần đóng góp là $q \cdot (i + (i+1) + \ldots + j) = q \cdot (i + j) \cdot (j - i + 1) / 2$. Ví dụ n = 20 cho tổng 339. Với $n = 10^{12}$, tổng này khoảng $8{,}2 \cdot 10^{23}$, vượt `long long`, nên đề bài thường hỏi theo mô-đun; tổng $(i + j)(j - i + 1)$ có thể tới 10^{24}, phải tính bằng `__int128`:

Tổng σ(1) + ... + σ(n) mod $10^9 + 7$ (với n = 10^{12} cho 146462362)

```cpp
const long long M = 1000000007;
long long s = 0;
for (long long i = 1, j; i <= n; i = j + 1) {
    long long q = n / i;
    j = n / q;
    __int128 t = (__int128)(i + j) * (j - i + 1) / 2;  // i + ... + j
    s = (s + (long long)(t % M) * (q % M)) % M;
}
```

Mở rộng 3: Tổng các số dư Σ n mod i

Vì $n \bmod i = n - i \cdot \lfloor n / i \rfloor$, ta có $\Sigma_{i=1..n} (n \bmod i) = n^2 - \Sigma\, i \cdot \lfloor n / i \rfloor$: quy về ngay Mở rộng 2. Ví dụ n = 20: 400 − 339 = 61.

Mở rộng 4: Hai thương cùng lúc

Với tổng có dạng $\Sigma\, f(\lfloor n / i \rfloor, \lfloor m / i \rfloor)$, cả hai thương cùng không đổi trên đoạn [i, j] với $j = \min(\lfloor n / \lfloor n / i \rfloor \rfloor, \lfloor m / \lfloor m / i \rfloor \rfloor)$. Số khối không quá $2(\sqrt{n} + \sqrt{m})$. Kĩ thuật này xuất hiện rất nhiều trong các bài đếm cặp số có ƯCLN cho trước ở Tập 2.

BẪY THƯỜNG GẶP

- **Chia cho 0:** chỉ tính j = n / q khi q ≥ 1, tức khi i ≤ n. Điều kiện vòng lặp `i <= n` bảo đảm điều đó.
- **Dùng `int` cho i, j:** với n = 10^{12} thì i, j, n đều vượt 2^{31}. Dùng `long long` cho mọi biến.
- **Tính √n bằng số thực:** `sqrt(n)` có thể lệch 1 khi n gần một số chính phương lớn. Luôn sửa bằng hai vòng `while`.
- **Tràn ở tổng có trọng số:** Σ i · ⌊n / i⌋ và (i + j)(j − i + 1) vượt `long long` khi n = 10^{12}. Dùng `__int128` hoặc lấy dư.
- **Quên rằng khối có thể dài 1:** ở phần i ≤ √n mỗi khối chỉ có một chỉ số; công thức vẫn đúng, không cần xử lí riêng.

GHI NHỚ

- $\Sigma_{x \le n}$ d(x) = $\Sigma_{i \le n}$ ⌊n / i⌋ = số điểm nguyên (i, j) với i · j ≤ n.
- Dãy ⌊n / i⌋ có không quá 2√n giá trị khác nhau.
- Khối chứa i kết thúc tại j = ⌊n / ⌊n / i⌋⌋; nhảy i ← j + 1 cho O(√n).
- Hyperbol: S(n) = 2 · $\Sigma_{i \le r}$ ⌊n / i⌋ − r^2 với r = ⌊√n⌋.

Luyện tập

Bài	Tên	Gợi ý liên hệ
16.1	**TỔNG ƯỚC**	σ(1) + ... + σ(n) mod 10^9 + 7, n ≤ 10^{12}: chia khối, tổng cấp số cộng trong mỗi khối.
16.2	**TỔNG DƯ**	Σ (n mod i) với i từ 1 đến k (k có thể khác n), mod 10^9 + 7: n mod i = n − i · ⌊n / i⌋, chia khối trên [1, min(n, k)].
16.3	**HAI THƯƠNG**	Σ ⌊n / i⌋ · ⌊m / i⌋ với i từ 1 đến min(n, m), n, m ≤ 10^{12}: chia khối đồng thời theo hai thương.

Tự kiểm tra nhanh

1) Tính S(100) = Σ ⌊100 / i⌋. Thuật toán gom khối chạy bao nhiêu lượt? *(Đáp số: S(100) = 482, 19 lượt)*

2) Liệt kê các giá trị khác nhau của ⌊36 / i⌋ với i = 1..36. Có bao nhiêu giá trị? *(Đáp số: 1, 2, 3, 4, 5, 6, 7, 9, 12, 18, 36: 11 giá trị)*

3) Tính 20 mod 1 + 20 mod 2 + ... + 20 mod 20. *(Đáp số: 400 − 339 = 61)*

4) Dùng phương pháp hyperbol tính S(12). *(Đáp số: r = 3, 2 · (12 + 6 + 4) − 9 = 35)*

CHƯƠNG 5. KĨ THUẬT SỐ HỌC TRONG LẬP TRÌNH

Bốn chương trước cho em những định lí đẹp của số học. Chương cuối này trả lời một câu hỏi rất thực tế: **khi đem số học vào máy tính, ta cần những kĩ thuật gì?** Máy chỉ có những ô nhớ 32 hay 64 bit, nên với kết quả dài hàng nghìn chữ số ta phải tự xây "số lớn". Mọi số trong máy đều là dãy bit, nên hiểu hệ nhị phân giúp ta đếm nhanh tới mức khó tin. Và hai công cụ nhỏ là **tổng tiền tố** và **ƯCLN** lại mở ra lời giải cho những bài toán tưởng như phải duyệt hàng tỉ trường hợp.

Mỗi bài học trong chương là một "mẹo nghề" mà học sinh giỏi Tin học nào cũng cần thuộc lòng: số nguyên lớn, phép toán bit và đếm theo cột, tổng tiền tố kết hợp số dư, điểm nguyên và định lí Pick. Em sẽ thấy các kiến thức cũ (đồng dư ở Bài 1, Euclid ở Bài 3, tổ hợp ở Bài 13) xuất hiện trở lại ở khắp nơi.

TRONG CHƯƠNG NÀY

Bài 17. Số nguyên lớn: khi kết quả dài hơn mọi kiểu số của C++

Bài 18. Hệ cơ số và biểu diễn nhị phân: đếm bit 1 của 10^{18} số trong 60 bước

Bài 19. Tổng tiền tố và số dư: đếm đoạn con chia hết cho k trong một lượt duyệt

Bài 20. Điểm nguyên và hình học số nguyên: ƯCLN đếm điểm trên đoạn thẳng, định lí Pick

BÀI 17 Số nguyên lớn

Học xong bài này, em sẽ:

- ✓ Biết giới hạn của `int`, `long long`, `unsigned long long`, `__int128` và nhận ra lúc nào kết quả **không thể** chứa trong chúng.
- ✓ Biểu diễn một số rất lớn bằng **mảng chữ số đảo ngược** và hiểu vì sao phải đảo ngược.
- ✓ Nhân một số lớn với một số nhỏ bằng cách mang **số nhớ**, chứng minh được số nhớ không bao giờ vượt quá số nhân.
- ✓ Tăng tốc gần 9 lần bằng **cơ số 10^9** và in kết quả đúng định dạng.
- ✓ Viết được phép cộng, nhân hai số lớn, chia số lớn cho số nhỏ.

KHỞI ĐỘNG

Em bấm 25! trên máy tính bỏ túi và nhận được $1{,}551121004 \cdot 10^{25}$. Máy tính bỏ túi chỉ cho em **10 chữ số đầu**; 16 chữ số còn lại đã bị làm tròn mất. Thử với C++: `unsigned long long f = 1; for (i = 2..25) f *= i;` in ra 7034535277573963776, một con số trông rất "thật" nhưng **sai hoàn toàn**.

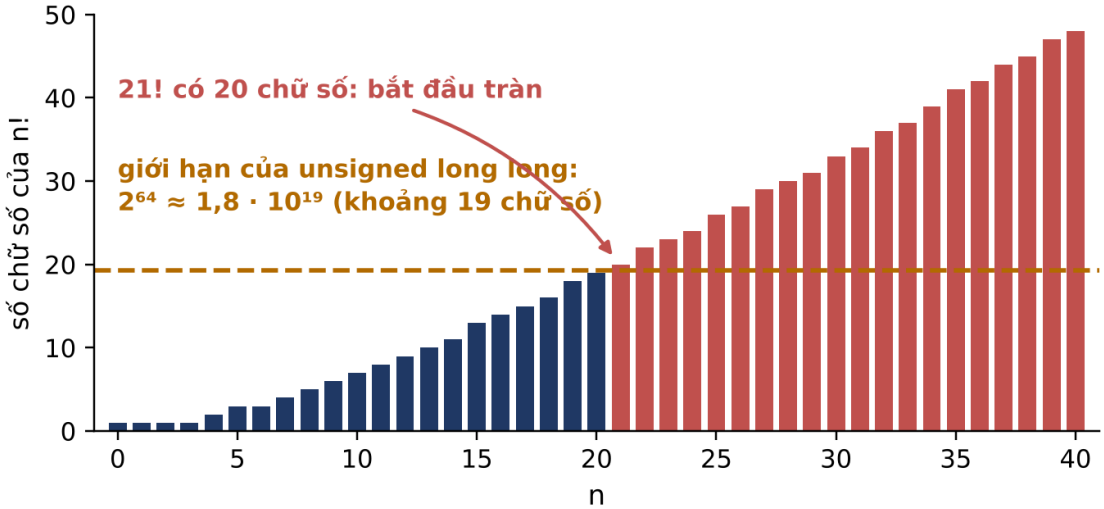
Vậy làm sao in được chính xác cả 26 chữ số của 25!, hay 5736 chữ số của 2000!? Câu trả lời nằm ở phép nhân em đã học từ lớp 3: nhân từng chữ số, viết hàng đơn vị, **nhớ** phần còn lại sang hàng bên trái. Bài học này dạy máy tính làm đúng như vậy.

1. Mỗi kiểu số có một "trần"

Mỗi biến số nguyên của C++ chiếm một số bit cố định, nên chỉ chứa được các số trong một khoảng cố định. Khi kết quả vượt quá khoảng ấy, máy **không báo lỗi**: nó lặng lẽ bỏ đi phần bit vượt quá (với kiểu không dấu, đó chính là lấy dư cho 2^{64}). Hãy thuộc bảng sau như thuộc bảng cửu chương:

Kiểu	Giá trị lớn nhất	Số chữ số	Xấp xỉ
`int`	$2^{31} - 1 = 2\,147\,483\,647$	10	$2{,}1 \cdot 10^{9}$
`long long`	$2^{63} - 1 = 9\,223\,372\,036\,854\,775\,807$	19	$9{,}2 \cdot 10^{18}$
`unsigned long long`	$2^{64} - 1 = 18\,446\,744\,073\,709\,551\,615$	20	$1{,}8 \cdot 10^{19}$
`__int128`	$2^{127} - 1$	39	$1{,}7 \cdot 10^{38}$

Giai thừa $n! = 1 \cdot 2 \cdot \ldots \cdot n$ tăng nhanh khủng khiếp: mỗi lần nhân thêm một số có hai chữ số, $n!$ dài thêm một hoặc hai chữ số. Hình 17.1 cho thấy chỉ tới $n = 20$ là $n!$ còn nằm dưới "trần" của `unsigned long long`.



Hình 17.1. *Số chữ số của n! với n = 0..40. Cột đỏ: n! đã vượt 2^{64}, không kiểu số nguyên 64 bit nào chứa nổi.*

n	n!	Số chữ số	Vừa `unsigned long long`?
12	479 001 600	9	có (cả `int`)
13	6 227 020 800	10	có (vượt `int`)
20	2 432 902 008 176 640 000	19	có
21	51 090 942 171 709 440 000	20	**không**
25	15 511 210 043 330 985 984 000 000	26	**không**
100	9332...0000	158	không
2000	3316...0000	5736	không

Ngay cả `__int128` cũng chỉ giữ được tới 33! (37 chữ số). Muốn đi xa hơn, ta phải tự xây "kiểu số" của riêng mình: một **mảng các chữ số**.

2. Biểu diễn số lớn bằng mảng chữ số

ĐỊNH NGHĨA (SỐ LỚN TRONG CƠ SỐ B)

Một số tự nhiên N được lưu bằng mảng a[0], a[1], ..., a[L − 1] với $0 \le a[i] < B$ và

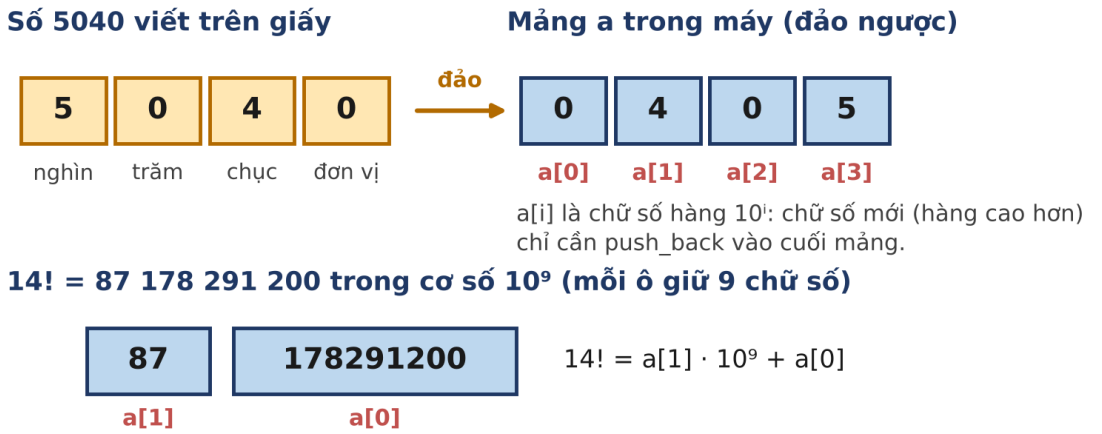
$$N = a[0] + a[1] \cdot B + a[2] \cdot B^2 + \ldots + a[L-1] \cdot B^{L-1}$$

Với B = 10, a[i] chính là **chữ số hàng 10^i**: a[0] là hàng đơn vị, a[1] hàng chục... Chữ số có nghĩa cao nhất nằm ở **cuối** mảng.

Vì sao lưu ngược, hàng đơn vị ở đầu mảng? Hãy nghĩ tới phép nhân trên giấy: ta luôn bắt đầu từ hàng đơn vị, và khi kết quả dài thêm thì chữ số mới xuất hiện ở **bên trái**, tức hàng cao nhất. Nếu hàng cao nhất ở cuối mảng thì thêm chữ số chỉ là một lệnh `push_back`, còn nếu lưu xuôi thì phải dồn cả mảng sang phải.

HÃY HÌNH DUNG

Hãy hình dung mảng a như một tờ giấy ghi số được **lật gương**: em đọc nó từ phải sang trái thì được số đúng. Chỉ khi in ra màn hình mới cần lật lại một lần.



Hình 17.2. Trên: số 5040 trong cơ số 10, lưu đảo ngược. Dưới: 14! trong cơ số 10^9, mỗi ô giữ 9 chữ số.

3. Bài toán GIAITHUA

BÀI TOÁN: GIAITHUA

Cho số tự nhiên n. Hãy in ra **chính xác** giá trị của n! = 1 · 2 · ... · n (quy ước 0! = 1).

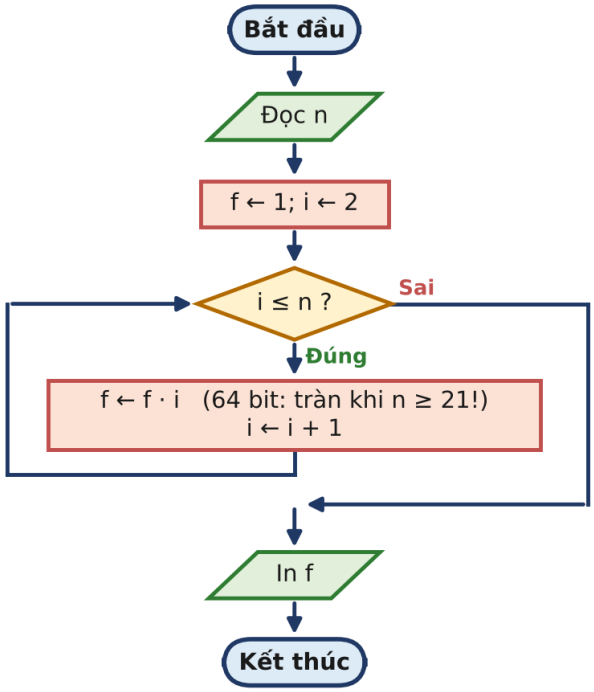
Dữ liệu vào: Một số nguyên n ($0 \le n \le 2000$).

Kết quả: Giá trị của n!, không có chữ số 0 thừa ở đầu.

GIAITHUA.INP	GIAITHUA.OUT	Giải thích
`7`	`5040`	7! = 1 · 2 · 3 · 4 · 5 · 6 · 7 = 5040.
`25`	`15511210043330985984000000`	25! có 26 chữ số, vượt xa $2^{64} \approx 1{,}8 \cdot 10^{19}$.
`0`	`1`	Theo quy ước, 0! = 1 (tích rỗng).

4. Ý tưởng ngây thơ: Nhân thẳng trong một biến

Cách đầu tiên ai cũng nghĩ ra: một biến f bắt đầu bằng 1, lần lượt nhân với 2, 3, ..., n. Ta dùng kiểu "to nhất" có sẵn là `unsigned long long`.



Hình 17.3. *Sơ đồ khối của cách nhân thẳng.*

GIAITHUA.CPP (ngây thơ): chỉ đúng khi n ≤ 20

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("GIAITHUA.INP", "r")) {
        freopen("GIAITHUA.INP", "r", stdin);
        freopen("GIAITHUA.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    unsigned long long f = 1;    // chỉ đúng khi n <= 20
    for (int i = 2; i <= n; i++)
        f *= i;                  // từ 21! trở đi: tràn số, sai âm thầm
    cout << f << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Chỉ $n - 1$ phép nhân: $O(n)$, nhanh hơn mọi cách khác. Chương trình này **cố ý sai** với $n \geq 21$: nó chỉ dùng để thấy lỗi tràn số. Trong phần kiểm tra, ta chỉ đối chiếu nó với lời giải đúng trên các test $n \leq 20$.

BẾ TẮC

Với n = 21 chương trình in 14197454024290336768, trong khi 21! = 51 090 942 171 709 440 000. Con số in ra chính là 21! mod 2^{64}: phần vượt quá 64 bit đã bị cắt bỏ âm thầm. Không có kiểu số có sẵn nào đủ chỗ cho 5736 chữ số của 2000!. Ta phải tự lưu từng chữ số. Nhưng khi đó, **làm sao nhân cả một mảng chữ số với x**?

5. Phá bế tắc: Nhân như trên giấy, mang số nhớ

Bước 1 Nhân từng chữ số, mang số nhớ sang hàng kế tiếp

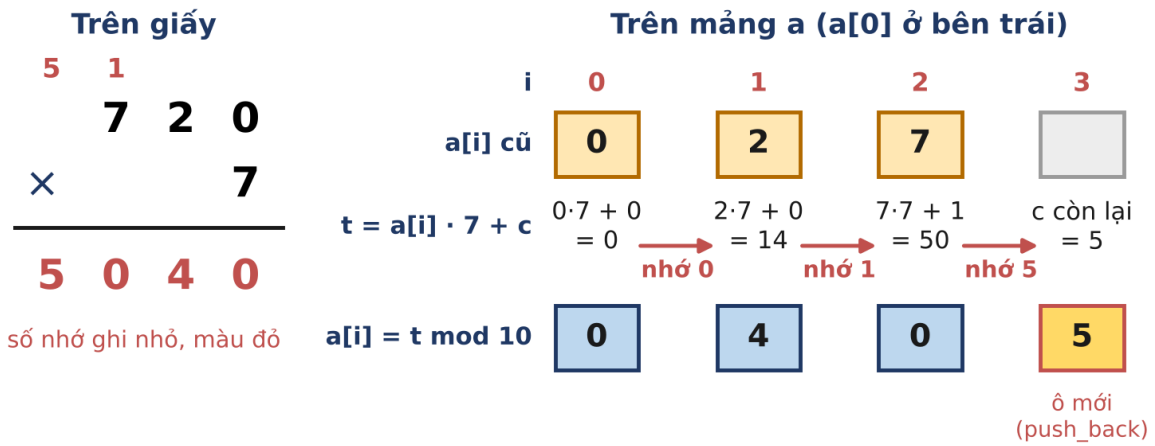
Giả sử mảng a đang lưu (x − 1)! và ta cần nhân nó với x (x ≤ 2000 là một số "nhỏ", vừa `int`). Đi từ hàng đơn vị (i = 0) lên, gọi c là **số nhớ** từ hàng trước (ban đầu c = 0). Tại hàng i:

$$t = a[i] \cdot x + c, \quad a[i] \leftarrow t \bmod 10, \quad c \leftarrow \lfloor t / 10 \rfloor$$

Hết mảng mà c vẫn khác 0 thì các chữ số của c trở thành những hàng mới ở cuối mảng. Đây đúng là phép nhân trên giấy, chỉ khác là ta nhân cả số x một lần chứ không tách x thành từng chữ số.

Bước 2 Chạy tay: 7! = 720 · 7

Sau khi nhân tới 6, mảng là a = [0, 2, 7] (tức 720). Nhân với x = 7:



Hình 17.4. Bên trái: phép nhân quen thuộc trên giấy. Bên phải: cùng phép nhân trên mảng đảo ngược; mũi tên đỏ là số nhớ đi sang hàng kế tiếp.

i	a[i] cũ	t = a[i] · 7 + c	a[i] mới	c mới
0	0	0 · 7 + 0 = 0	0	0
1	2	2 · 7 + 0 = 14	4	1
2	7	7 · 7 + 1 = 50	0	5
3 (mới)	—	c còn lại = 5	**5**	0

Mảng thành [0, 4, 0, 5], đọc ngược là 5040 = 7!. Toàn bộ quá trình tính 7! được ghi lại trong bảng dưới (sinh bằng cách chạy đúng thuật toán):

Nhân với x	Mảng a (a[0] trước)	Số được lưu = x!
2	[2]	2
3	[6]	6
4	[4, 2]	24
5	[0, 2, 1]	120
6	[0, 2, 7]	720
7	[0, 4, 0, 5]	**5040**

VÌ SAO SỐ NHỚ LUÔN NHỎ HƠN x?

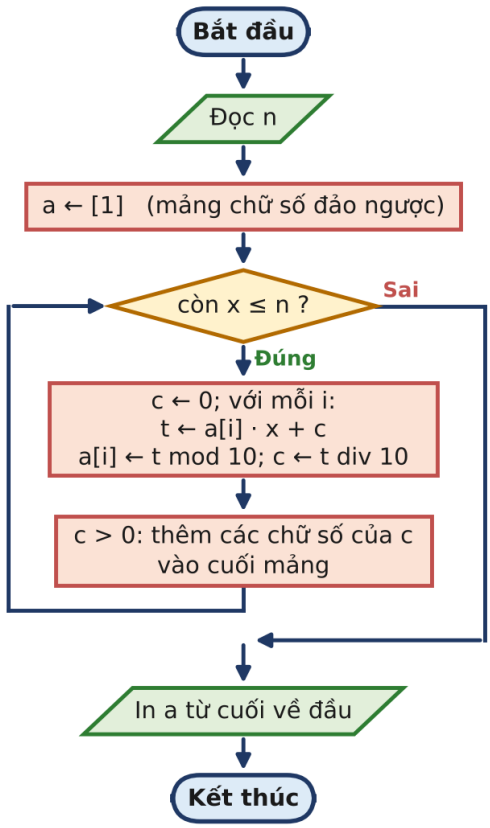
Bất biến: trước khi xử lí mỗi hàng, $0 \le c \le x - 1$.

Ban đầu $c = 0$. Giả sử $c \le x - 1$ trước hàng i. Vì $a[i] \le 9$ nên $t = a[i] \cdot x + c \le 9x + x - 1 = 10x - 1$, suy ra c mới $= \lfloor t / 10 \rfloor \le \lfloor (10x - 1) / 10 \rfloor = x - 1$. Bất biến được giữ.

Hệ quả: t luôn nhỏ hơn $10x \le 20\,000$, nên kiểu `int` là đủ, **không bao giờ tràn**. Số nhớ cuối cùng nhỏ hơn x nên có không quá 4 chữ số: mỗi lần nhân, số dài thêm tối đa 4 chữ số.

Tính đúng: tổng $a[i] \cdot 10^i + c \cdot 10^{i+1}$ (tính trên các hàng đã xử lí, cộng với phần chưa xử lí nhân x) luôn bằng (số cũ) · x; đó là phép phân phối $a \cdot x = \Sigma\, (a[i] \cdot x) \cdot 10^i$ viết lại theo từng hàng.

Bước 3 Chương trình cơ số 10



Hình 17.5. *Sơ đồ khối của phép nhân số lớn với số nhỏ.*

GIAITHUA.CPP (cơ số 10): mảng chữ số đảo ngược

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("GIAITHUA.INP", "r")) {
        freopen("GIAITHUA.INP", "r", stdin);
        freopen("GIAITHUA.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    vector<int> a(1, 1);     // a[0] là chữ số hàng đơn vị; ban đầu a = 1
    for (int x = 2; x <= n; x++) {
        int c = 0;           // số nhớ
        for (size_t i = 0; i < a.size(); i++) {
            int t = a[i] * x + c;
            a[i] = t % 10;   // chữ số giữ lại
            c = t / 10;      // phần nhớ sang hàng kế tiếp
        }
        while (c > 0) {      // số nhớ cuối cùng thành các chữ số mới
            a.push_back(c % 10);
            c /= 10;
        }
    }
    for (int i = (int)a.size() - 1; i >= 0; i--) cout << a[i];
    cout << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Lần nhân với x duyệt qua mọi chữ số của (x − 1)!. Đếm bằng máy với n = 2000: vòng lặp trong chạy tổng cộng **5 300 965** lần, tức khoảng n · L / 2 với L = 5736 là số chữ số của kết quả: O(n · L). Chương trình chạy khoảng một phần trăm giây.

Bước 4 Tăng tốc: cơ số 10^9

Mỗi ô `int` có thể chứa tới hai tỉ, vậy mà ta chỉ cất vào đó một chữ số từ 0 đến 9: thật lãng phí! Nếu mỗi ô giữ **9 chữ số** (cơ số $B = 10^9$), số ô giảm 9 lần và số phép tính cũng giảm theo. Công thức nhân giữ nguyên, chỉ thay 10 bằng B:

$$\mathbf{t = a[i] \cdot x + c, \quad a[i] \leftarrow t \bmod 10^9, \quad c \leftarrow \lfloor t / 10^9 \rfloor}$$

Lập luận ở khung "Vì sao" vẫn đúng với mọi cơ số B: số nhớ luôn nhỏ hơn x. Do đó $t < B \cdot x \le 10^9 \cdot 2000 = 2 \cdot 10^{12}$: vượt `int` nhưng rất an toàn trong `long long`. Và vì $c < x < B$ nên khi hết mảng chỉ cần thêm **đúng một ô**.

Chạy tay một bước: 13! = 6 227 020 800 được lưu thành a = [227020800, 6]. Nhân với 14:

i	a[i] cũ	t = a[i] · 14 + c	a[i] mới	c mới
0	227 020 800	3 178 291 200	178 291 200	3
1	6	6 · 14 + 3 = 87	87	0

Kết quả a = [178291200, 87], tức $14! = 87 \cdot 10^9 + 178\,291\,200 = 87\,178\,291\,200$.

LƯU Ý KHI IN SỐ CƠ SỐ 10^9

Ô cao nhất in bình thường, còn **mọi ô phía sau phải in đủ 9 chữ số**, kể cả các số 0 ở đầu ô. Số 1 000 000 005 được lưu là [5, 1]; nếu in mỗi ô bằng `%lld` ta được "15" thay vì "1000000005". Dùng `printf("%09lld", a[i])` để thêm số 0 bên trái cho đủ 9 chữ số.

GIAITHUA.CPP: số lớn cơ số 10^9

```cpp
#include <bits/stdc++.h>
using namespace std;

const long long B = 1000000000;   // cơ số 10^9: mỗi ô giữ 9 chữ số

int main() {
    if (fopen("GIAITHUA.INP", "r")) {
        freopen("GIAITHUA.INP", "r", stdin);
        freopen("GIAITHUA.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    vector<long long> a(1, 1);    // a[0] là khối 9 chữ số thấp nhất
    for (int x = 2; x <= n; x++) {
        long long c = 0;          // số nhớ, luôn nhỏ hơn x
        for (size_t i = 0; i < a.size(); i++) {
            long long t = a[i] * x + c;   // < 10^9 · 2001: vừa long long
            a[i] = t % B;
            c = t / B;
        }
        if (c > 0) a.push_back(c);        // c < x < B: đúng một ô mới
    }
    printf("%lld", a.back());             // khối đầu: không in số 0 thừa
    for (int i = (int)a.size() - 2; i >= 0; i--)
        printf("%09lld", a[i]);           // các khối sau: đủ 9 chữ số
    printf("\n");
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Với n = 2000, 2000! chỉ cần 638 ô thay vì 5736 ô, và vòng lặp trong chạy **589 883** lần thay vì 5 300 965 lần: nhanh gần 9 lần. Độ phức tạp vẫn là O(n · L), nhưng L tính theo số ô.

Với n = 2000	Cơ số 10	Cơ số 10^9
Số chữ số mỗi ô	1	9
Số ô của 2000!	5 736	638
Số lần lặp vòng trong	5 300 965	589 883
Kiểu của t = a[i] · x + c	`int` (t < 20 000)	`long long` (t < $2 \cdot 10^{12}$)

6. Mở rộng

Mở rộng 1: Cộng hai số lớn

Cộng hai số dài L_1 và L_2 chữ số cũng đi từ hàng đơn vị, mỗi hàng cộng hai chữ số và số nhớ: $x \le 9 + 9 + 1 = 19$, nên số nhớ chỉ là 0 hoặc 1. Ví dụ 98765 + 4789: hàng đơn vị 5 + 9 = 14 (viết 4 nhớ 1), rồi 6 + 8 + 1 = 15, 7 + 7 + 1 = 15, 8 + 4 + 1 = 13, 9 + 0 + 1 = 10, cuối cùng còn nhớ 1: kết quả 103554. Độ phức tạp $O(\max(L_1, L_2))$.

CONGLON.CPP: cộng hai số tự nhiên rất dài

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("CONGLON.INP", "r")) {
        freopen("CONGLON.INP", "r", stdin);
        freopen("CONGLON.OUT", "w", stdout);
    }
    string s, t;
    cin >> s >> t;                  // hai số tự nhiên rất dài
    vector<int> a, b, c;            // chữ số đảo ngược: a[0] hàng đơn vị
    for (int i = (int)s.size() - 1; i >= 0; i--) a.push_back(s[i] - '0');
    for (int i = (int)t.size() - 1; i >= 0; i--) b.push_back(t[i] - '0');
    int nho = 0;
    for (size_t i = 0; i < max(a.size(), b.size()) || nho; i++) {
        int x = nho;
        if (i < a.size()) x += a[i];
        if (i < b.size()) x += b[i];
        c.push_back(x % 10);        // x <= 9 + 9 + 1 = 19
        nho = x / 10;               // nhớ 0 hoặc 1
    }
    for (int i = (int)c.size() - 1; i >= 0; i--) cout << c[i];
    cout << "\n";
    return 0;
}
```

Phép **trừ** a − b (với a ≥ b) làm y hệt, thay "nhớ" bằng "mượn": nếu a[i] − b[i] − mượn < 0 thì cộng thêm 10 và mượn 1 từ hàng kế tiếp; cuối cùng xóa các số 0 thừa ở cuối mảng (đầu số). Để **so sánh** hai số lớn không có số 0 thừa: số nào nhiều chữ số hơn thì lớn hơn; nếu dài bằng nhau thì so từ chữ số cao nhất xuống, đúng như so hai xâu cùng độ dài.

Mở rộng 2: Nhân hai số lớn

Chữ số hàng 10^i của số thứ nhất nhân với chữ số hàng 10^j của số thứ hai cho đóng góp vào hàng 10^{i+j}. Ta cộng dồn mọi tích vào c[i + j] trước, rồi dồn nhớ **một lượt** ở cuối. Với 123 · 45: c[0] = 3 · 5 = 15, c[1] = 3 · 4 + 2 · 5 = 22, c[2] = 2 · 4 + 1 · 5 = 13, c[3] = 1 · 4 = 4. Dồn nhớ: 15 → 5 nhớ 1; 22 + 1 = 23 → 3 nhớ 2; 13 + 2 = 15 → 5 nhớ 1; 4 + 1 = 5. Kết quả 5535.

NHANLON.CPP: nhân hai số lớn, O(L1 · L2)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("NHANLON.INP", "r")) {
        freopen("NHANLON.INP", "r", stdin);
        freopen("NHANLON.OUT", "w", stdout);
    }
    string s, t;
    cin >> s >> t;
    int L1 = s.size(), L2 = t.size();
    vector<long long> c(L1 + L2, 0);   // tích có không quá L1 + L2 chữ số
    for (int i = 0; i < L1; i++)
        for (int j = 0; j < L2; j++)   // chữ số hàng 10^i nhân hàng 10^j
            c[i + j] += (s[L1 - 1 - i] - '0') * (t[L2 - 1 - j] - '0');
    for (int k = 0; k + 1 < L1 + L2; k++) {   // dồn nhớ một lượt
        c[k + 1] += c[k] / 10;
        c[k] %= 10;
    }
    int k = L1 + L2 - 1;
    while (k > 0 && c[k] == 0) k--;    // bỏ các số 0 thừa ở đầu
    for (; k >= 0; k--) cout << c[k];
    cout << "\n";
    return 0;
}
```

Mỗi c[k] là tổng của không quá $\min(L_1, L_2)$ tích, mỗi tích ≤ 81, nên `long long` không thể tràn dù số dài cả triệu chữ số. Có những thuật toán nhân nhanh hơn $O(L_1 \cdot L_2)$ (Karatsuba, biến đổi Fourier nhanh), nhưng với vài nghìn chữ số thì cách "trên giấy" này là đủ.

Mở rộng 3: Chia số lớn cho số nhỏ

Chia thì đi **ngược chiều**: từ chữ số cao nhất xuống, đúng như phép chia trên giấy và đúng như sơ đồ Horner ở Bài 1. Giữ r là số dư của phần đầu; thêm một chữ số thì $r \leftarrow r \cdot 10 +$ chữ số, chữ số tiếp theo của thương là $\lfloor r / b \rfloor$, rồi $r \leftarrow r \bmod b$. Với 2026 : 7, r lần lượt nhận 2, 20, 62, 66 và thương nhận các chữ số 0, 2, 8, 9; kết quả thương 289, dư 3.

CHIANHO.CPP: chia số lớn cho số b ≤ 10^9

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("CHIANHO.INP", "r")) {
        freopen("CHIANHO.INP", "r", stdin);
        freopen("CHIANHO.OUT", "w", stdout);
    }
    string s;
    long long b;                    // chia số lớn s cho số nhỏ b <= 10^9
    cin >> s >> b;
    string q;                       // thương, tính từ chữ số cao nhất
    long long r = 0;                // số dư của phần đầu (như Bài 1)
    for (char ch : s) {
        r = r * 10 + (ch - '0');
        q.push_back('0' + r / b);  // r < 10 · b nên r / b là một chữ số
        r %= b;
    }
    size_t k = 0;
    while (k + 1 < q.size() && q[k] == '0') k++;   // bỏ số 0 ở đầu
    cout << q.substr(k) << " " << r << "\n";
    return 0;
}
```

Mở rộng 4: Biết độ dài của n! mà không cần tính n!

Một số dương N có $\lfloor \log_{10} N \rfloor + 1$ chữ số, và $\log_{10}(n!) = \log_{10} 1 + \log_{10} 2 + ... + \log_{10} n$. Với n = 2000, tổng này bằng 5735,52... nên 2000! có **5736** chữ số: biết trước để cấp phát mảng. Số chữ số 0 tận cùng của n! thì tính bằng công thức Legendre (Bài 7): $\lfloor 2000/5 \rfloor + \lfloor 2000/25 \rfloor + \lfloor 2000/125 \rfloor + \lfloor 2000/625 \rfloor = 400 + 80 + 16 + 3 =$ **499**.

BẪY THƯỜNG GẶP

- **Tin vào kết quả tràn số:** `unsigned long long` cho 21! ra 14197454024290336768 mà không báo lỗi gì. Luôn ước lượng độ lớn của kết quả trước khi chọn kiểu.
- **Quên số nhớ cuối cùng:** hết mảng mà c > 0 thì phải thêm chữ số mới. Với cơ số 10, c có thể có tới 4 chữ số nên phải dùng `while`, không phải `if`.
- **Thiếu số 0 khi in cơ số 10^9:** các ô sau ô đầu tiên phải in bằng `%09lld`; nếu không, 1 000 000 005 in thành "15".
- **Tràn trong cơ số 10^9:** t = a[i] · x + c có thể tới $2 \cdot 10^{12}$; khai báo a là `vector<int>` và nhân `int` với `int` sẽ tràn. Dùng `long long`.
- **Lưu xuôi chữ số:** lưu chữ số cao nhất ở a[0] làm việc thêm chữ số mới rất tốn kém và dễ nhầm hàng khi cộng hai số khác độ dài.
- **Quên trường hợp 0:** 0! = 1, và số 0 trong phép nhân hai số lớn phải in ra "0" chứ không phải xâu rỗng.

GHI NHỚ

- `unsigned long long` chứa tới khoảng $1{,}8 \cdot 10^{19}$ (20! là giai thừa lớn nhất vừa); `__int128` tới khoảng $1{,}7 \cdot 10^{38}$.
- Số lớn = mảng chữ số **đảo ngược**: a[i] là hệ số của B^i, hàng mới thêm bằng `push_back`.
- Nhân số lớn với số nhỏ x: t = a[i] · x + c, giữ t mod B, nhớ $\lfloor t / B \rfloor$; số nhớ luôn < x.
- Cơ số 10^9 nhanh gần 9 lần; khi in, ô đầu in thường, các ô sau in `%09lld`.
- Cộng, trừ, nhân đi từ hàng thấp lên; chia (cho số nhỏ) đi từ hàng cao xuống, như sơ đồ Horner.

Luyện tập

Bài	Tên	Gợi ý liên hệ
17.1	**TỔNG LỚN**	Cộng hai số có tới 10^5 chữ số: Mở rộng 1, nhớ trường hợp hai số khác độ dài.
17.2	**LŨY THỪA ĐẦY ĐỦ**	In chính xác a^b ($a \le 1000$, $b \le 3000$): nhân số lớn với số nhỏ b lần, cơ số 10^9.
17.3	**TÍCH LỚN**	Nhân hai số $2 \cdot 10^4$ chữ số: cộng dồn c[i + j] trong cơ số 10^4 để giảm 16 lần số phép nhân.

Tự kiểm tra nhanh

1) 30! có bao nhiêu chữ số? Có vừa `__int128` không? *(Đáp số: 30! = 265 252 859 812 191 058 636 308 480 000 000 có 33 chữ số; vừa `__int128` (tới 39 chữ số))*

2) 100! có 158 chữ số. Trong cơ số 10^9 cần bao nhiêu ô? *(Đáp số: $\lceil 158 / 9 \rceil = 18$ ô)*

3) Số 1 000 000 005 lưu trong cơ số 10^9 là mảng nào? Nếu in mỗi ô bằng `%lld` thì được gì? *(Đáp số: [5, 1]; in ra "15")*

4) Khi nhân số lớn (cơ số 10) với x = 2000, số nhớ lớn nhất có thể là bao nhiêu? t lớn nhất là bao nhiêu? *(Đáp số: $c \le 1999$; $t \le 9 \cdot 2000 + 1999 = 19\,999$)*

BÀI 18 Hệ cơ số và biểu diễn nhị phân

Học xong bài này, em sẽ:

- ✓ Hiểu cách viết một số trong cơ số b bất kì, đổi qua lại giữa các cơ số bằng chia liên tiếp và sơ đồ Horner.
- ✓ Sử dụng thành thạo các phép toán bit `&`, `|`, `^`, `<<`, `>>` và vài "mẹo" kinh điển như `n & (n - 1)`.
- ✓ Phát hiện tính **tuần hoàn** của từng cột bit và dùng nó để đếm tổng số bit 1 của 1..n trong O(log n).
- ✓ Nhận ra khi nào đáp số vượt 2^{64} và xử lí bằng `__int128`.

KHỞI ĐỘNG

Một trò ảo thuật: em nghĩ một số từ 0 đến 15. Ảo thuật gia đưa ra 4 tấm thẻ, mỗi tấm ghi 8 số, và hỏi "số của em có trên thẻ này không?". Sau 4 câu trả lời "có/không", ông ấy đoán trúng ngay. Bí mật: tấm thẻ thứ k ghi các số có **bit thứ k bằng 1**; bốn câu trả lời chính là bốn chữ số nhị phân của số em nghĩ.

Nếu xếp các số 0, 1, 2, ..., 15 thành bảng nhị phân, mỗi cột bit lặp lại theo một nhịp rất đều: 0101..., 0011..., 00001111... Bài học này dùng chính nhịp ấy để đếm tổng số bit 1 của **một tỉ tỉ** số đầu tiên chỉ trong vài chục phép tính.

1. Hệ cơ số

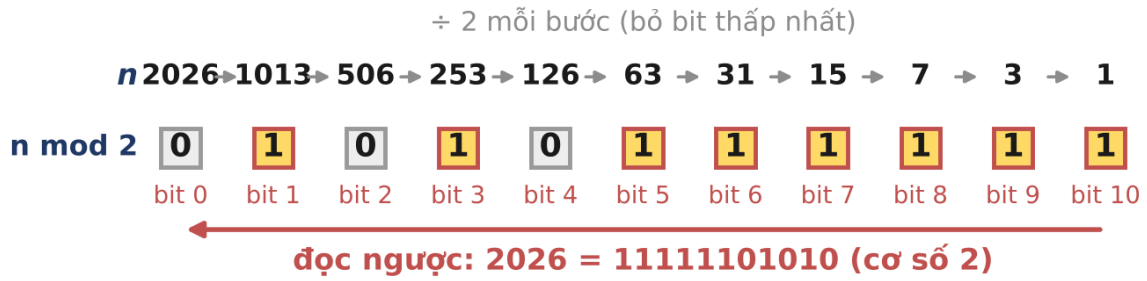
ĐỊNH NGHĨA (BIỂU DIỄN TRONG CƠ SỐ b)

Cho số nguyên $b \geq 2$. Mỗi số tự nhiên N viết được **duy nhất** dưới dạng

$$\mathbf{N = d_k \cdot b^k + d_{k-1} \cdot b^{k-1} + \ldots + d_1 \cdot b + d_0, \quad 0 \leq d_i < b, \quad d_k \neq 0}$$

Ta viết $N = (d_k\, d_{k-1} \ldots d_0)_b$. Các d_i là **chữ số** trong cơ số b; với $b > 10$ người ta dùng thêm chữ cái A = 10, B = 11, ..., Z = 35.

Làm sao tìm các chữ số? Hãy để ý: mọi số hạng trừ d_0 đều chia hết cho b, nên $\mathbf{d_0 = N \bmod b}$. Bỏ chữ số cuối đi (lấy $\lfloor N / b \rfloor$) thì được số $(d_k \ldots d_1)_b$, và lặp lại. Các chữ số sinh ra theo thứ tự từ hàng thấp lên, nên cuối cùng phải đọc ngược.



Hình 18.1. *Đổi 2026 sang cơ số 2: chia liên tiếp cho 2, ghi lại số dư, rồi đọc ngược.*

Cơ số	2026 viết là	Kiểm tra
2	11111101010	1024 + 512 + 256 + 128 + 64 + 32 + 8 + 2
8	3752	$3 \cdot 512 + 7 \cdot 64 + 5 \cdot 8 + 2$
10	2026	
16	7EA	$7 \cdot 256 + 14 \cdot 16 + 10$

Chiều ngược lại, từ các chữ số dựng lại N, chính là **sơ đồ Horner** ở Bài 1: $N \leftarrow N \cdot b + d_i$ từ chữ số cao nhất xuống. Cơ số 8 và 16 được dùng nhiều vì $8 = 2^3$ và $16 = 2^4$: mỗi chữ số thập lục phân ứng với đúng một nhóm 4 bit (7EA = 0111 1110 1010).

DOICOSO.CPP: đổi $n \le 10^{18}$ sang cơ số b ($2 \le b \le 36$)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DOICOSO.INP", "r")) {
        freopen("DOICOSO.INP", "r", stdin);
        freopen("DOICOSO.OUT", "w", stdout);
    }
    long long n;
    int b;                          // 2 <= b <= 36
    cin >> n >> b;
    const string D = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    string s;
    do {
        s.push_back(D[n % b]);      // chữ số hàng thấp nhất
        n /= b;                     // bỏ hàng thấp nhất
    } while (n > 0);                // do-while: n = 0 vẫn in "0"
    reverse(s.begin(), s.end());    // các chữ số sinh ra theo thứ tự ngược
    cout << s << "\n";
    return 0;
}
```

2. Phép toán bit

Trong máy tính mọi số nguyên đã được lưu ở dạng nhị phân, nên C++ cho phép thao tác **trực tiếp trên từng bit**, nhanh như một phép cộng. Bảng sau dùng $a = 12 = (1100)_2$ và $b = 10 = (1010)_2$:

Biểu thức	Nhị phân	Giá trị	Ý nghĩa
`a & b`	1000	8	bit 1 khi cả hai bit là 1 (AND)
`a | b`	1110	14	bit 1 khi ít nhất một bit là 1 (OR)
`a ^ b`	0110	6	bit 1 khi hai bit khác nhau (XOR)
`a << 2`	110000	48	dịch trái k bit: nhân 2^k
`a >> 2`	11	3	dịch phải k bit: $\lfloor a / 2^k \rfloor$
`(a >> 2) & 1`	1	1	đọc bit thứ 2 của a
`a & (a - 1)`	1000	8	xóa bit 1 thấp nhất
`a & -a`	100	4	giữ lại bit 1 thấp nhất
`__builtin_popcount(a)`		2	số bit 1 của a

Bit thứ k của n (tính từ 0, bên phải) là `(n >> k) & 1`; bật bit k là `n | (1 << k)`. Với số 64 bit phải viết `1LL << k`: hằng `1` có kiểu `int`, nên `1 << 40` là **tràn** chứ không phải 2^{40}. Họ hàm đếm bit có sẵn: `__builtin_popcount` (cho `int`) và `__builtin_popcountll` (cho `long long`).

3. Bài toán DEMBIT

BÀI TOÁN: DEMBIT

Gọi popcount(i) là số bit 1 trong biểu diễn nhị phân của i. Cho n, hãy tính tổng popcount(1) + popcount(2) + ... + popcount(n).

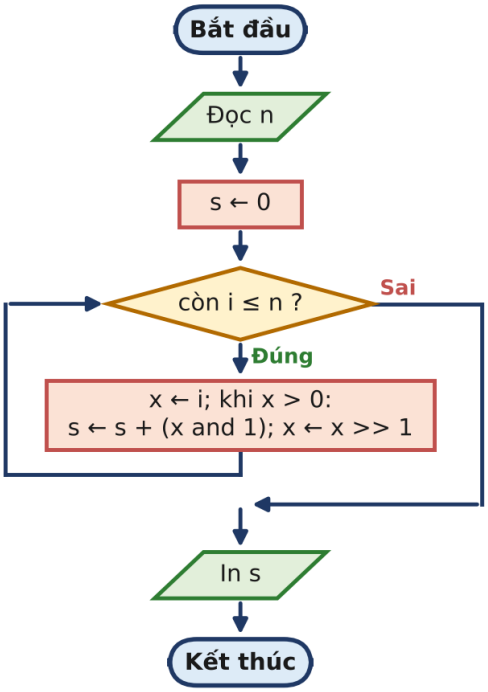
Dữ liệu vào: Một số nguyên n ($0 \le n \le 10^{18}$).

Kết quả: Tổng số bit 1 của các số 1, 2, ..., n.

DEMBIT.INP	DEMBIT.OUT	Giải thích
`12`	`22`	Số bit 1 của 1..12 lần lượt là 1, 1, 2, 1, 2, 2, 3, 1, 2, 2, 3, 2; tổng 22.
`1000000000000000000`	`29761222783429247000`	Đáp số khoảng $2{,}98 \cdot 10^{19}$, **lớn hơn 2^{64}**.
`1`	`1`	Chỉ có số 1 = $(1)_2$.

4. Ý tưởng ngây thơ: Đếm bit của từng số

Duyệt i từ 1 tới n; với mỗi i, tách lần lượt bit thấp nhất (`x & 1`) rồi dịch phải cho tới khi x = 0.



Hình 18.2. *Sơ đồ khối của cách đếm từng số.*

DEMBIT.CPP (ngây thơ): đếm bit của từng số

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DEMBIT.INP", "r")) {
        freopen("DEMBIT.INP", "r", stdin);
        freopen("DEMBIT.OUT", "w", stdout);
    }
    long long n;
    cin >> n;
    long long s = 0;              // tổng số bit 1 của 1..n
    for (long long i = 1; i <= n; i++) {
        long long x = i;
        while (x > 0) {           // đếm bit 1 của i: xét từng bit
            s += x & 1;           // bit thấp nhất
            x >>= 1;              // bỏ bit thấp nhất
        }
    }
    cout << s << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Số i có khoảng $\log_2 i$ bit, nên tổng số bước là khoảng $n \cdot \log_2 n$. Với $n = 10^6$: khoảng $2 \cdot 10^7$ bước, rất nhanh. Với $n = 10^{18}$: khoảng $6 \cdot 10^{19}$ bước.

BẾ TẮC

Máy làm khoảng 10^8 phép tính mỗi giây, nên $6 \cdot 10^{19}$ bước cần hơn **mười nghìn năm**. Duyệt từng số là vô vọng: phải đếm bit 1 mà **không** nhìn từng số. Ta đang đếm theo **hàng** (từng số một). Còn nếu đếm theo **cột** thì sao?

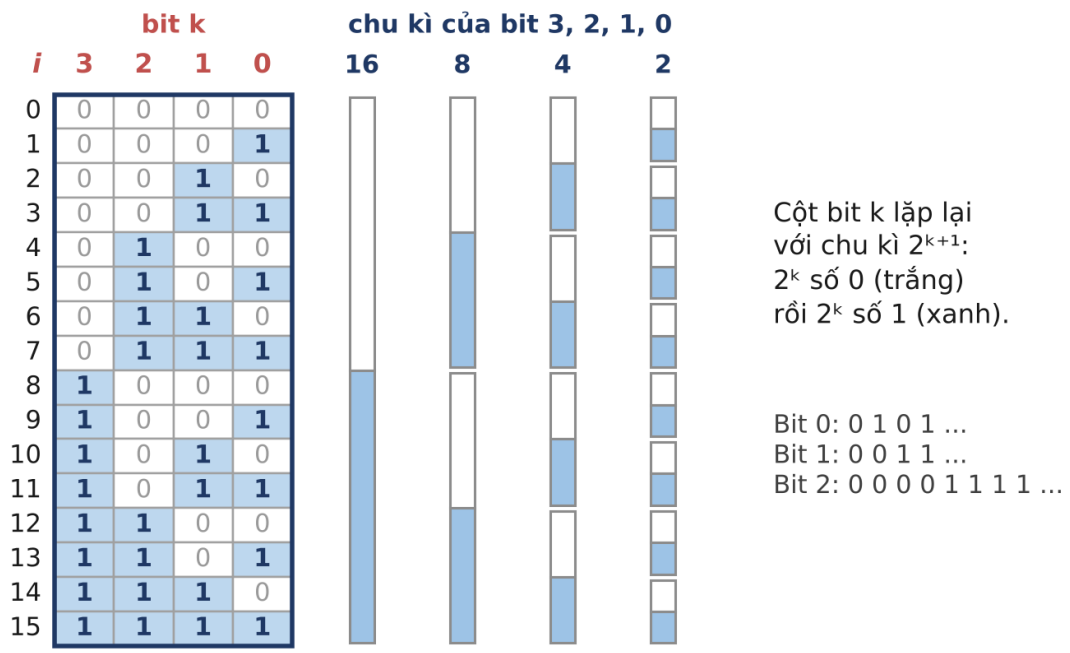
5. Phá bế tắc: Đếm theo cột thay vì theo hàng

Bước 1 Nhìn bảng bit theo cột

Viết các số 0, 1, ..., 15 thành bảng, mỗi hàng một số, mỗi cột một bit. Tổng cần tìm là **tổng số ô 1 của cả bảng**, và ta được tự do cộng theo cột thay vì theo hàng.

HÃY HÌNH DUNG

Hãy hình dung mỗi cột bit như một **dải đèn nhấp nháy**: cột bit 0 tắt-sáng-tắt-sáng; cột bit 1 tắt hai nhịp rồi sáng hai nhịp; cột bit k tắt 2^k nhịp rồi sáng 2^k nhịp, lặp mãi mãi.



Hình 18.3. *Bảng nhị phân của 0..15. Bên phải: mỗi cột bit là một dải tuần hoàn, nửa đầu chu kì là 0, nửa sau là 1.*

ĐỊNH LÍ (TÍNH TUẦN HOÀN CỦA CỘT BIT)

Trong dãy 0, 1, 2, ..., bit thứ k tuần hoàn với chu kì 2^{k+1}: trong mỗi chu kì, 2^k số đầu có bit k bằng 0 và 2^k số sau có bit k bằng 1.

VÌ SAO ĐÚNG?

Bit thứ k của i chỉ phụ thuộc vào k + 1 bit thấp nhất của i, tức vào i mod 2^{k+1}. Cộng thêm 2^{k+1} vào i không làm thay đổi i mod 2^{k+1}, nên bit k lặp lại sau mỗi 2^{k+1} số. Trong một chu kì, i mod 2^{k+1} chạy qua 0, 1, ..., $2^{k+1} - 1$; bit k của r (với $0 \le r < 2^{k+1}$) bằng 1 khi và chỉ khi $r \ge 2^k$, tức đúng nửa sau của chu kì.

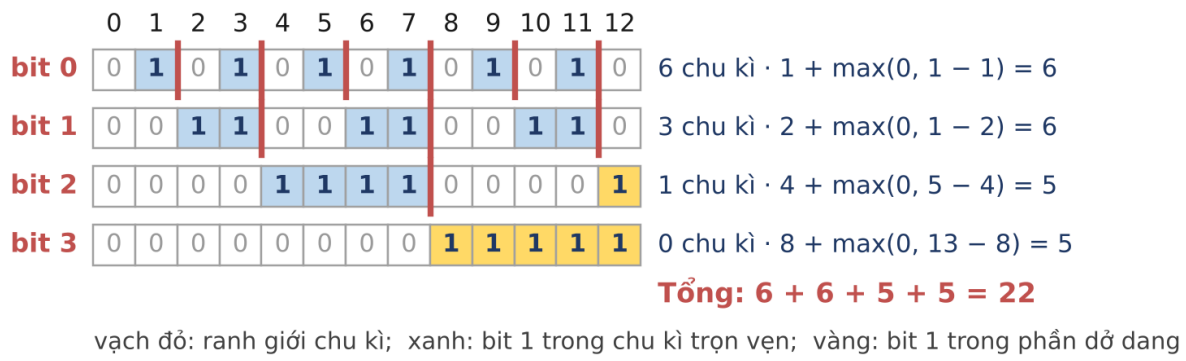
Bước 2 Công thức cho một cột

Các số 0, 1, ..., n gồm **n + 1** số (thêm số 0 không làm thay đổi tổng vì 0 không có bit 1). Với cột bit k, đặt $p = 2^k$, $T = 2p$, rồi chia n + 1 số ấy thành các chu kì trọn vẹn và một phần dở dang:

$$\mathbf{q = \lfloor (n+1) / T \rfloor, \quad r = (n+1) \bmod T, \quad \text{số bit 1 ở cột } k = q \cdot p + \max(0, r - p)}$$

Mỗi chu kì trọn vẹn góp đúng p bit 1. Phần dở dang gồm r số đầu của một chu kì: p số đầu là 0, nên chỉ có r − p số mang bit 1 nếu r > p, và không có số nào nếu r ≤ p. Chỉ cần xét các k có $2^k \le n$, tức khoảng $\log_2 n$ cột: với $n \le 10^{18}$ là không quá 60 cột.

Bước 3 Chạy tay với n = 12



Hình 18.4. *Với n = 12 (13 số 0..12), mỗi hàng là một cột bit. Vạch đỏ ngăn các chu kì; ô xanh nằm trong chu kì trọn vẹn, ô vàng trong phần dở dang.*

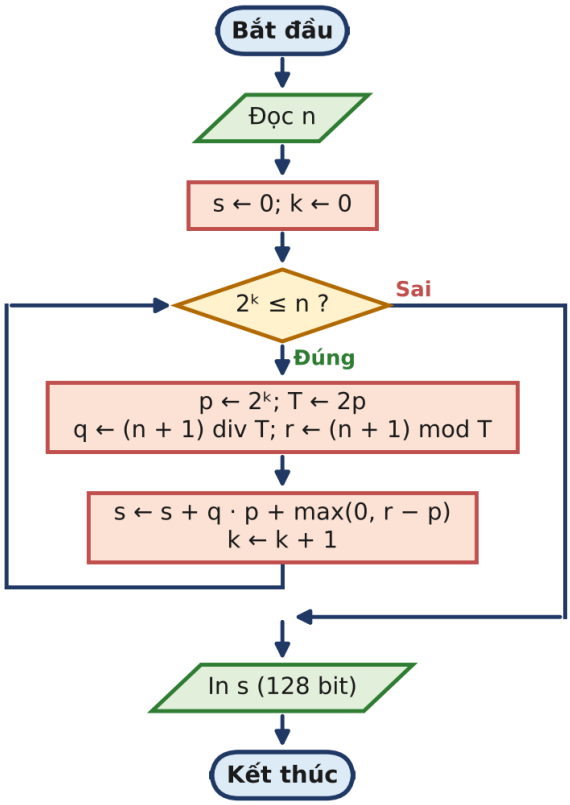
k	$p = 2^k$	T	$q = \lfloor 13 / T \rfloor$	$r = 13 \bmod T$	$\max(0, r - p)$	Cột k
0	1	2	6	1	0	6
1	2	4	3	1	0	6
2	4	8	1	5	1	5
3	8	16	0	13	5	5
					Tổng	**22**

Đáp số 22 khớp với ví dụ 1. Đến k = 4 thì $2^4 = 16 > 12$: mọi số 0..12 đều có bit 4 bằng 0, dừng lại.

Bước 4 Đáp số vượt 2^{64}

Mỗi cột đóng góp khoảng (n + 1) / 2 bit 1, và có khoảng $\log_2 n$ cột, nên đáp số xấp xỉ $n \cdot \log_2 n / 2$. Với $n = 10^{17}$ đáp số là 2 804 216 299 269 586 964, vẫn vừa `long long`. Nhưng với $n = 10^{18}$ đáp số là **29 761 222 783 429 247 000**, lớn hơn cả $2^{64} = 18\,446\,744\,073\,709\,551\,616$: tràn cả `unsigned long long`! Ta cộng dồn vào `unsigned __int128`. Vì `cout` không in được kiểu này, ta tự in bằng đệ quy: in $\lfloor x / 10 \rfloor$ trước, rồi in chữ số x mod 10, chính là ý tưởng "số lớn" của Bài 17.

Bước 5 Chương trình



Hình 18.5. *Sơ đồ khối của cách đếm theo cột.*

DEMBIT.CPP: đếm theo từng cột bit, O(log n)

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef unsigned __int128 u128;   // đáp số có thể vượt 2^64

void inso(u128 x) {               // cout không in được __int128
    if (x >= 10) inso(x / 10);
    putchar('0' + (int)(x % 10));
}

int main() {
    if (fopen("DEMBIT.INP", "r")) {
        freopen("DEMBIT.INP", "r", stdin);
        freopen("DEMBIT.OUT", "w", stdout);
    }
    long long n;
    cin >> n;
    u128 s = 0;
    for (int k = 0; (1LL << k) <= n; k++) {   // bit k của các số 0..n
        long long p = 1LL << k;     // nửa chu kì: 2^k số 0, 2^k số 1
        long long T = 2 * p;        // chu kì 2^(k+1)
        long long q = (n + 1) / T;  // số chu kì trọn vẹn
        long long r = (n + 1) % T;  // số phần tử của chu kì dở dang
        s += (u128)q * p + max(0LL, r - p);
    }
    inso(s);
    putchar('\n');
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Vòng lặp chạy qua các k với $2^k \le n$: không quá 60 lần với $n \le 10^{18}$, mỗi lần vài phép chia. Từ hơn 10^{19} bước xuống **60 bước**. Tất cả các số trung gian ($n + 1$, $T \le 2^{60}$) đều vừa `long long`; chỉ tổng s cần 128 bit.

6. Mở rộng

Mở rộng 1: Cách thứ hai, tách bit cao nhất

Gọi f(n) là tổng số bit 1 của 0..n. Các số $0..2^k - 1$ là mọi dãy k bit, mỗi bit bằng 1 ở đúng một nửa số đó, nên $f(2^k - 1) = k \cdot 2^{k-1}$. Với $2^k \le n < 2^{k+1}$, các số $2^k..n$ đều có bit k bằng 1 ($n - 2^k + 1$ bit), phần còn lại của chúng giống hệt các số $0..n - 2^k$. Do đó:

$$\mathbf{f(n) = k \cdot 2^{k-1} + (n - 2^k + 1) + f(n - 2^k)}$$

Mỗi lần đệ quy xóa đi bit cao nhất của n, nên độ sâu không quá 60. Ví dụ $f(12) = 3 \cdot 4 + 5 + f(4) = 17 + (2 \cdot 2 + 1 + f(0)) = 22$. Biết f thì tổng bit 1 của đoạn [a, b] bằng $f(b) - f(a - 1)$.

DEMBIT.CPP (cách 2): đệ quy theo bit cao nhất

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef unsigned __int128 u128;

u128 f(long long n) {              // tổng số bit 1 của 0, 1, ..., n
    if (n <= 0) return 0;
    int k = 63 - __builtin_clzll(n);  // bit cao nhất: 2^k <= n < 2^(k+1)
    long long p = 1LL << k;
    // 0..p-1 có k · 2^(k-1) bit 1; p..n có (n - p + 1) bit cao nhất,
    // phần còn lại của p..n giống hệt 0..n-p
    return (u128)k * (p / 2) + (n - p + 1) + f(n - p);
}

void inso(u128 x) {
    if (x >= 10) inso(x / 10);
    putchar('0' + (int)(x % 10));
}

int main() {
    if (fopen("DEMBIT.INP", "r")) {
        freopen("DEMBIT.INP", "r", stdin);
        freopen("DEMBIT.OUT", "w", stdout);
    }
    long long n;
    cin >> n;
    inso(f(n));
    putchar('\n');
    return 0;
}
```

Mở rộng 2: XOR các số từ 1 đến n

Kí hiệu ⊕ là phép XOR (toán tử ^ của C++). Tổng XOR 1 ⊕ 2 ⊕ ... ⊕ n cũng có quy luật tuần hoàn, lần này theo n mod 4:

n	1	2	3	4	5	6	7	8	...
1 ⊕ ... ⊕ n	1	3	0	4	1	7	0	8	...
n mod 4	1	2	3	0	1	2	3	0	...

Quy luật: n mod 4 = 0 cho kết quả n; = 1 cho 1; = 2 cho n + 1; = 3 cho 0. Lí do: với mọi số chẵn m, m ⊕ (m + 1) = 1 (hai số chỉ khác bit 0), nên các cặp (0, 1), (2, 3), ... triệt tiêu theo từng đôi. Ví dụ 2026 mod 4 = 2, nên 1 ⊕ 2 ⊕ ... ⊕ 2026 = 2027.

Mở rộng 3: Cơ số −2

Cơ số không nhất thiết phải dương! Trong cơ số −2, mỗi số nguyên (kể cả số âm) viết được duy nhất bằng các chữ số 0, 1. Ví dụ $6 = (11010)_{-2} = 16 - 8 + 0 - 2 + 0$. Cách đổi vẫn là chia liên tiếp cho −2, nhưng số dư phải thuộc {0, 1}; khi C++ cho số dư −1 (vì phép chia làm tròn về 0, xem Bài 1) thì sửa: r ← r + 2 và thương tăng thêm 1, vì $n = (-2) \cdot q + r = (-2) \cdot (q + 1) + (r + 2)$.

COSOAM2.CPP: đổi số nguyên sang cơ số −2

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("COSOAM2.INP", "r")) {
        freopen("COSOAM2.INP", "r", stdin);
        freopen("COSOAM2.OUT", "w", stdout);
    }
    long long n;                  // |n| <= 10^18, có thể âm
    cin >> n;
    string s;
    while (n != 0) {
        long long r = n % (-2);   // C++: r thuộc {-1, 0, 1}
        n /= -2;
        if (r < 0) {              // sửa số dư âm: r += 2, thương tăng 1
            r += 2;
            n += 1;
        }
        s.push_back('0' + r);
    }
    if (s.empty()) s = "0";
    reverse(s.begin(), s.end());
    cout << s << "\n";
    return 0;
}
```

Mở rộng 4: Duyệt mọi tập con bằng số nhị phân

Mỗi số mask từ 0 tới $2^n - 1$ tương ứng một tập con của {0, 1, ..., n − 1}: phần tử i thuộc tập khi bit i của mask bằng 1. Đây là cách viết ngắn nhất để duyệt mọi tập con, dùng nhiều ở Bài 14 (bù trừ):

Duyệt 2^n tập con, $O(2^n \cdot n)$

```cpp
for (int mask = 0; mask < (1 << n); mask++) {
    long long s = 0;
    for (int i = 0; i < n; i++)
        if ((mask >> i) & 1) s += a[i];   // phần tử i có trong tập
    // ... xử lí tập con mask có tổng s
}
```

BẪY THƯỜNG GẶP

- **`1 << k` với k ≥ 31:** hằng 1 có kiểu `int`; phải viết `1LL << k` khi cần số 64 bit.
- **Thứ tự ưu tiên của toán tử bit:** `x & 1 == 0` được hiểu là `x & (1 == 0)`. Luôn đặt ngoặc: `(x & 1) == 0`.
- **Quên số 0 trong đếm chu kì:** công thức dùng n + 1 số 0..n; dùng n thay cho n + 1 sẽ lệch cả chu kì.
- **Đáp số tràn 2^{64}:** với $n = 10^{18}$ tổng số bit 1 vượt $1{,}8 \cdot 10^{19}$. Dùng `unsigned __int128` và tự viết hàm in.
- **Quên đảo ngược khi đổi cơ số:** chia liên tiếp sinh chữ số từ hàng thấp lên; với n = 0 vẫn phải in "0" (dùng `do ... while`).

GHI NHỚ

- Chữ số cuối trong cơ số b là N mod b; bỏ chữ số cuối là $\lfloor N / b \rfloor$. Dựng lại số bằng Horner.
- Bit k của n là `(n >> k) & 1`; `n & (n - 1)` xóa bit 1 thấp nhất; dùng `1LL << k` cho số 64 bit.
- Cột bit k của dãy 0, 1, 2, ... tuần hoàn chu kì 2^{k+1}: 2^k số 0 rồi 2^k số 1.
- Tổng bit 1 của 1..n = $\Sigma_k [q \cdot 2^k + \max(0, r - 2^k)]$ với q, r là thương và dư của n + 1 cho 2^{k+1}: O(log n).
- Đổi góc nhìn "theo hàng" sang "theo cột" là một kĩ thuật đếm rất mạnh.

Luyện tập

Bài	Tên	Gợi ý liên hệ
18.1	**ĐỔI CƠ SỐ**	Đổi một số từ cơ số a sang cơ số b ($2 \le a, b \le 36$): Horner để về giá trị, rồi chia liên tiếp.
18.2	**BIT TRONG ĐOẠN**	Q truy vấn tổng số bit 1 của các số trong [L, R] ≤ 10^{18}: f(R) − f(L − 1) với hàm f của bài.
18.3	**SỐ CÓ k BIT 1**	Đếm số trong [1, n] có đúng k bit 1: duyệt bit 1 của n từ cao xuống, cộng tổ hợp C(số bit còn lại, k còn lại) (Bài 13).

Tự kiểm tra nhanh

1) Tính tổng số bit 1 của các số 1..100 bằng công thức theo cột. *(Đáp số: 319)*

2) Số 2026 có bao nhiêu bit 1? Biểu thức `2026 & 2025` cho giá trị nào? *(Đáp số: 8 bit 1 (11111101010); 2026 & 2025 = 2024 (xóa bit 1 thấp nhất))*

3) Tính 1 ⊕ 2 ⊕ 3 ⊕ ... ⊕ 2026 (⊕ là XOR). *(Đáp số: 2026 mod 4 = 2 nên kết quả là 2027)*

4) Viết −3 trong cơ số −2. *(Đáp số: $(1101)_{-2} = -8 + 4 + 0 + 1$)*

BÀI 19 Tổng tiền tố và số dư

Học xong bài này, em sẽ:

- ✓ Dùng **tổng tiền tố** để tính tổng một đoạn con bất kì trong O(1).
- ✓ Chuyển điều kiện "tổng đoạn chia hết cho k" thành "hai tổng tiền tố **cùng số dư**".
- ✓ Đếm số đoạn con có tổng chia hết cho k trong O(n + k), kể cả khi các số có thể âm.
- ✓ Hiểu **nguyên lí Dirichlet** và dùng nó để chứng minh sự tồn tại, tìm chu kì của dãy số dư.

KHỞI ĐỘNG

Trong một trò chơi, mỗi ngày em được cộng hoặc trừ điểm: +2, −7, +5, +4, −3, +1. Cửa hàng chỉ đổi quà cho những **chuỗi ngày liên tiếp** có tổng điểm chia hết cho 3. Có bao nhiêu chuỗi ngày như vậy? Với 6 ngày, em có thể thử cả 21 chuỗi. Với một triệu ngày thì có khoảng $5 \cdot 10^{11}$ chuỗi: thử hết là không thể.

Mẹo của bài học này: thay vì nhìn từng chuỗi ngày, hãy nhìn **tổng điểm tích lũy** sau mỗi ngày, và chỉ để ý số dư của nó khi chia cho 3. Hai ngày có cùng số dư "tích lũy" sẽ cho ngay một chuỗi đổi được quà.

1. Tổng tiền tố

ĐỊNH NGHĨA (TỔNG TIỀN TỐ)

Cho dãy $a_1, a_2, ..., a_n$. Tổng tiền tố là dãy $S_0 = 0$, $S_i = a_1 + a_2 + ... + a_i = S_{i-1} + a_i$ $(1 \le i \le n)$.

Khi đó tổng của đoạn con $a_l, ..., a_r$ $(1 \le l \le r \le n)$ bằng

$$\mathbf{a_l + a_{l+1} + ... + a_r = S_r - S_{l-1}}$$

Vì sao? S_r là tổng r số đầu, S_{l-1} là tổng l − 1 số đầu; hiệu của chúng chính là phần "ở giữa", từ a_l tới a_r. Tính dãy S mất O(n) một lần, sau đó tổng của **bất kì** đoạn nào cũng chỉ là một phép trừ. Phần tử $S_0 = 0$ (tiền tố rỗng) không phải là chi tiết thừa: nhờ nó mà các đoạn bắt đầu từ l = 1 cũng dùng được công thức.

i	0	1	2	3	4	5	6
a_i		2	−7	5	4	−3	1
S_i	0	2	−5	0	4	1	2

Ví dụ: $a_3 + a_4 = 5 + 4 = 9 = S_4 - S_2 = 4 - (-5)$. Còn $a_2 + ... + a_6 = S_6 - S_1 = 2 - 2 = 0$.

2. Khi tổng tiền tố gặp số dư

Đoạn [l, r] có tổng chia hết cho k khi và chỉ khi k | $S_r - S_{l-1}$. Theo định nghĩa đồng dư ở Bài 1, điều đó có nghĩa là:

> **ĐỊNH LÍ (ĐOẠN CHIA HẾT VÀ TIỀN TỐ CÙNG DƯ)**
>
> Tổng $a_l + ... + a_r$ chia hết cho k $\Leftrightarrow$ $S_r \equiv S_{l-1} \pmod k$ $\Leftrightarrow$ $S_r \bmod k = S_{l-1} \bmod k$.

Như vậy, mỗi đoạn con cần đếm ứng với đúng một **cặp chỉ số** i < j (i = l − 1, j = r) trong 0..n có S_i và S_j cùng số dư khi chia cho k. Bài toán đếm đoạn trở thành bài toán đếm cặp cùng số dư.

> **LƯU Ý VỚI SỐ ÂM**
>
> Ở bảng trên, $S_2 = -5$. Trong C++, `-5 % 3` bằng −2, còn số dư theo nghĩa toán học là 1. Nếu dùng thẳng `%` thì S_2 và $S_4 = 4$ (dư 1) bị xếp vào hai "ngăn" khác nhau và ta đếm sót đoạn [3, 4]. Phải dùng công thức của Bài 1: `((x % k) + k) % k`.

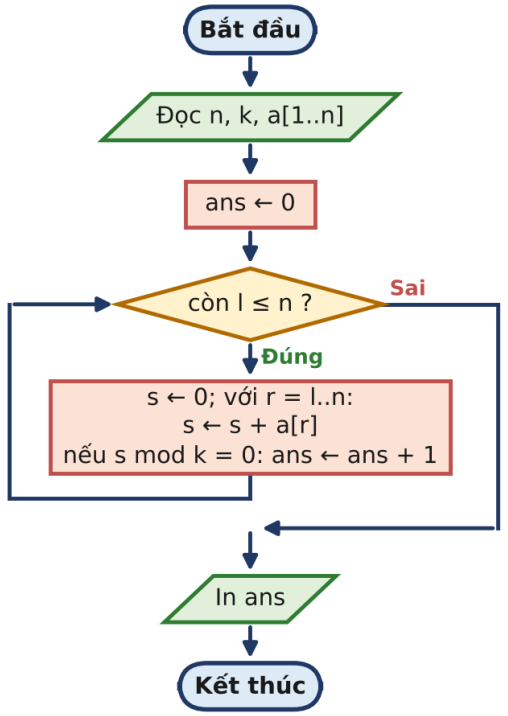
3. Bài toán DOANCHIA

> **BÀI TOÁN: DOANCHIA**
>
> Cho dãy n số nguyên $a_1, a_2, ..., a_n$ (có thể âm) và số nguyên dương k. Hãy đếm số cặp (l, r) với $1 \le l \le r \le n$ sao cho tổng $a_l + a_{l+1} + ... + a_r$ chia hết cho k.
>
> **Dữ liệu vào:** Dòng đầu chứa n và k ($1 \le n \le 10^6$, $1 \le k \le 10^6$). Dòng thứ hai chứa n số a_i ($|a_i| \le 10^9$).
>
> **Kết quả:** Số đoạn con có tổng chia hết cho k.

DOANCHIA.INP	DOANCHIA.OUT	Giải thích
`6 3` `2 -7 5 4 -3 1`	`5`	5 đoạn: [1, 3] (tổng 0), [2, 6] (tổng 0), [3, 4] (tổng 9), [3, 5] (tổng 6), [5, 5] (tổng −3).
`4 5` `5 0 -10 5`	`10`	Mọi tổng tiền tố 0, 5, 5, −5, 0 đều chia hết cho 5, nên cả 10 đoạn con đều thỏa mãn.
`3 2` `1 1 1`	`2`	Đoạn có tổng chẵn là đoạn có độ dài chẵn: [1, 2] và [2, 3].

4. Ý tưởng ngây thơ: Thử mọi đoạn con

Với mỗi điểm đầu l, đi dần điểm cuối r từ l tới n và cộng dồn tổng; mỗi lần kiểm tra tổng có chia hết cho k không. Cộng dồn giúp ta không phải tính lại tổng từ đầu cho mỗi đoạn.



Hình 19.1. *Sơ đồ khối của cách thử mọi đoạn con.*

DOANCHIA.CPP (ngây thơ): thử mọi đoạn, O(n²)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DOANCHIA.INP", "r")) {
        freopen("DOANCHIA.INP", "r", stdin);
        freopen("DOANCHIA.OUT", "w", stdout);
    }
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long k;
    cin >> n >> k;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    long long ans = 0;
    for (int l = 0; l < n; l++) {
        long long s = 0;              // tổng a[l..r]
        for (int r = l; r < n; r++) {
            s += a[r];
            if (s % k == 0) ans++;    // đúng cả khi s âm
        }
    }
    cout << ans << "\n";
    return 0;
}
```

Ở đây `s % k == 0` vẫn đúng khi s âm, vì một bội của k có số dư 0 theo cả hai quy ước. Tổng s có thể tới $10^6 \cdot 10^9 = 10^{15}$ nên phải dùng `long long`.

ĐẾM SỐ PHÉP TÍNH

Có $n \cdot (n + 1) / 2$ đoạn con. Với $n = 10^6$ đó là $500\,000\,500\,000 \approx 5 \cdot 10^{11}$ lần kiểm tra: hàng chục phút chạy máy, trong khi giới hạn thường chỉ 1 giây.

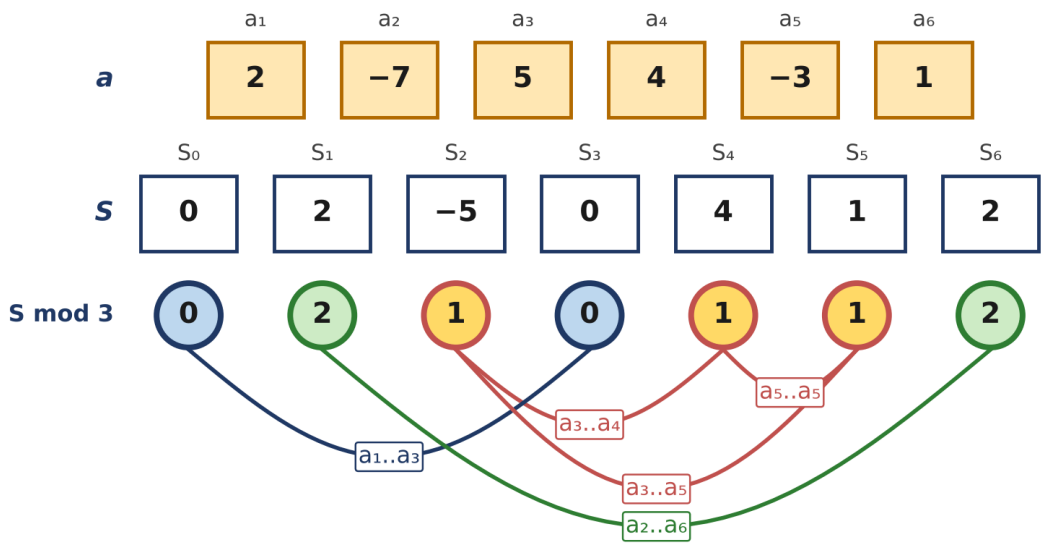
BẾ TẮC

Ta đang xét từng đoạn con một, mà số đoạn con lại là $n^2 / 2$. Để nhanh hơn, ta phải **đếm nhiều đoạn cùng một lúc** mà không liệt kê chúng. Định lí ở mục 2 gợi ý: điều duy nhất quan trọng về mỗi tổng tiền tố là **số dư** của nó.

5. Phá bế tắc: Gom tổng tiền tố theo số dư

Bước 1 Đổi đoạn con thành cặp tiền tố cùng màu

Tô màu mỗi tổng tiền tố S_0, ..., S_n theo số dư của nó khi chia cho k. Mỗi đoạn con có tổng chia hết cho k là một cặp hai tiền tố **cùng màu**, và ngược lại.

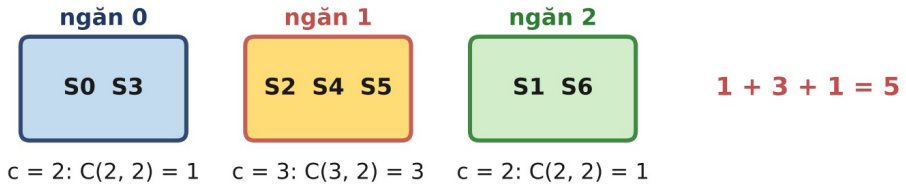


Hình 19.2. *Dãy của ví dụ 1 với k = 3: các tổng tiền tố được tô màu theo số dư; mỗi cung nối hai tiền tố cùng màu là một đoạn con có tổng chia hết cho 3.*

Bước 2 Đếm cặp trong từng ngăn

Bỏ các tổng tiền tố vào k "ngăn" theo số dư. Nếu ngăn d có c_d phần tử thì nó tạo ra $C(c_d, 2) = c_d \cdot (c_d - 1) / 2$ cặp (Bài 13), và các cặp ở hai ngăn khác nhau không bao giờ được tính. Vậy:

$$\textbf{Số đoạn} = \Sigma_{d=0..k-1}\, c_d \cdot (c_d - 1) / 2$$



Hình 19.3. *Bảy tổng tiền tố của ví dụ 1 rơi vào ba ngăn; số cặp trong các ngăn là 1 + 3 + 1 = 5.*

Bước 3 Đếm ngay khi đọc

Ta không cần chờ đọc hết dãy rồi mới dùng công thức. Duy trì mảng cnt[d] = số tiền tố **đã gặp** có số dư d. Khi tính xong S_r với số dư s, mọi tiền tố trước đó cùng số dư đều ghép với S_r thành một đoạn kết thúc tại r, nên ta cộng cnt[s] vào đáp số **trước**, rồi mới tăng cnt[s]. Ban đầu cnt[0] = 1 vì $S_0 = 0$. Chạy tay ví dụ 1:

r	a_r	S_r	s % 3 (C++)	s = S_r mod 3	cnt[s] trước	ans
0		0	0	0	(khởi tạo)	0
1	2	2	2	2	0	0
2	−7	−5	**−2**	**1**	0	0
3	5	0	0	0	1	1
4	4	4	1	1	1	2
5	−3	1	1	1	2	4
6	1	2	2	2	1	**5**

Ở hàng r = 2, phép % của C++ cho −2; nếu dùng giá trị này làm chỉ số mảng thì chương trình truy cập cnt[−2] và chạy sai (hoặc bị dừng). Sửa thành số dư không âm 1 thì S_2 được xếp đúng ngăn với S_4, S_5.

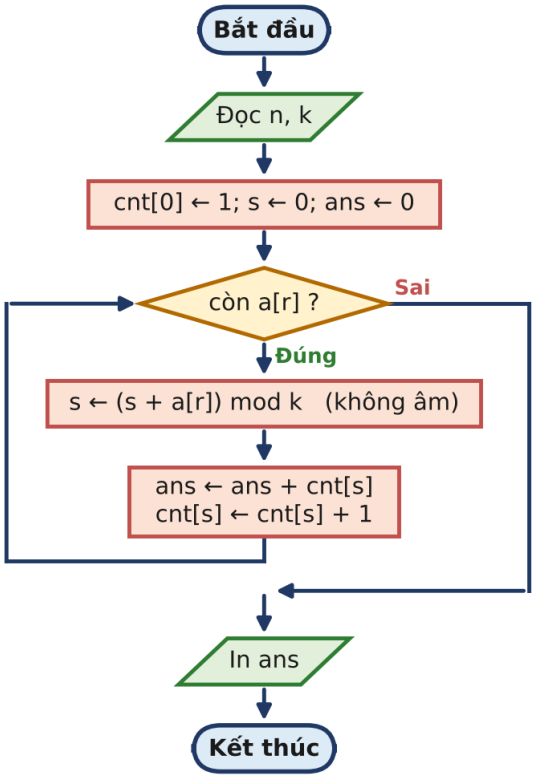
VÌ SAO ĐÚNG?

Bất biến: sau khi xử lí r, ans bằng số cặp $i < j \le r$ có $S_i \equiv S_j \pmod k$, và cnt[d] bằng số chỉ số $i \le r$ có $S_i \bmod k = d$.

Khi chuyển từ r − 1 sang r, các cặp mới chính là các cặp (i, r) với $i < r$ và $S_i \equiv S_r$; số cặp này đúng bằng cnt[s] trước khi tăng. Theo định lí ở mục 2, mỗi cặp (i, j) như vậy ứng với đúng một đoạn [i + 1, j] có tổng chia hết cho k. Khi r = n, ans là đáp số.

Ta cũng không cần lưu S_r thật: chỉ cần $s = S_r \bmod k$, cập nhật bằng $s \leftarrow (s + a_r) \bmod k$ theo tính chất cộng của đồng dư (Bài 1). Nhờ vậy s luôn nhỏ hơn k, không bao giờ tràn.

Bước 4 Chương trình



Hình 19.4. *Sơ đồ khối của cách đếm theo số dư.*

DOANCHIA.CPP: đếm cặp tiền tố cùng số dư, O(n + k)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DOANCHIA.INP", "r")) {
        freopen("DOANCHIA.INP", "r", stdin);
        freopen("DOANCHIA.OUT", "w", stdout);
    }
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long k;
    cin >> n >> k;
    vector<long long> cnt(k, 0);  // cnt[d]: số tổng tiền tố có dư d
    cnt[0] = 1;                   // S[0] = 0 (tiền tố rỗng)
    long long s = 0, ans = 0;     // s = S[r] mod k, luôn trong [0, k)
    for (int r = 1; r <= n; r++) {
        long long a;
        cin >> a;
        s = ((s + a) % k + k) % k;    // số dư không âm (Bài 1)
        ans += cnt[s];            // ghép với mọi S[l - 1] cùng số dư
        cnt[s]++;
    }
    cout << ans << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Một lượt đọc dãy, mỗi phần tử vài phép tính: O(n), cộng O(k) để khởi tạo mảng cnt. Với $n = 10^6$: khoảng một triệu bước thay vì $5 \cdot 10^{11}$. Đáp số có thể tới $5 \cdot 10^{11}$ nên ans phải là `long long`.

6. Mở rộng

Mở rộng 1: Nguyên lí Dirichlet

ĐỊNH LÍ (NGUYÊN LÍ DIRICHLET)

Nếu bỏ n + 1 vật vào n cái hộp thì có ít nhất một hộp chứa từ 2 vật trở lên.

Nghe hiển nhiên, nhưng kết hợp với tổng tiền tố nó cho một kết quả bất ngờ: **trong n số nguyên bất kì luôn có một đoạn con (khác rỗng) có tổng chia hết cho n.** Thật vậy, n + 1 tổng tiền tố $S_0, ..., S_n$ chỉ có n số dư có thể khi chia cho n, nên có hai tiền tố $S_i \equiv S_j$ (i < j), và đoạn [i + 1, j] là đoạn cần tìm. Ví dụ với dãy 2, 7, 4, 6, 8 (n = 5): các tổng tiền tố 0, 2, 9, 13, 19, 27 có số dư 0, 2, 4, 3, 4, 2. Hai cặp cùng dư cho hai đoạn: 7 + 4 + 6 + 8 = 25 và 4 + 6 = 10, đều chia hết cho 5.

Chương trình tìm đoạn ấy chính là DOANCHIA với k = n, chỉ thay vì đếm thì lưu lại **vị trí đầu tiên** first[d] gặp mỗi số dư d; lần đầu gặp lại một số dư là có ngay đáp án.

Mở rộng 2: Đoạn con dài nhất có tổng chia hết cho k

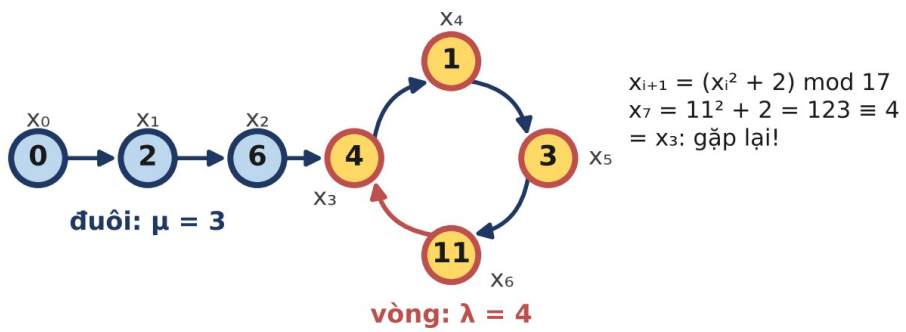
Cũng với mảng first[d] (vị trí nhỏ nhất i có $S_i \bmod k = d$), đoạn dài nhất kết thúc tại r và có tổng chia hết cho k là [first[s] + 1, r], dài r − first[s]. Lấy max theo mọi r: vẫn O(n + k). Nhớ khởi tạo first[0] = 0 cho S_0.

Mở rộng 3: Khi k quá lớn

Nếu k tới 10^{18} thì không thể khai báo mảng cnt kích thước k. Nhưng chỉ có n + 1 số dư thực sự xuất hiện: ta tính hết các số dư, **sắp xếp** chúng, rồi đếm các nhóm giá trị bằng nhau, O(n log n). Cũng có thể dùng `map<long long, long long>`. Chú ý khi đó tính $s \leftarrow (s + a_r) \bmod k$ phải dùng `__int128` hoặc lấy dư a_r trước để tránh tràn.

Mở rộng 4: Tìm chu kì của dãy $x_{i+1} = f(x_i) \bmod m$

Dãy x_0, $x_1 = f(x_0)$, $x_2 = f(x_1)$, ... chỉ nhận m giá trị 0..m − 1, nên theo nguyên lí Dirichlet, trong m + 1 phần tử đầu phải có hai phần tử bằng nhau. Từ lần lặp đầu tiên $x_\mu = x_{\mu+\lambda}$ trở đi, dãy lặp lại mãi mãi với chu kì λ. Hình dạng của dãy giống chữ ρ (rho): một "đuôi" dài μ rồi một "vòng" dài λ.



Hình 19.5. *Dãy $x_{i+1} = (x_i^2 + 2) \bmod 17$ bắt đầu từ $x_0 = 0$: đuôi 0, 2, 6 rồi vòng 4, 1, 3, 11.*

i	0	1	2	3	4	5	6	7
x_i	0	2	6	**4**	1	3	11	**4**

Để tìm μ và λ, lưu first[v] = chỉ số đầu tiên mà $x_i = v$. Khi gặp một giá trị đã có first, ta có μ = first[x_i] và λ = i − μ. Ở ví dụ trên, $x_7 = 4 = x_3$ nên μ = 3, λ = 4. Khi m quá lớn để lưu mảng, người ta dùng thuật toán **Floyd "rùa và thỏ"** chỉ cần O(1) bộ nhớ.

CHUKI.CPP: tìm đuôi μ và chu kì λ, O(m)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("CHUKI.INP", "r")) {
        freopen("CHUKI.INP", "r", stdin);
        freopen("CHUKI.OUT", "w", stdout);
    }
    long long x, c, m;            // x[i+1] = (x[i]^2 + c) mod m
    cin >> x >> c >> m;           // m <= 10^6, 0 <= x, c < m
    vector<int> first(m, -1);     // first[v]: chỉ số đầu tiên có x[i] = v
    for (int i = 0;; i++) {
        if (first[x] != -1) {     // gặp lại một giá trị: đã vào vòng
            int mu = first[x], lam = i - first[x];
            cout << mu << " " << lam << "\n";
            break;
        }
        first[x] = i;
        x = (x * x + c) % m;
    }
    return 0;
}
```

BẪY THƯỜNG GẶP

- **Số dư âm làm chỉ số mảng:** `S % k` có thể âm khi S âm; `cnt[S % k]` truy cập ngoài mảng. Dùng `((s + a) % k + k) % k`.
- **Quên tiền tố rỗng $S_0 = 0$:** thiếu `cnt[0] = 1` thì mọi đoạn bắt đầu từ vị trí 1 đều bị bỏ sót.
- **Tăng cnt trước khi cộng:** nếu tăng cnt[s] rồi mới cộng vào ans, ta đếm cả "đoạn rỗng" (i = j) và đáp số dư ra n.
- **Tràn đáp số:** số đoạn con có thể tới $5 \cdot 10^{11}$, vượt `int`. Dùng `long long` cho ans và cho cnt.
- **Cộng dồn tổng thật mà không lấy dư:** tổng tiền tố tới 10^{15} vẫn vừa `long long`, nhưng nếu a_i tới 10^{18} thì tràn; lấy dư ngay sau mỗi bước.

GHI NHỚ

- $S_0 = 0$, $S_i = S_{i-1} + a_i$; tổng đoạn $[l, r] = S_r - S_{l-1}$.
- Tổng đoạn [l, r] chia hết cho k $\Leftrightarrow$ $S_r \equiv S_{l-1} \pmod k$.
- Đếm đoạn = đếm cặp tiền tố cùng số dư: $\Sigma\, C(c_d, 2)$, hoặc cộng cnt[s] trước khi tăng, O(n + k).
- Luôn dùng số dư không âm `((x % k) + k) % k` khi số dư làm chỉ số mảng.
- Nguyên lí Dirichlet: n + 1 tiền tố, n số dư, nên luôn có đoạn con chia hết cho n; dãy số dư luôn tuần hoàn.

Luyện tập

Bài	Tên	Gợi ý liên hệ
19.1	**TỔNG BẰNG KHÔNG**	Đếm đoạn con có tổng bằng 0: đếm cặp tiền tố bằng nhau (sắp xếp hoặc `map`).
19.2	**ĐOẠN DÀI NHẤT**	Đoạn con dài nhất có tổng chia hết cho k: lưu vị trí đầu tiên của mỗi số dư (Mở rộng 2).
19.3	**HÌNH CHỮ NHẬT CHIA HẾT**	Đếm hình chữ nhật con của ma trận m × n có tổng chia hết cho k: cố định cặp hàng, quy về DOANCHIA trên dãy tổng cột, $O(m^2 \cdot n)$.

Tự kiểm tra nhanh

1) Dãy 1, 2, 3, 4, 5 có bao nhiêu đoạn con có tổng chia hết cho 3? *(Đáp số: các tiền tố có số dư 0, 1, 0, 0, 1, 0, nên C(4, 2) + C(2, 2) = 6 + 1 = 7)*

2) Trong C++, `-7 % 3` bằng bao nhiêu? Số dư dùng làm chỉ số cnt phải là bao nhiêu? *(Đáp số: −1 và 2)*

3) Dãy 1, 1, 1, 1 với k = 2 có bao nhiêu đoạn con có tổng chẵn? *(Đáp số: tiền tố dư 0, 1, 0, 1, 0, nên C(3, 2) + C(2, 2) = 4)*

BÀI 20 Điểm nguyên và hình học số nguyên

Học xong bài này, em sẽ:

- ✓ Biết một điểm nguyên là gì và tham số hóa các điểm trên một đoạn thẳng.
- ✓ Chứng minh được: đoạn thẳng nối hai điểm nguyên chứa đúng gcd(|dx|, |dy|) + 1 điểm nguyên.
- ✓ Xử lí đúng các trường hợp đặc biệt: đoạn thẳng đứng, nằm ngang, suy biến thành một điểm, tọa độ âm.
- ✓ Tính diện tích đa giác bằng công thức dây giày và đếm điểm nguyên bên trong bằng **định lí Pick**.

KHỞI ĐỘNG

Một vườn cây được trồng theo ô bàn cờ: mỗi gốc cây đứng ở một điểm có tọa độ nguyên. Người làm vườn căng một sợi dây thẳng từ gốc (1, 1) tới gốc (13, 9). Sợi dây chạm vào bao nhiêu gốc cây, tính cả hai đầu?

Em có thể vẽ ra giấy kẻ ô và đếm: 5 gốc. Nhưng nếu hai đầu dây là $(-10^9, -10^9)$ và $(10^9, 10^9)$ thì sao? Đáp số nằm ở một người bạn cũ: **ước chung lớn nhất**.

1. Điểm nguyên và bước nhảy trên đoạn thẳng

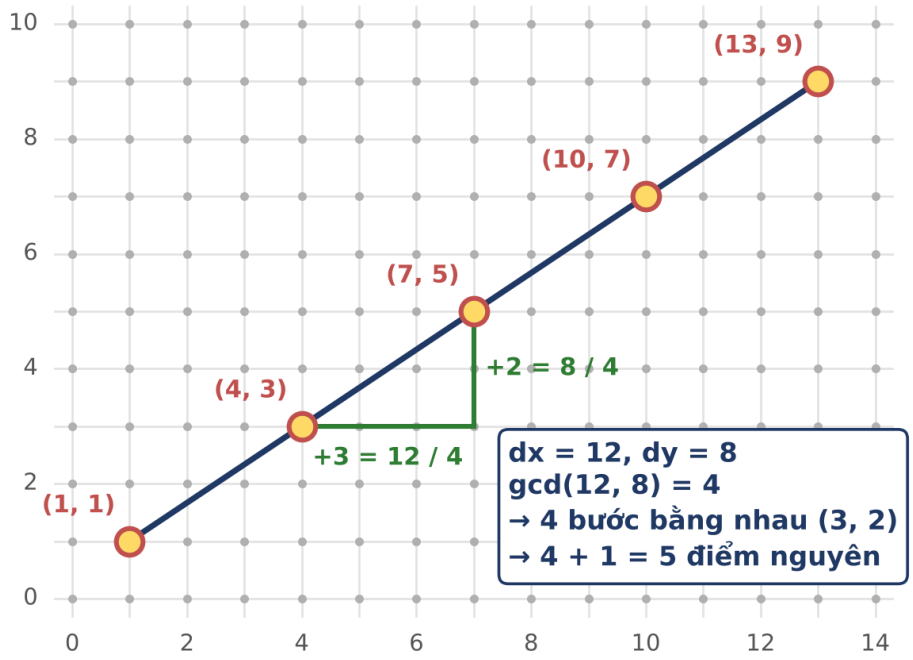
ĐỊNH NGHĨA (ĐIỂM NGUYÊN)

Điểm (x, y) trên mặt phẳng tọa độ gọi là **điểm nguyên** (điểm lưới) nếu cả x và y đều là số nguyên.

Cho hai điểm nguyên $A(x_1, y_1)$ và $B(x_2, y_2)$. Đặt $dx = x_2 - x_1$, $dy = y_2 - y_1$. Mọi điểm trên đoạn AB có dạng

$$\mathbf{(x_1 + t \cdot dx,\ y_1 + t \cdot dy)\ \ với\ \ 0 \le t \le 1}$$

Với t = 0 ta ở A, với t = 1 ta ở B. Câu hỏi là: với những giá trị t nào thì **cả hai** tọa độ đều nguyên? Với A(1, 1), B(13, 9) thì dx = 12, dy = 8. Nếu chia vectơ (12, 8) cho g = gcd(12, 8) = 4, ta được "bước nhảy" (3, 2): từ A nhảy 4 bước (3, 2) là tới B, và mỗi điểm dừng đều là điểm nguyên.



Hình 20.1. *Đoạn từ (1, 1) tới (13, 9): vectơ (12, 8) chia thành 4 bước bằng nhau (3, 2), cho 5 điểm nguyên.*

Liệu còn điểm nguyên nào nằm **giữa** hai điểm dừng không? Không: bước (3, 2) không thể chia nhỏ hơn được nữa vì gcd(3, 2) = 1. Mục 4 sẽ chứng minh điều này một cách chặt chẽ.

2. Bài toán DIEMNGUYEN

BÀI TOÁN: DIEMNGUYEN

Cho hai điểm nguyên $A(x_1, y_1)$ và $B(x_2, y_2)$. Hãy đếm số điểm nguyên nằm trên đoạn thẳng AB, kể cả hai đầu mút.

Dữ liệu vào: Một dòng gồm bốn số nguyên x_1, y_1, x_2, y_2 ($|x_1|, |y_1|, |x_2|, |y_2| \le 10^9$). A và B có thể trùng nhau.

Kết quả: Số điểm nguyên trên đoạn AB.

DIEMNGUYEN.INP	DIEMNGUYEN.OUT	Giải thích
`1 1 13 9`	`5`	dx = 12, dy = 8, gcd = 4: các điểm (1, 1), (4, 3), (7, 5), (10, 7), (13, 9).
`-3 7 -3 -2`	`10`	Đoạn thẳng đứng: mọi điểm (−3, y) với −2 ≤ y ≤ 7, có |−2 − 7| + 1 = 10 điểm.
`-1000000000 -1000000000 1000000000 1000000000`	`2000000001`	dx = dy = $2 \cdot 10^9$ vượt `int`; mọi điểm (t, t) với $\|t\| \le 10^9$.

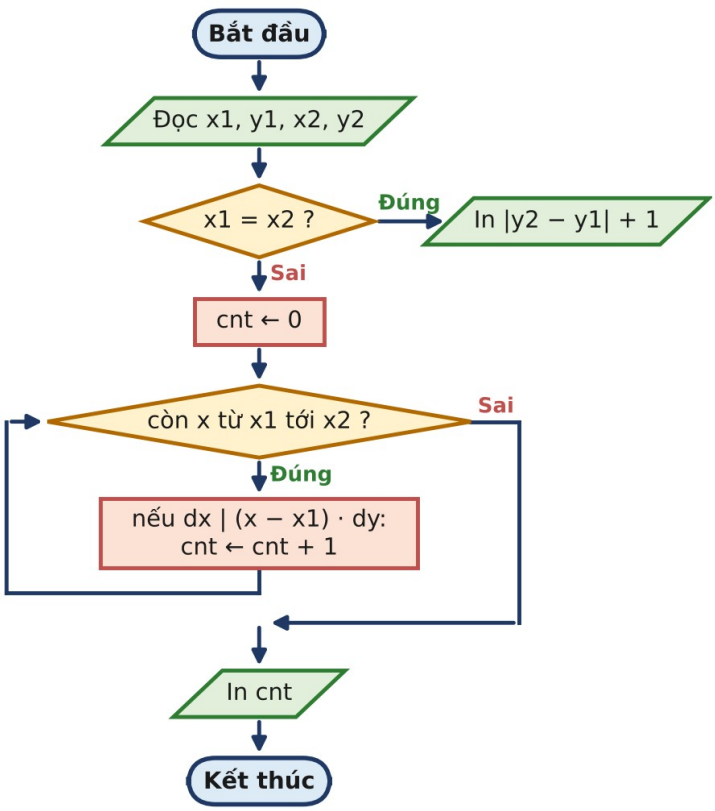
3. Ý tưởng ngây thơ: Duyệt từng hoành độ

Nếu đoạn không thẳng đứng ($dx \neq 0$), mỗi hoành độ x giữa x_1 và x_2 ứng với đúng một điểm trên đường thẳng, có tung độ $y = y_1 + (x - x_1) \cdot dy / dx$. Với x nguyên, điểm ấy là điểm nguyên khi và chỉ khi dx chia hết $(x - x_1) \cdot dy$. Kiểm tra bằng số nguyên như vậy thì tránh được hoàn toàn sai số của số thực. Đoạn thẳng đứng ($dx = 0$) có $|dy| + 1$ điểm nguyên, xét riêng.

Chạy tay với A(1, 1), B(13, 9): $dx = 12$, $dy = 8$.

x	1	2	3	4	5	6	7	8	9	10	11	12	13
$(x - 1) \cdot 8$	0	8	16	24	32	40	48	56	64	72	80	88	96
mod 12	**0**	8	4	**0**	8	4	**0**	8	4	**0**	8	4	**0**
y	**1**			**3**			**5**			**7**			**9**

Số dư lặp lại với chu kì 3 (0, 8, 4, 0, ...): cứ 3 bước theo x lại gặp một điểm nguyên. Đó chính là $12 / \gcd(12, 8) = 3$.



Hình 20.2. *Sơ đồ khối của cách duyệt từng hoành độ.*

DIEMNGUYEN.CPP (ngây thơ): duyệt x, O(|dx|)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("DIEMNGUYEN.INP", "r")) {
        freopen("DIEMNGUYEN.INP", "r", stdin);
        freopen("DIEMNGUYEN.OUT", "w", stdout);
    }
    long long x1, y1, x2, y2;
    cin >> x1 >> y1 >> x2 >> y2;
    if (x1 == x2) {                    // đoạn thẳng đứng (hoặc một điểm)
        cout << llabs(y2 - y1) + 1 << "\n";
        return 0;
    }
    if (x1 > x2) {                     // cho x1 < x2
        swap(x1, x2);
        swap(y1, y2);
    }
    long long dx = x2 - x1, dy = y2 - y1, cnt = 0;
    for (long long x = x1; x <= x2; x++)
        // y = y1 + (x - x1) · dy / dx nguyên <=> dx | (x - x1) · dy
        if ((x - x1) * dy % dx == 0) cnt++;
    cout << cnt << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Vòng lặp chạy $|dx| + 1$ lần. Tích $(x - x_1) \cdot dy$ có thể tới $2 \cdot 10^9 \cdot 2 \cdot 10^9 = 4 \cdot 10^{18}$: vẫn vừa `long long`, nhưng chỉ vừa sát nút.

BẾ TẮC

Với $|dx|$ tới $2 \cdot 10^9$, vòng lặp chạy hai tỉ lần, mỗi lần một phép chia lấy dư 64 bit: khoảng 6–7 giây, vượt xa giới hạn 1 giây. Bảng chạy tay cho thấy các điểm nguyên **cách đều nhau**, chu kì 3 = 12 / 4. Nếu biết trước khoảng cách đều ấy thì không cần duyệt: chỉ cần **chia**.

4. Phá bế tắc: Bước nhảy nhỏ nhất là (dx / g, dy / g)

Bước 1 **Định lí**

ĐỊNH LÍ (ĐIỂM NGUYÊN TRÊN ĐOẠN THẲNG)

Đoạn thẳng nối hai điểm nguyên $A(x_1, y_1)$, $B(x_2, y_2)$ chứa đúng

$$\mathbf{gcd(|x_2 - x_1|, |y_2 - y_1|) + 1}$$

điểm nguyên, kể cả hai đầu mút (quy ước $\gcd(d, 0) = d$ và $\gcd(0, 0) = 0$).

VÌ SAO ĐÚNG?

Nếu A ≡ B thì g = 0 và có 1 điểm: đúng. Giả sử A ≠ B, đặt g = gcd(|dx|, |dy|) > 0, dx = g · u, dy = g · v với gcd(u, v) = 1.

Các điểm $P_t = (x_1 + t \cdot u, y_1 + t \cdot v)$, t = 0, 1, ..., g đều là điểm nguyên trên đoạn AB: chúng ứng với tham số t / g ∈ [0, 1], và tọa độ là số nguyên. Có g + 1 điểm như vậy, đôi một khác nhau.

Không còn điểm nguyên nào khác: giả sử $(x_1 + X, y_1 + Y)$ là một điểm nguyên trên đoạn, với X = s · dx, Y = s · dy (0 ≤ s ≤ 1) là hai số nguyên. Vì X · dy = s · dx · dy = Y · dx nên X · g · v = Y · g · u, tức X · v = Y · u. Như vậy u chia hết X · v, mà gcd(u, v) = 1 nên u | X (bổ đề ở Bài 3–4: nếu a | b · c và gcd(a, b) = 1 thì a | c).

Nếu u ≠ 0, đặt t = X / u (số nguyên). Khi đó s = X / dx = t / g và Y = s · g · v = t · v, nên điểm ấy chính là P_t; điều kiện 0 ≤ s ≤ 1 cho 0 ≤ t ≤ g. Nếu u = 0 thì |v| = gcd(0, v) = 1, và lập luận y hệt với vai trò của X, Y đổi cho nhau. Vậy có đúng g + 1 điểm nguyên.

Bước 2 Các trường hợp đặc biệt

Công thức gcd(|dx|, |dy|) + 1 tự động đúng cho mọi trường hợp, miễn là hàm gcd xử lí đúng số 0:

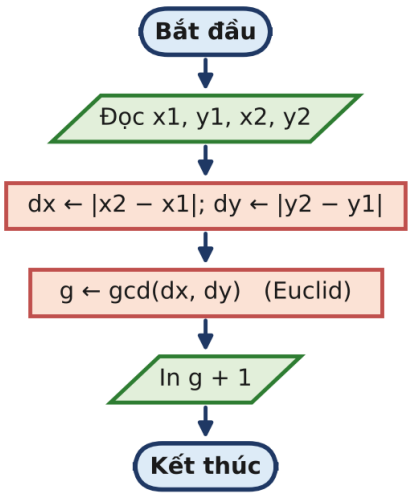
Trường hợp	Ví dụ	gcd	Số điểm
Thẳng đứng (dx = 0)	(−3, 7) → (−3, −2)	gcd(0, 9) = 9	10
Nằm ngang (dy = 0)	(0, 0) → (5, 0)	gcd(5, 0) = 5	6
Suy biến (A ≡ B)	(4, 4) → (4, 4)	gcd(0, 0) = 0	1
dx, dy nguyên tố cùng nhau	(0, 0) → (3, 2)	gcd(3, 2) = 1	2 (chỉ hai đầu)
Tọa độ âm, đi ngược	(13, 9) → (1, 1)	gcd(12, 8) = 4	5

Phải lấy **trị tuyệt đối** của dx, dy: với hàm gcd tự viết, số âm có thể làm kết quả âm. Và dx có thể tới $2 \cdot 10^9$, vượt `int`.

Bước 3 Chạy tay ví dụ 1

dx = 13 − 1 = 12, dy = 9 − 1 = 8. Thuật toán Euclid (Bài 3): gcd(12, 8) = gcd(8, 12 mod 8 = 4) = gcd(4, 8 mod 4 = 0) = 4. Đáp số 4 + 1 = 5, khớp với bảng của cách ngây thơ.

Bước 4 Chương trình



Hình 20.3. Sơ đồ khối của cách dùng ƯCLN.

DIEMNGUYEN.CPP: gcd(|dx|, |dy|) + 1

```cpp
#include <bits/stdc++.h>
using namespace std;

long long gcd(long long a, long long b) {   // Euclid (Bài 3)
    return b == 0 ? a : gcd(b, a % b);
}

int main() {
    if (fopen("DIEMNGUYEN.INP", "r")) {
        freopen("DIEMNGUYEN.INP", "r", stdin);
        freopen("DIEMNGUYEN.OUT", "w", stdout);
    }
    long long x1, y1, x2, y2;
    cin >> x1 >> y1 >> x2 >> y2;
    long long dx = llabs(x2 - x1), dy = llabs(y2 - y1);
    cout << gcd(dx, dy) + 1 << "\n";   // gcd(0, 0) = 0: đoạn suy biến
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Thuật toán Euclid trên các số $\le 2 \cdot 10^9$ chỉ cần vài chục phép chia (Bài 3: số bước tỉ lệ với log của số nhỏ hơn). Từ hai tỉ bước xuống **dưới 50 bước**.

5. Mở rộng

Mở rộng 1: Diện tích đa giác, công thức dây giày

Cho đa giác (không tự cắt) với các đỉnh (x_0, y_0), ..., (x_{n-1}, y_{n-1}) liệt kê theo thứ tự đi quanh biên. Diện tích của nó là

$$S = \left| \sum_{i=0..n-1} (x_i \cdot y_{i+1} - x_{i+1} \cdot y_i) \right| / 2, \quad \text{với } (x_n, y_n) = (x_0, y_0)$$

Mỗi số hạng là hai lần diện tích có dấu của tam giác tạo bởi gốc tọa độ và một cạnh; khi đi hết một vòng, phần nằm ngoài đa giác triệt tiêu nhau. Tên "dây giày" đến từ cách viết các tọa độ thành hai cột rồi nhân chéo như xỏ dây giày. Với đỉnh nguyên, **2S luôn là số nguyên**: hãy tính 2S bằng `long long` và chỉ chia 2 khi in.

Đỉnh i	Đỉnh i + 1	$x_i \cdot y_{i+1} - x_{i+1} \cdot y_i$	gcd của cạnh
(0, 0)	(6, 0)	$0 \cdot 0 - 6 \cdot 0 = 0$	6
(6, 0)	(6, 4)	$6 \cdot 4 - 6 \cdot 0 = 24$	4
(6, 4)	(2, 6)	$6 \cdot 6 - 2 \cdot 4 = 28$	2
(2, 6)	(0, 3)	$2 \cdot 3 - 0 \cdot 6 = 6$	1
(0, 3)	(0, 0)	$0 \cdot 0 - 0 \cdot 3 = 0$	3
	Tổng	**58**, nên S = 29	**16**

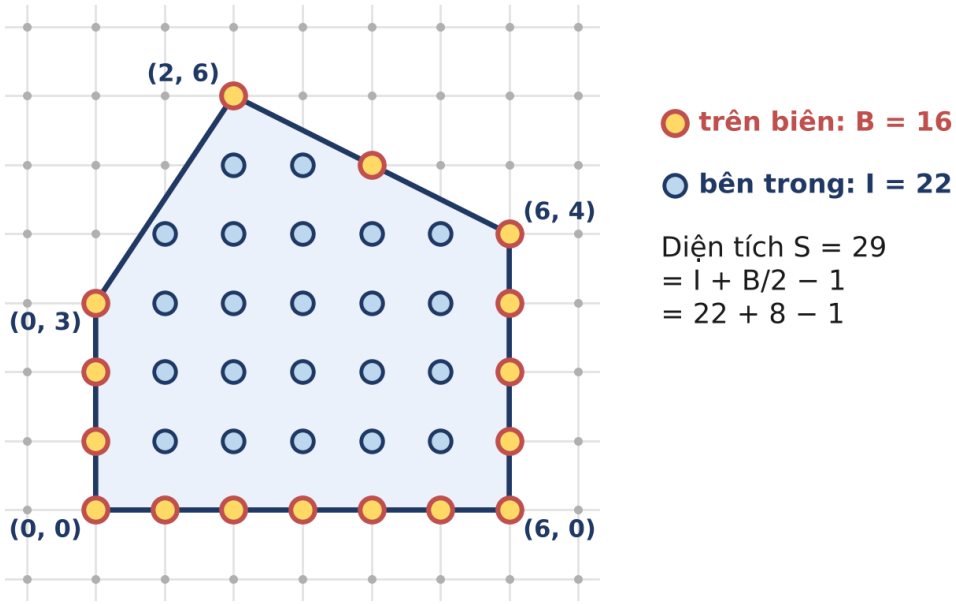
Mở rộng 2: Định lí Pick

ĐỊNH LÍ (PICK)

Với đa giác không tự cắt có mọi đỉnh là điểm nguyên, gọi S là diện tích, B là số điểm nguyên trên biên, I là số điểm nguyên nằm hẳn bên trong. Khi đó

$$S = I + B / 2 - 1$$

Số B tính được ngay nhờ bài toán DIEMNGUYEN: mỗi cạnh có gcd(|dx|, |dy|) + 1 điểm, nhưng mỗi đỉnh bị đếm ở hai cạnh, nên B = Σ gcd(|dx|, |dy|) theo các cạnh. Từ đó **I = (2S − B + 2) / 2**: đếm được hàng tỉ điểm bên trong mà không cần duyệt điểm nào.



Hình 20.4. *Đa giác 5 đỉnh: 16 điểm trên biên, 22 điểm bên trong, diện tích 29 = 22 + 16/2 − 1.*

VÌ SAO ĐÚNG (VỚI HÌNH CHỮ NHẬT)?

Xét hình chữ nhật $a \times b$ có cạnh song song trục tọa độ. Bên trong có $(a-1)\cdot(b-1)$ điểm, trên biên có $2a + 2b$ điểm. Khi đó $I + B/2 - 1 = (a-1)(b-1) + (a+b) - 1 = ab = S$. Chứng minh đầy đủ đi theo hai bước: kiểm tra cho tam giác vuông (một nửa hình chữ nhật), rồi chỉ ra đại lượng $I + B/2 - 1$ **cộng được** khi ghép hai đa giác dọc một cạnh chung, giống như diện tích.

Chương trình dưới đọc đa giác (số đỉnh n rồi tọa độ các đỉnh theo thứ tự), in diện tích S (thêm đuôi `.5` khi 2S lẻ), B và I. Với đa giác trên (5 đỉnh (0, 0), (6, 0), (6, 4), (2, 6), (0, 3)), chương trình in `29 16 22`. Để tích $x_i \cdot y_{i+1}$ không tràn, giả thiết |tọa độ| ≤ 10^6 (với tọa độ tới 10^9 hãy dùng `__int128`).

PICK.CPP: dây giày + Pick, O(n log C)

```cpp
#include <bits/stdc++.h>
using namespace std;

long long gcd(long long a, long long b) {
    return b == 0 ? a : gcd(b, a % b);
}

int main() {
    if (fopen("PICK.INP", "r")) {
        freopen("PICK.INP", "r", stdin);
        freopen("PICK.OUT", "w", stdout);
    }
    int n;
    cin >> n;                           // đa giác n đỉnh, liệt kê theo thứ tự
    vector<long long> x(n), y(n);
    for (int i = 0; i < n; i++) cin >> x[i] >> y[i];
    long long S2 = 0, B = 0;            // S2 = 2 lần diện tích
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;            // cạnh nối đỉnh i với đỉnh i + 1
        S2 += x[i] * y[j] - x[j] * y[i];          // công thức dây giày
        B += gcd(llabs(x[j] - x[i]), llabs(y[j] - y[i]));
    }
    S2 = llabs(S2);
    long long I = (S2 - B + 2) / 2;   // Pick: S = I + B/2 - 1
    cout << S2 / 2 << (S2 % 2 ? ".5" : "") << " " << B << " " << I << "\n";
    return 0;
}
```

Mở rộng 3: Điểm nguyên trên đường thẳng ax + by = c

Các điểm nguyên trên đường thẳng ax + by = c chính là các nghiệm nguyên của phương trình ở Bài 4. Nếu (x_0, y_0) là một nghiệm và g = gcd(a, b) thì mọi nghiệm có dạng $(x_0 + t \cdot b / g, y_0 - t \cdot a / g)$: lại là bước nhảy nhỏ nhất (b / g, −a / g), đúng như vectơ (dx / g, dy / g) của bài này. Hai bài toán là hai mặt của cùng một ý tưởng.

BẪY THƯỜNG GẶP

- **Quên trị tuyệt đối:** dx, dy âm khi B nằm bên trái hoặc phía dưới A. Lấy `llabs` trước khi gọi gcd.
- **Dùng `int` cho hiệu tọa độ:** $x_2 - x_1$ có thể tới $2 \cdot 10^9 > 2^{31} - 1$.
- **Dùng số thực để kiểm tra điểm nguyên:** $y = y_1 + (x - x_1) \cdot dy / dx$ tính bằng `double` có sai số; kiểm tra bằng phép chia hết số nguyên.
- **Đếm trùng đỉnh khi tính B:** mỗi cạnh góp gcd(|dx|, |dy|) điểm (không cộng 1), vì mỗi đỉnh thuộc hai cạnh.
- **Áp dụng Pick sai chỗ:** định lí chỉ đúng khi mọi đỉnh nguyên và đa giác không tự cắt, không có lỗ.

GHI NHỚ

- Đoạn nối hai điểm nguyên có gcd(|dx|, |dy|) + 1 điểm nguyên, cách đều nhau bởi bước (dx / g, dy / g).
- Quy ước gcd(d, 0) = d, gcd(0, 0) = 0 làm công thức đúng cả khi đoạn đứng, ngang hay suy biến.
- Công thức dây giày: $2S = |\Sigma (x_i\, y_{i+1} - x_{i+1}\, y_i)|$, luôn nguyên với đỉnh nguyên.
- Định lí Pick: $S = I + B/2 - 1$, với $B = \Sigma$ gcd các cạnh; suy ra $I = (2S - B + 2) / 2$.

Luyện tập

Bài	Tên	Gợi ý liên hệ
20.1	**TRỒNG CÂY**	Đếm điểm nguyên trên đường gấp khúc qua n điểm: cộng gcd từng đoạn, trừ các đỉnh chung.
20.2	**ĐA GIÁC LƯỚI**	Cho đa giác $n \le 10^5$ đỉnh nguyên, đếm điểm nguyên nằm hẳn bên trong: dây giày + Pick.
20.3	**TAM GIÁC NGUYÊN**	Đếm điểm nguyên trong tam giác tọa độ $\le 10^9$: Pick với 2S tính bằng `__int128`.

Tự kiểm tra nhanh

1) Đoạn thẳng nối (0, 0) và (6, 9) chứa bao nhiêu điểm nguyên? Liệt kê chúng. *(Đáp số: gcd(6, 9) + 1 = 4 điểm: (0, 0), (2, 3), (4, 6), (6, 9))*

2) Đoạn thẳng nối (−4, 7) và (8, −2) chứa bao nhiêu điểm nguyên? *(Đáp số: gcd(12, 9) + 1 = 4)*

3) Tam giác có đỉnh (0, 0), (4, 0), (0, 4): tính S, B, I. *(Đáp số: S = 8, B = 4 + 4 + 4 = 12, I = (16 − 12 + 2) / 2 = 3)*

PHỤ LỤC A. TỔNG HỢP THUẬT TOÁN VÀ CÔNG THỨC

Bảng dưới đây tóm tắt các thuật toán và công thức chính của cuốn sách, cùng độ phức tạp và bài học tương ứng. Hãy dùng nó như một "bản đồ" khi ôn tập: nhìn vào cột đầu, em có nhớ lại được ý tưởng và tự viết được chương trình không?

Chương 1. CHIA HẾT VÀ ƯỚC SỐ

Thuật toán / công thức	Độ phức tạp	Bài
Đếm ước, tổng ước bằng cặp ước (d, n / d)	$O(\sqrt{n})$	Bài 2
Sàng điều hòa: d(k), σ(k) với mọi k ≤ N	O(N log N)	Bài 2
Thuật toán Euclid: gcd(a, b) = gcd(b, a mod b)	O(log min(a, b))	Bài 3
BCNN không tràn: a / gcd(a, b) · b, kiểm tra trước khi nhân	O(log min(a, b))	Bài 3
gcd tiền tố / hậu tố cho bài "bỏ một phần tử"	O(n log max)	Bài 3
Euclid mở rộng: x, y với ax + by = gcd(a, b)	O(log min(a, b))	Bài 4
Phương trình ax + by = c: Bézout + họ nghiệm x_0 + (b/g)t	O(log min(a, b))	Bài 4

Chương 2. SỐ NGUYÊN TỐ

Thuật toán / công thức	Độ phức tạp	Bài
Kiểm tra nguyên tố: thử 2, 3 và 6k ± 1 tới $\sqrt{n}$	$O(\sqrt{n})$	Bài 5
Miller–Rabin tất định (12 cơ sở, $n < 2^{64}$)	O(12 · log n) phép nhân	Bài 5
Sàng Eratosthenes tới N	O(N log log N)	Bài 6
Đếm số nguyên tố trên đoạn bằng mảng tiền tố	O(1) mỗi truy vấn	Bài 6
Sàng tuyến tính (kèm ước nguyên tố nhỏ nhất)	O(N)	Bài 6
Phân tích thừa số bằng thử chia tới $\sqrt{n}$	$O(\sqrt{n})$	Bài 7
Sàng ước nguyên tố nhỏ nhất spf tới N	O(N log log N)	Bài 7
Phân tích bằng mảng spf	O(log n)	Bài 7
Số ước d(n), tổng ước σ(n) từ phân tích	O(số thừa số)	Bài 7
Công thức Legendre: số mũ của p trong n!	$O(\log_p n)$	Bài 7
Sàng trên đoạn [L, R]	$O(\sqrt{R} \log\log \sqrt{R} + (R - L) \log\log R)$	Bài 8

Chương 3. SỐ HỌC ĐỒNG DƯ

Thuật toán / công thức	Độ phức tạp	Bài
Lũy thừa nhanh a^b mod m (bình phương liên tiếp)	O(log b)	Bài 9
Nhân Ấn Độ a · b mod m (không cần `__int128`)	O(log b)	Bài 9
Tổng cấp số nhân 1 + a + ... + a^{n-1} mod m	O(log n)	Bài 9

Thuật toán / công thức	Độ phức tạp	Bài
Lũy thừa với số mũ là xâu L chữ số	O(L)	Bài 9
Nghịch đảo theo mô-đun nguyên tố: $b^{p-2} \bmod p$ (định lí Fermat)	O(log p)	Bài 10
Nghịch đảo theo mô-đun bất kì: Euclid mở rộng	O(log m)	Bài 10
Nghịch đảo của 1..n: inv[i] = (p − ⌊p/i⌋) · inv[p mod i]	O(n)	Bài 10
Phương trình đồng dư a · x ≡ c (mod m)	O(log m)	Bài 10
φ(n) bằng thử chia tới √n, res ← res − res / p	O(√n)	Bài 11
Sàng φ cho mọi số 1..N	O(N log log N)	Bài 11
a mũ $(b^c) \bmod p$: rút gọn số mũ theo p − 1	O(log c + log p)	Bài 11
Đếm phân số tối giản mẫu ≤ N: tổng φ(2..N)	O(N log log N)	Bài 11
Hệ đồng dư: gộp từng cặp bằng Euclid mở rộng (mô-đun bất kì)	O(k log m)	Bài 12
Công thức Tôn Tử (mô-đun đôi một nguyên tố cùng nhau)	O(k log m)	Bài 12
Duyệt nghiệm theo bước m_1 (ngây thơ)	$O(k \cdot L / m_1)$	Bài 12

Chương 4. TỔ HỢP VÀ ĐẾM

Thuật toán / công thức	Độ phức tạp	Bài
Tam giác Pascal C(n, k) mod m	$O(n^2)$	Bài 13
C(n, k) mod p bằng giai thừa và nghịch đảo giai thừa	O(n + log p) chuẩn bị, O(1) mỗi truy vấn	Bài 13
C(n, k) mod p nhân dần (n lớn, k nhỏ)	O(k + log p)	Bài 13
Định lí Lucas (p nguyên tố nhỏ)	$O(p + \log_p n)$	Bài 13
Nguyên lí bù trừ trên k số (đếm bội trong [1, N])	$O(2^k \cdot \log N)$	Bài 14
Đếm số nguyên tố cùng nhau với n trong [1, N]	$O(\sqrt{n} + 2^{\omega(n)})$	Bài 14
Số hoán vị không điểm cố định D(n)	O(n)	Bài 14
Số Fibonacci bằng lũy thừa nhanh ma trận	O(log n)	Bài 15
Truy hồi tuyến tính bậc k (ma trận đồng hành)	$O(k^3 \log n)$	Bài 15
Số đường đi độ dài n trên đồ thị V đỉnh	$O(V^3 \log n)$	Bài 15
Fibonacci bằng công thức nhân đôi	O(log n)	Bài 15
Chia khối ⌊n / i⌋ (tổng số ước của 1..n)	O(√n)	Bài 16
Phương pháp hyperbol	O(√n)	Bài 16
Tổng σ(1..n), tổng n mod i bằng chia khối	O(√n)	Bài 16

Chương 5. KĨ THUẬT SỐ HỌC TRONG LẬP TRÌNH

Thuật toán / công thức	Độ phức tạp	Bài
Nhân số lớn với số nhỏ x (mang số nhớ, số nhớ < x)	O(L)	Bài 17
In chính xác n! (cơ số 10^9)	O(n · L), L = số ô	Bài 17
Cộng, trừ, so sánh hai số lớn	O(L)	Bài 17
Nhân hai số lớn (cộng dồn c[i + j])	$O(L_1 \cdot L_2)$	Bài 17
Chia số lớn cho số nhỏ (Horner từ hàng cao)	O(L)	Bài 17
Đổi sang cơ số b: chia liên tiếp, đọc ngược số dư	$O(\log_b n)$	Bài 18
Tổng số bit 1 của 1..n: đếm theo cột, chu kì 2^{k+1}	O(log n)	Bài 18
Tổng số bit 1 của 0..n: tách bit cao nhất (đệ quy)	O(log n)	Bài 18
XOR 1..n theo n mod 4	O(1)	Bài 18
Duyệt mọi tập con bằng bit mask	$O(2^n \cdot n)$	Bài 18
Tổng tiền tố, tổng đoạn $S_r - S_{l-1}$	O(n) chuẩn bị, O(1) mỗi truy vấn	Bài 19
Đếm đoạn con có tổng chia hết cho k (đếm theo số dư)	O(n + k)	Bài 19
Đoạn con dài nhất chia hết cho k (vị trí đầu của mỗi số dư)	O(n + k)	Bài 19
Tìm đuôi μ và chu kì λ của dãy $x_{i+1} = f(x_i) \bmod m$	O(m)	Bài 19
Số điểm nguyên trên đoạn thẳng: gcd(|dx|, |dy|) + 1	O(log C)	Bài 20
Diện tích đa giác: công thức dây giày (tính 2S)	O(n)	Bài 20
Định lí Pick: I = (2S − B + 2) / 2, B = Σ gcd cạnh	O(n log C)	Bài 20

PHỤ LỤC B. NHỮNG CON SỐ ĐÁNG NHỚ

Khi đọc ràng buộc của đề bài, những con số dưới đây giúp em ước lượng ngay: vòng lặp chạy bao nhiêu lần, mảng cần bao nhiêu phần tử, kết quả có vừa kiểu dữ liệu không. Mọi giá trị đều được tính lại bằng máy.

Giới hạn của các kiểu số nguyên

Kiểu	Giá trị lớn nhất	Xấp xỉ
`int`	$2^{31} - 1 = 2\,147\,483\,647$	$2{,}1 \cdot 10^{9}$
`unsigned int`	$2^{32} - 1 = 4\,294\,967\,295$	$4{,}3 \cdot 10^{9}$
`long long`	$2^{63} - 1 = 9\,223\,372\,036\,854\,775\,807$	$9{,}2 \cdot 10^{18}$
`unsigned long long`	$2^{64} - 1$	$1{,}8 \cdot 10^{19}$
`__int128`	$2^{127} - 1$	$1{,}7 \cdot 10^{38}$

Số lượng số nguyên tố π(N) và số ước lớn nhất

N	π(N): số nguyên tố ≤ N	Số ước nhiều nhất của một số ≤ N	Đạt tại
10	4	4	6
10^2	25	12	60
10^3	168	32	840
10^4	1 229	64	7 560
10^5	9 592	128	83 160
10^6	78 498	240	720 720
10^7	664 579	448	8 648 640
10^8	5 761 455	768	73 513 440
10^9	50 847 534	1 344	735 134 400
10^{12}	37 607 912 018	6 720	963 761 198 400
10^{18}	24 739 954 287 740 860	103 680	897 612 484 786 617 600

Các mô-đun nguyên tố hay gặp

Số	Ghi chú
$10^9 + 7 = 1\,000\,000\,007$	Số nguyên tố; tích hai số dư < 10^{18} vừa `long long`.
$10^9 + 9$	Số nguyên tố, hay dùng cùng $10^9 + 7$ trong băm hai mô-đun.
$998\,244\,353 = 119 \cdot 2^{23} + 1$	Số nguyên tố, dùng trong biến đổi số học nhanh (NTT).
$2^{61} - 1$	Số nguyên tố Mersenne, dùng trong băm, cần nhân `__int128`.